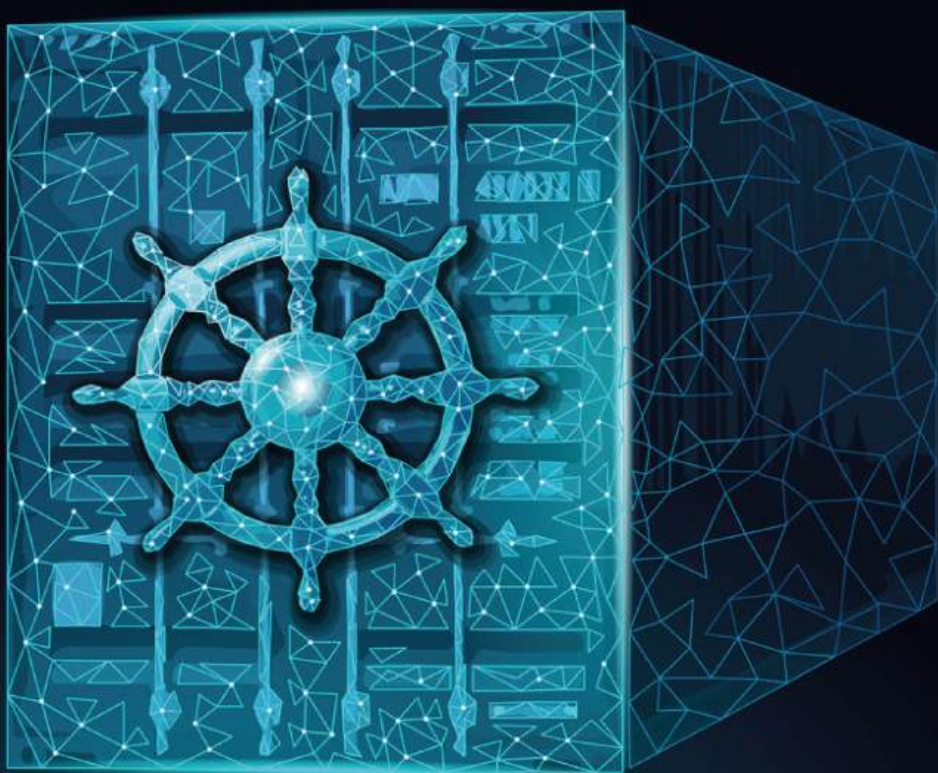


Аймен Эль Амри

ОБЛАЧНЫЕ микросервисы



***Разработка и применение
на платформе Kubernetes***

Аймен Эль Амри

Облачные микросервисы

Aymen El Amri

Cloud Native Microservices With Kubernetes

**A Comprehensive Guide to Building, Scaling,
Deploying, Observing, and Managing
Highly-Available Microservices in Kubernetes**

Аймен Эль Амри

Облачные микросервисы

Разработка и применение
на платформе Kubernetes



Москва, 2025

УДК 004.738.5:004.42Kubernetes
ББК 16.263
А62

Аймен Эль Амри

А62 Облачные микросервисы / пер. с англ. В. С. Яценкова. – М.: ДМК Пресс, 2024. – 276 с.: ил.

ISBN 978-5-93700-324-9

Данная книга раскроет весь потенциал Kubernetes – самой популярной платформы оркестровки контейнеров в мире. Вы узнаете, как можно использовать Kubernetes для создания надежной, масштабируемой и отказоустойчивой микросервисной архитектуры. Подробно раскрыты особенности применения микросервисов, стратегий высокой доступности, CI/CD, GitOps и наблюдаемости в мире облачных технологий. Представлены практические примеры применения таких инструментов, как Docker, Rancher, Terraform, Operators, Helm, Prometheus, Istio, Grafana, OpenTelemetry, Jaeger, Loki, и других.

Издание предназначено инженерам DevOps, архитекторам ПО, а также будет полезно системным администраторам и разработчикам микросервисов.

УДК 004.738.5:004.42Kubernetes
ББК 16.263

Title of English-language edition: “Cloud Native Microservices With Kubernetes”, published by FAUN – www.faun.dev.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-5-93700-324-9

Copyright © 2024 All rights reserved, Aymen EL Amri.
© Перевод, оформление, издание, ДМК Пресс, 2024

Содержание

От издательства	12
Глава 1. Облачные микросервисы – почему они появились	13
1.1. Общие принципы и закономерности	13
1.2. Методология двенадцати факторов	14
1.2.1. Фактор I. Кодовая база	14
1.2.2. Фактор II. Зависимости	14
1.2.3. Фактор III. Конфигурация	15
1.2.4. Фактор IV. Внешние сервисы	15
1.2.5. Фактор V. Сборка, релиз, выполнение	16
1.2.6. Фактор VI. Процессы	17
1.2.7. Фактор VII. Привязка портов	17
1.2.8. Фактор VIII. Параллелизм	18
1.2.9. Фактор IX. Утилизируемость	19
1.2.10. Фактор X. Паритет разработки и производства	19
1.2.11. Фактор XI. Журналирование	19
1.2.12. Фактор XII. Административные процессы	20
1.3. Микросервисы	21
1.3.1. Отдельная база данных для каждого сервиса	22
1.3.2. API Composition	22
1.3.3. Отдельный экземпляр сервиса на каждый контейнер	22
1.3.4. Внешняя конфигурация	23
1.3.5. Обнаружение сервисов на стороне сервера	23
1.3.6. Автоматический предохранитель	23
1.3.7. Облачный подход	24
1.4. От монолита к облачным микросервисам	24
Глава 2. Технические требования	26
2.1. Сервер разработки	26
2.2. Установка kubectl	28
Глава 3. Кластеры Kubernetes	29
3.1. Создание кластера Kubernetes для разработки с помощью Minikube	29
3.1.1. Установка Minikube	30
3.1.2. Создание кластера Minikube	31
3.1.3. Профили Minikube	32
3.1.4. Использование панели управления K8s с Minikube	33
3.1.5. Создание развертывания	34

3.1.6. События Kubernetes	34
3.1.7. Раскрытие развертывания	34
3.1.8. Удаление ресурсов K8s	35
3.1.9. Дополнения Minikube	35
3.1.10. Использование kubectl без расширения minikube	36
3.1.11. Удаление кластеров	36
3.2. Создание кластера Kubernetes для разработки с помощью Rancher	37
3.2.1. Технические требования	37
3.2.2. Использование Terraform для запуска кластера	37
3.2.3. Создание ресурсов Kubernetes с помощью пользовательского интерфейса Rancher	38
3.3. Создание локального кластера Kubernetes с помощью Rancher	40
3.3.1. Начальные требования	40
3.3.2. Создание кластера с помощью сервера Rancher	40
3.3.3. Примечание о высокой доступности	42
3.4. Создание локального кластера Kubernetes: другие варианты	43
3.5. Управляемые кластеры	43
3.6. Создание управляемого кластера DOK с помощью Terraform	44

Глава 4. Обзор архитектуры Kubernetes

4.1. Введение	47
4.2. Плоскость управления	47
4.2.1. Система хранения etcd	47
4.2.2. Сервер API (kube-apiserver)	48
4.2.3. Менеджер контроллеров (kube-controller-manager)	48
4.2.4. Облачный менеджер контроллеров (cloud-controller-manager)	48
4.2.5. Планировщик (kube-scheduler)	48
4.3. Узлы	48
4.3.1. Kubelet	48
4.3.2. Среда выполнения контейнера	49
4.3.3. Kube-proxy	49
4.4. Пулы узлов	49
4.5. Обзор архитектуры	50

Глава 5. Микросервисы без сохранения и с сохранением состояния

5.1. Введение	51
5.2. Рабочие нагрузки без сохранения состояния	51
5.3. Рабочие нагрузки с сохранением состояния	52

Глава 6. Развертывание микросервисов без сохранения состояния

6.1. Технические требования	54
6.2. Создание пространства имен	57
6.3. Создание развертывания	58
6.4. Проверка подов и развертываний	60
6.5. Доступ к подам	61

6.6. Предоставление доступа к разворачиванию	62
6.6.1. Сервис ClusterIP	62
6.6.2. Сервис NodePort	64
6.6.3. Сервис LoadBalancer	66
6.6.4. Сервис Headless	68
6.6.5. Сервис Ingress	70

Глава 7. Микросервисы с сохранением состояния в Kubernetes

7.1. Технические требования	76
7.2. Создание пространства имен	77
7.3. Создание ConfigMap для базы данных PostgreSQL	77
7.3.1. Что такое ConfigMap?	77
7.3.2. ConfigMap для PostgreSQL	78
7.4. Постоянное хранение данных в PostgreSQL	79
7.4.1. Тома Kubernetes	79
7.4.2. Объект VolumeClaims	79
7.4.3. Объект StorageClass	79
7.4.4. Добавление хранилища в PostgreSQL	80
7.4.5. Создание разворачивания для PostgreSQL	81
7.4.6. Создание сервиса для PostgreSQL	83
7.4.7. Создание разворачивания для нашего приложения	83
7.4.8. Создание сервиса для нашего приложения	86
7.4.9. Создание внешнего сервиса для нашего приложения	86
7.4.10. Создание Ingress для нашего приложения	87
7.5. Просмотр журналов и проверка работоспособности	87
7.6. Заключение	89

Глава 8. Разворачивание микросервисов с сохранением состояния: StatefulSet

8.1. Что такое StatefulSet?	90
8.2. StatefulSet или разворачивание?	91
8.3. Создание StatefulSet	91
8.4. Создание сервиса для StatefulSet	92
8.5. Задачи после разворачивания	93
8.6. StatefulSet или разворачивание: постоянное хранилище	94
8.7. StatefulSet или разворачивание: связанный сервис	96

Глава 9. Паттерны микросервисов: внешние конфигурации

9.1. Хранение конфигураций в среде	97
9.2. Секреты и переменные среды Kubernetes: зачем они нужны?	98
9.3. Секреты и переменные среды Kubernetes: как это работает?	98

Глава 10. Важные приемы работы с микросервисами: проверки состояния

10.1. Проверки состояния приложения	104
---	-----

10.2. Зонды жизнеспособности и готовности	105
10.3. Типы зондов.....	106
10.4. Реализация зондов.....	106

Глава 11. Стратегии управления ресурсами микросервисов

11.1. Управление ресурсами и риски: от Docker до Kubernetes.....	110
11.2. Запросы и лимиты	111
11.3. Единицы измерения ресурсов ЦП.....	112
11.4. Единицы измерения ресурсов памяти	114
11.5. Принципы настройки запросов и ограничений ресурсов	116
11.6. Резервированные и выделяемые ресурсы узла.....	117
11.7. Классы качества обслуживания (QoS)	118
11.7.1. Guaranteed	119
11.7.2. Burstable.....	119
11.7.3. BestEffort.....	121
11.7.4. Проверка класса пода по QoS	121
11.7.5. Порядок вытеснения	122
11.7.6. Пользовательский класс PriorityClass	122

Глава 12. Автоматическое масштабирование микросервисов в Kubernetes: введение

12.1. Лучшие методы масштабирования микросервисов	125
12.1.1. Использование реестра для обнаружения сервисов.....	125
12.1.2. Реализация проверок работоспособности.....	126
12.1.3. Проектирование с учетом масштабируемости и другие передовые практики	126

Глава 13. Горизонтальное автомасштабирование микросервисов в Kubernetes

13.1. Что такое горизонтальное масштабирование	128
13.2. Горизонтальное автомасштабирование подов.....	132
13.3. Автоматическое масштабирование на основе пользовательских метрик Kubernetes.....	136
13.4. Специализированные пользовательские метрики Kubernetes.....	137
13.5. Использование нескольких метрик	139
13.6. Автоматическое масштабирование на основе пользовательских метрик, отличных от Kubernetes.....	140
13.7. Автомасштабирование кластера.....	141

Глава 14. Вертикальное автомасштабирование микросервисов в Kubernetes

14.1. Вертикальное масштабирование – в чем отличие.....	144
14.2. Вертикальное автомасштабирование пода.....	146
14.3. Режимы VPA	151
14.3.1. Auto (автоматический).....	151

14.3.2. Initial (начальный)	151
14.3.3. Recreate (повторное создание)	152
14.3.4. Off (выключен)	152
14.4. Рекомендации VPA.....	152
14.4.1. Ограничения VPA.....	154

Глава 15. Масштабирование микросервисов, сохраняющих состояние на примере PostgreSQL..... 156

15.1. StatefulSet и масштабирование.....	156
15.2. Stolon: знакомство	158
15.3. Stolon: установка	160
15.4. Stolon: применение	161

Глава 16. Стратегии развертывания микросервисов: один сервис на узел..... 165

16.1. Назначение и варианты использования DaemonSet	165
16.2. Создание и управление DaemonSet.....	166

Глава 17. Стратегии развертывания микросервисов: назначение рабочих нагрузок определенным узлам 169

17.1. Зачем назначать рабочие нагрузки определенным узлам?.....	169
17.2. Ограничения и допуски.....	171
17.2.1. Определение	171
17.2.2. Пример ограничений и допусков	172
17.3. nodeSelector	175
17.3.1. Простейшая форма привязки к узлам	175
17.3.2. Пример привязки через nodeSelector	175
17.4. Сходство и антисходство узлов	176
17.4.1. Сходство узлов: как nodeSelector, но с большим количеством опций	176
17.4.2. Сходство узлов: пример	177
17.4.3. Антисходство узлов: пример	179
17.4.4. Вес сходства.....	181
17.4.5. Типы сходства и антисходства	182

Глава 18. Kubernetes: управление обновлениями инфраструктуры и режим обслуживания..... 184

18.1. Зачем нам нужно обновлять инфраструктуру?	184
18.2. Что нужно обновлять?	184
18.3. Обновление рабочих узлов.....	185
18.4. Обновление рабочих узлов: отмена кордона	186
18.5. Обновление пулов узлов	187
18.6. Обновления с нулевым временем простоя: бюджеты сбоев пода	187

Глава 19. Стратегии развертывания микросервисов:

управление обновлениями и развертывание приложений	190
19.1. Изначально облачная система: концепция	190
19.2. Стратегии развертывания	191
19.2.1. Сине-зеленое развертывание: введение.....	191
19.2.2. Канареечное развертывание: введение	193
19.2.3. Канареечное развертывание: пример использования Istio.....	194
19.2.4. Канареечное развертывание: тестирование в рабочей среде	202
19.3. Скользящее обновление.....	204
19.3.1. Скользящие обновления: пример	205

Глава 20. Наблюдаемость микросервисов в мире Kubernetes 211

20.1. Введение	211
20.2. Что такое мониторинг?	211
20.3. Что такое наблюдаемость?.....	212
20.4. Мониторинг: белый ящик или черный?	212
20.5. Основные компоненты наблюдаемости.....	213
20.5.1. Журналы.....	213
20.5.2. Метрики	213
20.5.3. Трассировка	214
20.5.4. Компоненты наблюдаемости на практике	214
20.6. Четыре золотых сигнала мониторинга	215
20.6.1. Задержка	215
20.6.2. Трафик.....	215
20.6.3. Ошибки.....	215
20.6.4. Насыщенность.....	216
20.7. Мониторинг и наблюдаемость: в чем разница?	216

Глава 21. Средства наблюдения: Prometheus, Grafana, Loki, Promtail, OpenTelemetry и Jaeger 217

21.1. Что такое Prometheus?.....	217
21.2. Как работает Prometheus.....	217
21.3. Установка Prometheus.....	219
21.4. Доступ к веб-интерфейсу Prometheus.....	220
21.5. Метрики, доступные в Prometheus	220
21.6. Использование Grafana для визуализации метрик Prometheus	221
21.7. Извлечение информации из логов Kubernetes с помощью Promtail.....	222
21.8. стек журналирования Loki	223
21.9. Использование Loki для запроса и просмотра журналов	225
21.10. Использование Jaeger и OpenTelemetry для распределенной трассировки	228

Глава 22. GitOps: непрерывная поставка в облачном стиле..... 235

22.1. GitOps: введение и определения	235
22.2. Преимущества и недостатки GitOps.....	235
22.3. Инструменты GitOps.....	236

Глава 23. Пример рабочего процесса GitOps с использованием

Argo CD	238
23.1. Знакомство с Argo CD	238
23.2. Argo CD: установка и настройка	238
23.3. Argo CD: создание приложения	239
23.4. Argo CD: автоматическая синхронизация и самовосстановление	242
23.5. Откат изменений с помощью Argo CD	243
23.6. Декларативный способ использования Argo CD	243
23.7. Управление конфигурацией с помощью Argo CD	244
23.8. Argo CD: управление различными средами.....	250
23.9. Argo CD: хуки развертывания	252

Глава 24. Создание конвейеров CI/CD для микросервисов 255

24.1. Непрерывная интеграция, поставка и развертывание микросервисов	255
24.2. Инструменты CI/CD	256
24.2.1. Jenkins	256
24.2.2. Spinnaker	256
24.2.3. Argo CD	257
24.2.4. GitHub Actions.....	257
24.2.5. GitLab CI/CD	257
24.3. Создание конвейера CI/CD для микросервиса	258
24.3.1. Установка и настройка Argo CD	258
24.3.2. Создание репозитория GitHub для нашего микросервиса.....	260
24.3.3. Создание учетной записи Docker Hub	263
24.3.4. Настройка GitHub Actions	264
24.3.5. Создание чарта Helm.....	266
24.3.6. Создание приложения Argo CD.....	269
24.3.7. Автоматизация развертывания новых версий.....	271

Глава 25. Послесловие..... 273

25.1. Что дальше?.....	273
25.2. Благодарность.....	273
25.3. Об авторе	273
25.4. Присоединяйтесь к сообществу	274
25.5. Обратная связь	274

Предметный указатель..... 275

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Глава 1

Облачные микросервисы – почему они появились

1.1. Общие принципы и закономерности

Отрасль разработки программного обеспечения постоянно развивается и во многом живет по своим законам. В соответствии с этими законами каждое приложение, решающее реальную задачу, постоянно претерпевает изменения, его сложность непрерывно растет, а качество норовит стать хуже.

Еще в 1974 г. Леман сформулировал законы эволюции программного обеспечения¹. Эти законы описывают баланс между противоположными силами, одни из которых движут разработки вперед, а другие замедляют прогресс. Впоследствии эти законы неоднократно пересматривали и дополняли.

Основные законы эволюции программного обеспечения Лемана звучат так:

- *закон постоянных изменений*: удовлетворенность программным обеспечением со временем снижается, если его не адаптируют для удовлетворения новых потребностей;
- *закон возрастающей сложности*: поскольку большой программный продукт постоянно изменяется, он становится все сложнее, если не предпринимать специальных действий по сохранению или снижению сложности;
- *закон снижения качества*: качество системы, как правило, снижается, если она не адаптируется к изменениям операционной среды.

Поскольку программное обеспечение по своей природе подвержено изменениям, Леман стремился определить законы, которые управляют такими изменениями, и найти способы сохранения жизнеспособности программных систем.

¹ https://ru.wikipedia.org/wiki/Законы_Лемана.

Программные системы постоянно развиваются, но три основных закона остаются актуальными. Поэтому разработчики ищут новые подходы к управлению развитием своих систем, придерживаясь этих принципов.

К наиболее распространенным подходам относятся *методология двенадцати факторов*, микросервисы и облачные вычисления.

1.2. Методология двенадцати факторов

Эта методология представляет собой набор лучших практик для создания веб-приложений или программного обеспечения как услуги. Она была разработана командой провайдера облачной платформы Heroku¹ и получила широкое распространение в сообществе разработчиков. Как можно понять из ее названия, эта методология выделяет двенадцать факторов разработки успешного и долгоживущего приложения.

1.2.1. Фактор I. Кодовая база

В соответствии с методологией приложение должно иметь только одну кодовую базу, которая отслеживается в системе контроля версий, такой как Git или Mercurial. Кодовая база должна представлять собой единственный репозиторий или набор репозиторий, которые совместно используют корневой коммит. Наличие нескольких кодовых баз указывает на *распределенную систему*, в которой каждый компонент следует рассматривать как *отдельное приложение*. Совместное использование кода несколькими приложениями нарушает принцип двенадцати факторов. Общий код следует выделить в библиотеки и затем включать их в приложение через менеджер зависимостей.

Приложение может быть представлено различными развертываниями, включая производственные, промежуточные и локальные среды разработки. Хотя в каждом развертывании могут быть активны разные версии, кодовая база остается одинаковой во всех развертываниях. Разработчики всегда должны иметь копию единой кодовой базы в своей локальной среде разработки.

Подробнее об этом принципе можно прочитать по ссылке².

1.2.2. Фактор II. Зависимости

Второй фактор подчеркивает необходимость явного объявления и изоляции зависимостей.

¹ <https://www.heroku.com/>.

² <https://12factor.net/ru/codebase>.

В соответствии с методологией следует объявлять все зависимости через *манифест объявления зависимостей* (dependency declaration manifest) и использовать инструмент изоляции зависимостей во время выполнения, чтобы никакие неявные зависимости не «просочились» из окружающей системы.

Этот принцип применяется как к производственным, так и к локальным средам разработки.

Объявление и изоляция зависимостей всегда должны применяться вместе. Явное объявление зависимостей упрощает настройку среды для новых разработчиков и гарантирует, что приложение будет одинаково корректно работать в разных средах.

Методология двенадцати факторов также не полагается на неявное существование каких-либо системных инструментов и требует явно поставлять любые необходимые системные инструменты в приложение для обеспечения совместимости.

Подробнее об этом принципе можно прочитать по ссылке¹.

1.2.3. Фактор III. Конфигурация

В соответствии с третьим фактором конфигурацию следует хранить в среде, а не в коде.

Конфигурация приложения должна храниться в переменных среды, которые легко меняются между развертываниями, не зависят от языка и ОС и вряд ли будут случайно добавлены в репозиторий кода.

Конфигурации не должны храниться как константы в коде, поскольку текущая конфигурация может существенно зависеть от развертывания, а код – нет. Но при этом *внутреннюю* конфигурацию приложения, например способ подключения подов, лучше реализовать в коде.

Не рекомендуется группировать конфигурации в именованные среды. Переменные среды должны быть детализированными элементами управления, которые задаются независимо для каждого развертывания.

Эта модель плавно масштабируется по мере расширения приложения на большее количество развертываний в течение его жизненного цикла.

Подробнее об этом принципе можно прочитать по ссылке².

1.2.4. Фактор IV. Внешние сервисы

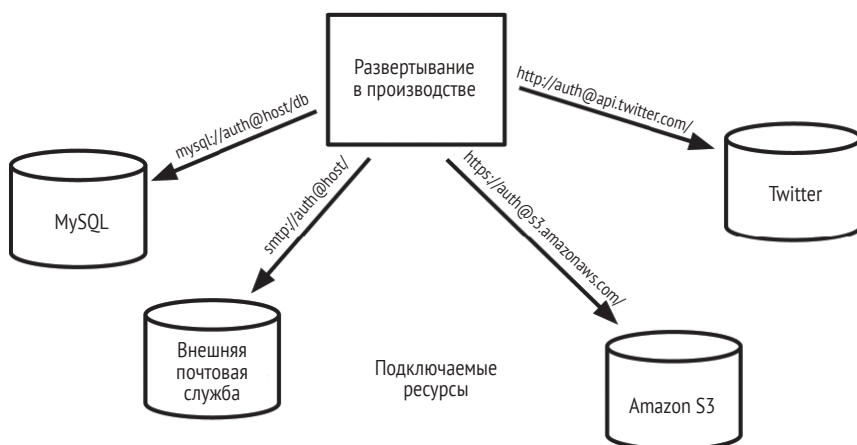
Этот фактор методологии выделяет необходимость обработки *внешних сервисов* (backing service) как подключаемых ресурсов, доступ к которым осуществляется через URL или другой локатор / учетные данные, хранящиеся в конфигурации.

¹ <https://12factor.net/ru/dependencies>.

² <https://12factor.net/ru/config>.

К внешним сервисам относятся любые сервисы, которые приложение использует по сети в рамках своей обычной работы, такие как хранилища данных (например, MySQL), системы обмена сообщениями/очередями (например, AWS SQS) и системы кеширования (например, Redis).

Код приложения обрабатывает локальные и внешние сервисы одинаково. При замене локального ресурса на внешние необходимо изменить только дескриптор ресурса в конфигурации. Каждый внешний сервис обрабатывается как подключаемый ресурс, который можно легко присоединить или отсоединить от развертываний без необходимости менять код приложения.



Источник изображения: <https://12factor.net/>

Подробнее об этом принципе можно прочитать по ссылке¹.

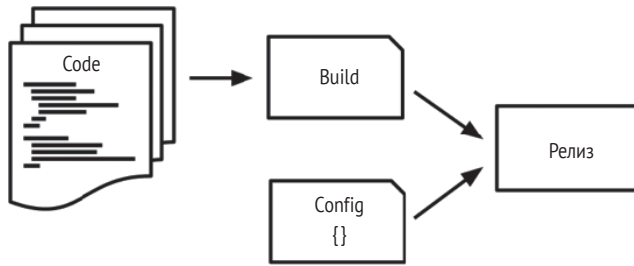
1.2.5. Фактор V. Сборка, релиз, выполнение

Пятый фактор указывает на важность строгого разделения этапов сборки, релиза и выполнения.

На этапе сборки репозиторий кода преобразуют в исполняемый пакет, который так и называется сборкой, на этапе выпуска объединяют сборку с текущей конфигурацией развертывания, а на последнем этапе запускают приложение в среде выполнения.

Код невозможно изменить во время выполнения, а каждый релиз должен иметь уникальный идентификатор, который нельзя менять после его создания. Создание сборки инициируется разработчиками приложения, в то время как выполнение может запускаться автоматически. Поэтому этап запуска должен быть максимально простым, чтобы избежать поломки приложения в отсутствие разработчиков.

¹ <https://12factor.net/ru/backing-services>.



Источник изображения: <https://12factor.net/>

Подробнее об этом принципе можно прочитать по ссылке¹.

1.2.6. Фактор VI. Процессы

Этот фактор определяет выполнение приложения как один или несколько процессов без сохранения состояния, запускаемых в среде выполнения.

Согласно этому принципу процессы не имеют состояния и ничем не делятся, то есть любые данные, которые необходимо сохранить, должны храниться во внешнем сервисе хранения состояний, например в хранилище данных.

Пространство памяти или файловая система процесса может играть роль краткосрочного кеша на одну транзакцию. Тем не менее приложение не должно предполагать, что какие-либо кешированные данные будут доступны в будущем. Использование «липких сессий» (sticky session), которые кешируют данные сеанса пользователя в памяти, нарушает методологию двенадцати факторов. Данные о состоянии сеанса должны храниться в хранилище данных, которое отслеживает истечение срока действия, например Memcached или Redis^{2,3}.

Подробнее об этом принципе можно прочитать по ссылке⁴.

1.2.7. Фактор VII. Привязка портов

Следующий фактор – экспорт сервисов через *привязку портов* (port binding).

В отличие от традиционных веб-приложений, которые полагаются на контейнер веб-сервера, приложение, соответствующее принципам двенадцати факторов, является самодостаточным и обходится без веб-сервера для создания веб-службы.

Приложение должно экспортировать HTTP как сервис, привязываясь к порту и прослушивая входящие запросы. Привязку портов также можно

¹ <https://12factor.net/ru/build-release-run>.

² <https://memcached.org/>.

³ <https://redis.com/>.

⁴ <https://12factor.net/ru/processes>.

использовать для экспорта других типов серверного программного обеспечения, позволяя одному приложению выступать в качестве внешнего сервиса для другого.

При развертывании обработка запросов маршрутизации от общедоступного имени хоста к веб-процессам, привязанным к порту, выполняется на *уровне маршрутизации* (routing layer).

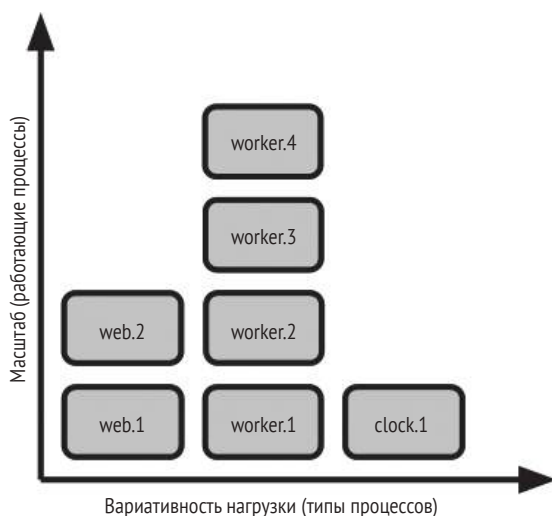
Подробнее об этом принципе можно прочитать по ссылке¹.

1.2.8. Фактор VIII. Параллелизм

В соответствии с принципом *параллелизма* процессы являются сущностями первого уровня, а их масштабирование выражается в количестве запущенных процессов.

Разработчики могут сопоставлять каждый тип задания с типом процесса. Расширение параллелизма становится простой операцией благодаря несоместному использованию и горизонтальной разделимости процессов.

Процессы никогда не должны выполняться как демоны или записывать файлы PID, а диспетчер процессов операционной системы должен управлять выходными потоками и обрабатывать перезапуски и завершения работы.



Источник изображения: <https://12factor.net/>

Подробнее об этом принципе можно прочитать по ссылке².

¹ <https://12factor.net/ru/port-binding>.

² <https://12factor.net/ru/concurrency>.

1.2.9. Фактор IX. Утилизируемость

Принцип *утилизируемости* (disposability) означает способность процессов быстро запускаться и останавливаться, что позволяет быстро развертывать изменения и эластично масштабировать систему без ущерба для надежности.

Процессы должны быть спроектированы так, чтобы они быстро запускались и корректно завершались при получении сигнала SIGTERM. Воркеры должны возвращать все текущие задания в очередь перед выходом, а процессы должны быть устойчивы к внезапным сбоям.

Кроме того, приложение должно уметь обрабатывать неожиданные, некорректные завершения.

Подробнее об этом принципе можно прочитать по ссылке¹.

1.2.10. Фактор X. Паритет разработки и производства

Принцип *паритета разработки и производства* (dev/prod parity) направлен на сокращение разрывов между средой разработки и производства, включая время, персонал и инструменты.

Принцип подразумевает развертывание кода в течение нескольких часов или даже минут, минимизацию разрыва между написанием и развертыванием кода, вовлечение одних и тех же людей в оба процесса и обеспечение максимальной схожести сред разработки и производства.

Также рекомендуется использовать одни и те же тип и версию внешних сервисов во всех развертываниях.

Легкие локальные сервисы не так привлекательны, как раньше, поскольку современные системы упаковки и декларативные инструменты предоставления позволяют разработчикам запускать приложения в локальных средах, очень похожих на производственные.

Подробнее об этом принципе можно прочитать по ссылке².

1.2.11. Фактор XI. Журналирование

В соответствии с принципом *журналирования* (logging) приложение обрабатывает журналы как *потoki событий*³, которые упорядочены по времени и обеспечивают видимость поведения работающего приложения.

¹ <https://12factor.net/ru/disposability>.

² <https://12factor.net/ru/dev-prod-parity>.

³ https://adam.herokuapp.com/past/2011/4/1/logs_are_streams_not_files/.

Приложение, разработанное по методологии двенадцати факторов, записывает свой поток событий без буферизации в `STDOUT`¹ и никогда не должно беспокоиться о маршрутизации или хранении своего выходного потока.

Среда выполнения захватывает поток каждого процесса, сопоставляет его со всеми другими потоками из приложения и направляет его в одно или несколько конечных мест назначения для просмотра и долгосрочного архивирования.

Для этой цели можно применять маршрутизаторы журналов с открытым исходным кодом, такие как `Logplex`², `Fluentd`³ и др.

При желании поток событий для приложения можно направить в файл или в систему индексации, анализа и визуализации журналов, такую как `ELK`⁴. Это позволяет разработчику самостоятельно выполнять мощный и гибкий анализ поведения приложения на всем протяжении его работы, включая возможность находить определенные события в прошлом, отображать тенденции в больших масштабах и настраивать активные оповещения.

Подробнее об этом принципе можно прочитать по ссылке⁵.

1.2.12. Фактор XII. Административные процессы

Двенадцатый, и последний, фактор фокусируется на запуске административных или управленческих задач как однократных процессов (`one-off process`). Типичным примером однократного административного процесса является перенос базы данных или сбор статических файлов.

Эти процессы должны выполняться в той же среде, что и обычные долгосрочные процессы приложения, с использованием той же кодовой базы и конфигурации.

Код, отвечающий за административные процессы, должен поставляться с кодом приложения, чтобы избежать проблем с синхронизацией.

Однократные административные процессы должны выполняться с теми же методами изоляции зависимостей, что и обычные процессы.

Методология двенадцати факторов рекомендует использовать языки, которые предоставляют оболочку `REPL`⁶ в пакете поставки и упрощают запуск одноразовых скриптов. В производстве разработчики могут использовать для запуска одноразовых административных процессов `SSH` или другие механизмы удаленного выполнения команд, предоставляемые средой развертывания.

Подробнее об этом принципе можно прочитать по ссылке⁷.

¹ https://en.wikipedia.org/wiki/Standard_streams.

² <https://devcenter.heroku.com/articles/logplex>.

³ <https://www.fluentd.org/>.

⁴ <https://www.elastic.co/what-is/elk-stack>.

⁵ <https://12factor.net/ru/logs>.

⁶ http://en.wikipedia.org/wiki/Read-eval-print_loop.

⁷ <https://12factor.net/admin-processes>.

1.3. Микросервисы

Методология двенадцати факторов оказала значительное влияние на современные методы разработки программного обеспечения. Она вдохновила разработчиков на переход к разработке приложений как слабосвязанных сервисов – так называемых микросервисов.

Микросервисы (microservice) – это архитектура для создания программных систем. Она разбивает различные компоненты системы на более мелкие, более управляемые части, называемые микросервисами. Каждый микросервис отвечает за одну задачу и может быть разработан и развернут независимо от других.

Подход микросервисов решает проблемы, определенные в законах эволюции программного обеспечения Лемана:

- закон постоянных изменений реализуется путем упрощения адаптации и обновления отдельных микросервисов без необходимости изменения всей системы;
- закон возрастающей сложности компенсируется путем разбиения системы на более мелкие, более управляемые компоненты;
- закон снижения качества компенсируется путем упрощения обслуживания и обновления отдельных микросервисов с меньшими ограничениями для всей системы и предоставления механизмов для мониторинга и управления системой в целом.

Микросервисная архитектура также обеспечивает дополнительные преимущества, такие как масштабируемость, гибкость, более простое тестирование и развертывание. Эти преимущества делают ее оптимальным подходом к созданию больших и сложных программных систем.

До появления шаблонов микросервисов разработчикам приходилось иметь дело с монолитными системами.

При монолитном подходе все функции и модули упаковывают в одно большое приложение. Хотя этот подход может хорошо работать в небольших проектах, он становится все более проблематичным по мере увеличения размера и сложности приложения.

Другими словами, по мере увеличения монолитного приложения его становится сложнее обслуживать и обновлять. Кроме того, различные компоненты приложения могут начать мешать друг другу, вызывая проблемы, которые трудно диагностировать и устранять.

В свою очередь, микросервисы – это архитектурный подход к созданию приложений, в котором различные компоненты разбиваются на более мелкие, более управляемые части.

Каждый микросервис решает одну задачу и может быть разработан и развернут независимо от других. Это упрощает разработку и поддержку приложения, а также повышает его гибкость и масштабируемость.

Микросервисы позволяют командам работать независимо над различными частями приложения, используя языки программирования, фреймворки, инструменты и хранилища данных, которые им наиболее удобны. При правильно организованном управлении это может привести к значительному

повышению производительности. Кроме того, микросервисы могут быть разработаны и развернуты независимо, что упрощает тестирование и развертывание новых функций с небольшим или нулевым временем простоя.

Одной из самых заметных публикаций, посвященных микросервисам, является книга Криса Ричардсона «Микросервисы. Паттерны разработки и рефакторинга»¹. Эти паттерны свободно доступны на microservices.io². Кратко рассмотрим самые распространенные из них.

1.3.1. Отдельная база данных для каждого сервиса

В микросервисной архитектуре каждый сервис должен иметь свою собственную частную базу данных, доступ к которой возможен только через ее API. Такой подход обеспечивает слабую связь между сервисами. Поскольку сервисы могут иметь разные требования к хранению данных, они могут использовать разные типы баз данных.

Однако реализация транзакций и запросов, включающих несколько сервисов, становится сложнее и может потребовать использования таких паттернов, как Saga, API Composition и CQRS^{3,4,5}. Следует избегать паттерна общей базы данных, поскольку он может привести к проблемам при масштабировании и обслуживании системы.

1.3.2. API Composition

При этом подходе запросы в микросервисной архитектуре реализуют с использованием API Composer для совместной работы сервисов.

Этот инструмент упрощает получение данных из нескольких сервисов, но может генерировать неэффективные объединения больших наборов данных.

Альтернативной реализацией подхода с отдельными БД для каждого сервиса является CQRS.

1.3.3. Отдельный экземпляр сервиса на каждый контейнер

Этот подход подчеркивает важность использования контейнеров, таких как Docker, для упаковки и развертывания сервисов в микросервисной архитек-

¹ Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга. СПб.: Питер, 2022. ISBN 978-5-4461-0996-8.

² <https://microservices.io/patterns/index.html>.

³ <https://microservices.io/patterns/data/saga.html>.

⁴ <https://microservices.io/patterns/data/api-composition.html>.

⁵ <https://microservices.io/patterns/data/cqrs.html>.

туре. Подход предусматривает упаковку каждого сервиса как образа Docker и развертывание каждого экземпляра сервиса как контейнера.

Использование контейнеров обеспечивает такие преимущества, как масштабируемость, изоляция и ограничения ресурсов. Однако по сравнению с развертыванием виртуальных машин у контейнеров есть недостаток в виде менее богатой инфраструктуры.

1.3.4. Внешняя конфигурация

Согласно этому подходу данные конфигурации извлекаются из внешнего источника, чтобы сервис мог работать в нескольких средах без изменений. Это важно, поскольку в разных средах могут быть разные экземпляры инфраструктуры и внешних сервисов.

Сервисы должны быть настроены на чтение конфигурации из внешнего источника во время запуска.

Преимущество этого подхода заключается в возможности использовать приложение, способное работать в нескольких средах без необходимости внесения изменений или перекомпиляции. Однако важно понимать, как сделать так, чтобы полученная конфигурация гарантированно соответствовала ожиданиям при развертывании приложения. Для решения этой проблемы применяют паттерны обнаружения сервисов на стороне сервера и на стороне клиента.

1.3.5. Обнаружение сервисов на стороне сервера

Микросервисному приложению требуется паттерн межсервисного взаимодействия и обнаружения сервисов.

Поскольку количество и местоположение экземпляров сервисов может динамически меняться, клиентам сервисов нужен механизм для отправки запросов к изменяющемуся набору эфемерных (вытесняемых) экземпляров сервисов.

Решение заключается в использовании маршрутизатора обнаружения на стороне сервера, который запрашивает реестр сервисов и пересылает запрос доступным экземплярам.

Этот прием упрощает клиентский код и поддерживается облачными средами, такими как AWS Elastic Load Balancer¹. Однако для него требуются дополнительные системные компоненты и сетевые переходы.

1.3.6. Автоматический предохранитель

Этот паттерн предназначен для обработки сбоев удаленных сервисов в микросервисной архитектуре.

¹ <https://aws.amazon.com/elasticloadbalancing/>.

Автоматический предохранитель действует как прокси-сервер для предотвращения каскадного распространения сбоев на другие сервисы. Когда количество последовательных сбоев превышает определенное пороговое значение, срабатывает предохранитель, и все попытки вызвать удаленный сервис отвергаются в течение некоторого периода времени. Если ограниченное количество тестовых запросов в течение периода ожидания успешно выполнено, предохранитель возобновляет нормальную работу; в противном случае период ожидания начинается снова. Этот паттерн можно реализовать с помощью библиотек, таких как Netflix Hystrix¹ (в настоящее время в режиме обслуживания), или альтернативных решений, таких как Istio². Его можно использовать в сервисах, разработанных с помощью Microservice Chassis, API Gateway или маршрутизатора обнаружения на стороне сервера.

Задача разработчика заключается в выборе правильных значений тайм-аута, не вызывающих ложных срабатываний или чрезмерной задержки.

1.3.7. Облачный подход

Микросервисный подход требует не только новых способов мышления и проектирования программных систем, но также новых инструментов и технологий для управления развертыванием, координацией, мониторингом и другими аспектами разработки программного обеспечения.

С появлением микросервисов появилась новая концепция *облачных приложений*.

Технология облачных приложений объединяет в себе преимущества контейнеров облачных вычислений, таких как Docker, и инструментов оркестровки, таких как Kubernetes, и применяется для создания и эксплуатации приложений, ориентированных на работу в облаке.

Облачные приложения создаются с использованием контейнерных микросервисов, что обеспечивает простоту развертывания, масштабирования и администрирования в облачной среде.

1.4. От монолита к облачным микросервисам

Сдвиг парадигмы в архитектуре программного обеспечения от монолита к микросервисам и облачным приложениям стал феноменальным опытом, который повлиял на проектирование и поставку приложений. Микросервисы разбивают монолиты на более мелкие независимые компоненты, а с появле-

¹ <https://github.com/Netflix/Hystrix>.

² <https://istio.io/>.

ния облачных микросервисов началась новая эра проектирования и поставки облачных систем. Kubernetes вместе со своей экосистемой инструментов и технологий сыграл важную роль в реализации этих изменений, позволив разработчикам быстро создавать, масштабировать и организовывать свои платформы на основе микросервисов. Его функциональные возможности, такие как оркестровка контейнеров, обнаружение сервисов и автоматическое масштабирование, хорошо подходят для ускорения разработки облачных микросервисов.

В этой книге рассматриваются фреймворк Kubernetes и его экосистема, обсуждаются новые проблемы и возможности, которые он представляет, и говорится о том, как использовать его для развертывания, управления, защиты, мониторинга и масштабирования облачных микросервисов.

Мы также обсудим некоторые передовые методы и паттерны создания устойчивых, масштабируемых и обслуживаемых систем на основе микросервисов с помощью Kubernetes.

Глава 2

Технические требования

2.1. Сервер разработки

Для простоты мы будем использовать в качестве сервера разработки сервер Ubuntu 22.04. Если вы используете другую ОС, изменятся только инструкции по установке, приведенные в этом руководстве.

Я также рекомендую создать учетную запись провайдера DigitalOcean, вы можете использовать эту ссылку¹ или мою реферальную ссылку², чтобы получить кредит в размере 200 долл. США.

После создания учетной записи DigitalOcean вам нужно будет вручную создать машину Ubuntu.

Вы также можете применить Terraform с помощью кода из листинга 2.1:

Листинг 2.1

```
# ssh-keygen -t rsa -b 4096 -C "mydigitaloceankey" -f ~/.ssh/mykey
# export DIGITALOCEAN_TOKEN=xxxxxx

terraform {
  required_providers {
    digitalocean = {
      source = "digitalocean/digitalocean"
      version = "~> 2.0"
    }
  }
}
#
variable "do_token" {
  type = string
}
```

¹ <https://cloud.digitalocean.com/registrations/new>.

² <https://m.do.co/c/9e1ec82fb675>.

```

variable "region" {
  default = "<REGION>"
}

variable "vpc_uuid" {
  default = "<VPC_UUID>"
}

data "digitalocean_project" "playground" {
  name = "<PROJECT_NAME>"
}

resource "digitalocean_ssh_key" "my_ssh_key" {
  name = "mykey"
  public_key = file("~/ssh/mykey.pub")
}

resource "digitalocean_project_resources" "playground" {
  project = data.digitalocean_project.playground.id
  resources = [for droplet in digitalocean_droplet.mydroplets : droplet\.urn]
}

// Определяем список имен дроплетов
variable "names" {
  default = [
    "<NAME_OF_DROPLET>",
  ]
}

// Используем цикл for_each, чтобы создать дроплет для каждого имени в списке
resource "digitalocean_droplet" "mydroplets" {
  for_each = { for name in var.names : name => name }

  image = "ubuntu-22-04-x64"
  name = each.value
  region = var.region
  size = "s-1vcpu-1gb"
  ssh_keys = [digitalocean_ssh_key.my_ssh_key.id]
  monitoring = false
  vpc_uuid = var.vpc_uuid
  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y net-tools",
    ]
  }
}

// Используем цикл for_each для вывода IP-адреса каждого дроплета
output "droplet_ip_addresses" {
  value = {
    for name, droplet in digitalocean_droplet.mydroplets : name => drop\
let.ipv4_address
  }
}

```

Перед запуском Terraform вам понадобятся:

- по крайней мере один проект в вашей учетной записи DigitalOcean;
- токен, который вы сгенерировали на панели управления DigitalOcean.

Обязательно измените значения в файле Terraform HCL на свои значения (например: REGION, PROJECT_NAME и т. д.).

Если вы не используете Windows, обязательно обновите каталог, в котором сгенерирован ваш ключ SSH (~/.ssh/mykey). Вам также понадобится клиент SSH, например PuTTY или встроенный клиент SSH.

Затем выполните следующие команды:

```
ssh-keygen -t rsa -b 4096 -C "mydigitaloceankey" -f ~/.ssh/mykey
export DIGITALOCEAN_TOKEN=xxxxxx
terraform init
terraform plan
terraform apply
```

2.2. Установка kubectl

Kubectl – это инструмент командной строки, который широко используется для управления и развертывания приложений в кластерах Kubernetes. Он разработан для того, чтобы упростить разработчикам, операторам и администраторам взаимодействие с кластерами Kubernetes, и обладает широкой функциональностью.

Установите этот инструмент для связи с сервером API Kubernetes и управления нашими кластерами и ресурсами Kubernetes:

```
curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Активируйте функцию автодополнения:

```
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl > /dev/null
sudo chmod a+r /etc/bash_completion.d/kubectl
source ~/.bashrc
```

Глава 3

Кластеры Kubernetes

3.1. Создание кластера Kubernetes для разработки с помощью Minikube

Minikube – это инструмент, позволяющий запустить кластер Kubernetes на локальной машине за считанные минуты. При всем удобстве у него есть ограничения:

- *масштабируемость*: Minikube предназначен для тестирования и разработки и не рекомендуется для производственных сред с высоким трафиком;
- *ограниченное распределение ресурсов*: поскольку Minikube предназначен для работы на одном узле, вам доступны ресурсы только на этом узле;
- *ограниченная функциональность*: Minikube разработан для предоставления легкой и упрощенной среды Kubernetes, в которой некоторые расширенные функции Kubernetes могут быть недоступны. Например, в ней нет службы балансировки нагрузки;
- *ограниченные параметры хранения*: Minikube поддерживает только ограниченное количество параметров хранения и не всегда подходит для приложений со сложными требованиями к хранению;
- *ограниченные параметры сети*: инструмент предоставляет упрощенную сетевую среду и не всегда подходит для приложений со сложными требованиями к сети;
- *ограниченная безопасность*: поскольку инструмент предназначен для целей тестирования/разработки, он не обеспечивает тот же уровень безопасности, что и производственная среда Kubernetes.

Тем не менее это хороший способ начать экспериментировать с Kubernetes. Вам понадобится машина со следующими параметрами:

- 2 ЦП или более;
- 2 ГБ свободной памяти;

- 20 ГБ свободного места на диске;
- начальный конфигурактор, такой как Docker¹, QEMU², Hyperkit³, Hyper-V⁴, KVM⁵, Parallels⁶, Podman⁷, VirtualBox⁸ или VMware Fusion/Workstation⁹.

3.1.1. Установка Minikube

В следующем примере мы будем использовать Docker. Это означает, что различные компоненты нашего кластера Kubernetes будут работать внутри контейнеров.

Для установки Minikube загрузите его двоичный файл:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Поскольку мы собираемся использовать Docker, его тоже нужно установить:

```
sudo apt-get remove -y docker docker-engine docker.io containerd runc

sudo apt-get update

sudo apt-get install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg\
--dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
"deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings\
/docker.gpg] https://download.docker.com/linux/ubuntu \
"$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update

sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin\
docker-compose-plugin
```

¹ <https://minikube.sigs.k8s.io/docs/drivers/docker/>.

² <https://minikube.sigs.k8s.io/docs/drivers/qemu/>.

³ <https://minikube.sigs.k8s.io/docs/drivers/hyperkit/>.

⁴ <https://minikube.sigs.k8s.io/docs/drivers/hyperv/>.

⁵ <https://minikube.sigs.k8s.io/docs/drivers/kvm2/>.

⁶ <https://minikube.sigs.k8s.io/docs/drivers/parallels/>.

⁷ <https://minikube.sigs.k8s.io/docs/drivers/podman/>.

⁸ <https://minikube.sigs.k8s.io/docs/drivers/virtualbox/>.

⁹ <https://minikube.sigs.k8s.io/docs/drivers/vmware/>.

```
sudo service docker start

sudo groupadd docker

sudo usermod -aG docker $USER

newgrp docker
```

Теперь мы можем запустить кластер, но перед этим, поскольку драйвер Docker (предоставленный по умолчанию) не должен использоваться с привилегиями root, мы создадим пользователя, который не является пользователем root, и добавим его в группу docker:

```
adduser user
usermod -aG sudo user
su - user
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

3.1.2. Создание кластера Minikube

Затем запустите наш первый кластер:

```
minikube start --driver=docker
```

Выполните следующую команду, чтобы убедиться, что кластер работает:

```
minikube kubectl -- get pods -A
```

`kubectl` – это инструмент командной строки, используемый для взаимодействия с кластерами Kubernetes. Он позволяет пользователям развертывать, проверять и управлять приложениями и ресурсами в кластере Kubernetes.

С обычными удаленными кластерами Kubernetes мы используем команду следующим образом:

```
kubectl get pods -A
```

Но при работе с Minikube мы всегда начинаем с команды `minikube`, а затем добавляем `kubectl -- <опции>`.

Также можно запустить кластер Minikube с другой конфигурацией, например с параметром `memory`, который ограничивает использование памяти кластером:

```
minikube start --driver=docker --memory=<memory>
```

или:

```
minikube start --driver=docker --extra-config=kubeadm.ignore-preflight-\
errors=NumCPU --force --cpus=1
```

или:

```
minikube start --driver=docker --memory=1977mb --cpus=2 --disk-size=20g\  
--kubernetes-version=v1.25.0
```

Теперь удалим этот кластер:

```
minikube delete
```

3.1.3. Профили Minikube

Далее запустим другой кластер с именем профиля. Вы можете создать несколько локальных кластеров, поэтому полезно иметь разные профили:

```
minikube start --driver=docker -p c1  
minikube start --driver=docker -p c2  
minikube profile list
```

Теперь у нас есть два кластера: c1 и c2.

Чтобы переключить контекст на первый кластер, выполните команду

```
minikube profile c1
```

Теперь все наши команды Kubectl будут выполняться на первом кластере:

```
minikube kubectl -- get pods -A  
minikube kubectl -- cluster-info
```

Вы можете проверить журналы кластера с помощью команды

```
minikube logs
```

или перенаправить журналы в файл:

```
minikube logs --file=logs.txt
```

Давайте попробуем другие команды:

```
# Получить поды во всех пространствах имен  
minikube kubectl -- get pods -A  
# Получить узлы  
minikube kubectl -- get nodes  
# получить поды в пространстве имен по умолчанию  
minikube kubectl -- get pods  
# получить поды в пространстве имен по умолчанию с широким выводом  
minikube kubectl -- get pods -o wide  
# получить поды во всех пространствах имен с широким выводом  
minikube kubectl -- get pods -o wide --all-namespaces  
# получить поды во всех пространствах имен с широким выводом и наблюдать  
minikube kubectl -- get pods -o wide --all-namespaces --watch  
# получить поды во всех пространствах имен с широким выводом и отсортировать по имени  
minikube kubectl -- get pods -o wide --all-namespaces --sort-by=.metadata.name
```

```
# получить поды во всех пространствах имен с широким выводом и отсортировать по имени
с фильтром по статусу
minikube kubectl -- get pods -o wide --all-namespaces --sort-by=.metadata.name --field-
selector=status.phase=Running
```

3.1.4. Использование панели управления K8s с Minikube

У Kubernetes есть *панель управления* (dashboard), которую мы также можем использовать с Minikube. Для этого введите следующую команду:

```
minikube dashboard
```

После ее выполнения вы должны увидеть примерно такой вывод:

```
[..]
[..]
[..]
Opening http://127.0.0.1:44485/api/v1/namespaces/kubernetes-dashboard
/services/http:kubernetes-dashboard:/proxy/ in your default browser...
http://127.0.0.1:44485/api/v1/namespaces/kubernetes-dashboard/services
/http:kubernetes-dashboard:/proxy/
[..]
[..]
```

где 127.0.0.1:44485 – адрес и порт для доступа к панели.

Поскольку мы запускаем удаленную виртуальную машину Ubuntu, одно из решений, которое мы можем сделать, – это создать туннель SSH с вашей локальной машины для доступа к удаленному серверу. Туннель перенаправляет любой трафик с локального порта, который вы задали, на порт 44485 удаленного сервера:

```
export IP=<REMOTE_IP_ADDRESS>
ssh -NfL 44485:127.0.0.1:44485 root@$IP
```

Если панель управления была открыта на другом порту, измените команду, заменив 44485 на правильный номер порта.

После создания туннеля откройте браузер и перейдите по адресу, выведенному командой `minikube dashboard`. Вывод должен выглядеть так:

```
http://127.0.0.1:37925/api/v1/namespaces/kubernetes-dashboard/services/\
http:kubernetes-dashboard:/proxy/
```

Теперь вы можете просматривать все ресурсы (сервисы, поды и т. д.), которые есть в кластере.

Например, чтобы просмотреть все поды в системном пространстве имен, переключитесь на пространство имен **kube-system**, затем выберите **Pods** (Поды) на боковой панели **Workloads** (Рабочие нагрузки).

Вывод:

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node	8080	http://192.168.49.2:31914

Чтобы получить доступ к приложению с помощью его URL из нашего браузера, нам нужно создать туннель:

```
ssh -NfL 31914:192.168.49.2:31914 root@$IP
```

Вам следует заменить 31914 на правильный порт, используемый вашим кластером.

Теперь веб-страница должна быть доступна по следующему адресу:

```
http://127.0.0.1:31914
```

Здесь также следует заменить 31914 на правильный порт, используемый вашим кластером.

3.1.8. Удаление ресурсов K8s

Чтобы очистить узел, удалите сервис, а затем развертывание:

```
minikube kubectl -- delete service hello-node
minikube kubectl -- delete deployment hello-node
```

3.1.9. Дополнения Minikube

Minikube имеет систему *дополнений* (addon), которая позволяет устанавливать дополнительное программное обеспечение на ваш кластер.

Вы можете просмотреть список этих дополнений с помощью следующей команды:

```
minikube addons list
```

Чтобы включить/отключить дополнение, введите такие команды:

```
minikube addons enable <ADDON_NAME>
minikube addons disable <ADDON_NAME>
```

Например, мы можем включить дополнение `metrics-server`¹:

```
minikube addons enable metrics-server
```

Проверьте, успешно ли прошла установка:

```
minikube -p c1 kubectl -- get deploy,svc -n kube-system | egrep metrics-server
```

¹ <https://github.com/kubernetes-sigs/metrics-server>.

Затем отобразите некоторые метрики с помощью следующих команд:

```
minikube kubectl -- top nodes
minikube kubectl -- top pods
```

3.1.10. Использование kubectl без расширения minikube

Если вы предпочитаете использовать kubectl без вызова команды minikube kubectl, вам сначала нужно будет установить его:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)
/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Активируйте автодополнение:

```
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl > /dev/null
sudo chmod a+r /etc/bash_completion.d/kubectl
source ~/.bashrc
```

Теперь вы можете увидеть контексты с помощью команды

```
kubectl config get-contexts
```

Контекст должен указывать на c1 – кластер, который мы используем. Если это не так, используйте следующую команду:

```
kubectl config set-context c1
```

Затем вы можете выполнить обычные команды kubectl, такие как

```
kubectl get pods -A
kubectl get nodes
kubectl get pods
kubectl get pods -o wide
kubectl -n kube-system get pods -o wide
```

3.1.11. Удаление кластеров

Чтобы удалить кластер, созданный с помощью minikube, вам нужно всего лишь удалить профиль:

```
minikube delete -p c1 # или kubectl delete
minikube delete -p c2
```

3.2. Создание кластера Kubernetes для разработки с помощью Rancher

3.2.1. Технические требования

Следующие инструкции были проверены с использованием виртуальных машин на DigitalOcean, подготовленных с помощью Ubuntu 22.04.

Хотя я использую машины провайдера DigitalOcean, те же инструкции должны работать на других типах виртуальных машин.

3.2.2. Использование Terraform для запуска кластера

Нам предстоит запустить Rancher Server и тестовый кластер.

Есть несколько способов сделать это, но самый быстрый и простой – использовать Terraform. Выполните следующие шаги:

```
git clone https://github.com/rancher/quickstart
cd quickstart/rancher/do
mv terraform.tfvars.example terraform.tfvars
```

Теперь отредактируйте файл `terraform.tfvars` и обновите следующие переменные:

- `do_token`: ключ доступа к вашей учетной записи DigitalOcean;
- `rancher_server_admin_password`: пароль администратора для Rancher Server;
- `do_region`: регион DigitalOcean, который вы хотите использовать;
- `prefix`: префикс для использования для всех созданных ресурсов;
- `droplet_size`: размер используемого дроплета. Для большей производительности можно использовать минимальный – `s-2vcpu-4gb-s-4vcpu-8gb`.

Теперь выполните команду `terraform init`, а затем `terraform apply --auto-approve`. Последняя команда создаст два кластера Kubernetes, один из которых будет работать под управлением Rancher Server, а другой предназначен для экспериментальных развертываний.

Вывод команды должен содержать следующие данные:

- `rancher_node_ip`: IP-адрес созданного узла (Rancher Server);
- `rancher_server_url`: URL панели управления, доступный с помощью `sslip.io`¹;
- `workload_node_ip`: IP-адрес созданного узла (Rancher Server).

Используйте `rancher_server_url` для доступа к панели управления Rancher Server. Логин – `admin`, а пароль – тот, который вы задали для `rancher_server_admin_password` в файле Terraform `terraform.tfvars`.

¹ <https://sslip.io/>.

Если вы хотите подключиться к Rancher Server по ssh, вы можете использовать ключ `id_rsa`, автоматически сгенерированный и помещенный в каталог `quickstart/rancher/do`.

3.2.3. Создание ресурсов Kubernetes с помощью пользовательского интерфейса Rancher

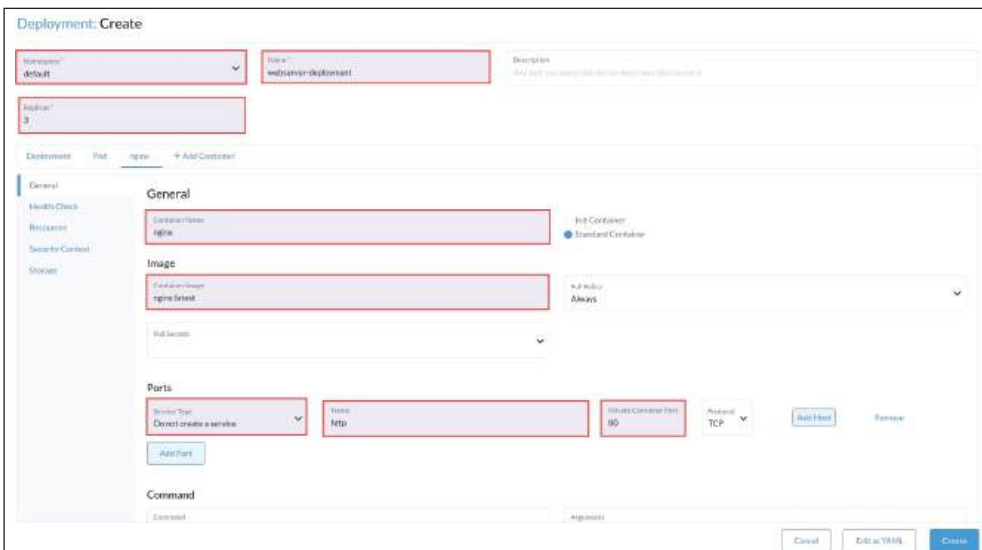
Теперь попробуем создать некоторые ресурсы Kubernetes с помощью экспериментального кластера.

Для начала перейдите на панель управления Rancher и выберите кластер **quickstart-do-custom** на главной странице. Там нажмите **Workloads** на боковой панели, затем выберите **Deployments** (Развертывания) и нажмите **Create** (Создать).



В пространстве имен `default` создайте развертывание с именем `webserver-deployment` с 3 репликами.

Используйте `nginx` в качестве имени контейнера и `nginx:latest` в качестве образа контейнера. Добавьте порт без сервиса, назовите его `http` и задайте частный порт `80`.



Нажмите **Pod** и добавьте метку `app` со значением `webserver`. Мы будем использовать ее в качестве селектора на следующем шаге.

Оставьте все остальные настройки без изменений, затем нажмите **Create**. Подождите, пока будут созданы три реплики.

Веб-сервер пока недоступен публично. Чтобы сделать его доступным, нам нужно создать для развертывания сервис `ClusterIP`, а затем входящий ресурс.

Нажмите **Service Discovery** (Просмотр сервисов) на боковой панели, нажмите **Services** (Сервисы), потом **Create**. Выберите опцию **Cluster IP** (IP кластера).

Создайте сервис `Cluster IP` в пространстве имен по умолчанию, назовите его `nginx-clusterip`.

Добавьте порт с именем `http`, прослушивающий порт `8000` и имеющий `80` в качестве целевого порта (это порт, используемый в конфигурации развертывания).

Перейдите в **Selectors** (Опции выбора) и выберите пару `app` со значением `webserver`, так вы выберете поды, созданные предыдущим развертыванием и имеющие ту же метку.

Создайте сервис.

Теперь нажмите **Ingresses** в разделе **Service Discovery** и задайте имя `ingress`. Мы можем применить `sslip.io`¹ для создания поддомена, который будем использовать в **Request Host** (Запрос хоста). Например, если узел, на котором развернут наш кластер Kubernetes (а именно `quickstart-quickstartnode`), имеет IP `165.22.82.137`, то мы можем использовать следующее имя:

`165.22.82.137.sslip.io`²

Не забудьте подставить в это имя свой IP-адрес.

Добавьте настройки **Target Service** (Целевой сервис), **Path** (Путь) и **Port** (Порт), чтобы указать на ранее созданную службу / порт IP-кластера.



¹ <http://sslip.io>.

² <http://165.22.82.137.sslip.io>.

Не забудьте выбрать опцию **Ingress Class**. Выберите `nginx` и нажмите **Create**.

Перейдите на `<ВАШ_IP>[.sslip.io](http://165.22.82.137.sslip.io/)`, чтобы проверить доступ к сервису ClusterIP через Ingress. Вы должны увидеть веб-страницу «Welcome to nginx!»

3.3. Создание локального кластера Kubernetes с помощью Rancher

3.3.1. Начальные требования

Следующие инструкции были проверены с использованием виртуальных машин на DigitalOcean, подготовленных с Ubuntu 22.04.

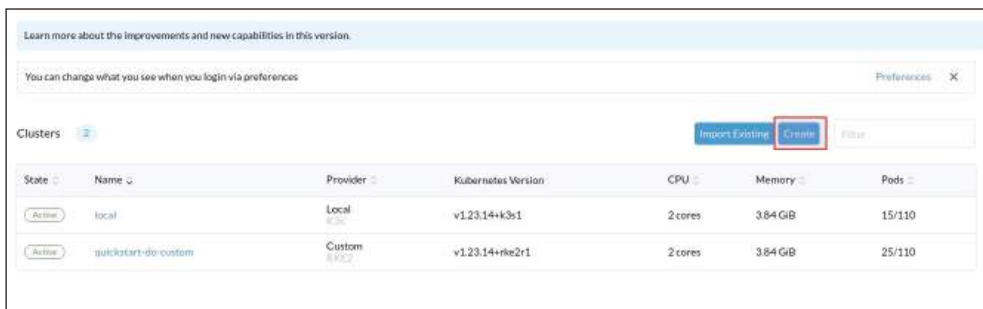
Нам предстоит создать кластер Kubernetes с помощью сервера Rancher и нескольких рабочих узлов (воркеров, `worker`).

Нам нужны как минимум три машины:

- *плоскость управления* (`control plane`). Имя хоста, которое мы используем для этой машины, – `controlplane`;
- *воркер 1*. Имя хоста, которое мы здесь используем, – `worker1`;
- *воркер 2*. Имя хоста, которое мы здесь используем, – `worker2`. Это не обязательно, но если вам нужны два узла в кластере Kubernetes, добавьте его;
- *воркер n*: при желании вы можете добавить столько воркеров, сколько вам нужно для вашего кластера.

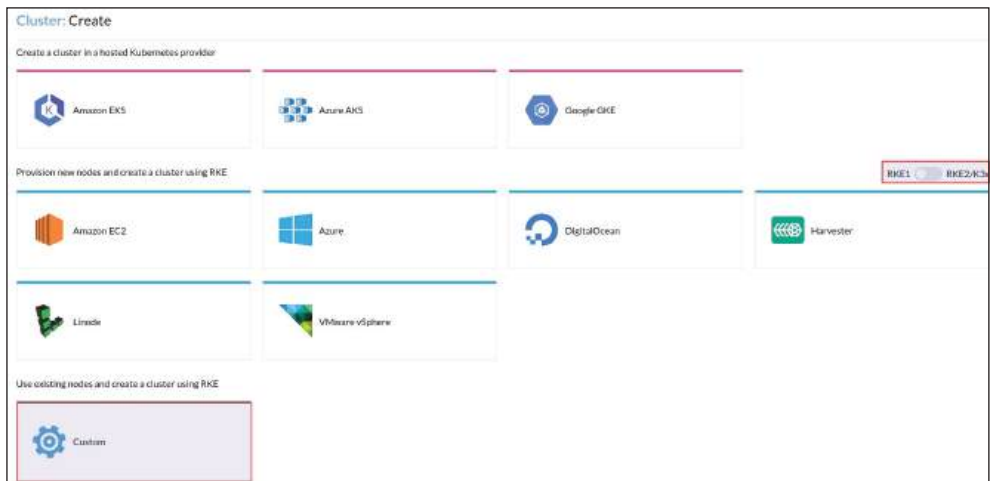
3.3.2. Создание кластера с помощью сервера Rancher

Теперь перейдите в веб-интерфейс сервера Rancher и нажмите **Create**, чтобы создать новый кластер.



При создании кластера с помощью Rancher у вас есть выбор из нескольких вариантов в зависимости от используемой инфраструктуры. Например, вы можете развернуть кластер Amazon EKS, если вы используете AWS, или кластер Google GKE, если используете Google Cloud.

Мы используем DigitalOcean, поэтому можно выбрать провайдера Rancher для этого облака, и Rancher потребует ваши ключи API DigitalOcean для создания кластера, включая создание узлов и развертывание компонентов Kubernetes. Однако наша цель – развернуть *локальный* кластер, поэтому мы воспользуемся опцией **Custom** (Пользовательский), которую следует выбрать при создании локального кластера.



В этом случае вам нужно предоставить Rancher готовые виртуальные машины, поскольку он не будет создавать их за вас.

Выберите **RKE1**, нажмите **Custom** и дайте кластеру имя, например mycluster. Мы будем использовать в примерах v1.24.10-rancher4-1 в соответствии с версией Kubernetes; измените это значение и оставьте все остальное как есть.

Мы начнем с развертывания узла плоскости управления, поэтому выберите роли **etcd** и **Control Plane**, потом на узле controlplane запустите агента Docker, предоставленного Rancher.



Затем выберите **Worker** и запустите агента Docker на каждом рабочем узле, который вы хотите использовать. Вы должны увидеть подтверждение регистрации узла в веб-интерфейсе.

Rancher создаст различные компоненты Kubernetes, такие как сервер API для панели управления и kubelet на рабочих узлах. Подготовка кластера займет какое-то время.

RKE (Rancher Kubernetes Engine) – это сертифицированный CNCF дистрибутив Kubernetes, который полностью работает в контейнерах Docker. Если вы перейдете на узел controlplane и введете команду `watch docker ps`, то сможете увидеть etcd и другие компоненты, работающие внутри контейнеров Docker.

Если по какой-то причине у вас возникнут проблемы, вы можете очистить свои узлы с помощью одного из двух готовых скриптов^{1,2}.

3.3.3. Примечание о высокой доступности

Наша конфигурация не идеальна с точки зрения высокой доступности. Например, рекомендуется запускать базу данных etcd на более чем одном сервере. Вы можете выбрать либо стекированную топологию, либо внешнюю топологию etcd³.

В любом случае, роль worker не должна использоваться или добавляться к узлам с ролью etcd или controlplane.

Кроме того, база данных etcd требует наличия нечетного количества узлов. Поэтому рекомендуется использовать три узла, чтобы предотвратить явление *расщепления ведущих* (split brain) в схеме с ведущим и ведомыми узлами. Количество узлов, которые вы можете потерять без утраты работоспособности кластера, зависит от того, сколько узлов назначено роли etcd. Поэтому рекомендуется создавать узлы etcd в трех разных зонах доступности.

Вам также потребуется настроить балансировщик нагрузки, дабы предотвратить отключение связи с сервером управления Rancher из-за сбоя одного узла.

Чтобы обеспечить высокую доступность главного компонента, у вас должно быть не менее двух узлов с ролью controlplane. Рекомендуется, чтобы эти узлы находились в разных зонах доступности.

Чтобы включить перепланирование рабочей нагрузки в случае отказа узла, вам необходимо не менее двух узлов с ролью worker. Чем больше узлов вы добавите, тем более устойчивыми к сбоям станут ваши рабочие нагрузки.

Дополнительная информация доступна в официальной документации Rancher⁴.

¹ <https://github.com/rancherlabs/support-tools/blob/master/extended-rancher-2-cleanup/extended-cleanuprancher2.sh>.

² <https://gist.github.com/l1eriayo/1bef407602208911e86f42d5d208c1fb>.

³ <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/ha-topology/>.

⁴ <https://ranchermanager.docs.rancher.com/how-to-guides/new-user-guides/infrastructure-setup/ha-rke-kubernetes-cluster>.

3.4. Создание локального кластера Kubernetes: другие варианты

Мы запустили кластер с помощью сервера Rancher, но есть и другие способы.

Существует множество инструментов для создания локального кластера Kubernetes, в том числе:

- **kubeadm**: это инструмент, предоставляемый Kubernetes, который автоматизирует создание кластера на основе набора узлов. Это хороший выбор, если вы знакомы с Kubernetes и хотите лучше контролировать процесс создания;
- **kops**: это инструмент, предоставляемый Kubernetes, который автоматизирует создание кластера на AWS. Выберите его, если вы хотите использовать AWS и вам нужен инструмент, специально разработанный для этой среды;
- **OpenShift**: это дистрибутив Kubernetes, предоставляемый Red Hat, который содержит дополнительные функции и инструменты для управления кластерами Kubernetes. Это хороший выбор, если вам нужно более корпоративное решение;
- **Kubespray**: подходит, если вам нужна большая гибкость и настраиваемость. Это также хороший выбор, если вы знакомы с фреймворком Ansible, поскольку он активно используется для развертывания.

3.5. Управляемые кластеры

Управляемые кластеры Kubernetes, такие как Google Kubernetes Engine (GKE), Amazon Elastic Kubernetes Service (EKS), Azure Kubernetes Service (AKS) и Digital Ocean Kubernetes (DOK), обеспечивают более простой способ запуска кластеров Kubernetes без каких-либо операционных издержек. Благодаря наличию управляемых сервисов разработчики могут сосредоточиться на рабочих нагрузках своих приложений, возлагая остальные задачи на платформу для управления кластером и масштабирования. Иными словами, разработчики могут спокойно заниматься своими основными обязанностями и достигать эффективных результатов, не беспокоясь о базовой инфраструктуре. Но следует помнить, что за это удобство иногда приходится платить значительные суммы.

Вышеупомянутые сервисы предлагают такие функции, как автоматическое масштабирование, балансировка нагрузки и мониторинг, и к ним можно получить доступ через веб-консоль или интерфейс командной строки.

Например, GKE предоставляет полностью управляемую среду Kubernetes с такими функциями, как автоматическое масштабирование узлов, автоматические обновления и интеграция с сервисами Google Cloud, такими как Cloud Monitoring.

EKS также предоставляет управляемую среду Kubernetes, которая интегрируется с сервисами Amazon Web Services (AWS).

AKS предлагает управляемый сервис Kubernetes со встроенным мониторингом, аналитикой журналов и поддержкой гибридных развертываний в локальных и облачных средах.

DOK – это управляемый сервис Kubernetes от DigitalOcean, имеющий гибкие тарифные планы и удобный интерфейс.

Для создания управляемого кластера достаточно нескольких щелчков мыши.

3.6. Создание управляемого кластера DOK с помощью Terraform

В следующих примерах DigitalOcean будет выступать в роли облачного провайдера для создания управляемого кластера Kubernetes. У нас есть разные варианты создания кластера:

- с помощью веб-интерфейса облачного провайдера;
- с помощью интерфейса командной строки DigitalOcean `doctl`¹;
- с помощью Terraform.

Мы применим третий вариант. Вот почему вам нужно сначала создать файл `main.tf` (листинг 3.1):

Листинг 3.1 ❖ Файл `main.tf`

```
// Токен автоматически извлекается из переменной окружения
// execute: export DIGITALOCEAN_TOKEN=dop_v1_XXXXXXXXXXXXXXXXXXXXXXXXXXXX\
XXXXXX

terraform {
  required_providers {
    digitalocean = {
      source = "digitalocean/digitalocean"
      version = "~> 2.0"
    }
  }
}

variable "region" {
  default = "fra1"
}

data "digitalocean_kubernetes_versions" "versions" {
  version_prefix = "1.26."
}

// Задаем имена кластеров, которые хотим создать
// Для создания одного кластера задаем одно имя,
// для создания двух кластеров задаем два имени
// и т. д.
```

¹ <https://docs.digitalocean.com/reference/doctl/>.

```

variable "names" {
  default = [
    "my_cluster",
    # "my_other_cluster"
  ]
}

// Создаем кластер для каждого имени в списке
resource "digitalocean_kubernetes_cluster" "cluster" {
  for_each = { for name in var.names : name => name }

  name = each.value
  region = var.region
  version = data.digitalocean_kubernetes_versions.versions.latest_version

  node_pool {
    name = "default"
    size = "s-2vcpu-2gb"
    auto_scale = true
    min_nodes = 1
    max_nodes = 3
  }
}

```

Измените имя кластера и пула узлов на любые имена, которые вы хотите использовать. Если вы собираетесь создать несколько кластеров, добавьте больше имен в список.

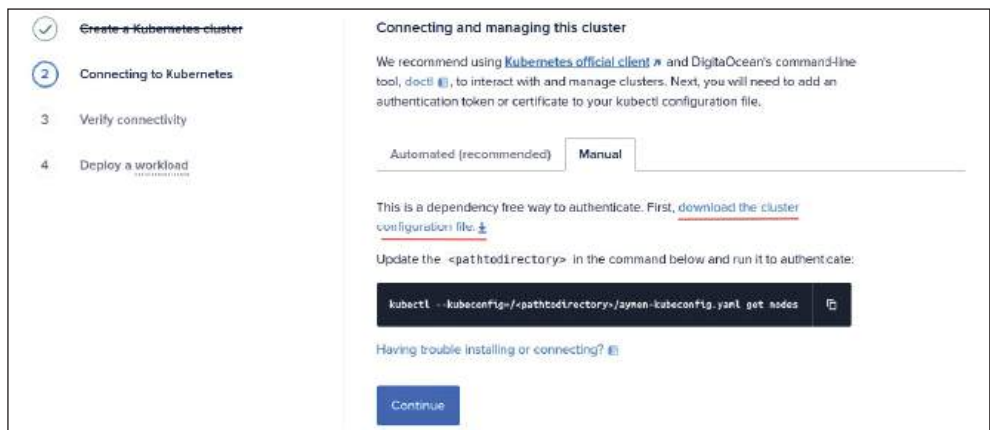
Создайте токен API, затем выполните следующие команды, чтобы экспортировать токен и создать кластер:

```

export DIGITALOCEAN_TOKEN=dop_v1_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
terraform init
terraform plan
terraform apply

```

Теперь вы можете загрузить файл kubeconfig из кластера, как показано на следующем снимке экрана:



Загрузите файл `kubeconfig` на свой сервер разработки и поместите его в каталог `$HOME/.kube/config`.

Например, если вы используете пользователя `root`, выполните следующие команды:

```
export IP=<your_development_server_ip>
scp kubeconfig.yaml root@$IP:/root/.kube/config
```

Вам нужно будет удаленно создать каталог `.kube`, если он еще не создан.

Если вы используете другое имя пользователя, скажем `myuser`, то вам следует выполнить такие команды:

```
export IP=<your_development_server_ip>
scp kubeconfig.yaml myuser@$IP:/home/myuser/.kube/config
```

Чтобы убедиться, что `kubectl` работает с загруженным файлом `kubeconfig`, выполните такие команды:

```
ssh root@$IP
kubectl get services -A
```

Вы должны увидеть сервисы по умолчанию, запущенные в вашем кластере DigitalOcean.

Глава 4

Обзор архитектуры Kubernetes

4.1. Введение

Kubernetes – это распределенная система, состоящая из нескольких компонентов, которые работают вместе, образуя высокодоступную, масштабируемую и отказоустойчивую платформу.

Архитектура Kubernetes построена на кластере узлов, где каждый узел представляет собой отдельную машину в кластере. Эти узлы вместе формируют высокодоступную и отказоустойчивую систему, способную запускать большое количество контейнеризированных приложений.

Kubernetes состоит из двух основных элементов:

- плоскость управления;
- узлы.

Каждый кластер имеет как минимум одну плоскость управления и один рабочий узел.

4.2 Плоскость управления

Плоскость управления сама состоит из нескольких компонентов, каждый из которых играет определенную роль в управлении и мониторинге кластера.

Вот краткое описание каждого компонента.

4.2.1. Система хранения etcd

Компонент etcd – это распределенная система хранения данных «ключ–значение», используемая для хранения конфигурации и состояния кластера.

4.2.2. Сервер API (kube-apiserver)

Сервер API – это основная точка входа для взаимодействия с кластером. Он предоставляет RESTful API, позволяющий пользователям создавать, изменять и удалять объекты в кластере с помощью CLI, например `kubectl`, SDK или путем прямого вызова API.

4.2.3. Менеджер контроллеров (kube-controller-manager)

Менеджер контроллеров отвечает за управление контроллерами Kubernetes. Эти контроллеры отслеживают состояние объектов в кластере и предпринимают действия для поддержания желаемого состояния.

4.2.4. Облачный менеджер контроллеров (cloud-controller-manager)

Это облачный компонент, который управляет службами, связанными с определенными платформами, такими как AWS, GCP и т. д.

4.2.5. Планировщик (kube-scheduler)

Планировщик распределяет поды по узлам в кластере, гарантируя, что они размещены на узлах с необходимыми ресурсами для их запуска.

4.3. Узлы

Узлы – это машины, на которых Kubernetes выполняет пользовательские рабочие нагрузки. Каждый узел в кластере запускает следующие компоненты.

4.3.1. Kubelet

Kubelet – это агент Kubernetes, который работает на каждом узле. Он отвечает за связь с плоскостью управления и за управление подами и контейнерами на узле.

4.3.2. Среда выполнения контейнера

Среда выполнения контейнера (container Runtime) – это инструмент, используемый для запуска контейнеров. Наиболее распространенными средами выполнения контейнеров являются Docker, containerd и CRI-O^{1,2,3}.

4.3.3. Kube-proxy

Kube-proxy – это сетевой прокси, который работает на каждом узле. Он отвечает за управление сетевым трафиком между подами в кластере.

4.4. Пулы узлов

Пул узлов (node pool) – это группа идентичных узлов (или воркеров) в кластере Kubernetes. Он позволяет группировать узлы на основе их операционных возможностей и роли в кластере.

Пул узлов можно использовать для определенных задач, таких как выполнение рабочих нагрузок, требующих высокой вычислительной мощности, или размещение приложений, требующих большого объема хранилища данных.

Каждый пул узлов имеет определенную конфигурацию и метку, которые позволяют идентифицировать его и управлять им независимо от других узлов в кластере.

Пулы узлов обеспечивают более эффективное и детализированное управление ресурсами в кластере Kubernetes, позволяя администраторам развертывать и управлять узлами на основе конкретных потребностей приложений и сервисов.

Типичный случай использования пулов узлов в Kubernetes – когда вам нужно горизонтально масштабировать приложение, добавляя или удаляя рабочие узлы для обработки рабочей нагрузки. Пулы узлов позволяют создавать группы узлов со схожими характеристиками и назначать им различные рабочие нагрузки. Например, вы можете создать один пул узлов для задач с интенсивным использованием ЦП, а другой пул – для задач с интенсивным использованием памяти, а также назначать поды этим пулам узлов на основе их требований к ресурсам.

Если вам нужен еще более детальный контроль над планированием распределения подов, вы можете использовать *окраску узлов (node taint)*, чтобы пометить узлы как не подходящие для планирования определенных подов,

¹ <https://github.com/moby/moby>.

² <https://github.com/containerd/containerd>.

³ <https://github.com/cri-o/cri-o>.

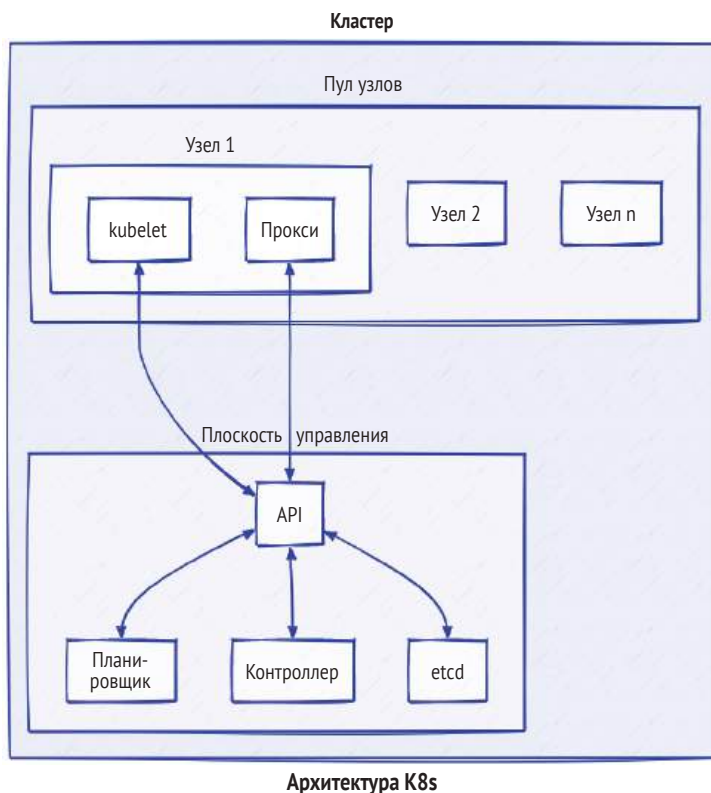
или *допуски узлов* (node toleration), чтобы разрешить планирование определенных подов на определенных узлах, несмотря на их окраску.

В кластере отдельные пулы узлов могут быть созданы, обновлены и удалены, не затрагивая весь кластер или другие узлы в других пулах узлов. Однако при внесении изменений в конфигурацию пула узлов изменения будут применены ко *всем* узлам в этом пуле; невозможно настроить только один узел в пуле.

Пулы узлов также облегчают обновление Kubernetes. Создав новый пул узлов с обновленной версией Kubernetes, администратор может постепенно перемещать рабочие нагрузки из старого пула в новый.

4.5. Обзор архитектуры

На следующей схеме показаны различные компоненты кластера Kubernetes.



Глава 5

Микросервисы без сохранения и с сохранением состояния

5.1. Введение

В микросервисной архитектуре различные компоненты работают вместе, но независимо, для достижения определенной цели или решения задачи. Эти сервисы можно разделить на два типа: *без сохранения состояния* (stateless) и *с сохранением состояния* (stateful).

При проектировании и развертывании приложений в облаке важно понимать разницу между этими категориями рабочих нагрузок.

В следующих главах этой книги вы узнаете, чем они отличаются, когда дело доходит до развертывания и управления ими в кластере Kubernetes.

5.2. Рабочие нагрузки без сохранения состояния

К рабочим нагрузкам без сохранения состояния относятся приложения, которым не требуется информация о предыдущих взаимодействиях или событиях для выполнения задачи. Каждый запрос обрабатывается независимо, без знания каких-либо предыдущих запросов.

Хорошим примером рабочей нагрузки без сохранения состояния является веб-сервер, который обслуживает статический контент, такой как файлы HTML, JS или CSS. Другим примером является контейнер, который запускает пакетное задание для преобразования данных.

Когда рабочей нагрузке без сохранения состояния необходимо сохранить состояние, она обычно полагается на *внешний* источник данных.

Эти приложения не хранят никаких данных в промежутке времени между запросами, поэтому их можно легко масштабировать, создавая дополнительные экземпляры приложения или службы без каких-либо особых ограничений.

Развертывание или масштабирование такого типа рабочей нагрузки не требует выполнения сложных операций.

Благодаря высокой эксплуатационной гибкости рабочие нагрузки этого типа стали очень популярными в облачную эпоху. Контейнеры, такие как Docker, например, по умолчанию не имеют состояния, и когда контейнер выходит из строя, все данные, хранящиеся в нем, теряются. Когда вы создаете новый контейнер с тем же образом, рабочая нагрузка сбрасывается в свое исходное состояние, описанное в файле конфигурации `dockerfile`.

Чем больше ваше приложение полагается на внешние источники данных, тем проще становится их развертывание, масштабирование и обслуживание.

Однако эти преимущества влекут за собой и некоторые недостатки, в первую очередь из-за делегирования ответственности за данные внешним источникам. Это может привести к более высоким задержкам, поскольку вычисления не выполняются рядом с хранилищем.

5.3. Рабочие нагрузки с сохранением состояния

Несмотря на высокую популярность приложений, не сохраняющих состояние, для большинства приложений, создаваемых разработчиками, по-прежнему необходимы данные. В большинстве случаев наши приложения полагаются на различные хранилища данных: Redis, PostgreSQL, MySQL и т. д.

Вот когда в игру вступают рабочие нагрузки с сохранением состояния.

Обычно это приложения, которым требуется постоянное хранение данных и которые нуждаются в поддержании состояния в нескольких экземплярах. Эти приложения, как правило, сложнее, чем приложения без сохранения состояния, и их труднее разворачивать и управлять в Kubernetes, учитывая дополнительную сложность управления хранилищем.

Примерами рабочих нагрузок с сохранением состояния являются базы данных, очереди сообщений и файловые серверы. Базы данных (например, MongoDB) и очереди сообщений (например, Kafka) основаны на сохранении состояния.

Хотя разработчики в первую очередь рассматривают подходы без сохранения состояния, они могут привести к дополнительным финансовым и операционным расходам при необходимости увеличить производительность, тогда как подход с сохранением состояния более эффективен при создании высокопроизводительных и потоковых систем, работающих в режиме, близком к реальному времени.

Глава 6

Развертывание микросервисов без сохранения состояния

6.1. Технические требования

Мы будем использовать наш сервер разработки Ubuntu, но сперва установим `kubectl` и загрузим файл `kubeconfig`.

Для следующих примеров мы будем использовать Python в качестве основного языка программирования и Flask в качестве фреймворка. Код Python и Flask легко понять, даже если вы не программируете на языке Python, поэтому примеры будут хорошо понятны всем разработчикам.

Начнем с установки на сервер разработки следующих пакетов:

- `pip3`: официальный менеджер пакетов и команда `pip` для Python 3;
- `virtualenvwrapper`: набор расширений для создания и удаления виртуальных сред разработки Python.

```
apt get update && apt install -y python3-pip
pip install virtualenvwrapper
export WORKON_HOME=~/.Envs
mkdir -p $WORKON_HOME
export VIRTUALENVWRAPPER_PYTHON='/usr/bin/python3'
source /usr/local/bin/virtualenvwrapper.sh
```

Далее создадим новую виртуальную среду:

```
mkvirtualenv stateless-flask
```

Теперь создадим каталоги для нашего приложения Flask и установим его зависимости.

```
mkdir -p stateless-flask
cd stateless-flask
mkdir -p app
mkdir -p kubernetes
pip install Flask==2.2.3
pip freeze > app/requirements.txt
```

Следующий код создает простое todo-приложение:

Листинг 6.1 ❖ Файл app/app.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Определяем список заданий
tasks = []

# Путь для получения всех заданий
@app.route('/tasks', methods=['GET'])
def get_tasks():
    return jsonify({'tasks': tasks})

# Путь для получения одного задания
@app.route('/tasks', methods=['POST'])
def add_task():
    task = {
        'id': len(tasks) + 1,
        'title': request.json['title'],
        'description': request.json['description'],
    }
    tasks.append(task)
    return jsonify(task), 201

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Этот код создает приложение Flask, которое определяет список задач. У него есть два маршрута: один для получения всех задач, а другой для добавления задачи.

Функция `get_tasks()` возвращает ответ JSON, содержащий все задачи в списке.

Функция `add_task()` создает новую задачу с идентификатором, заголовком и описанием, добавляет ее в список задач и возвращает ответ JSON, содержащий новую задачу.

Следующий блок кода запускает приложение Flask и делает его доступным на локальном хосте на порту 5000. Для облегчения разработки включен режим отладки.

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Затем создадим файл `dockerfile`:

Листинг 6.2 ❖ Файл `app/Dockerfile`

```
# Используем официальный дистрибутив Python
FROM python:3.9-slim-buster
# Задаем рабочий каталог /app
WORKDIR /app
# Копируем содержимое текущего каталога в контейнер в /app
COPY . /app
# Устанавливаем все пакеты, указанные в requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
# Открываем порт 5000 для доступа к контейнеру извне
EXPOSE 5000
# Определяем переменную окружения
CMD ["python", "app.py"]
```

Теперь мы можем собрать образ и запустить его для тестирования.

```
docker build -t stateless-flask:v0 -f app/Dockerfile app
docker run -it -p 5000:5000 stateless-flask:v0
```

Поскольку мы открываем контейнер на порту 5000 хоста, мы запросим API, сгенерированный Flask, с помощью инструмента наподобие Postman через публичный IP нашего дроплета, за которым следует порт 5000.

Загрузите и установите Postman¹.



Используйте запрос POST со следующими данными:

```
{
  "description": "Master the art of using containers, Kubernetes and microservices",
  "id": 1,
  "title": "Kubernetes"
}
```

Не забудьте добавить заголовок `Content-Type` и задайте ему значение `application/json`, чтобы получить ответ в формате JSON.

После сохранения первого элемента вы можете выполнить запрос GET к `/tasks`, и получите те же данные JSON, которые вы сохранили. Однако при

¹ <https://www.postman.com/downloads/>.

остановке контейнера данные будут стерты, что нормально, поскольку приложение не имеет состояния и не сохраняет данные ни в каком внешнем хранилище данных.

Теперь создайте учетную запись на Docker Hub¹ и отправьте образ.

```
docker login
docker tag stateless-flask:v0 <dockerhub_username>/stateless-flask:v0
docker push <dockerhub_username>/stateless-flask:v0
```

Замените <dockerhub_username> на свое имя пользователя Docker Hub.

6.2. Создание пространства имен

Мы можем развернуть приложение в пространстве имен по умолчанию, однако это не рекомендуется, особенно если у вас несколько приложений, работающих в одном кластере. В этом случае рекомендуемый подход – создать отдельное пространство имен для каждого приложения.

Мы развернем API Flask в пространстве имен, отличном от пространства по умолчанию. Сначала создайте его (листинг 6.3).

Листинг 6.3 ❖ Файл kubernetes/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: stateless-flask
```

Теперь примените YAML:

```
kubectl apply -f kubernetes/namespace.yaml
```

Выведите список всех пространств имен:

```
kubectl get ns
```

Вы должны увидеть только что созданное пространство имен.



Пространство имен обеспечивает виртуальное разделение ресурсов внутри кластера. Оно позволяет нескольким командам или приложениям использовать один и тот же кластер Kubernetes, имея собственные изолированные наборы ресурсов, такие как поды, службы и тома. Это виртуальное разделение помогает управлять ресурсами и избегать конфликтов имен между командами или приложениями.

Например, если вы хотите запустить несколько приложений на одном кластере, вы можете развернуть каждое приложение в другом пространстве имен с ресурсом Ingress, который направляет трафик всем этим приложениям, находящимся в разных пространствах имен.

¹ <https://hub.docker.com/>.

6.3. Создание развертывания

Чтобы развернуть контейнер Flask, нам нужно создать *развертывание* (deployment). Развертывание создаст ReplicaSet, а последний отвечает за поддержание стабильного набора реплик – пода.

i *Под* (pod) – это наименьшая и самая простая единица объектной модели Kubernetes, представляющая собой один экземпляр запущенного процесса в кластере.

Под может содержать один или несколько контейнеров, которые гарантированно будут работать вместе на одном хосте и совместно использовать одно и то же сетевое пространство имен.

Все контейнеры в поде совместно используют один и тот же набор пространств имен Linux, включая сетевое пространство имен, что означает, что они могут взаимодействовать друг с другом через сетевой интерфейс localhost.

Контейнеры в одном поде имеют одинаковый жизненный цикл: когда создается под, создаются все контейнеры, а когда под уничтожается или удаляется, все контейнеры внутри этого пода исчезают.

Например, контейнеры container_a и container_b входят в один и тот же под, у них обоих будет одинаковый IP пода. Контейнер container_a будет обращаться к container_b, как если бы он обращался к localhost¹, и наоборот.

В следующем манифесте YAML обязательно измените <your-docker-registry> на свое имя пользователя Docker Hub.

Листинг 6.4 ❖ Файл kubernetes/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateless-flask
  namespace: stateless-flask
spec:
  replicas: 3
  selector:
    matchLabels:
      app: stateless-flask
  template:
    metadata:
      labels:
        app: stateless-flask
    spec:
      containers:
        - name: stateless-flask
          image: <your-docker-registry>/stateless-flask:v0
          ports:
            - containerPort: 5000
```

¹ <http://localhost/>.

Давайте разберем по строкам, какую информацию содержит файл манифеста развертывания:

- `apiVersion` указывает версию API Kubernetes для этого объекта, в данном случае это `apps/v1`;
- `kind` указывает тип создаваемого объекта, который является развертыванием;
- `metadata` содержит некоторые метаданные о развертывании, включая имя и пространство имен развертывания;
- `spec` указывает желаемое состояние развертывания. Он содержит количество создаваемых реплик (в данном случае 3), а также селектор для сопоставления меток подов, которые должны управляться развертыванием;
- поле `selector` в развертывании Kubernetes указывает метки, используемые для идентификации и выбора пода, которыми управляет развертывание. В нашем коде селектор указывает, что развертывание будет управлять подом с меткой `app: stateless-flask`. Это означает, что любые поды, имеющие эту метку, будут включены в набор реплик, поддерживаемых развертыванием, а любые поды, не имеющие этой метки, не будут включены. Поле `matchLabels` в селекторе указывает метки, которые должны совпадать для включения пода в набор;
- `template` содержит спецификацию для подов, которые должны быть созданы развертыванием. Он указывает метки, которые применяются к поду и должны соответствовать селектору в `spec`;
- `containers` содержит список контейнеров, которые будут развернуты в поде. В данном случае у него есть только один контейнер с именем `stateless-flask`, который использует образ Docker, указанный в `image`, и открывает порт 5000.

Мы можем применить файл YAML и получить список развертываний в пространстве имен `stateless-flask`. Не забудьте заменить `<your-docker-registry>` на свое имя пользователя Docker Hub.

```
kubectl apply -f kubernetes/deployment.yaml
kubectl get pods -n stateless-flask
```

Следующая схема показывает различные ресурсы Kubernetes, которые мы создали.

Обратите внимание, что наш кластер имеет один узел. Если бы у нас было два узла, то три созданных нами пода, скорее всего, были бы распределены по узлам.



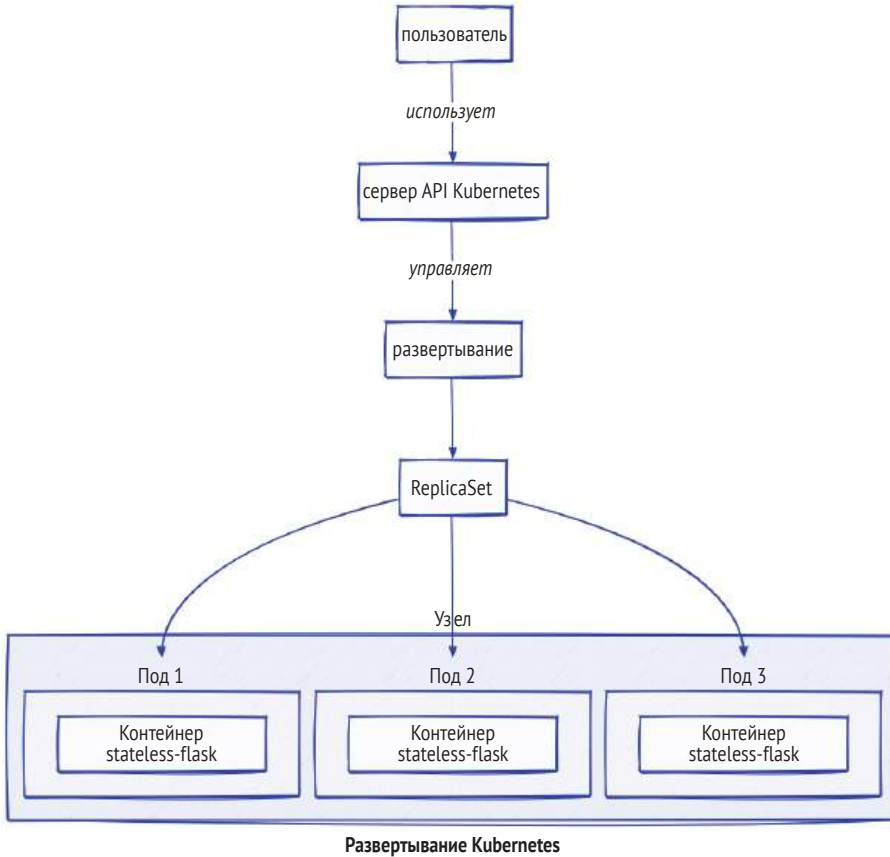
Развертывание отвечает за управление набором реплик определенного пода.

Развертывание делает так, чтобы в любой момент времени работало нужное количество реплик, и предоставляет способ выполнять скользящие обновления и откаты базового шаблона пода.

Развертывание предоставляет декларативный интерфейс для создания, обновления и удаления подов и `ReplicaSet`.

i ReplicaSet – это еще один объект Kubernetes, который отвечает за поддержание стабильного набора реплик подов. Он гарантирует, что определенное количество реплик подов на основе определенного шаблона (обычно определенного в Deployment) работает.

ReplicaSets часто используются как часть развертывания, при этом развертывание управляет ReplicaSet, чтобы обеспечить наличие желаемого количества реплик. Поэтому не рекомендуется создавать ReplicaSet напрямую. Вместо этого создайте развертывание.



6.4. Проверка подов и развертываний

Пора познакомиться с полезной командой `kubectl describe`, которую мы можем использовать для проверки состояния, событий и конфигурации пода.

Если вы использовали Docker, вы наверняка знакомы с командой `docker inspect <container>`. Команда `kubectl describe pod <pod>` чем-то похожа на нее.

```
kubectl -n stateless-flask describe pod <pod>
```

Замените `<pod>` на имя любого пода. Вы можете получить список подов с помощью команды `kubectl -n stateless-flask get pods`.

Вы также можете получить описание развертывания с помощью команды `kubectl -n stateless-flask describe deployment <deployment>`

Одной из самых полезных разновидностей информации, которую предоставляет команда `describe`, является *список событий*.

События предоставляют способ отслеживания состояния различных объектов в системе. Этими объектами могут быть узлы, поды, развертывания, сервисы и т. д. Регистрация событий формирует аудиторский след изменений, которые происходят с этими объектами с течением времени.

Журналы подов (pod's log) – это еще один способ понять, что происходит внутри пода. С помощью следующей команды мы можем просмотреть журналы подов:

```
kubectl -n stateless-flask logs <pod>
```

Вам нужно заменить `<pod>` на имя нужного пода.

Если вы хотите получить журналы всех подов в одной команде, добавьте фильтр по метке. Вот пример:

```
kubectl -n stateless-flask logs -l app=stateless-flask
```

Мы используем `app=stateless-flask`, потому что мы задали эту метку в шаблоне наших подов в файле развертывания.

Чтобы отобразить журналы и отслеживать изменения, добавьте флаг `-f`.

```
kubectl -n stateless-flask logs <pod> -f
kubectl -n stateless-flask logs -l app=stateless-flask -f
```

6.5. Доступ к подам

Под не исследует какой-либо внешний адрес/порт, поэтому если мы хотим увидеть запущенное приложение Flask, нам нужно перенаправить внутренний порт пода на один из портов на нашем сервере разработки. Вот как это делается:

```
kubectl port-forward stateless-flask-<pod_id> 5000:5000 -n stateless-flask
```

Обязательно замените `stateless-flask-<pod_id>` на имя нужного пода. Вы можете получить список подов с помощью команды `kubectl get pods -n stateless-flask`.

Вы также можете использовать AWK, чтобы получить первый под, который появляется в списке, когда вы вводите `kubectl get pods` следующим образом:

```
export pod=$(kubectl get pods -n stateless-flask | awk 'FNR==2{print $1}
```

```
}' )
```

```
kubectl port-forward $pod 5000:5000 -n stateless-flask
```

В обоих случаях под теперь доступен через наш локальный сервер, и мы можем проверить его ответ с помощью команды

```
curl http://0.0.0.0:5000/tasks
```

6.6. Предоставление доступа к развертыванию

Если мы хотим использовать наше приложение Flask обычным способом, мы должны сделать его доступным через веб-браузер. Однако у нас не будет внешнего доступа к нашему поду, если мы не создадим переадресацию портов. Вот почему нам нужно создать сервис.

i *Сервис* – это мощный уровень абстракции, который обеспечивает простой способ раскрытия подов в качестве сетевой службы.

Развертывание – это основа вашего приложения, позволяющая легко управлять созданием и масштабированием набора идентичных подов. В свою очередь, сервис предоставляет вам IP-адрес и имя DNS для надежного доступа к этим подам.

Определив сервис для развертывания, вы можете легко получить доступ к подам из других приложений в кластере или извне кластера через IP-адрес или имя DNS сервиса.

Это интеллектуальное отделение подов от сервисов обеспечивает простое управление подами и большую гибкость доступа к ним.

В Kubernetes существует несколько способов раскрытия приложения в сети, включая сервисы ClusterIP, NodePort, LoadBalancer, Ingress и Headless.

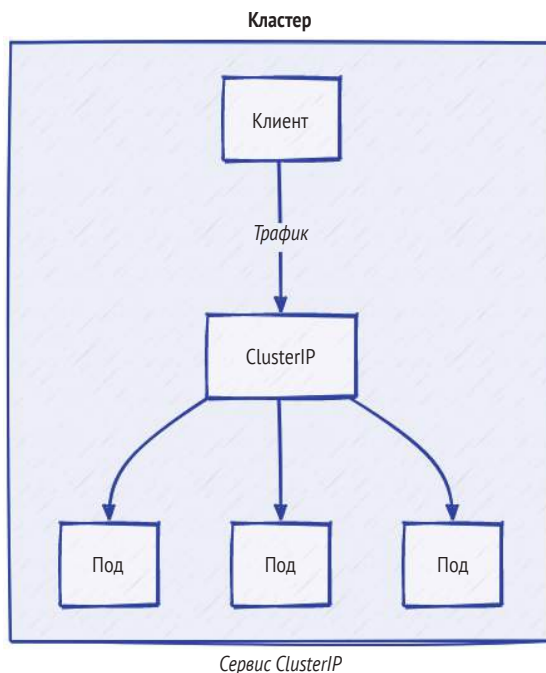
6.6.1. Сервис ClusterIP

ClusterIP – это тип сервиса по умолчанию, который предоставляет стабильный IP-адрес для доступа к сервису в кластере. Он раскрывает под в кластере другим объектам в том же кластере, используя внутренний IP-адрес, что обеспечивает бесперебойную связь между компонентами.

Сервис ClusterIP применяется для обеспечения связи между различными компонентами или микросервисами в кластере Kubernetes. Он действует как виртуальный IP-адрес, назначенный набору подов¹.

Структура сервиса ClusterIP показана на следующем рисунке.

¹ https://en.wikipedia.org/wiki/Virtual_IP_address.



Так можно создать собственный сервис ClusterIP:

Листинг 6.5 ❖ Файл `kubernetes/clusterip-service.yaml`

```

apiVersion: v1
kind: Service
metadata:
  name: stateless-flask-clusterip-service
  namespace: stateless-flask
spec:
  type: ClusterIP
  selector:
    app: stateless-flask
  ports:
    - port: 5000
      protocol: TCP
      targetPort: 5000

```

- `apiVersion`: версия API Kubernetes, в которой написан этот манифест;
- `kind`: тип объекта Kubernetes, который описывает этот манифест. В данном случае это `Service` (сервис);
- `metadata`: метаданные о сервисе, включая его имя и пространство имен;
- `spec`: спецификация сервиса, которая включает его тип, селектор и порты;
- `type`: указывает, что этот сервис относится к типу `ClusterIP`, следовательно, он доступен только в пределах кластера;

- `selector`: определяет селектор, идентифицирующий под, на который этот сервис должен направлять трафик. В данном случае он выбирает под с меткой `app: stateless-flask`;
- `ports`: указывает конфигурацию порта сервиса. В данном случае он отображает порт 5000 пода как порт 5000 сервиса;
- `port` обозначает порт, который будет прослушивать сервис, в то время как `targetPort` обозначает порт, который прослушивают внутренние службы пода.

Примените файл YAML и проверьте список сервисов:

```
kubectl apply -f kubernetes/cluserip-service.yaml  
kubectl get svc -n stateless-flask
```

Чтобы получить доступ к приложению, нам нужно использовать сервис ClusterIP. Для этого мы применим переадресацию портов Kubernetes:

```
kubectl port-forward svc/stateless-flask-clusterip-service 5000:5000 -n stateless-flask
```

Теперь мы можем проверить подключение к службе с помощью команды `cURL`.

```
curl [http://127.0.0.1:5000/tasks](http://127.0.0.1:5000/tasks)
```

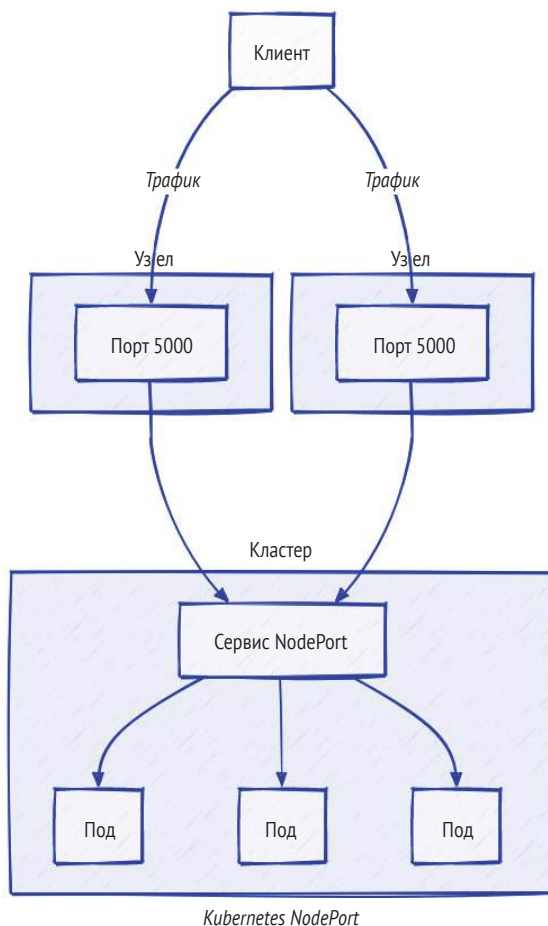
6.6.2. Сервис NodePort

NodePort – это сервис, который раскрывает развертывание для доступа извне на статическом порту на каждом узле в кластере. Диапазон портов для сервисов NodePort составляет 30000–32767. Внешний трафик поступает на эти порты, а затем перенаправляется в под.

Одним из преимуществ сервиса NodePort является простота, так как он не требует никаких внешних балансировщиков нагрузки или сетевых компонентов. Он также прост в использовании и не требует особой настройки.

Однако у NodePort есть и недостатки. Он представляет сервис на статическом порту на каждом узле в кластере, что может вызвать проблемы, если у вас есть потенциальные уязвимости системы. Для него также нужно открыть диапазон портов на всех узлах в кластере, что может послужить потенциальным вектором атаки.

NodePort полезен для тестирования и отладки, но если у вас есть производственная нагрузка и вы хотите разрешить внешний трафик, это не самое подходящее решение.



Вот как мы можем создать сервис NodePort:

Листинг 6.6 ❖ Файл `kubernetes/nodeport-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: stateless-flask-nodeport-service
  namespace: stateless-flask
spec:
  type: NodePort
  selector:
    app: stateless-flask
  ports:
    - port: 5000
      protocol: TCP
      targetPort: 5000
      nodePort: 30000
```

Порты, которые будет прослушивать этот сервис, определяются в `ports`, включая `port` (порт, который прослушивает сервис), `protocol` (протокол, используемый портом), `targetPort` (порт пода, на который должен отправляться трафик) и `nodePort` (порт, который сервис предоставляет на каждом узле в кластере). В данном случае сервис прослушивает порт 5000 и отображается на каждом узле кластера на порту 30000.

Теперь вы можете применить этот манифест YAML:

```
kubectl apply -f kubernetes/nodeport-service.yaml
```

Проверим список сервисов:

```
kubectl get svc -n stateless-flask
```

Получим внешний IP-адрес любого из узлов вашего кластера:

```
kubectl get nodes -o wide
```

С помощью веб-браузера, такого инструмента, как Postman или cURL, вы можете проверить ответ API, используя внешний IP-адрес узла и порт 30000.

```
curl http://<external_ip>:30000/tasks
```

Все это можно сделать одной командой:

```
curl HTTP://$(kubectl get nodes -o wide | awk '{print $7}' | tail -n +2):30000/tasks
```

Команда `kubectl get nodes -o wide | awk '{print $7}' | tail -n +2` получит внешний IP-адрес, не показывая весь вывод `kubectl get nodes -o wide`.

6.6.3. Сервис LoadBalancer

LoadBalancer – это еще один тип сервиса, доступный для использования в кластере Kubernetes. Он помогает распределять входящий трафик по нескольким репликам пода. Это реализация паттерна Load Balancing (балансировка нагрузки)¹, который используется для распределения трафика по нескольким узлам, службам или системам.

В Kubernetes это сделано немного по-другому: для реализации паттерна вам понадобятся две вещи:

- сервис LoadBalancer;
- машина балансировщика нагрузки.

Хотя сервис – это объект Kubernetes, который, в свою очередь, является частью кластера, машина балансировщика не входит в состав кластера и относится к внешним объектам. Если вы управляете физическим сервером Kubernetes или локальным кластером, реализация может оказаться сложной за-

¹ [https://en.wikipedia.org/wiki/Load_balancing_\(computing\)](https://en.wikipedia.org/wiki/Load_balancing_(computing)).

дачей, однако если вы используете управляемый кластер с использованием публичного облака, такого как AWS, GCP или DigitalOcean, все гораздо проще.

Обычно провайдер облачных услуг автоматически предоставляет вам машину балансировщика нагрузки.

Сервис LoadBalancer улучшает доступность и масштабируемость приложения за счет распределения трафика по нескольким репликам, что снижает вероятность возникновения единой точки отказа.

Одним из его важных преимуществ является способность автоматически обнаруживать неработоспособные реплики и перенаправлять трафик на работоспособные, что добавляет еще один уровень надежности к вашей рабочей нагрузке.

Кроме того, при создании LoadBalancer вы можете добавлять расширенные функции, такие как завершение SSL или привязка сеанса.

Однако создание машины балансировщика нагрузки для каждого вашего сервиса может привести к высоким накладным расходам. Кроме того, добавление дополнительного сетевого уровня может привести к задержке, хотя в большинстве случаев она незначительна.



Балансировщик нагрузки Kubernetes

Чтобы создать сервис LoadBalancer, используйте следующий манифест YAML:

Листинг 6.7 ❖ Файл `kubernetes/loadbalancer-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: stateless-flask-loadbalancer-service
  namespace: stateless-flask
spec:
  type: LoadBalancer
  selector:
    app: stateless-flask
  ports:
    - port: 5000
      protocol: TCP
      targetPort: 5000
```

В этом файле определяется один порт со следующими свойствами:

- `port`: номер порта, на котором будет доступен сервис;
- `protocol`: протокол, используемый сервисом, в данном случае TCP;
- `targetPort`: номер порта, на который сервис будет направлять трафик для пода.

Примените этот манифест:

```
kubectl apply -f kubernetes/loadbalancer-service.yaml
```

Проверьте список своих сервисов с помощью команды `kubectl get svc -n stateless-flask -w`, и когда внешний IP-адрес вашей машины балансировщика нагрузки будет готов, начните тестирование API, используя тот же IP-адрес и порт 5000 (port).

Выполните проверку при помощи следующих команд:

```
load_balancer_ip = $(kubectl get svc -n stateless-flask | grep "LoadBalancer" | awk '{print $4}')
```

```
curl http://$load_balancer_ip:5000/tasks
```

6.6.4. Сервис Headless

Обычно сервис Kubernetes имеет виртуальный IP-адрес (ClusterIP) и имя DNS, которое разрешается в этот IP-адрес. Когда клиент подключается к IP-адресу сервиса, он прозрачно перенаправляется на один из подов, поддерживающих сервис. Что делать, если нам нужно получить доступ к каждому поду, используя его собственный IP-адрес?

Этого можно достичь с помощью функционала сервиса Headless Kubernetes.

Сервисы Headless создаются с использованием того же синтаксиса, что и обычные сервисы, но с параметром `clusterIP`, установленным на `None`.

Сервис Headless можно создать при помощи следующего кода.

Листинг 6.8 ❖ Файл `kubernetes/headless-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: stateless-flask-headless-service
  namespace: stateless-flask
spec:
  clusterIP: None
  selector:
    app: stateless-flask
  ports:
    - port: 5000
      protocol: TCP
      targetPort: 5000
```

Теперь вы можете создать сервис при помощи команды

```
kubectl apply -f kubernetes/headless-service.yaml
```

и вывести список сервисов:

```
kubectl get svc -n stateless-flask
```

Чтобы получить доступ к сервису Headless извне кластера, вы можете использовать публичный IP-адрес одного из подов в сервисе, а затем порт, указанный в манифесте Kubernetes, который мы развернули (5000 в нашем случае).

Чтобы протестировать разрешение DNS, давайте создадим временный под `tmp01`, который запускает образ `tutum/dnsutils`¹. Мы разворачиваем этот под, потому что в нем установлен набор инструментов `dnsutils`, которые мы можем использовать для тестирования разрешения DNS.

Воспользуемся командой `nslookup` для тестирования разрешения DNS сервиса `stateless-flask-headless-service.stateless-flask.svc.cluster.local`.

Обратите внимание, что у сервиса Headless нет IP, однако к нему можно получить доступ по его внутреннему имени DNS:

```
stateless-flask-headless-service.stateless-flask.svc.cluster.local
```

Разверните временный под `tmp01` с помощью `kubectl`:

```
kubectl run tmp01 --image=tutum/dnsutils -- sleep infinity
```

Выполните `nslookup` для проверки разрешения DNS сервиса Headless. У нас есть возможность выполнить команду из пода, не находясь внутри него:

¹ <https://hub.docker.com/r/tutum/dnsutils>.

```
kubectl exec -it tmp01 -- nslookup stateless-flask-headless-service.stateless-flask.svc.cluster.local
```

Вывод должен выглядеть примерно так:

```
Server: <IP of the DNS server>
```

```
Address: <IP of the DNS server>#53
```

```
Name: stateless-flask-headless-service.stateless-flask.svc.cluster.local
```

```
Address: <IP of the Pod 1>
```

```
Name: stateless-flask-headless-service.stateless-flask.svc.cluster.local
```

```
Address: <IP of the Pod 2>
```

Обратите внимание, что сервер DNS возвращает IP-адреса подов, на которых запущено приложение `stateless-flask`. ClusterIP никогда не вернет IP-адреса подов, он вернет только свой собственный IP-адрес.

Кластеризация баз данных – это распространенный вариант использования сервисов Headless. RabbitMQ, MySQL и PostgreSQL – примеры приложений, которые можно кластеризовать с помощью Headless.

i Внутренние имена доступны только из кластера. Они имеют формат `<service-name>.<namespace>.svc.cluster.local`. Например, чтобы получить доступ к сервису `stateless-flask-headless-service` из другого пространства имен (отличного от `stateless-flask`), мы используем имя `stateless-flask-headless-service.stateless-flask.svc.cluster.local`, где `stateless-flask` – это пространство имен, в котором запущен сервис, `stateless-flask-headless-service` – это имя сервиса, а `svc.cluster.local` – это доменное имя кластера.

6.6.5. Сервис Ingress

Ingress – это компонент, который разрешает входящие соединения с кластером. Он предоставляет способ управления внешним доступом к сервисам в кластере. Вместо создания отдельного балансировщика нагрузки для каждого сервиса Ingress позволяет вам определить набор правил, которые маршрутизируют трафик к различным сервисам на основе хоста или пути запроса.

Сервисы Ingress упрощают доступ к остальным вашим сервисам за счет использования одного IP-адреса и доменных имен, а также могут применять сертификат SSL/TLS для шифрования трафика. Они имеют гибкую конфигурацию, которая позволяет вам определять правила маршрутизации трафика.

Например, я могу перенаправить трафик в сервис `my-service`, когда в запросе указан хост `myhost.com` и путь `/my-service`. Но если запрос будет содержать хост `myhost.com` и путь `/my-other-service`, трафик будет перенаправлен в сервис `my-other-service`.

И хотя некоторые облачные провайдеры не поддерживают собственные сервисы Ingress, вы всегда можете использовать контроллер Ingress с открытым исходным кодом или коммерческий, такой как Traefik или Nginx^{1,2}.

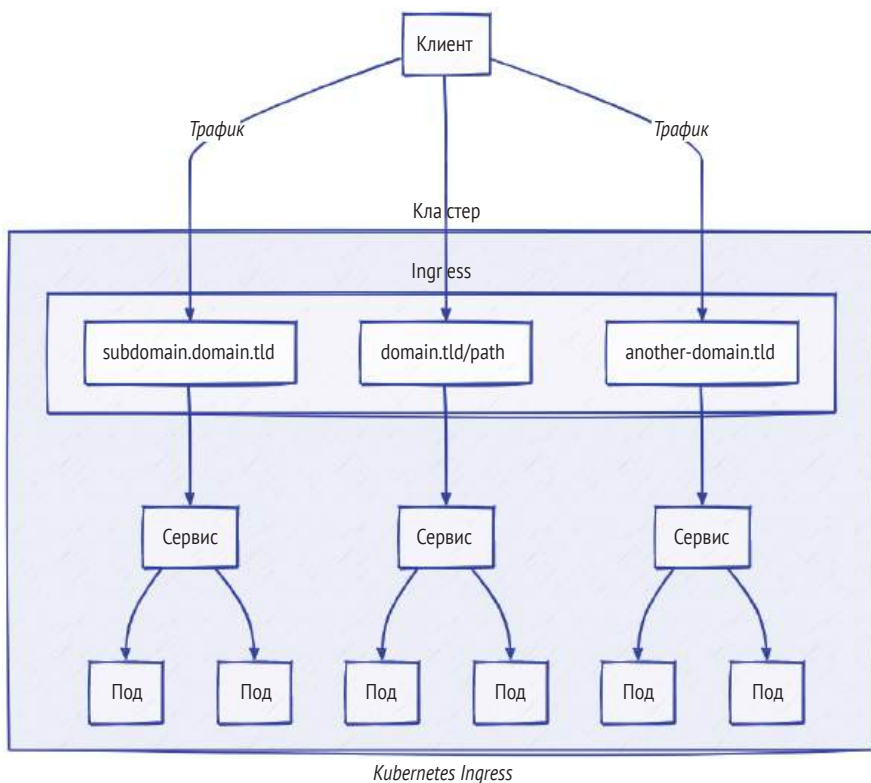
¹ <https://traefik.io/>.

² <https://www.nginx.com/>.

Вот список некоторых из них:

- Nginx Ingress: это один из самых популярных контроллеров, он использует веб-сервер Nginx для управления трафиком Ingress;
- Traefik Ingress: этот контроллер поддерживает несколько технологий прокси и предлагает простую интеграцию с поставщиками облачных услуг;
- HAProxy Ingress: этот контроллер использует прокси-сервер HAProxy для управления входящим трафиком;
- Istio Ingress: Istio – это платформа сервисной микросети, которая предлагает контроллер Ingress с расширенными функциями безопасности, такими как аутентификация, авторизация и шифрование;
- Contour Ingress: этот контроллер использует прокси-сервер Envoy для управления входящим трафиком и обеспечивает простую интеграцию с ресурсами Kubernetes.

Также важно отметить, что правила маршрутизации основаны на запросе HTTP(S), поэтому он не может маршрутизировать трафик TCP или UDP. Если вы хотите открыть порт TCP/UDP, вам нужно будет использовать LoadBalancer или NodePort.



Давайте очистим все ранее созданные сервисы перед созданием Ingress:

```
kubectl delete -f kubernetes/nodeport-service.yaml
kubectl delete -f kubernetes/loadbalancer-service.yaml
kubectl delete -f kubernetes/cluserip-service.yaml
```

Нам понадобится сервис ClusterIP, расположенный перед нашими подами. Этот сервис будет использоваться контроллером Ingress для маршрутизации трафика в поды. Создайте его при помощи следующего кода.

Листинг 6.9 ❖ Файл kubernetes/stateless-flask-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: stateless-flask
  namespace: stateless-flask
spec:
  selector:
    app: stateless-flask
  ports:
    - name: http
      protocol: TCP
      port: 5000
      targetPort: 5000
```

Примените эту конфигурацию:

```
kubectl apply -f kubernetes/stateless-flask-service.yaml
```

Теперь создадим ресурс Ingress.

Как вы уже видели, у нас есть выбор из нескольких контроллеров Ingress. Мы будем использовать контроллер Nginx Ingress. Этот контроллер развернут как в поде, так и в кластере.

Для установки контроллера Nginx Ingress нам понадобится менеджер Helm.

Helm¹ – это менеджер пакетов для Kubernetes. Он позволяет легко устанавливать приложения в kubernetes/clusterip-service.yaml.

Начните с установки Helm на вашем сервере разработки:

```
curl https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3 | bash
```

Затем добавьте репозиторий контроллера Nginx Ingress:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install nginx-ingress ingress-nginx/ingress-nginx --set controller.publishService.enabled=true
```

Хорошей практикой является создание выделенного пространства имен для контроллера Ingress, но здесь мы для простоты будем использовать про-

¹ <https://helm.sh/>.

странство имен по умолчанию. Если вы хотите использовать Helm для установки контроллера Ingress в выделенном пространстве имен, выполните следующую команду:

```
helm install nginx-ingress ingress-nginx/ingress-nginx --set controller.publishService.enabled=true --namespace ingress-nginx
```

Команда `helm install` устанавливает контроллер Nginx Ingress из стабильного репозитория. Она определяет параметр `publishService` как `true`, поэтому контроллер Ingress будет опубликован как сервис. Поскольку мы не указали пространство имен, Helm установит контроллер в пространство имен по умолчанию.

Теперь мы можем получить список подов и сервисов в пространстве имен по умолчанию:

```
kubectl get pods -n default
# или kubectl get pods
kubectl get services -n default
# или kubectl get services
```

Контроллер Ingress Nginx развертывается как под в пространстве имен по умолчанию. Он также публикуется как сервис в пространстве имен по умолчанию. Мы должны настроить контроллер Ingress для использования сервиса ClusterIP, который мы создали ранее. Для этого нам нужно создать ресурс Ingress.

Обратите внимание, что контроллер не является ресурсом Ingress. Ресурс Ingress – это ресурс Kubernetes, который определяет правила маршрутизации и используется контроллером Ingress для маршрутизации трафика в под через службу ClusterIP.

Вот пример ресурса Ingress:

Листинг 6.10 ❖ Файл `kubernetes/ingress.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
    - host: <host>
      http:
        paths:
          - path: <path>
            pathType: Prefix
            backend:
              service:
                name: <service-name>
                port:
                  number: 5000
  ingressClassName: nginx
```

- секция `spec` определяет правила Ingress. В данном случае есть одно правило, которое соответствует определенному имени `<host>` и префиксу пути `<path>`;
- секция `backend` указывает, в какой сервис Kubernetes направлять трафик для запросов, соответствующих указанному хосту и пути. В данном случае она указывает на сервис с именем `<service-name>` на порту 5000;
- полю `pathType` задано значение `Prefix`, что означает, что путь `/tasks` и любой путь, начинающийся с `/tasks/`, будут соответствовать этому правилу;
- поле `ingressClassName` содержит имя класса Ingress, который будет использоваться для этого сервиса Ingress. В данном случае оно указывает на контроллер Nginx.

Мы также можем добавить поле `namespace` в секцию `metadata`, чтобы указать пространство имен, в котором развернут сервис.

Если мы используем указанный выше ресурс Ingress, у нас будет сервис Ingress в пространстве имен по умолчанию, в то время как наше приложение было развернуто в пространстве имен `stateless-flask`. Это проблема, поскольку контроллер Ingress не сможет направлять трафик в под.

Чтобы решить эту проблему, нам нужно создать сервис `ExternalName` в пространстве имен `stateless-flask`, который указывает на сервис `ClusterIP` в пространстве имен по умолчанию. `ExternalName` – это особый тип сервиса, который позволяет направлять трафик в сервис, расположенный в другом пространстве имен.

Создайте следующий файл конфигурации.

Листинг 6.11 ❖ Файл `kubernetes/stateless-flask-service-externalname.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: stateless-flask-service-externalname
  namespace: default
spec:
  type: ExternalName
  externalName: stateless-flask.stateless-flask.svc.cluster.local
```

Теперь примените эту конфигурацию:

```
kubectl apply -f kubernetes/stateless-flask-service-externalname.yaml
```

Убедитесь, что служба `ExternalName` создана:

```
kubectl get services -n default
```

Теперь мы можем создать ресурс Ingress:

Листинг 6.12 ❖ Файл `kubernetes/ingress2.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```

metadata:
  name: my-ingress
spec:
  rules:
  - host: stateless-flask.<ingress-ip>.nip.io
    http:
      paths:
      - path: /tasks
        pathType: Prefix
        backend:
          service:
            name: stateless-flask-service-externalname
            port:
              number: 5000
  ingressClassName: nginx

```

В описанном выше ресурсе Ingress мы указали имя хоста `stateless-flask.<ingress-ip>.nip.io`. Обязательно замените `<ingress-ip>` на IP-адрес контроллера Ingress. Чтобы узнать IP-адрес контроллера Ingress, введите следующую команду:

```
kubectl get services nginx-ingress-ingress-nginx-controller | awk '{print $4}' | tail -n 1
```

Ingress работает с публичным DNS, однако у нас его нет. Мы будем использовать `nip.io`¹ для создания имени поддомена на лету из IP-адреса. Вы можете купить и использовать любое другое доменное имя у провайдера или использовать другой бесплатный сервис, например `sslip.io`².

Примените конфигурацию ресурса Ingress:

```
kubectl apply -f kubernetes/ingress.yaml
```

Проверьте наличие ресурса Ingress:

```
kubectl get ingress
```

Вы должны увидеть ресурс Ingress в выводе. Поле ADDRESS должно содержать IP-адрес контроллера Ingress.

Теперь вы можете открыть браузер и перейти по адресу:

```
http://stateless-flask.<ingress-ip>.nip.io/tasks
```

чтобы увидеть список задач.

Если ваш вариант использования не похож на описанные здесь, вы можете ознакомиться с официальной документацией для получения более подробной информации³.

¹ <https://nip.io/>.

² <https://sslip.io/>.

³ <https://kubernetes.github.io/ingress-nginx/user-guide/basic-usage/>.

Глава 7

Микросервисы с сохранением состояния в Kubernetes

7.1. Технические требования

Мы используем прежний сервер разработки Ubuntu 22.04 для создания и развёртывания ресурсов Kubernetes.

В этой главе мы развернем базу данных PostgreSQL и изменим код нашего приложения Flask, чтобы оно использовало базу данных для хранения своего состояния. Когда пользователь добавит элемент в список задач, он не исчезнет при перезапуске пода, поскольку все данные будут сохранены в базе данных PostgreSQL. Но – важный момент – это не превратит наш микросервис в сохраняющий состояние. Это будет по-прежнему микросервис без сохранения состояния, который хранит свое состояние во *внешнем* хранилище данных.

Установите `virtualenvwrapper`, если он еще не установлен:

```
apt get update && apt install -y python3-pip

pip install virtualenvwrapper
export WORKON_HOME=~/.Envs
mkdir -p $WORKON_HOME
export VIRTUALENVWRAPPER_PYTHON='/usr/bin/python3'
source /usr/local/bin/virtualenvwrapper.sh
```

Затем активируйте виртуальную среду:

```
mkvirtualenv stateful-flask
```

Теперь создайте каталог для нового приложения:

```
cd $HOME
mkdir -p stateful-flask
```

```
cd stateful-flask
mkdir -p app
mkdir -p kubernetes
```

Мы будем использовать в качестве имени `stateful-flask`, а не `stateless-flask` в предыдущей части. Однако, как уже было сказано, это не означает, что новое приложение Flask, которое мы собираемся создать, будет иметь состояние – оно все еще не имеет внутреннего состояния, но теперь для хранения своего состояния использует внешнее хранилище данных (PostgreSQL). С другой стороны, база данных – это сервис, сохраняющий состояние.

7.2. Создание пространства имен

Начнем с создания пространства имен для базы данных:

Листинг 7.1 ❖ Файл `kubernetes/postgres-namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: postgres
```

Затем создадим пространство имен для приложения Flask:

Листинг 7.2 ❖ Файл `kubernetes/namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: stateful-flask
```

Применим конфигурации пространства имен:

```
kubectl apply -f kubernetes/postgres-namespace.yaml
kubectl apply -f kubernetes/namespace.yaml
```

7.3. Создание ConfigMap для базы данных PostgreSQL

7.3.1. Что такое ConfigMap?

ConfigMap – это ресурс, который хранит неконфиденциальные данные конфигурации в виде пар «ключ–значение». Он позволяет отделить данные конфигурации от контейнерных приложений, что упрощает обновление

и управление конфигурациями без необходимости перестраивать и повторно развертывать все приложение.

ConfigMap может хранить данные конфигурации, в основном переменные среды, файлы конфигурации и любые другие неконфиденциальные данные, которые могут понадобиться вашим приложениям. Когда под запускается, он может считывать данные конфигурации контейнера из ConfigMap в виде переменных среды или файлов, смонтированных в томе.

Этот ресурс особенно полезен в некоторых сценариях, например когда несколько приложений совместно используют одну и ту же конфигурацию, когда данные конфигурации необходимо часто менять без повторного развертывания пода или когда вы хотите организовать свои развертывания, отделив операции от данных. Вместо того чтобы управлять конфигурацией для каждого приложения отдельно, можно использовать один ресурс ConfigMap для хранения данных конфигурации, к которым затем могут получить доступ несколько приложений. ConfigMap можно создать с помощью инструмента командной строки `kubectl` или путем объявления объекта ConfigMap в файле манифеста.

7.3.2. ConfigMap для PostgreSQL

В этом разделе мы создадим ConfigMap для PostgreSQL. Ресурс будет содержать следующие файлы:

- `POSTGRES_DB` – имя создаваемой базы данных;
- `POSTGRES_USER` – имя создаваемого пользователя;
- `POSTGRES_PASSWORD` – пароль создаваемого пользователя.

Листинг 7.3 ❖ Файл `kubernetes/postgres-config.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-config
  namespace: postgres
  labels:
    app: postgres
data:
  POSTGRES_DB: stateful-flask-db
  POSTGRES_USER: stateful-flask-user
  POSTGRES_PASSWORD: stateful-flask-password
```

Примените конфигурацию ConfigMap:

```
kubectl apply -f kubernetes/postgres-config.yaml
```

7.4. Постоянное хранение данных в PostgreSQL

7.4.1. Тома Kubernetes

Том (volume) – это уровень абстракции между контейнером и физическим хранилищем. Он позволяет контейнерам хранить и получать доступ к данным независимо от базовой инфраструктуры (AWS, GCP, Azure и т. д.).

Тома предоставляют способ хранения данных в контейнере за пределами жизненного цикла пода и позволяют обмениваться данными между подами.

Тома Kubernetes могут быть созданы из различных компонентов, таких как локальный диск, сетевая файловая система или блочное хранилище облачного провайдера. Их можно монтировать в контейнер как каталог или файл, а также получать к ним доступ и управлять ими, как и в любой другой файловой системе.

7.4.2. Объект VolumeClaims

Заявка на том (volume claim) – это запрос пользователя на хранилище. Она указывает объем хранилища, к которому под должен иметь доступ, и использует StorageClass для определения того, как хранилище должно быть предоставлено.

7.4.3. Объект StorageClass

StorageClass – это объект, определяющий тип хранилища, которое будет использоваться для динамического предоставления *постоянного тома* (persistent volume, PV) в ответ на заявку на постоянный том (persistent volume claim, PVC), сделанную подом и настроенную пользователем.

Некоторые примеры StorageClass в Kubernetes:

- aws-ebs: класс хранилища для томов Amazon Elastic Block Store (EBS);
- azure-disk: класс хранилища для дисков Azure;
- csi-cephfs: класс хранилища для томов CephFS с использованием драйвера интерфейса контейнерного хранилища (CSI);
- do-block-storage: тип постоянного хранилища, предлагаемый DigitalOcean (DO), который предоставляет тома хранения на основе SSD для хранения данных;
- local-storage: класс хранилища для локального хранения на узле.

7.4.4. Добавление хранилища в PostgreSQL

Чтобы добавить хранилище в PostgreSQL, необходимо создать `PersistentVolumeClaim` и `PersistentVolume`.

Листинг 7.4 ❖ Файл `kubernetes/postgres-pvc-pv.yaml`

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: postgres-pv-volume
  labels:
    app: postgres
spec:
  storageClassName: do-block-storage
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/mnt/data"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pv-claim
  namespace: postgres
  labels:
    app: postgres
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: do-block-storage
```

Этот файл манифеста YAML состоит из двух частей:

- `PersistentVolume`:
 - PV определен с параметрами `kind` и `apiVersion`, установленными на `PersistentVolume` и `v1` соответственно;
 - он имеет имя `name` и метки `labels` для целей идентификации;
 - параметру `storageClassName` присвоено значение `do-block-storage`, что определяет тип используемого класса хранилища;
 - задана емкость 5 ГБ, а параметру `accessModes` присвоено значение `ReadWriteMany`, т. е. к тому могут обращаться для чтения и записи несколько подов одновременно;
 - `HostPath` указывает физический путь к тому на хост-машине.

○ **PersistentVolumeClaim:**

- PVC определяется с параметром `kind`, установленным на `PersistentVolumeClaim`, и метаданными, определяющими имя `name`, пространство имен `namespace` и метки `labels`;
- параметру `accessModes` задано значение `ReadWriteOnce`, что означает, что том может быть прочитан и записан только одним подом за раз;
- параметр `resources` определяет объем хранилища, запрашиваемого для PVC;
- наконец, параметру `storageClassName` присвоено значение `do-block-storage`, что указывает на класс хранилища, используемого PV.

Согласно официальной документации для `PersistentVolumes` и `PersistentVolumeClaims` существуют следующие режимы доступа:

- `ReadWriteOnce`: том может быть смонтирован на чтение–запись одним узлом. Режим доступа `ReadWriteOnce` по-прежнему может позволить нескольким подам получать доступ к тому, когда поды работают на одном узле;
- `ReadOnlyMany`: том может быть смонтирован только на чтение одновременно несколькими узлами;
- `ReadWriteMany`: том может быть смонтирован на чтение и запись одновременно несколькими узлами;
- `ReadWriteOncePod`: том может быть смонтирован как чтение–запись одним подом. Используйте режим доступа `ReadWriteOncePod`, если вы хотите, чтобы только один под во всем кластере мог читать этот PVC или записывать в него. Этот параметр поддерживается только для томов CSI¹ и Kubernetes версии 1.22+.

Создайте `PersistentVolume` и `PersistentVolumeClaim`:

```
kubectl apply -f kubernetes/postgres-pvc-pv.yaml
```

Вы можете проверить статус `PersistentVolume` и `PersistentVolumeClaim` с помощью следующих команд:

```
kubectl get pv
kubectl get pvc -n postgres
```

7.4.5. Создание развертывания для PostgreSQL

В этом разделе мы создадим развертывание для PostgreSQL. Хотя это не лучший ресурс для использования с PostgreSQL, пока мы ограничимся этим подходом.

¹ <https://kubernetes.io/blog/2019/01/15/container-storage-interface-ga/>.

Листинг 7.5 ❖ Файл `kubernetes/postgres-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:10.1
          imagePullPolicy: Always
      ports:
        - containerPort: 5432
      envFrom:
        - configMapRef:
            name: postgres-config
      volumeMounts:
        - mountPath: /var/lib/postgresql/data
          subPath: postgres
          name: postgreddb
  volumes:
    - name: postgreddb
      persistentVolumeClaim:
        claimName: postgres-pv-claim
```

- Поле `spec` определяет желаемое состояние развертывания.
- `replicas` указывает количество реплик, которые мы хотим создать.
- `selector` указывает, как выбирать поды, которыми должно управлять развертывание. В данном случае `selector` сопоставляет поды с меткой `app: postgres`.
- Поле `template` указывает шаблон пода, используемый для создания новых подов в развертывании.
- `metadata` содержит метки, которые идентифицируют под как члена группы `app: postgres`.
- Поле `spec` содержит конфигурацию контейнера.
- Контейнер запускает образ PostgreSQL версии 10.1 и предоставляет порт 5432 для связи.
- `envFrom` определяет переменную среды из ресурса ConfigMap с именем `postgres-config`.

- Контейнеру требуется том для постоянного хранения, определенный в поле `volumes`.
- Том называется `postgredb` и поддерживается `PersistentVolumeClaim` (PVC) с именем `postgres-pv-claim`.
- Поле `volumeMounts` в конфигурации контейнера указывает том и его путь монтирования.
- В данном случае путь монтирования – `/var/lib/postgresql/data`, а подпуть – `postgres`, что означает, что данные базы данных PostgreSQL хранятся в каталоге `postgres` тома.

Применим развертывание:

```
kubectl apply -f kubernetes/postgres-deployment.yaml
```

7.4.6. Создание сервиса для PostgreSQL

В этом разделе мы создадим сервис для PostgreSQL. Для этого воспользуемся следующим файлом YAML:

Листинг 7.6 ❖ Файл `kubernetes/postgres-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: postgres # Имя сервиса
  namespace: postgres
  labels:
    app: postgres # Метки и селекторы
spec:
  type: NodePort # Тип сервиса
  ports:
    - port: 5432 # Порт для запуска приложения postgres
  selector:
    app: postgres
```

Теперь создадим сервис:

```
kubectl apply -f kubernetes/postgres-service.yaml
```

7.4.7. Создание развертывания для нашего приложения

Далее мы изменим код приложения таким образом, чтобы оно подключалось к базе данных PostgreSQL и сохраняло каждый элемент `todo` в базе данных.

Убедитесь, что новая виртуальная среда активирована:

```
workon stateful-flask
# или mkvirtualenv stateful-flask
```

Затем установите необходимые зависимости:

```
pip install Flask==2.2.3
pip install Flask-SQLAlchemy==3.0.3
pip install psycopg2-binary==2.9.6
pip install Flask-Migrate==4.0.4
```

«Заморозьте» зависимости:

```
pip freeze > requirements.txt
```

Теперь изменим код приложения и подготовим следующий файл app.py:

Листинг 7.7 ❖ Файл app/app.py

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://stateful-flask-user:stateful-flask-
password@postgres.postgres.svc.cluster.local:5432/stateful-flask-db'
db = SQLAlchemy(app)
migrate = Migrate(app, db)

class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(80), nullable=False)
    description = db.Column(db.String(200))

@app.route('/tasks', methods=['GET'])
def get_tasks():
    tasks = Task.query.all()
    return jsonify({'tasks': [{'id': task.id, 'title': task.title, 'description': task.
description} for task in tasks]})

@app.route('/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    title = data['title']
    description = data['description']
    task = Task(title=title, description=description)
    db.session.add(task)
    db.session.commit()
    return jsonify({'task': {'id': task.id, 'title': task.title, 'description': task.
description}})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Для компоновки образа нам также понадобится следующий файл Docker:

Листинг 7.8 ❖ Файл app/Dockerfile

```
FROM python:3.9-slim-buster
WORKDIR /app
```

```
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Скомпонуйте образ:

```
docker build -t stateful-flask:v0 -f app/Dockerfile app
```

Запустите контейнер для тестирования приложения:

```
docker run -it -p 5000:5000 stateful-flask:v0
```

Вы должны увидеть ошибку при подключении к базе данных, что нормально, поскольку мы не развернули приложение в кластере, это просто локальный тест.

Переименуйте образ и отправьте его в Docker Hub:

```
docker login
# замените <dockerhub_username> на ваше имя пользователя Docker Hub
export DOCKERHUB_USERNAME=<dockerhub_username>
docker tag stateful-flask:v0 $DOCKERHUB_USERNAME/stateful-flask:v0
docker push $DOCKERHUB_USERNAME/stateful-flask:v0
```

Создайте развертывание:

Листинг 7.9 ❖ Файл kubernetes/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          ports:
            - containerPort: 5000
          imagePullPolicy: Always
```

Я использую образ eon01/stateful-flask:v0, который я отправил в Docker Hub. Вы можете использовать свой собственный образ.

Затем мы создадим развертывание:

```
kubectl apply -f kubernetes/deployment.yaml
```

Мы можем проверить статус развертывания:

```
kubectl get pods -n stateful-flask
```

Наконец, нам нужно перенести базу данных:

```
export pod=$(kubectl get pods -n stateful-flask -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it $pod -n stateful-flask -- flask db init
kubectl exec -it $pod -n stateful-flask -- flask db migrate
kubectl exec -it $pod -n stateful-flask -- flask db upgrade
```

7.4.8. Создание сервиса для нашего приложения

Мы воспользуемся сервисом ClusterIP для организации доступа к приложению в кластер. Для этого нам понадобится следующий файл YAML:

Листинг 7.10 ❖ Файл `kubernetes/stateful-flask-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  selector:
    app: stateful-flask
  ports:
    - name: http
      protocol: TCP
      port: 5000
      targetPort: 5000
```

Применим файл конфигурации:

```
kubectl apply -f kubernetes/stateful-flask-service.yaml
```

7.4.9. Создание внешнего сервиса для нашего приложения

Поскольку контроллер Ingress находится в другом пространстве имен, нам нужно создать внешний сервис, чтобы связать кластер с приложением. Для этого мы будем использовать следующий файл YAML:

Листинг 7.11 ❖ Файл `kubernetes/stateful-flask-service-externalname.yaml`

```
apiVersion: v1
kind: Service
```

```

metadata:
  name: stateful-flask-service-externalname
  namespace: default
spec:
  type: ExternalName
  externalName: stateful-flask.stateful-flask.svc.cluster.local

```

Применим этот файл:

```
kubectl apply -f kubernetes/stateful-flask-service-externalname.yaml
```

7.4.10. Создание Ingress для нашего приложения

Для создания Ingress мы воспользуемся следующим файлом YAML:

Листинг 7.12 ❖ Файл kubernetes/ingress.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
    - host: stateful-flask.<ip>.nip.io
      http:
        paths:
          - path: /tasks
            pathType: Prefix
            backend:
              service:
                name: stateful-flask-service-externalname
                port:
                  number: 5000
  ingressClassName: nginx

```

Обязательно замените <ip> на IP-адрес контроллера Ingress. Примените файл конфигурации:

```
kubectl apply -f kubernetes/ingress.yaml
```

7.5. Просмотр журналов и проверка работоспособности

Для просмотра журналов приложения выполните команду `kubectl logs`. Например, чтобы просмотреть журналы пода `hello-world`, введите команду

```
kubectl logs hello-world
```

Если под находится в другом пространстве имен, можно указать пространство имен с помощью флага `-n`:

```
kubectl logs hello-world -n my-namespace
```

Чтобы применить команду к приложению `stateful-flask`, введите

```
kubectl -n stateful-flask logs <POD_NAME>
```

Вам нужно будет заменить `<POD_NAME>` на имя пода, для которого вы хотите просмотреть журналы. В качестве альтернативы вы можете получить имя пода из развертывания и использовать его в команде:

```
export pod=$(kubectl -n stateful-flask get pods -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')
kubectl -n stateful-flask logs $pod
```

Но если у вас несколько подов и вы хотите увидеть все журналы, добавьте флаг `-l`, чтобы указать селектор метки:

```
kubectl -n stateful-flask logs -l app=stateful-flask
```

Вы также можете использовать флаг `-f`, чтобы следить за журналами:

```
kubectl -n stateful-flask logs -f -l app=stateful-flask
```

Теперь можно перейти по адресу `http://stateful-flask.<ip>.nip.io/tasks`, чтобы увидеть приложение в действии.

Чтобы сделать POST-запрос к приложению и добавить задачу, нам понадобится `curl` (или Postman). Например, чтобы добавить задачу с заголовком `My first task` и описанием `This is my first task`, введите команду

```
curl -X POST -H "Content-Type: application/json" -d '{"title":"My first task",
"description":"This is my first task"}' "http://stateful-flask.<ip>.nip.io/tasks"
```

Обязательно замените `http://stateful-flask.<ip>.nip.io` на URL вашего приложения, который можно найти в выводе команды `kubectl get ingress` или выполнив следующую команду:

```
kubectl get ingress | awk '{print $3}' | tail -n 1
```

Вы также можете объединить обе команды в одну:

```
curl -X POST -H "Content-Type: application/json" -d '{"title":"My first task",
"description":"This is my first task"}' "http://$(kubectl get ingress | awk '{print $3}' | tail -n 1)/tasks"
```

Теперь вы можете сделать запрос GET к приложению, чтобы увидеть добавленные вами задачи:

```
curl "http://$(kubectl get ingress | awk '{print $3}' | tail -n 1)/tasks"
```

Если мы удалим все поды приложения и все поды базы данных, мы не потеряем данные, поскольку они хранятся в `PersistentVolumeClaim`.

Чтобы удалить все поды приложения, выполните команды:

```
kubect! delete -f kubernetes/deployment.yaml
kubect! delete -f kubernetes/postgres-deployment.yaml
```

Повторное развертывание приложения:

```
kubect! apply -f kubernetes/deployment.yaml
kubect! apply -f kubernetes/postgres-deployment.yaml
```

Если мы сделаем запрос GET к приложению после повторного развертывания, мы увидим задачи, которые добавили ранее:

```
curl "http://$(kubect! get ingress | awk '{print $3}' | tail -n 1)/tasks"
```

7.6. Заключение

Даже если приложение Flask не хранит свое состояние внутри (stateless), оно хранит состояние всего приложения в базе данных. Поэтому нам нужно убедиться, что база данных является постоянной.

В этой части книги вы узнали, как развернуть в Kubernetes базу данных с сохранением состояния, используемую микросервисом Flask без сохранения состояния. Вы также увидели, как использовать `PersistentVolumeClaim` для хранения состояния приложения в `PersistentVolume`. Даже если пример кажется простым, это хорошая отправная точка для понимания того, как можно объединить микросервисы с сохранением состояния и без сохранения состояния в Kubernetes.

Глава 8

Развертывание микросервисов с сохранением состояния: StatefulSet

8.1. Что такое StatefulSet?

Проще говоря, StatefulSet – это объект API рабочей нагрузки, который управляет приложениями с сохранением состояния.

При запуске приложений с сохранением состояния в Kubernetes крайне важно обеспечить стабильность, масштабируемость и целостность данных.

Приложение с сохранением состояния должно:

- работать стабильно и надежно;
- масштабироваться вверх и вниз без потери данных;
- обновляться без потери данных;
- откатываться до предыдущей версии без потери данных;
- перезапускаться без потери данных.

В Kubernetes StatefulSet – это объекты API, предназначенные для управления приложениями с сохранением состояния, например базами данных. Они обеспечивают такую функциональность, как масштабируемость и возможность обновления, сохраняя при этом порядок и уникальность подов.

Таким образом, объекты StatefulSet полезны для приложений, которым требуются особые функции, такие как уникальные и стабильные сетевые идентификаторы, постоянное хранилище, которое остается неизменным при перезапуске подов, а также способ развертывания и масштабирования в определенном порядке.

8.2. StatefulSet или развертывание?

В отличие от развертывания, StatefulSet назначает уникальный и постоянный идентификатор каждому поду. Это позволяет поду сохранять свою идентичность даже при перепланировании.

StatefulSet особенно хорошо подходит для рабочих нагрузок, нуждающихся в томах хранения.

Постоянные идентификаторы, назначенные подам, упрощают сопоставление томов с подами, которые заменяют произвольные отказавшие поды, гарантируя, что данные не будут потеряны в случае сбоя.

Если приложению не требуются уникальные и стабильные сетевые идентификаторы и постоянное хранилище, которое не изменяется при перезапуске подов, лучше использовать развертывание.

8.3. Создание StatefulSet

В предыдущих примерах мы создали базу данных PostgreSQL с сохранением состояния с помощью развертывания. В этом примере мы создадим ту же базу данных, используя StatefulSet. Этот способ больше подходит для нашего случая, поскольку мы используем постоянную базу данных PostgreSQL.

Прежде чем продолжить, удалите развертывание и сервис, которые мы создали в предыдущем примере.

```
cd $HOME/stateful-flask
```

```
kubectl delete -f kubernetes/postgres-deployment.yaml
```

```
kubectl delete -f kubernetes/postgres-service.yaml
```

Мы также можем удалить объекты Volume и PersistentVolumeClaim, которые мы создали в предыдущем примере:

```
kubectl delete -f kubernetes/postgres-pvc-pv.yaml
```

Теперь создайте следующий файл манифеста StatefulSet:

Листинг 8.1 ❖ Файл kubernetes/postgres-statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
```

```
template:
  metadata:
    labels:
      app: postgres
  spec:
    containers:
      - name: postgres
        image: postgres:10.1
        imagePullPolicy: Always
        ports:
          - containerPort: 5432
        envFrom:
          - configMapRef:
              name: postgres-config
        volumeMounts:
          - name: postgreddb-volume
            mountPath: /var/lib/postgresql/data
            subPath: postgres
        env:
          - name: PGDATA
            value: /var/lib/postgresql/data/pgdata
    volumeClaimTemplates:
      - metadata:
          name: postgreddb-volume
        spec:
          accessModes: [ "ReadWriteOnce" ]
          storageClassName: "do-block-storage"
          resources:
            requests:
              storage: 5Gi
```

Затем создайте StatefulSet:

```
kubectl apply -f kubernetes/postgres-statefulset.yaml
```

Проверьте StatefulSet:

```
kubectl get statefulset -n postgres
```

Если вы удалите и заново создадите StatefulSet, у подов будут те же самые идентификаторы, что и прежде.

8.4. Создание сервиса для StatefulSet

Объекты StatefulSets нуждаются в сервисе Headless. Напомню, что Kubernetes позволяет клиентам обнаруживать IP-адреса подов с помощью DNS-запросов, и это возможно благодаря сервису Headless. Без этого сервиса IP-адреса подов не могут быть обнаружены напрямую, вместо этого сервер DNS возвращает один IP-адрес, который является адресом самого сервиса. Затем сервис может распределить нагрузку трафика между базовыми подами.

Мы создадим сервис Headless при помощи следующего кода:

Листинг 8.2 ❖ Файл `kubernetes/postgres-headless-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - name: postgres
      port: 5432
```

Применим файл конфигурации для создания сервиса Headless:

```
kubectl apply -f kubernetes/postgres-headless-service.yaml
```

8.5. Задачи после развертывания

У нас есть новый StatefulSet, и теперь нужно заново создать схему базы данных и применить любые миграции SQL.

```
kubectl exec -it <pod> -n stateful-flask -- flask db init
kubectl exec -it <pod> -n stateful-flask -- flask db migrate
kubectl exec -it <pod> -n stateful-flask -- flask db upgrade
```

Замените `<pod>` на имя одного из подов приложения. Вы можете получить имя пода с помощью следующей команды:

```
kubectl get pods -n stateful-flask
```

Можно также объединить две команды:

```
export pod=$(kubectl get pods -n stateful-flask | awk '{print $1}' | tail -n 1)
kubectl exec -it $pod -n stateful-flask -- flask db init
kubectl exec -it $pod -n stateful-flask -- flask db migrate
kubectl exec -it $pod -n stateful-flask -- flask db upgrade
```

Убедитесь, что приложение работает:

```
export url="http://$(kubectl get ingress | awk '{print $3}' | tail -n 1)/tasks"
```

```
# добавляем новое задание
curl -X POST -H "Content-Type: application/json" -d '{"title":"New task",
"description":"New description"}' $url
```

```
# получаем все задания
curl $url
```

Вы также можете удалить, а затем заново создать StatefulSet и проверить, сохраняются ли данные.

```
kubectl delete -f kubernetes/postgres-statefulset.yaml
kubectl apply -f kubernetes/postgres-statefulset.yaml
```

```
curl $url
```

8.6. StatefulSet или развертывание: постоянное хранилище

Ранее мы создали развертывание, которое использовало PersistentVolumeClaim для сохранения данных. Вот как мы это сделали:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pv-claim
  namespace: postgres
  labels:
    app: postgres
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: do-block-storage
```

Затем мы смонтировали PersistentVolumeClaim в файле манифеста развертывания:

```
apiVersion: apps/v1
kind: Deployment
[...]
spec:
  [...]
  template:
    [...]
    spec:
      containers:
        - name: postgres
          image: postgres:10.1
          [...]
      volumes:
        - name: postgreddb
```

```

persistentVolumeClaim:
  claimName: postgres-pv-claim

```

Развертывание будет работать нормально, но это не лучший способ сохранения данных. После масштабирования развертывания новые поды могут не иметь доступа к тому же PersistentVolumeClaim. Это означает, что новые поды не будут иметь доступа к тем же данным. Если вы посмотрите на конфигурацию PersistentVolumeClaim, то заметите, что мы использовали строки

```

accessModes:
- ReadWriteOnce

```

Это означает, что PersistentVolumeClaim может быть смонтирован только на чтение–запись одним узлом. Поэтому поды, созданные на новых узлах, не будут иметь доступа к тому же PersistentVolumeClaim.

Что, если мы изменим accessModes на ReadWriteMany?

```

accessModes:
- ReadWriteMany

```

В этом случае развертывание все равно не сможет сохранять данные должным образом. Поскольку ReadWriteMany позволяет работать на нескольких узлах, наше приложение должно иметь возможность обрабатывать одновременное чтение–запись одного и того же файла несколькими узлами, но это не так.

Чтобы избежать этих проблем и поскольку мы не хотим управлять параллелизмом данных на уровне приложения, мы будем использовать VolumeClaimTemplates. Вот как мы это сделали в файле манифеста StatefulSet:

```

volumeClaimTemplates:
- metadata:
  name: postgresdb-volume
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "do-block-storage"
    resources:
      requests:
        storage: 5Gi

```

VolumeClaimTemplates будет динамически запрашивать PersistentVolumeClaim из StorageClass. Если у вас есть *x* подов, StatefulSet создаст *x* заявок PersistentVolumeClaims, каждую с именем в следующем формате:

```
<volumeClaimTemplate-name>-<pod-name>-<ordinal-index>
```

Когда мы удаляем StatefulSet, тома, связанные с ним, не будут удалены. Под, который использовал PersistentVolumeClaim, будет повторно использовать тот же том при повторном создании StatefulSet.

Это делается для обеспечения безопасности и целостности данных.

8.7. StatefulSet или развертывание: связанный сервис

В самом первом примере развертывания PostgreSQL мы создали ClusterIP. Это связано с необходимостью создать сервис для развертывания, доступный только из кластера. Вот как мы это сделали:

```
apiVersion: v1
kind: Service
metadata:
  name: postgres # Задаем имя сервиса
  namespace: postgres
  labels:
    app: postgres # Метки и селекторы
spec:
  type: NodePort # Задаем тип сервиса
  ports:
    - port: 5432 # Задаем порт для выполнения приложения Postgres
  selector:
    app: postgres
```

Однако в файле манифеста StatefulSet мы не только задаем доступ к поду PostgreSQL с помощью сервиса. Мы также хотим иметь возможность масштабировать количество подов без влияния на целостность данных, например выполнять одновременное чтение–запись одного и того же файла.

Чтобы решить эту задачу, мы использовали StatefulSet и VolumeClaimTemplates и таким образом позволили всем подам совместно использовать каталог данных `/var/lib/postgresql/data`. Однако доступ к подам через сервис оставался проблематичным. Обычная реализация сервиса в Kubernetes не работает с PostgreSQL, поскольку он использует балансировщик нагрузки или прокси для направления трафика в под. Вместо этого каждый под должен быть обнаружен непосредственно клиентом. Вот почему мы использовали сервис Headless.

Наличие сервиса Headless гарантирует, что приложение с отслеживанием состояния функционирует должным образом, обеспечивая стабильную сетевую идентичность для кластера базы данных. Даже если произойдет что-то непредвиденное, например отказ пода и предоставление нового с другим IP-адресом, Headless гарантирует, что новый под будет иметь прежнюю сетевую идентичность.

Глава 9

Паттерны микросервисов: внешние конфигурации

9.1. Хранение конфигураций в среде

Одним из основных преимуществ микросервисов является возможность масштабирования и независимого управления каждым сервисом.

Однако управление конфигурациями для нескольких сервисов и нескольких сред может быть сложной задачей. Например, два или более сервиса могут совместно использовать одни и те же конфигурации, и если мы решим изменить конфигурацию, это изменение отразится на всех связанных сервисах.

Другая проблема возникает из-за того, что изменение жестко закодированных конфигураций требует повторного развертывания кода, что включает в себя различные дополнительные шаги, и это может занять много времени, если изменения происходят часто.

Реализация паттерна внешней конфигурации решает эту проблему путем хранения всех данных конфигурации приложения за пределами кодовой базы. В соответствии с принципами 12 факторов¹ конфигурации приложений хранятся в среде.

Этот подход не зависит от языка и ОС и плавно масштабируется по мере расширения приложения на большее количество развертываний. Благодаря выносу конфигурации во внешнюю среду микросервисы могут работать в нескольких средах без модификации или перекомпиляции, что упрощает независимое управление и масштабирование каждого сервиса.

¹ <https://12factor.net/config>.

9.2. Секреты и переменные среды Kubernetes: зачем они нужны?

В предыдущем примере у нас был такой код:

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://stateful-flask-user:stateful-flask-
password@postgres.postgres.svc.cluster.local:5432/stateful-flask-db'
..
..
..
```

У этого кода есть три недостатка:

- пароль базы данных находится в коде, что является нарушением безопасности;
- всякий раз, когда мы хотим изменить переменную, например имя базы данных, нам нужно изменить ее в нескольких местах в нашем коде;
- нам нужно изменить код и повторно развернуть приложение.

Эти проблемы можно решить, используя *переменные среды* и *секреты*.

9.3. Секреты и переменные среды Kubernetes: как это работает?

Давайте начнем с изменения кода и заставим его использовать переменные среды в строке подключения.

Перейдите в каталог приложения:

```
cd $HOME/stateful-flask
```

Не забудьте установить необходимые зависимости:

```
workon stateful-flask

pip install Flask==2.2.3
pip install Flask-SQLAlchemy==3.0.3
pip install psycopg2-binary==2.9.6
pip install Flask-Migrate==4.0.4
```

«Заморозьте» зависимости:

```
pip freeze > requirements.txt
```

Затем измените код, чтобы использовать переменные среды:

Листинг 9.1 ❖ Файл app/app.py

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
import os

app = Flask(__name__)
DB_USER = os.environ.get('DB_USER')
DB_PASSWORD = os.environ.get('DB_PASSWORD')
DB_HOST = os.environ.get('DB_HOST')
DB_NAME = os.environ.get('DB_NAME')
DB_PORT = os.environ.get('DB_PORT')
app.config['SQLALCHEMY_DATABASE_URI'] = "postgresql://{user}:{password}@{host}/{db}".format(DB_USER, DB_
PASSWORD, DB_HOST, DB_PORT, DB_NAME)
db = SQLAlchemy(app)
migrate = Migrate(app, db)

class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(80), nullable=False)
    description = db.Column(db.String(200))

@app.route('/tasks', methods=['GET'])
def get_tasks():
    tasks = Task.query.all()
    return jsonify({'tasks': [{'id': task.id, 'title': task.title, 'description': task.
description} for task in tasks]})

@app.route('/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    title = data['title']
    description = data['description']
    task = Task(title=title, description=description)
    db.session.add(task)
    db.session.commit()
    return jsonify({'task': {'id': task.id, 'title': task.title, 'description': task.
description}})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Следующая строка, которую мы обновили в нашем приложении Flask, устанавливает URI для подключения к базе данных SQLAlchemy.

```
app.config['SQLALCHEMY_DATABASE_URI'] = "postgresql://{user}:{password}@{host}/{db}".format(DB_USER, DB_
PASSWORD, DB_HOST, DB_PORT, DB_NAME)
```

Здесь мы применяем метод форматирования строк для создания URI с использованием переменных, определенных ранее в коде (DB_USER, DB_PASSWORD, DB_HOST, DB_PORT и DB_NAME). Результирующий URI имеет формат:

```
postgresql://<user>:<password>@<host>:<port>/<database>
```

где каждая переменная заменяется соответствующим значением.

Скомпонуйте и выгрузите приложение, изменив <DOCKERHUB_USERNAME> на свое имя пользователя Docker Hub:

```
export DOCKERHUB_USERNAME=<DOCKERHUB_USERNAME>
docker build -t stateful-flask:v0 -f app/Dockerfile app
docker tag stateful-flask:v0 $DOCKERHUB_USERNAME/stateful-flask:v0
docker push $DOCKERHUB_USERNAME/stateful-flask:v0
```

Теперь создадим так называемый секрет (secret) для пользователя и пароля базы данных.

Секреты Kubernetes – это объекты, которые позволяют хранить и управлять конфиденциальной информацией, такой как пароли, токены и ключи. Они предоставляют безопасный способ хранения конфиденциальных данных вместо жесткого кодирования в вашем приложении или файлах конфигурации.

Мы будем использовать того же пользователя и пароль, что и раньше.

При создании секрета в Kubernetes нам нужно закодировать данные с помощью base64. Это можно сделать с помощью команды echo и канала | base64.

```
echo -n 'stateful-flask-user' | base64
echo -n 'stateful-flask-password' | base64
```

Результат должен быть таким:

```
c3RhZGVmdWwtZmxhc2stdXNlcg==
c3RhZGVmdWwtZmxhc2stcGFzc3dvcmQ=
```

Секреты Kubernetes хранятся в закодированном формате из соображений безопасности. Это связано с тем, что секреты могут содержать неотображаемые символы, такие как перевод строки или специальные символы и т. д. Команда echo -n, за которой следует команда | base64, кодирует данные в формате base64 без неотображаемых символов.

Однако важно отметить, что конфиденциальные данные не шифруются. Они **только кодируются** в base64, что является обратимой процедурой. Для шифрования данных опишем ресурс секрета:

Листинг 9.2 ❖ Файл kubernetes/stateful-flask-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: stateful-flask-secret
  namespace: stateful-flask
type: Opaque
data:
  DB_USER: c3RhZGVmdWwtZmxhc2stdXNlcg==
  DB_PASSWORD: c3RhZGVmdWwtZmxhc2stcGFzc3dvcmQ=
```

Затем создайте объект секрета:

```
kubectl apply -f kubernetes/stateful-flask-secret.yaml
```

Чтобы этот секрет начал действовать, нам нужно обновить манифест разворачивания и указать использование секрета в качестве переменных среды:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: stateful-flask-secret
                  key: DB_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: stateful-flask-secret
                  key: DB_PASSWORD
```

- в этом коде мы определили переменные среды для нового разворачивания. В частности, создаются две переменные среды – DB_USER и DB_PASSWORD;
- вместо того чтобы напрямую задавать их значения, поле valueFrom используется для извлечения значений из секрета Kubernetes, с именем stateful-flask-secret;
- поле secretKeyRef указывает, что значение переменной среды должно быть извлечено из ключа в секрете stateful-flask-secret, где DB_USER и DB_PASSWORD являются ключами для соответствующих значений.

Теперь добавим другие переменные среды в манифест разворачивания. Следующие переменные будут использоваться для подключения к базе данных: DB_HOST, DB_NAME и DB_PORT.

Листинг 9.3 ❖ Файл `kubernetes/deployment.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: stateful-flask-secret
                  key: DB_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: stateful-flask-secret
                  key: DB_PASSWORD
            - name: DB_HOST
              value: postgres.postgres.svc.cluster.local
            - name: DB_NAME
              value: stateful-flask-db
            - name: DB_PORT
              value: "5432"

```

Эти переменные не так конфиденциальны, как пользователь и пароль базы данных, поэтому мы можем жестко закодировать их в манифесте развертывания. Их также можно хранить как секреты, но сейчас мы этого делать не будем.

По умолчанию секреты Kubernetes являются обязательными, и если они не будут найдены, под не запустится. Однако мы можем сделать секрет необязательным, установив полю `optional` значение `true`:

```

env:
- name: DB_USER
  valueFrom:

```

```
secretKeyRef:  
  name: stateful-flask-secret  
  key: DB_USER  
optional: true
```

Это не относится к переменным `DB_USER` и `DB_PASSWORD`, поскольку они необходимы для подключения к базе данных, и приложение не будет работать без них. Вот почему в нашем примере полю `optional` не задано значение `true`.

Теперь создадим развертывание:

```
kubectl apply -f kubernetes/deployment.yaml
```

Вы также можете просмотреть секрет и развертывание с помощью команды `kubectl get`:

```
kubectl get secret,deployment -n stateful-flask
```

Теперь вы можете получить доступ к приложению, используя IP-адрес Ingress:

```
curl http://$(kubectl get ingress | awk '{print $3}' | tail -n 1)/tasks
```

Глава 10

Важные приемы работы с микросервисами: проверки состояния

10.1. Проверки состояния приложения

Одним из лучших приемов разработки микросервисов является реализация проверок работоспособности.

Проверки состояния приложения (health check) – это способ мониторинга работоспособности микросервиса. Они выполняются инфраструктурой для определения состояния микросервиса. Если микросервис неработоспособен, он отключается от сети и больше не получает запросы.



Принцип проверки исправности

Устойчивые системы используют проверки состояния для обнаружения и удаления неработоспособных микросервисов и замены их новыми.



Проверки состояния также нужны инфраструктуре для определения готовности микросервиса к приему запросов. Это важно при запуске микросервиса. Инфраструктура не будет отправлять запросы микросервису, пока он не будет готов их принимать. Вот почему существуют различные типы проверок состояния и приемы, которые следует учитывать при их реализации, например предоставление микросервису времени для запуска, прежде чем считать его неработоспособным, или отключение микросервиса после определенного количества неудачных проверок работоспособности и т. д.

К счастью, Kubernetes предоставляет способ реализации проверок состояния. Проверки Kubernetes реализуются с помощью *зондов* (probe).

10.2. Зонды жизнеспособности и готовности

Существует два типа проверок состояния Kubernetes: на *жизнеспособность* (liveness) и на *готовность* (readiness).

Зонд жизнеспособности используется для определения того, правильно ли работает контейнер в поде. Kubernetes периодически запускает зонд жизнеспособности, и если он сработал успешно, контейнер считается «живым». Если зонд не сработал, контейнер считается «мертвым», и Kubernetes перезапускает его.

Зонд готовности используется для определения того, готов ли контейнер в поде получать запросы. Как и в первом случае, Kubernetes периодически выполняет проверку готовности, и если она успешна, контейнер считается готовым. Если проверка не пройдена, контейнер считается еще не готовым.

10.3. Типы зондов

Существует три типа зондов: HTTP, TCP и Exec.

- HTTP-зонды применяются для определения активности (готовности) контейнера путем отправки HTTP-запроса на определенную конечную точку. Если конечная точка возвращает любой код, больший или равный 200 и меньший 400, проверка завершается успешно. Если конечная точка возвращает любой другой код, проверка считается неудачной;
- TCP-зонды применяются для определения активности (готовности) контейнера путем попытки установить TCP-соединение с определенным портом. Если соединение установлено, проверка завершается успешно. Если соединение не удастся, проверка считается неудачной;
- Exec-зонды применяются для определения активности (готовности) контейнера путем выполнения команды внутри контейнера. Если команда возвращает нулевой код, проверка завершается успешно. Если команда возвращает ненулевой код выхода, проверка считается неудачной.

Следующая таблица кратко резюмирует различия трех типов зондов:

Тип зонда	Описание
HTTP	Отправляет HTTP-запрос на указанную конечную точку
TCP	Открывает TCP-соединение с указанным портом
Exec	Выполняет указанную команду внутри контейнера

10.4. Реализация зондов

Зонды реализуют в спецификации пода.

Вот пример спецификации пода с HTTP-зондами жизнеспособности и готовности:

Листинг 10.1 ❖ Файл http-probe

```
kubectl apply -f
```

```
# создаем временное пространство имен для выполнения теста
apiVersion: v1
kind: Namespace
metadata:
  name: healthcheck

# создаем развертывание с зондами жизнеспособности и готовности
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateless-flask
spec:
```

```

replicas: 1
selector:
  matchLabels:
    app: stateless-flask
template:
  metadata:
    labels:
      app: stateless-flask
spec:
  containers:
  - name: stateless-flask
    image: eon01/stateless-flask
    ports:
    - containerPort: 5000
    livenessProbe:
      httpGet:
        path: /tasks
        port: 5000
      initialDelaySeconds: 5
      periodSeconds: 5
      timeoutSeconds: 3
      successThreshold: 1
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /tasks
        port: 5000
      initialDelaySeconds: 10
      periodSeconds: 15
      timeoutSeconds: 10
      successThreshold: 1
      failureThreshold: 15
  # terminationGracePeriodSeconds: 10

```

Проверки жизнеспособности и готовности настраиваются с помощью полей `livenessProbe` и `readinessProbe`, и обе используют HTTP-зонды:

- в поле `initialDelaySeconds` указывают количество секунд ожидания перед выполнением первой проверки. Значение по умолчанию – 0 с;
- в поле `periodSeconds` указывают количество секунд ожидания между проверками. Значение по умолчанию – 10 с;
- в поле `timeoutSeconds` указывают количество секунд ожидания выполнения зонда, перед тем как считать проверку неудачной. Значение по умолчанию – 1 с;
- в поле `successThreshold` указывают количество последовательных успешных проверок, перед тем как считать контейнер «живым» или готовым. Значение по умолчанию – 1;
- в поле `failureThreshold` указывают количество последовательных неудачных проверок, перед тем как считать контейнер «мертвым» или неготовым. Значение по умолчанию – 3;

- в поле `terminateGracePeriodSeconds` указывают количество секунд ожидания перед завершением работы контейнера после получения им сигнала SIGTERM. Значение по умолчанию – 30 с. Его можно задать на уровне пода или на уровне зонда контейнера. Значение, заданное на уровне зонда, переопределяет значение, заданное на уровне пода.

Назначение полей кратко резюмировано в следующей таблице.

Поле	Назначение
<code>initialDelaySeconds</code>	Количество секунд ожидания перед выполнением первой проверки
<code>periodSeconds</code>	Количество секунд ожидания между каждой проверкой
<code>timeoutSeconds</code>	Количество секунд ожидания успешного выполнения проверки, перед тем как считать ее неудачной
<code>successThreshold</code>	Количество последовательных успешных проверок, перед тем как считать контейнер «живым» или готовым
<code>failureThreshold</code>	Количество последовательных неудачных проверок, перед тем как считать контейнер мертвым или неготовым
<code>terminationGracePeriodSeconds</code>	Количество секунд ожидания перед завершением работы контейнера после получения им сигнала SIGTERM

Другие типы проверок настраиваются аналогично. Вот пример спецификации пода с зондами TCP для проверки жизнеспособности и готовности:

```
[..]
  livenessProbe:
    tcpSocket:
      port: 5000
  readinessProbe:
    tcpSocket:
      port: 5000
[..]
```

Эта проверка подходит для микросервисов, которые не предоставляют конечную точку HTTP, но прослушивают определенный порт.

Вот пример спецификации пода с пробами Exec для проверки жизнеспособности и готовности:

```
[..]
  livenessProbe:
    exec:
      command:
        - cat
        - /tmp/healthy
  readinessProbe:
    exec:
      command:
        - cat
[..]
```

```
- /tmp/healthy  
[..]
```

Эта проверка подходит для микросервисов, которые нуждаются в более сложных проверках. Например, микросервис следует считать неготовым, пока не будет готова используемая им база данных. В этом случае скрипт можно использовать для проверки готовности базы данных и возврата нулевого кода выхода, если она готова, и ненулевого кода выхода, если это не так.

Пример:

```
#!/bin/bash  
  
# ждем готовность базы данных  
while ! mysqladmin ping -h mysql -u root -p$MYSQL_ROOT_PASSWORD --silent; do  
    sleep 1  
done  
  
# возвращаем нулевой код, если база данных готова  
exit 0
```

Затем этот скрипт можно использовать в проверке готовности:

```
[..]  
  readinessProbe:  
    exec:  
      command:  
      - /bin/bash  
      - /tmp/wait-for-db.sh  
[..]
```

Глава 11

Стратегии управления ресурсами микросервисов

11.1. Управление ресурсами и риски: от Docker до Kubernetes

Если вы использовали автономные контейнеры Docker, то, вероятно, уже знаете о рисках, связанных с неправильным распределением памяти и ресурсов ЦП. Если нет, вы можете быть неприятно удивлены, когда ваши контейнеры выйдут из строя из-за нехватки памяти или ЦП.

Контейнеры, работающие на хосте, не должны потреблять слишком много памяти, так как ядро ОС может активировать исключение Out Of Memory (OOM) и начать завершать процессы, включая сам демон Docker. Docker пытается предотвратить это, регулируя приоритет OOM на демоне, что снижает вероятность его завершения. Но это не относится к процессам контейнеров, которые с большей вероятностью будут завершены.

Поэтому рекомендуется ограничивать доступ контейнера к ресурсам хоста. Вот несколько примеров:

```
# один раз в секунду проверяет, что нагрузка на ЦП не превышает 50 %  
docker run -it --cpus=".5" ubuntu /bin/bash
```

```
# ограничивает максимальный объем памяти для контейнера значением 256 Мбайт  
docker run -it --memory=256m ubuntu /bin/bash
```

Если узел, на котором запущен под, имеет достаточно доступных ресурсов, контейнеру возможно (и разрешено) использовать больше ресурсов, чем указано в его *запросе* на этот ресурс. Однако контейнеру не разрешено использовать больше своего *лимита* ресурсов.

11.2. Запросы и лимиты

Если узел, на котором запущен под, имеет достаточно доступных ресурсов, контейнер может использовать больше ресурсов, чем указано в его запросе. Однако контейнеру не разрешено использовать больше своего лимита ресурсов.

Планировщик Kubernetes учитывает запросы и лимиты при определении того, на какой узел поместить под. Он пытается уместить общее количество запросов ресурсов для всех запланированных контейнеров на доступные узлы. Например, если у пода запрос ЦП 100m и запрос памяти 200 МиБ, планировщик не разместит его на узле с менее чем 100m и 200 МиБ памяти.

Ниже приведен пример того, как мы можем задать запросы ресурсов и ограничения для контейнера в поде. Перейдите в рабочий каталог:

```
cd $HOME/stateful-flask
```

Приготовьте следующий файл манифеста:

Листинг 11.1 ❖ Файл `kubernetes/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
      imagePullPolicy: Always
      ports:
        - containerPort: 5000
      env:
        - name: DB_USER
```

```
valueFrom:
  secretKeyRef:
    name: stateful-flask-secret
    key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_PASSWORD
- name: DB_HOST
  value: postgres.postgres.svc.cluster.local
- name: DB_NAME
  value: stateful-flask-db
- name: DB_PORT
  value: "5432"
```

Примените развертывание:

```
kubectl apply -f kubernetes/deployment.yaml
```

В приведенном выше примере мы устанавливаем следующие запросы ресурсов и ограничения:

- запрос ЦП 250m и лимит ЦП 500m;
- запрос памяти 64 МиБ и лимит памяти 128 МиБ.

11.3. Единицы измерения ресурсов ЦП

Ресурсы процессора измеряются в единицах ЦП (CPU). Один условный ЦП в Kubernetes соответствует:

- 1 AWS vCPU;
- GCP Core;
- 1 Azure vCore;
- 1 гипертрединговое ядро на физическом процессоре Intel bare metal с гипертредингом.

Допускаются дробные значения ресурса. Буква m обозначает дробную приставку «милли-». Например, контейнеру с запросом ЦП 100m гарантировано 100 mCPU на каждом узле, на котором он запланирован, а контейнеру с запросом 1 ЦП будет выделен как минимум один полный ЦП на каждом узле, на котором он запланирован.

В файлах конфигурации допускается использовать 0.1 вместо 100m или 0.01 вместо 10m. Можно также писать 1000m вместо 1.

Пример:

```
resources:
  requests:
    cpu: "250m"
```

ЧТО ЭКВИВАЛЕНТНО

```
resources:
  requests:
    cpu: "0.25"
```

Оба варианта означают, что контейнеру гарантировано 250m на каждом узле, на котором он запланирован. Другими словами, выделенный ЦП составляет 25 % от единичного ЦП. При наличии нескольких запросов ЦП планировщик суммирует их, чтобы получить общий запрос ЦП для пода.

В нашем предыдущем примере мы установили ЦП на 250m. Это означает, что контейнеру гарантировано 250 mCPU на каждом узле, на котором он запланирован.

Мы можем установить более высокие запросы и лимиты, например:

```
resources:
  requests:
    cpu: "1.5"
  limits:
    cpu: "3"
```

Это означает, что контейнеру гарантировано 1.5 ЦП на каждом узле, на котором он запланирован, и он может использовать не более 3 ЦП на каждом таком узле. Если у нас есть 2 ЦП на каждом узле, контейнер сможет использовать до 150 % от общего ЦП на каждом узле, а если у нас есть 4 ЦП на каждом узле, контейнер сможет использовать до 75 % от общего ЦП на каждом узле.

Теперь, если вы запросите больше, чем доступно на узле, планировщик не разместит под на этом узле. Например, если у вас есть узел с 2 ЦП, а вы запрашиваете 3, планировщик не разместит под на этом узле. Если у вас есть другой узел с 4 ЦП, планировщик, вероятно, разместит под на нем. Это связано с тем, что Kubelet и другие компоненты Kubernetes, работающие на узле, сами нуждаются в ресурсах ЦП.

Когда ни один узел не доступен, под будет находиться в состоянии ожидания. Вот пример запроса 10 ядер ЦП:

Листинг 11.2 ❖ Файл `kubernetes/deployment-10cpu.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
```

```
  app: stateful-flask
spec:
  containers:
  - name: stateful-flask
    image: eon01/stateful-flask:v0
    resources:
      requests:
        cpu: "10"
      limits:
        cpu: "20"
    imagePullPolicy: Always
  ports:
  - containerPort: 5000
  env:
  - name: DB_USER
    valueFrom:
      secretKeyRef:
        name: stateful-flask-secret
        key: DB_USER
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: stateful-flask-secret
        key: DB_PASSWORD
  - name: DB_HOST
    value: postgres.postgres.svc.cluster.local
  - name: DB_NAME
    value: stateful-flask-db
  - name: DB_PORT
    value: "5432"
```

Если вы примените манифест с помощью команды `kubectl apply -f kubernetes/deployment-10cpu.yaml`, вы увидите, что под будет в состоянии ожидания.

11.4. Единицы измерения ресурсов памяти

Ресурсы памяти измеряются в байтах.

i Байт – это единица размера памяти, и он равен 8 битам. Бит – это единица данных, и он равен либо 0, либо 1. Байт – это наименьшая *адресуемая* единица памяти.

Вы можете указать объем памяти, используя один из следующих суффиксов: E, P, T, G, M, K, где:

- E = эксабайт (1 000 000 000 000 000 000 байт);
- P = петабайт (1 000 000 000 000 000 байт);
- T = терабайт (1 000 000 000 000 байт);

- G = гигабайт (1 000 000 000 байт);
- M = мегабайт (1 000 000 байт);
- K = килобайт (1000 байт).

Можно также применять следующие суффиксы: Ei, Pi, Ti, Gi, Mi, Ki, где:

- Ei = эксбибайт (1 152 921 504 606 846 976 байт);
- Pi = пебибайт (1 125 899 906 842 624 байта);
- Ti = тебибайт (1 099 511 627 776 байт);
- Gi = гибибайт (1 073 741 824 байта);
- Mi = мебибайт (1 048 576 байт);
- Ki = кибибайт (1024 байта).

Для обозначения тысячной доли (милли) применяется буква m. Например, 100m означает 100 милли, что равно $100/1000 = 0.1$. Таким образом, запись 100m эквивалентна 0.1 байта.

Давайте, например, создадим под с контейнером, запрашивающим 64Mi памяти, и лимитом в 128Mi:

Листинг 11.3 ❖ Файл kubernetes/deployment-64mi.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          resources:
            requests:
              memory: "64Mi"
            limits:
              memory: "128Mi"
          imagePullPolicy: Always
      ports:
        - containerPort: 5000
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: stateful-flask-secret
              key: DB_USER
```

```

- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_PASSWORD
- name: DB_HOST
  value: postgres.postgres.svc.cluster.local
- name: DB_NAME
  value: stateful-flask-db
- name: DB_PORT
  value: "5432"

```

11.5. Принципы настройки запросов и ограничений ресурсов

Безотносительно к Kubernetes, важно понимать, что память и ЦП ведут себя по-разному, когда речь идет об управлении ресурсами. Рассмотрим следующий пример:

- ЦП (вычислительная мощность) похожа на время, которое обнуляется каждый цикл. Если вы его не используете, вы его теряете. Его также называют *сжимаемым ресурсом* (compressible resource). Сжимаемость ресурса означает, что если его использование достигает предела, любой процесс, которому он нужен, сначала *должен будет дождаться*, пока ЦП станет доступным;
- память похожа на пространство тем, что является нециклическим и *несжимаемым* ресурсом. Это означает, что если использование памяти достигает предела, любой процесс, которому он требуется, просто не сможет работать.

Когда контейнер запрашивает определенный объем памяти, это не означает, что контейнер будет использовать ее всю. Этот объем памяти будет гарантированно востребован только при запуске нескольких контейнеров в системе. Контейнер может использовать меньше этого объема.

Другими словами, среда выполнения контейнера использует requests в качестве подсказки для установки `memory.min` и `memory.low` контейнера с помощью `cgroups v2`¹.

i cgroup (control group), или группа управления, – это функция ядра Linux, которая ограничивает, учитывает и изолирует использование ресурсов (ЦП, памяти, дискового ввода-вывода, сети и т. д.) набора процессов.

Если контейнер превышает свой запрос памяти, под, к которому он принадлежит, *может быть* вытеснен, если на узле закончится память.

¹ <https://docs.kernel.org/admin-guide/cgroup-v2.html>.

Если контейнер превышает свой лимит памяти, контейнер *будет* завершен.

Однако контейнер может превышать свой лимит ЦП в течение определенного времени без вытеснения. Это временное увеличение использования ЦП называется *скачком ЦП* (CPU burst).

i Скачок ЦП – это период времени, когда контейнеру разрешено использовать больше ЦП, чем позволено лимитом. Длительность скачка ЦП ограничена только разумным периодом времени.

Если вы не зададите лимит ЦП для контейнера, может возникнуть одна из двух ситуаций:

- 1) контейнер не будет иметь ограничений на количество ресурсов ЦП, которые он может использовать, и *потенциально* сможет потратить все доступные ресурсы ЦП на узле, где он запущен;
- 2) контейнер унаследует ограничение ЦП по умолчанию, если пространство имен, в котором он запущен, имеет один набор. Это значение по умолчанию может быть установлено администраторами кластера с помощью ресурса Kubernetes, называемого LimitRange.

i LimitRange – это ресурс Kubernetes, который определяет значения по умолчанию для ограничений ЦП и памяти для пространства имен.

Вот пример настройки LimitRange для пространства имен stateful-flask:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: stateful-flask-limit-range
  namespace: stateful-flask
spec:
  limits:
    - default:
        memory: 512Mi
        cpu: 500m
      defaultRequest:
        memory: 256Mi
        cpu: 250m
      type: Container
```

11.6. Резервированные и выделяемые ресурсы узла

В кластере Kubernetes каждый узел имеет ресурсы, зарезервированные для операционной системы и компонентов Kubernetes. Эти ресурсы называются *резервированными ресурсами узла*. Остальные ресурсы называются *выделяемыми*. Резервированные ресурсы предназначены для таких компонентов, как:

- операционная система;
- Kubelet;
- Kube-proxy;
- среда выполнения контейнера;
- агенты регистрации и мониторинга на уровне узла;
- сетевые плагины на уровне узла.

Эти ресурсы не должны использоваться подами. Поды имеют свои собственные выделяемые ресурсы.



- i** kube-reserved: эти ресурсы зарезервированы для системных демонов Kubernetes, таких как kubelet или контейнерная среда выполнения, для обеспечения их доступности и стабильности. Они не предназначены для выделения ресурсов системным демонам, которые запускаются как поды. Объем резервированных ресурсов обычно зависит от плотности подов на узлах.
- i** system-reserved: эти ресурсы зарезервированы для системных демонов ОС, таких как sshd или udev.
- i** eviction-threshold: чтобы избежать системных инцидентов OOM (out of memory), Kubelet обеспечивает управление нехваткой ресурсов, резервируя часть памяти. Kubelet пытается вытеснить поды, когда доступная память на узле падает ниже зарезервированного значения. Ресурсы, зарезервированные для вытеснения, недоступны для подов.

С учетом этих определений мы можем сделать вывод, что выделяемые ресурсы – это общие ресурсы за вычетом резервных ресурсов узла. Распределение ресурсов для контейнеров выполняется только в объеме выделяемых ресурсов.

11.7. Классы качества обслуживания (QoS)

Kubernetes использует понятие класса *качества обслуживания* (Quality of Service, QoS) для указания характеристик пода. Класс пода определяется запросами ресурсов и ограничениями его контейнеров.

Вот три класса QoS, которые поддерживает Kubernetes:

- Guaranteed (гарантированный);
- Burstable (пиковый);
- BestEffort (наилучший возможный).

11.7.1. Guaranteed

Чтобы под соответствовал классу Guaranteed, все его контейнеры должны отвечать следующим требованиям:

- установлены лимит и запрос памяти;
- лимит памяти равен запросу;
- установлены лимит и запрос ЦП;
- лимит ЦП равен запросу.

Пример:

```
apiVersion: v1
kind: Pod
metadata:
  name: guaranteed-pod
spec:
  containers:
  - name: guaranteed-container
    image: eon01/stateful-flask:v0
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "64Mi"
        cpu: "250m"
```

Guaranteed – самый строгий класс QoS. Когда под имеет статус Guaranteed, это означает, что среда выполнения контейнера гарантирует, что под никогда не превысит свои запросы ресурсов; если это произойдет, под будет остановлен.

11.7.2. Burstable

Чтобы под соответствовал классу Burstable, все его контейнеры должны отвечать следующим требованиям:

- под не относится к классу Guaranteed. Это означает, что запросы отличаются от лимитов для ЦП и памяти во всех контейнерах;
- по крайней мере один контейнер имеет запрос или лимит памяти/ЦП.

Вот пример:

```
apiVersion: v1
kind: Pod
metadata:
  name: guaranteed-pod
spec:
  containers:
  - name: guaranteed-container
    image: eon01/stateful-flask:v0
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Другой пример:

```
apiVersion: v1
kind: Pod
metadata:
  name: guaranteed-pod
spec:
  containers:
  - name: guaranteed-container
    image: eon01/stateful-flask:v0
    resources:
      requests:
        memory: "64Mi"
```

Или

```
apiVersion: v1
kind: Pod
metadata:
  name: burstable-pod
spec:
  containers:
  - name: burstable-container
    image: eon01/stateful-flask:v0
    resources:
      requests:
        cpu: 100m
      limits:
        cpu: 500m
```

Когда поду назначен класс `Burstable`, это означает, что у него есть некоторые минимальные гарантии ресурсов на основе запроса, но не заданы конкретные ограничения. Ограничением считается емкость узла, что позволяет поду использовать больше ресурсов, если они доступны.

Поды класса **Burstable** вытесняются только после того, как в случае нехватки ресурсов с узла вытеснены все поды класса **BestEffort**. Поскольку под класса **Burstable** может иметь контейнер без ограничений или запросов на ресурсы, он может попытаться использовать любое количество ресурсов узла.

11.7.3. BestEffort

Чтобы под соответствовал классу **BestEffort**, все его контейнеры должны иметь следующие характеристики:

- под не имеет никаких запросов или лимитов памяти/ЦП.

Класс **QoS BestEffort** имеет самый низкий приоритет в **Kubernetes**. Поды в этом классе могут использовать любые ресурсы, которые специально не назначены другим классам **QoS**.

Например, если узел имеет 16 ядер ЦП, а поду в классе **Guaranteed** назначено четыре из этих ядер, то любой под в классе **BestEffort** может использовать оставшиеся 12 ядер ЦП.

Однако если узел начинает испытывать нехватку ресурсов, **Kubelet** предпочтет сначала вытеснить поды класса **BestEffort**, поскольку они считаются наименее приоритетными. Это означает, что поды в этом классе имеют наименьшую гарантию ресурсов и могут быть первыми претендентами на «выселение», если на узле не хватает ресурсов.

Пример:

```
apiVersion: v1
kind: Pod
metadata:
  name: besteffort-pod
spec:
  containers:
  - name: besteffort-container
    image: eon01/stateful-flask:v0
```

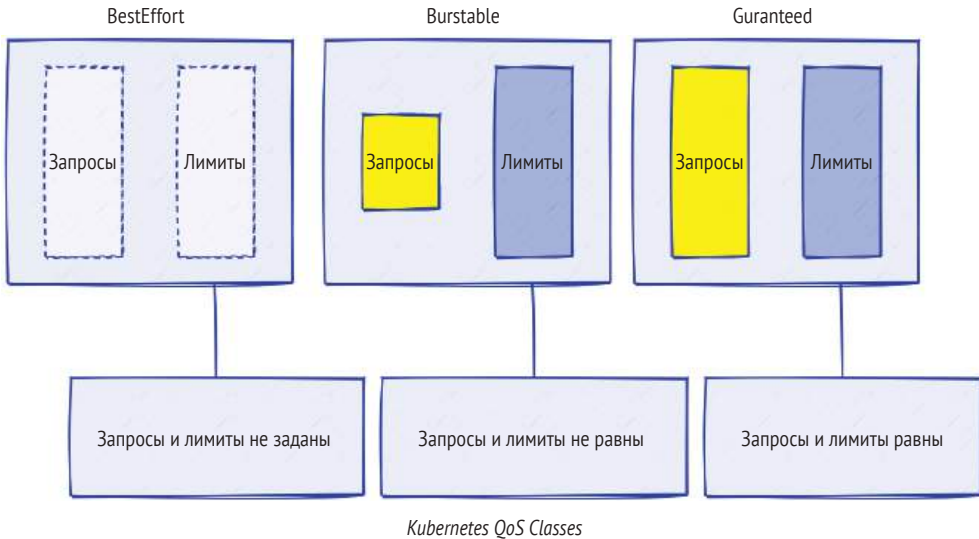
11.7.4. Проверка класса пода по QoS

Чтобы проверить класс пода по **QoS**, можно использовать следующую команду:

```
kubectl get pods <pod-name> -o jsonpath="{.status.qosClass}"
```

Результат будет одним из следующих:

- **Guaranteed**;
- **Burstable**;
- **BestEffort**.

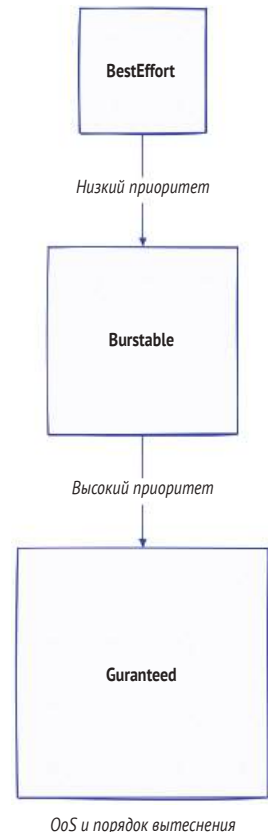


11.7.5. Порядок вытеснения

Как было сказано ранее, существуют три класса QoS. Когда узел испытывает нехватку ресурсов, Kubelet вытесняет поды в следующем порядке:

- 1) BestEffort;
- 2) Burstable;
- 3) Guaranteed.

Важно понимать порядок вытеснения, чтобы избежать непредвиденного поведения. Например, если у вас есть под с классом QoS Guaranteed и под с классом QoS BestEffort, а узел испытывает нехватку ресурсов, под с классом QoS BestEffort будет вытеснен первым.



11.7.6. Пользовательский класс PriorityClass

PriorityClass – это ресурс, который можно использовать для указания приоритета пода. Он определяет порядок вытеснения подов, когда узел испытывает нехватку ресурсов.

PriorityClass – это ресурс вне пространства имен в группе API `scheduling.k8s.io`. Вот пример:

```

apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
description: " Этот класс приоритета используется только для критических подов, таких как
наши агенты мониторинга "
preemptionPolicy: PreemptLowerPriority

```

В приведенном выше примере создается PriorityClass с именем high-priority со следующими характеристиками:

- value: значение приоритета PriorityClass. Это значение должно быть целым числом от -2 147 483 648 до 1 000 000 000;
- globalDefault: если установлено значение true, этот PriorityClass будет использоваться по умолчанию для всех подов, у которых не задан PriorityClassName. У вас может быть только один PriorityClass с этим полем, установленным в true;
- description: описание PriorityClass;
- preemptionPolicy: политика вытеснения PriorityClass. Может быть задано значение PreemptLowerPriority или Never. Используется для определения того, может ли под с PriorityClass вытеснять другие поды с более низким приоритетом. Эта конфигурация доступна с версии Kubernetes 1.24.

Следующий пример – класс с более низким приоритетом:

```

apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 1000
globalDefault: false
description: "Этот класс приоритета используется только для некритических подов, таких как
наши внутренние инструменты."
preemptionPolicy: Never

```

Чтобы применить PriorityClass к поду, добавьте поле priorityClassName в спецификацию пода:

```

apiVersion: v1
kind: Pod
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  priorityClassName: high-priority
  containers:
  - name: stateful-flask

```

```
image: eon01/stateful-flask:v0
```

```
resources:
```

```
  requests:
```

```
    memory: "64Mi"
```

```
    cpu: "250m"
```

```
  limits:
```

```
    memory: "128Mi"
```

```
    cpu: "500m"
```

Глава 12

Автоматическое масштабирование микросервисов в Kubernetes: введение

12.1. Лучшие методы масштабирования микросервисов

При проектировании микросервисов масштабируемость должна быть ключевым фактором. Для соблюдения этого условия необходимо знать и уметь применять современные методы. Наиболее важные из них мы рассмотрим в следующих разделах.

12.1.1. Использование реестра для обнаружения сервисов

Реестр сервисов – это центральный каталог, который позволяет сервисам обнаруживать и взаимодействовать друг с другом. Он обеспечивает эффективную связь между микросервисами, даже если количество сервисов растет.

Другими словами, добавление нового микросервиса в систему не должно требовать никаких изменений в существующих микросервисах. Реестр сервисов должен обладать способностью автоматически обрабатывать новый микросервис.

В Kubernetes эту функциональность реализует DNS кластера. Когда создается сервис или под, Kubernetes создает для него записи DNS.

Пример: при создании сервиса LoadBalancer с именем `service-name` в пространстве имен `namespace-name` все поды в этом сервисе будут доступны по имени DNS `service-name.namespace-name.svc.cluster.local`. При масштабировании развертывания запись DNS всегда будет указывать на текущий набор подов, независимо от узла, на котором они запущены. Это относится ко всем подам в именованном сервисе.

12.1.2. Реализация проверок работоспособности

Проверки работоспособности используются для определения состояния микросервиса. Они указывают, готов ли микросервис получать запросы или он неработоспособен и его необходимо перезапустить.

При масштабировании микросервиса обычно требуется некоторое время для запуска. В течение этого времени микросервис еще не готов получать запросы. Поэтому при масштабировании новые экземпляры не следует автоматически считать работоспособными. Их следует считать *неработоспособными*, пока они не будут готовы получать запросы.

В Kubernetes это можно сделать с помощью *зондирования готовности* (readiness probe). Зондирование готовности – это проверка работоспособности, которая определяет, готов ли микросервис получать запросы. Если она не пройдена, под не будет получать трафик от сервиса.

Еще одна полезная проверка называется *зондированием работоспособности* (liveness probe). Зондирование работоспособности определяет, продолжает ли микросервис исправно работать. Если проверка не пройдена, под будет перезапущен.

Мы обсудим эти проверки более подробно в следующих разделах.

12.1.3. Проектирование с учетом масштабируемости и другие передовые практики

Как уже было сказано, масштабируемость микросервисов должна быть ключевым фактором. Для соблюдения этого условия требуется реализовать набор передовых практик, таких как:

- *приоритет сервисов без сохранения состояния*: по сравнению с сервисами, сохраняющими состояние, их легче масштабировать, обеспечивая при этом высокую доступность. Это означает, что ваши микросервисы не должны хранить состояние в памяти. Вместо этого они должны хранить его в базе данных или другом внешнем хранилище;
- *идемпотентные сервисы*: свойство идемпотентности необходимо для масштабируемости. Это означает, что ваши микросервисы должны

иметь возможность обрабатывать один и тот же запрос несколько раз, не страдая от побочных эффектов;

- *приоритет асинхронной связи перед синхронной*. Это означает, что ваши микросервисы не должны ждать ответа от другого сервиса перед продолжением работы. Вместо этого они должны отправлять ему сообщение и продолжать обработку;
- *кеширование*: сервисы с кешированием работают лучше в динамической среде автоматического масштабирования. Поскольку сервисы без сохранения состояния не хранят состояние в памяти, им необходимо извлекать его из базы данных или другого внешнего хранилища. Это может быть медленно и затратно. Поэтому рекомендуется кешировать часто запрашиваемые данные в памяти для повышения производительности и снижения затрат;
- *балансировка нагрузки*: сервисы должны быть сбалансированы по нагрузке;
- *отказоустойчивость*: для масштабируемости требуется отказоустойчивость. Это означает, что ваши микросервисы должны уметь корректно обрабатывать сбои. Например, если микросервис выходит из строя, это не должно влиять на другие микросервисы. Вместо этого он должен автоматически перезапускаться;
- *наблюдаемость*: создавайте наблюдаемые сервисы. Это означает, что ваши микросервисы должны предоставлять метрики и журналы. Это поможет вам выявить узкие места и другие проблемы.

Существуют различные методики достижения оптимальной масштабируемости микросервисов. К счастью, создание микросервисов для Kubernetes естественным образом заставляет нас создавать рабочие нагрузки, совместимые с Kubernetes и готовые к Kubernetes, т. е. уже с учетом некоторых передовых методик.

Мы обсудим их в следующих разделах.

Глава 13

Горизонтальное автомасштабирование микросервисов в Kubernetes

13.1. Что такое горизонтальное масштабирование

На этом этапе у нас есть такой файл YAML для API:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v1
```

```
resources:
  requests:
    memory: "64Mi"
    cpu: "250m"
  limits:
    memory: "128Mi"
    cpu: "500m"
imagePullPolicy: Never
ports:
- containerPort: 5000
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_PASSWORD
- name: DB_HOST
  value: postgres.postgres.svc.cluster.local
- name: DB_NAME
  value: stateful-flask-db
- name: DB_PORT
  value: "5432"
```

И следующий StatefulSet для базы данных:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:10.1
          imagePullPolicy: Always
          ports:
            - containerPort: 5432
          envFrom:
            - configMapRef:
```

```
      name: postgres-config
    volumeMounts:
      - mountPath: /var/lib/postgresql/data
        subPath: postgres
        name: postgresdb-volume
    volumeClaimTemplates:
      - metadata:
          name: postgresdb-volume
        spec:
          accessModes: [ "ReadWriteOnce" ]
          storageClassName: "do-block-storage"
          resources:
            requests:
              storage: 5Gi
```

Если вы получите список всех подов во всех пространствах имен, вы увидите, что у нас есть один экземпляр API и два экземпляра базы данных:

```
kubectl get pods -A
#или kubectl get pods --all-namespaces
```

Мы уже масштабировали базу данных горизонтально. В определении StatefulSet видно, что у нас есть две реплики.

```
spec:
  replicas: 2
```

Для горизонтального масштабирования API нам нужно изменить определение развертывания. Например, чтобы иметь три реплики, используйте следующую конфигурацию:

```
spec:
  replicas: 3
```

В следующем листинге представлен полный код YAML:

Листинг 13.1 ❖ Файл kubernetes/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 3
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
```

```

containers:
- name: stateful-flask
  image: eon01/stateful-flask:v1
  resources:
    requests:
      memory: "64Mi"
      cpu: "250m"
    limits:
      memory: "128Mi"
      cpu: "500m"
  imagePullPolicy: Never
  ports:
  - containerPort: 5000
  env:
  - name: DB_USER
    valueFrom:
      secretKeyRef:
        name: stateful-flask-secret
        key: DB_USER
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: stateful-flask-secret
        key: DB_PASSWORD
  - name: DB_HOST
    value: postgres.postgres.svc.cluster.local
  - name: DB_NAME
    value: stateful-flask-db
  - name: DB_PORT
    value: "5432"

```

Примените конфигурацию и посмотрите изменения:

```

kubectl apply -f kubernetes/deployment.yaml
kubectl get pods -n stateful-flask -w

```

ReplicaSet – это объект, который будет создавать поды. Вы можете увидеть ReplicaSet, выполнив следующую команду:

```

kubectl get rs -n stateful-flask
# или kubectl get replicaset -n stateful-flask

```



ReplicaSet создается с определенными атрибутами. К ним относятся селектор, описывающий критерии для управляемых подов, количество реплик, указывающее, сколько подов должно поддерживаться, и шаблон пода, определяющий конфигурацию новых подов для соответствия требуемому количеству реплик.

ReplicaSet отвечает за создание и удаление подов по мере необходимости для достижения желаемого их числа с учетом текущего состояния системы.

С помощью командной строки также можно масштабировать развертывание:

```

kubectl scale --replicas=3 deployment/stateful-flask -n stateful-flask

```

13.2. Горизонтальное автомасштабирование подов

Kubernetes предлагает автоматическое масштабирование как функцию, позволяющую ресурсу рабочей нагрузки, такому как StatefulSet или развёртывание, подстраиваться под изменяющийся спрос. Функция *горизонтального автомасштабирования подов* (horizontal pod autoscaling, HPA) масштабирует рабочую нагрузку вверх или вниз на основе метрик, отслеживаемых контроллером HPA.

HPA добавляет больше подов к рабочей нагрузке для реагирования на возросшую нагрузку.

Для использования HPA в кластере должен быть запущен сервер метрик. Этот сервер отвечает за агрегацию данных об использовании ресурсов в кластере. Сервер метрик собирает метрики из различных ресурсов и предоставляет их через сервер Metrics API Kubernetes. Эти метрики сначала использует функция HPA, а затем контроллер *вертикального автомасштабирования подов* (vertical pod autoscaler, VPA).

Существуют различные способы установки сервера метрик:

- 1) непосредственно из YAML:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

- 2) с помощью Helm:

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update
helm install metrics-server metrics-server/metrics-server
```

- 3) с помощью DigitalOcean Marketplace¹. Если вы используете другого провайдера облачных услуг, вы можете использовать вариант, предоставленный вашим провайдером, если он доступен.

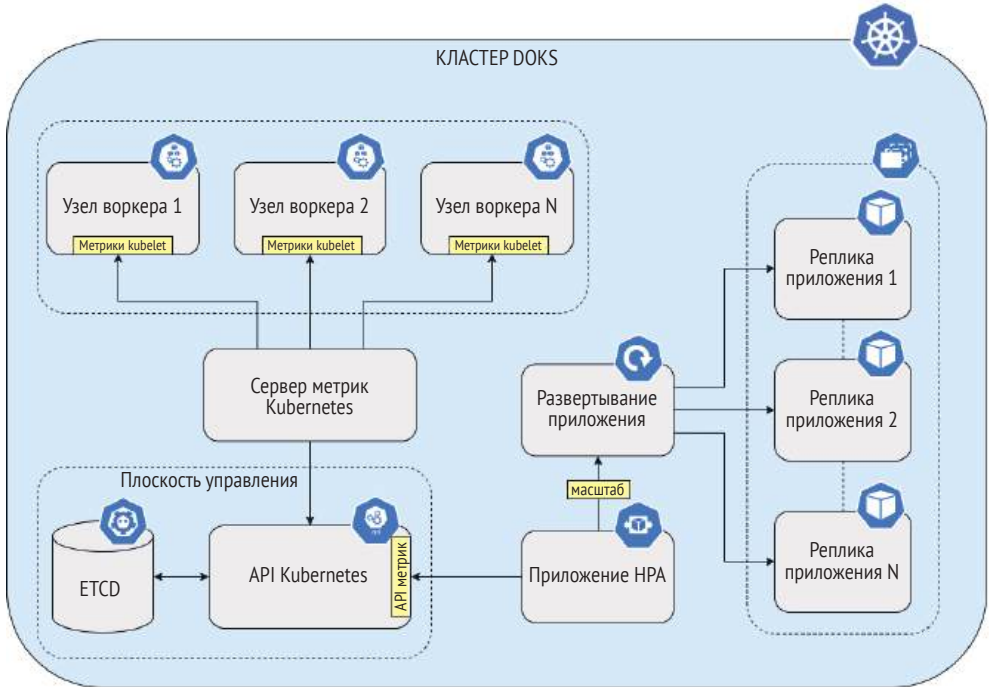
В нашем примере мы рассмотрим первый вариант. Выполните команду

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

На следующей диаграмме из официальной документации DigitalOcean² показано, как HPA работает в сочетании с metrics-server в кластере DigitalOcean Kubernetes:

¹ <https://marketplace.digitalocean.com/apps/kubernetes-metrics-server>.

² <https://marketplace.digitalocean.com/apps/kubernetes-metrics-server>.



Пояснение:

- разверните Metrics Server в вашем кластере DOKS;
- Metrics Server собирает метрики kubelet с каждого рабочего узла, получая данные об использовании ЦП и памяти для каждой рабочей нагрузки приложения;
- Metrics Server предоставляет метрики использования ЦП и памяти через сервер API Kubernetes;
- HPA извлекает метрики использования ЦП и памяти через сервер API Kubernetes. Затем, на основе наблюдения за метриками и целевого порогового значения, он решает, когда следует масштабировать поды развертывания приложения.

Источник: https://raw.githubusercontent.com/digitalocean/marketplace-kubernetes/master/stacks/metrics-server/assets/images/arch_hpa.png

Чтобы проверить, запущен ли сервер метрик, выполните следующую команду:

```
kubectl top nodes
```

Далее мы создадим HPA для API. HPA будет масштабировать API на основе использования ЦП. Если использование ЦП превысит 20 %, HPA добавит больше подов. Если загрузка ЦП упадет ниже 20 %, HPA удалит поды.

Вот пример:

Листинг 13.2 ❖ Файл `kubernetes/hpa.yaml`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa
  namespace: stateful-flask
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
```

Примените манифест НРА:

```
kubectl apply -f kubernetes/hpa.yaml
```

Вы можете увидеть НРА, выполнив следующую команду:

```
kubectl get hpa -n stateful-flask
# или kubectl get horizontalpodautoscalers -n stateful-flask
```

Давайте нагрузим API, чтобы увидеть НРА в действии. Выполните следующую команду:

```
kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/
sh -c "while sleep 0.01; do wget -q -O- http://stateful-flask.stateful-flask.svc.cluster.
local:5000/tasks; done"
```

Эта команда создает под, который нагружает API, вызывая увеличение использования ЦП. Откройте другой терминал и выполните следующую команду, чтобы посмотреть, как функционирует НРА:

```
kubectl get hpa -n stateful-flask -w
```

НРА добавляет больше подов, когда использование ЦП превышает 20 % от запрошенного объема ЦП. И наоборот, НРА удаляет поды, когда использование ЦП падает ниже 20 % от запрошенного объема ЦП. Поэтому важно указывать запросы для ваших подов при использовании НРА.

По умолчанию метрики проверяются каждые 15 с.

Мы также можем масштабировать поды на основе анализа использования памяти. Вот пример:

Листинг 13.3 ❖ Файл kubernetes/hpa-2.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
```

```

metadata:
  name: stateful-flask-hpa-memory
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 20

```

Или на основе анализа использования ЦП и памяти:

Листинг 13.4 ❖ Файл kubernetes/hpa-3.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-cpu-memory
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 20

```

13.3. Автоматическое масштабирование на основе пользовательских метрик Kubernetes

HPA также может принимать решение о масштабировании на основе пользовательских метрик. Вот пример:

Листинг 13.5 ❖ Файл `kubernetes/hpa-4.yaml`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-custom
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
```

В приведенном выше примере решение о масштабировании принимается на основе использования ЦП. Мы указываем HPA начать масштабирование, когда использование ЦП превышает 20 % от запроса. Вместо показателя `Utilization` мы можем использовать `AverageValue` или `Value`.

- `AverageValue` рассчитывается путем деления значения, возвращаемого сервером метрик, на количество подов;
- `Value` – это необработанное значение, возвращаемое сервером метрик.

Вот пример с `AverageValue`:

Листинг 13.6 ❖ Файл `kubernetes/hpa-5.yaml`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-custom
  namespace: stateful-flask
spec:
  scaleTargetRef:
```

```

    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
    minReplicas: 1
    maxReplicas: 10
    metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: AverageValue
          averageValue: 45m
    
```

Когда полю `target.type` задано `AverageValue` или `Value`, нам нужно использовать значения (например, `45m`), в отличие от предыдущего примера, где мы используем процент (20). Kubernetes поддерживает следующие типы метрик по умолчанию:

- `cpu`: использование ЦП подами в целевом масштабе;
- `memory`: использование памяти подами в целевом масштабе.

Обе они называются метриками ресурсов.

Kubernetes также поддерживает другие типы метрик, не связанные с ресурсами:

- `Object`: количество объектов в хранилище объектов Kubernetes, сообщаемое адаптером метрик;
- `Pods`: количество подов в целевом масштабе;
- `External`: метрика от внешнего провайдера метрик.

13.4. Специализированные пользовательские метрики Kubernetes

Мы также можем принимать решение о масштабировании на основе пользовательских типов метрик Kubernetes, таких как поды и объекты. Вот пример метрики на основе пода:

```

    apiVersion: autoscaling/v2
    kind: HorizontalPodAutoscaler
    metadata:
      name: stateful-flask-hpa-custom
      namespace: stateful-flask
    spec:
      scaleTargetRef:
        apiVersion: apps/v1
        kind: Deployment
        name: stateful-flask
    minReplicas: 1
    
```

```
maxReplicas: 10
metrics:
- type: Pods
  pods:
    metric:
      name: custom_metric
    target:
      type: AverageValue
      averageValue: 10k
```

Эти метрики описывают поды, как следует из их названия. Они всегда усредняются по всем подам и сравниваются с целевым значением для определения количества подов после масштабирования.

Они похожи на метрики ресурсов, за исключением того, что поддерживают только целевой тип `AverageValue`.

Вот пример метрики объекта:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-custom
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Object
    object:
      metric:
        name: custom_metric
      describedObject:
        apiVersion: networking.k8s.io/v1
        kind: Ingress
        name: my-ingress
      target:
        type: Value
        value: 10M
```

В отличие от описания подов, эти метрики описывают другие типы объектов в том же пространстве имен. Метрики объектов поддерживают два типа целей: `Value` и `AverageValue`.

При использовании `Value` целевое значение сравнивается напрямую с метрикой, возвращаемой API. В свою очередь, при использовании `AverageValue` значение, возвращаемое API пользовательских метрик, делится на количество подов, а затем сравнивается с целевым значением.

Все пользовательские метрики должны предоставляться внешним поставщиком метрик, который не является частью Kubernetes. Поставщик метрик отвечает за сбор значений метрик и предоставление их Kubernetes.

Нам доступно множество поставщиков, включая Prometheus, Datadog и New Relic.

Чтобы использовать Prometheus¹ в качестве примера, вам сначала нужно установить и настроить его для извлечения метрики `custom_metric` из вашего приложения или другого ресурса в вашем кластере. Затем установите Prometheus Adapter². Этот адаптер может заменить сервер метрик, который вы использовали ранее, поскольку он реализует API пользовательских, ресурсных и внешних метрик.

Нужно иметь в виду, что не каждый поставщик метрик поддерживает все типы метрик, и не все поставщики метрик могут заменить сервер метрик. Однако можно сохранить сервер метрик и использовать других поставщиков метрик одновременно.

13.5. Использование нескольких метрик

Мы можем использовать несколько метрик в одном HPA. Вот пример:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-custom
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
  - type: Pods
  pods:
    metric:
```

¹ <https://prometheus.io/>.

² <https://github.com/kubernetes-sigs/prometheus-adapter>.

```
name: custom_metric
target:
  type: AverageValue
  averageValue: 10k
```

В этом случае HPA будет рассматривать каждую метрику по очереди и выполнять масштабирование на основе метрики, которая требует больше всего реплик. Например, если загрузка ЦП составляет 20 %, а среднее значение `custom_metric` составляет 5k, HPA будет масштабировать, исходя из загрузки ЦП.

13.6. Автоматическое масштабирование на основе пользовательских метрик, отличных от Kubernetes

В предыдущих разделах мы рассмотрели масштабирование на основе стандартных метрик Kubernetes, таких как ЦП и память, а также пользовательских метрик Kubernetes, таких как поды и объекты. Все эти метрики предоставляются сервером метрик (Metric Server), причем метрики ресурсов предоставляются Kubernetes (в частности, Kubelet), а метрики подов и объектов предоставляются внешним поставщиком метрик, таким как Prometheus.

Однако также возможно масштабирование на основе метрик, никак не связанных с Kubernetes, таких как метрики из внешнего приложения или сервиса. Далее мы рассмотрим этот тип масштабирования.

Масштабирование на основе пользовательских метрик, не относящихся к Kubernetes, таких как `custom_metric`, показано в следующем примере:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateful-flask-hpa-custom
  namespace: stateful-flask
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateful-flask
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: External
    external:
      metric:
        name: custom_metric
```

```

selector:
  matchLabels:
    app: external-app
target:
  type: AverageValue
  averageValue: 30

```

В приведенном выше примере мы настраиваем внешнюю метрику с именем `custom_metric`, которая послужит основой для масштабирования. НРА будет использовать среднее значение (`type: AverageValue`) этой метрики, чтобы определить, в какую сторону нужно масштабировать – вверх или вниз. В частности, контроллер масштабирования будет сравнивать желаемое среднее значение (30 в данном случае) с текущим средним значением собранных метрик. Если текущее среднее значение больше желаемого среднего значения, НРА будет масштабировать ресурсы в сторону увеличения. В противном случае он будет масштабировать их в сторону уменьшения.

Обратите внимание, что раздел метрики должен быть определен с помощью селектора меток для идентификации соответствующей метрики. В данном случае это будет `app: external-app`. Если поставщик метрик возвращает несколько временных рядов, НРА отфильтрует их с помощью селектора меток. Затем НРА будет использовать среднее значение оставшихся временных рядов, чтобы определить направление масштабирования.

Доступные типы метрик для внешних метрик – `Value` и `AverageValue`.

13.7. Автомасштабирование кластера

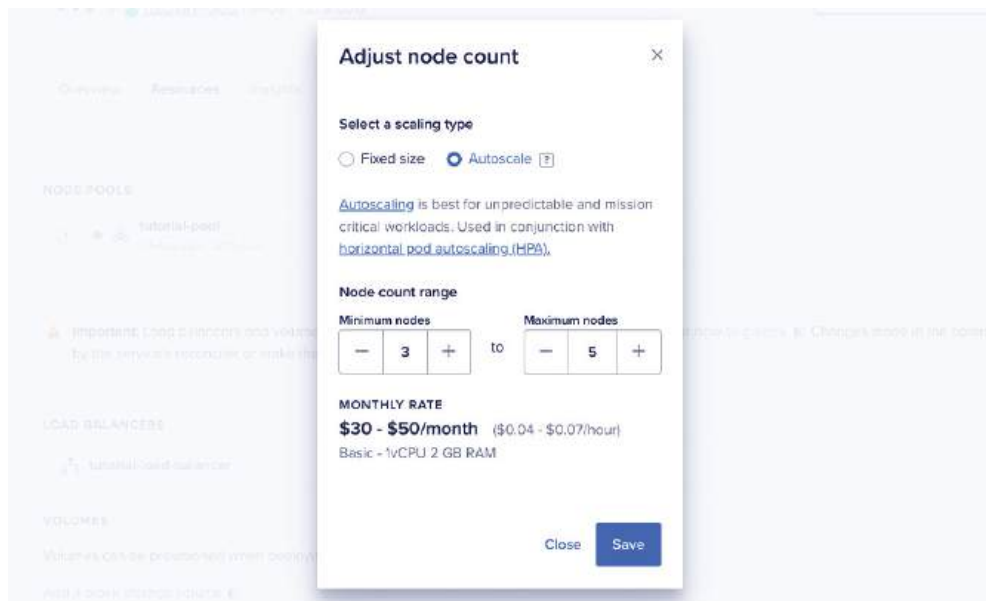
Контроллер *автомасштабирования кластера* (`cluster autoscaler`, CA) – это инструмент, который автоматически регулирует размер кластера Kubernetes в ответ на следующие ситуации:

- поды не запускаются в кластере из-за нехватки ресурсов;
- узлы в кластере остаются недоиспользованными в течение длительного периода времени, и их поды могут быть размещены на других существующих узлах.

CA – это отдельный компонент, не имеющий отношения к НРА, но его можно рассматривать как средство горизонтального автомасштабирования для самого кластера. Кроме того, он не является частью плоскости управления Kubernetes – это развертывание в пространстве имен `kube-system`.

Как правило, провайдеры облачных услуг имеют собственную реализацию CA, и в большинстве случаев вам следует использовать именно ее, поскольку она оптимизирована для инфраструктуры провайдера облачных услуг, ее проще установить (обычно из CLI или веб-интерфейса) и настроить.

В примерах этой книги нашим провайдером является DigitalOcean, поэтому мы можем использовать для установки и настройки CA веб-интерфейс



или CLI:

```
doctl kubernetes cluster node-pool update mycluster mypool --auto-scale --min-nodes 1
--max-nodes 10
```

где

- mycluster – имя кластера;
- mypool – имя пула узлов;
- --auto-scale включает СА;
- --min-nodes – минимальное количество узлов, до которого СА будет масштабировать;
- --max-nodes – максимальное количество узлов, до которого СА будет масштабировать.

Вы можете отключить его с помощью команды

```
doctl kubernetes cluster node-pool update mycluster mypool --auto-scale=false
```

По следующим ссылкам вы найдете официальные документы для СА поддерживаемых облачных провайдеров:

- GCE: <https://kubernetes.io/docs/concepts/cluster-administration/cluster-management/>;
- GKE: <https://cloud.google.com/container-engine/docs/cluster-autoscaler>;
- AWS: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/aws/README.md>;
- Azure: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/azure/README.md>;
- AliCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/alibabacloud/README.md>;

- BaiduCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/baiducloud/README.md>;
- BizflyCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/bizflycloud/README.md>;
- Brightbox: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/brightbox/README.md>;
- CherryServers: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/cherryservers/README.md>;
- Civo: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/civo/README.md>;
- CloudStack: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/cloudstack/README.md>;
- ClusterAPI: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/clusterapi/README.md>;
- DigitalOcean: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/digitalocean/README.md>;
- Exoscale: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/exoscale/README.md>;
- Equinix Metal: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/packet/README.md>;
- External gRPC: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/externalgrpc/README.md>;
- Hetzner: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/hetzner/README.md>;
- HuaweiCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/huaweicloud/README.md>;
- IonosCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/ionoscloud/README.md>;
- Kamatera: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/kamatera/README.md>;
- Linode: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/linode/README.md>;
- Magnum: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/magnum/README.md>;
- OracleCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/oci/README.md>;
- OVHcloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/ovhcloud/README.md>;
- Rancher: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/rancher/README.md>;
- Scaleway: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/scaleway/README.md>;
- TencentCloud: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/tencentcloud/README.md>;
- Vultr: <https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/vultr/README.md>.

Глава 14

Вертикальное автомасштабирование микросервисов в Kubernetes

14.1. Вертикальное масштабирование – в чем отличие

Вертикальное масштабирование отличается от горизонтального, которое подразумевает добавление большего количества подов для обработки возросшего спроса. Вертикальное масштабирование полезно для рабочих нагрузок, требующих больше ресурсов на под, таких как приложения с интенсивным использованием ЦП или памяти.

Другими словами, вертикальное масштабирование в Kubernetes означает увеличение ресурсов, выделяемых отдельному поду, таких как ЦП или память, для удовлетворения возросшего спроса.

Самый простой (но не самый практичный) способ сделать это – изменить лимиты и запросы пода в его спецификации.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
```

```

selector:
  matchLabels:
    app: stateful-flask
template:
  metadata:
    labels:
      app: stateful-flask
  spec:
    containers:
      - name: stateful-flask
        image: eon01/stateful-flask:v0
        resources:
          requests:
            memory: "64Mi"
            cpu: "250m"
          limits:
            memory: "128Mi"
            cpu: "500m"

```

В приведенном выше примере показано, как установить лимиты и запросы ресурсов для подов. Когда у нас высокий спрос, мы можем отредактировать спецификацию пода, чтобы увеличить эти ресурсы:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          resources:
            requests:
              memory: "128Mi"
              cpu: "500m"
            limits:
              memory: "256Mi"
              cpu: "1000m"

```

В этом примере мы удвоили лимиты и запросы ресурсов для пода. Когда под обновляется, Kubernetes завершает работу старого пода и создает новый с новыми значениями лимитов и запросов.

В Kubernetes запросы и лимиты устанавливаются на уровне контейнера в поде, а не на уровне пода или реплики. Когда под создается, каждый контейнер в нем может иметь собственные запросы и лимиты ЦП и памяти.

Как только эти значения изменяются, планировщик Kubernetes определяет, на какие узлы запланировать поды и сколько ресурсов следует выделить подам на каждом узле.

14.2. Вертикальное автомасштабирование пода

Описанную выше ручную операцию также можно автоматизировать с помощью *вертикального автомасштабирования подов* (vertical pod autoscaler, VPA). Этот инструмент активно разрабатывается группой SIG-Autoscaling¹, ответственной за горизонтальное автомасштабирование подов в Kubernetes.

Существуют разные способы установки VPA.

1. С помощью репозитория GitHub:

```
# Клонирование репозитория
git clone https://github.com/kubernetes/autoscaler.git
# Смена каталога
cd autoscaler/vertical-pod-autoscaler
# Установка VPA
./hack/vpa-up.sh
```

2. Средствами вашего облачного провайдера. Например, если вы используете GKE, вы можете установить VPA с помощью следующей команды:

```
gcloud container clusters update CLUSTER_NAME --enable-vertical-pod-autoscaling
```

Если вы используете DigitalOcean, установите VPA с помощью следующей команды:

```
doctl kubernetes cluster node-pool update CLUSTER_NAME NODE_POOL_NAME --auto-scale --min-nodes 1 --max-nodes 10
```

То же самое можно сделать с помощью веб-интерфейса вашего облачного провайдера. Обычно эта опция доступна при создании кластера или сразу после его создания.

В этой книге мы воспользуемся первым методом установки VPA:

```
# Клонирование репозитория
git clone
# Смена каталога
cd autoscaler/vertical-pod-autoscaler
```

¹ <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>.

```
# Установка VPA
./hack/vpa-up.sh
```

VPA устанавливается в пространстве имен kube-system. Проверьте статус VPA с помощью следующей команды:

```
kubectl get pods -n kube-system | grep vpa
```

VPA состоит из трех компонентов:

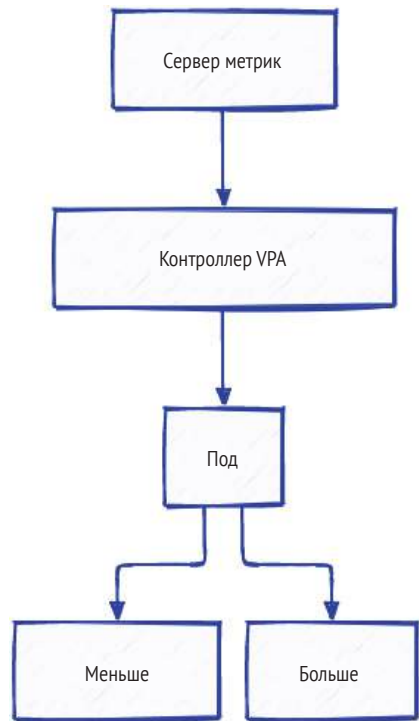
- vpa-admission-controller;
- vpa-recommender;
- vpa-updater.

Первый компонент – это двоичный файл, который регистрирует себя как Mutating Admission Webhook¹. Он перехватывает все события создания пода. Каждый раз, когда создается новый под, этот компонент получает запрос от сервера API. Затем он проверяет, доступна ли соответствующая конфигурация VPA. Если соответствующей конфигурации нет, он не предпринимает никаких действий. Однако если есть соответствующая конфигурация, он использует текущую рекомендацию для настройки запросов ресурсов в поде.

VPA Recommender является вторым компонентом и отвечает за генерацию рекомендаций для VPA Admission Controller. Он периодически запрашивает API метрик, чтобы получить текущую информацию об использовании ресурсов пода. Используя эти данные, VPA Recommender генерирует рекомендации для VPA Admission Controller.

Третий компонент – VPA Updater, который обновляет запросы и ограничения пода на основе рекомендаций от VPA Admission Controller. VPA Updater периодически опрашивает сервер Kubernetes API для проверки изменений в конфигурации VPA или статусе пода и применяет любые необходимые обновления к ресурсам пода. Это гарантирует, что ресурсы пода всегда настроены надлежащим образом на основе текущего спроса и использования приложения.

После установки VPA система может рекомендовать и устанавливать запросы



Kubernetes Vertical Autiscalling

¹ <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>.

ресурсов для подов. Чтобы включить автоматическое вычисление требований к ресурсам, нам нужно добавить ресурс VPA к каждому разворачиванию.

Вот пример того, как это делается:

```
cd $HOME/stateful-flask

cat <<EOF > kubernetes/vpa.yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: stateful-flask-vpa
  namespace: stateful-flask
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: stateful-flask
  updatePolicy:
    updateMode: "Auto"
```

Применим это разворачивание при создании двух реплик:

Листинг 14.1 ❖ Файл kubernetes/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 2
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v1
          resources:
            requests:
              memory: "1Mi"
              cpu: "1m"
            limits:
              memory: "128Mi"
              cpu: "100m"
          imagePullPolicy: Always
      ports:
        - containerPort: 5000
      env:
```

```

- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: stateful-flask-secret
      key: DB_PASSWORD
- name: DB_HOST
  value: postgres.postgres.svc.cluster.local
- name: DB_NAME
  value: stateful-flask-db
- name: DB_PORT
  value: "5432"

```

Теперь применим развертывание:

```
kubectl apply -f kubernetes/deployment.yaml
```

Затем тщательно проверим запросы и ограничения пода:

```

# получаем перечень имен подов
POD1=$(kubectl -n stateful-flask get pod -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')
POD2=$(kubectl -n stateful-flask get pod -l app=stateful-flask -o jsonpath='{.items[1].metadata.name}')

# получаем перечень ресурсов подов
kubectl -n stateful-flask get pod $POD1 -o=jsonpath='{.spec.containers[0].resources}'; echo
kubectl -n stateful-flask get pod $POD2 -o=jsonpath='{.spec.containers[0].resources}'; echo

```

Вы должны увидеть следующий вывод для обоих подов:

```
{"limits":{"cpu":"100m","memory":"128Mi"},"requests":{"cpu":"1m","memory":"1Mi"}}
```

Теперь применим VPA:

```
kubectl apply -f kubernetes/vpa.yaml
```

Проверим статус VPA:

```
kubectl get vpa -n stateful-flask
```

Чтобы увидеть VPA в действии, нам нужно сгенерировать некоторую нагрузку на приложение. Это можно сделать, выполнив следующую команду:

```

# Удалить под генератора нагрузки, если он уже существует
kubectl delete pod load-generator
# Создать новый под генератора нагрузки

kubectl run -i -tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c
"while sleep 0.01; do wget -q -O- http://stateful-flask.stateful-flask.svc.cluster.local:5000/tasks; done"

```

Через несколько минут повторно проверьте ресурсы подов:

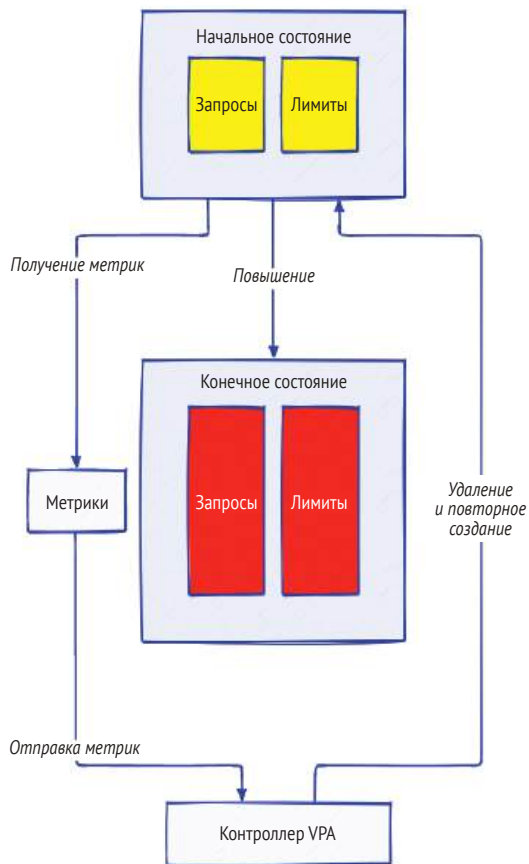
```
# получаем перечень имен подов
POD1=$(kubectl -n stateful-flask get pod -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')
POD2=$(kubectl -n stateful-flask get pod -l app=stateful-flask -o jsonpath='{.items[1].metadata.name}')

# получаем перечень ресурсов подов
kubectl -n stateful-flask get pod $POD1 -o=jsonpath='{.spec.containers[0].resources}'; echo
kubectl -n stateful-flask get pod $POD2 -o=jsonpath='{.spec.containers[0].resources}'; echo
```

Вы заметите, что запросы и лимиты пода были обновлены, чтобы отразить текущее использование приложения. Вот новые значения в моем случае:

```
{"limits":{"cpu":"2500m","memory":"32000Mi"},"requests":{"cpu":"25m","memory":"262144k"}}
{"limits":{"cpu":"2500m","memory":"32000Mi"},"requests":{"cpu":"25m","memory":"262144k"}}
```

VPA продолжит следить за приложением и обновлять запросы и лимиты подов при необходимости.



14.3. Режимы VPA

В предыдущем примере мы использовали VPA следующим образом:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: stateful-flask-vpa
  namespace: stateful-flask
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: stateful-flask
  updatePolicy:
    updateMode: "Auto"
```

Как видите, полю `updateMode` присвоено значение `Auto`. Это означает, что VPA будет автоматически обновлять запросы и ограничения пода независимо от того, в какую сторону происходит масштабирование.

Всего доступны четыре режима. Далее мы рассмотрим каждый из них.

14.3.1. Auto (автоматический)

VPA автоматически обновит запросы и лимиты пода независимо от направления масштабирования.

Этот режим может быть полезен для критических рабочих нагрузок, требующих высокой доступности и не допускающих простоя, а также для любых вариантов использования, когда вам нужно динамически корректировать требования к ресурсам, минимизируя влияние на работающие поды.

Данный режим может также подойти для приложений с отслеживанием состояния, которым требуется более изящный способ обновления их требований к ресурсам.

14.3.2. Initial (начальный)

VPA обновит запросы и лимиты подов только при их создании.

Этот режим может быть полезен, когда у вас есть предсказуемые рабочие нагрузки, которые не меняются значительно с течением времени, и вы хотите заранее задать запросы ресурсов, чтобы гарантировать их выполнение при планировании подов.

14.3.3. Recreate (повторное создание)

VPA обновит запросы и лимиты подов только при их повторном создании.

Этот режим может быть полезен, когда у вас есть переменные рабочие нагрузки, которые могут меняться со временем, и вы хотите динамически корректировать требования к ресурсам, чтобы гарантировать, что у пода достаточно ресурсов для эффективной работы. Этот режим может подойти для приложений без состояния, которые допускают короткие перерывы во время вытеснения подов.

14.3.4. Off (выключен)

VPA не будет обновлять запросы и лимиты подов, однако он все равно будет собирать метрики и давать рекомендации. Таким образом, VPA все равно сможет сообщать вам об избытке или нехватке подов.

Если вы хотите поэкспериментировать с рекомендациями VPA и оценить влияние на ваши приложения без автоматического изменения их требований к ресурсам, то выберите этот режим. Иногда полезно понять, как ваше приложение ведет себя в различных условиях нагрузки.

Мы собираемся использовать этот режим в следующем разделе и поэкспериментировать с рекомендациями VPA.

14.4. Рекомендации VPA

Как вы уже знаете, рекомендатель VPA является компонентом VPA, который собирает метрики и дает рекомендации. Он отвечает за расчет требований к ресурсам для подов на основе собираемых им метрик.

Рассмотрим следующий пример. Этот YAML-файл задает режим VPA Off:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: stateful-flask-vpa
  namespace: stateful-flask
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: stateful-flask
  updatePolicy:
    updateMode: "Off"
```

Примените эту конфигурацию:

```
kubectl apply -f kubernetes/vpa.yaml
```

Проведите нагрузочный тест, как мы делали ранее, и вы увидите, что запросы и лимиты подов не будут обновлены.

```
# Удаляем генератор нагрузки, если он существует
kubectl delete pod load-generator
# Создаем новый генератор нагрузки
```

```
kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/
sh -c "while sleep 0.01; do wget -q -O- http://stateful-flask.stateful-flask.svc.cluster.
local:5000/tasks; done"
```

Чтобы увидеть рекомендации, сделанные VPA, выполните следующую команду:

```
kubectl -n stateful-flask describe vpa stateful-flask-vpa
```

Так выглядит пример вывода:

```
Name:          stateful-flask-vpa
[...]
Spec:
  Target Ref:
    API Version:  apps/v1
    Kind:         Deployment
    Name:         stateful-flask
  Update Policy:
    Update Mode:  Off
Status:
  Conditions:
    Last Transition Time:  2023-05-05T15:37:17Z
    Status:               True
    Type:                 RecommendationProvided
  Recommendation:
    Container Recommendations:
      Container Name:     stateful-flask
      Lower Bound:
        Cpu:              25m
        Memory:           262144k
      Target:
        Cpu:              25m
        Memory:           262144k
      Uncapped Target:
        Cpu:              25m
        Memory:           262144k
      Upper Bound:
        Cpu:              25m
        Memory:           262144k
  Events:               <none>
```

Важно обратить внимание на раздел Recommendation. Здесь VPA сообщит вам, какими должны быть запросы и лимиты подов.

В этом разделе есть четыре поля:

- Lower Bound (нижняя граница) – это минимальный объем ресурсов, который должен иметь под;

- Target (целевой показатель) – это объем ресурсов, который должен иметь под;
- Unscapped Target (неограниченный целевой показатель) – это объем ресурсов, который должен иметь под, если бы не было ограничений;
- Upper Bound (верхняя граница) – это максимальный объем ресурсов, который должен иметь под.

Эти результаты основаны на метриках, собранных VPA, и они меняются в зависимости от нагрузки, которую получает приложение. Поэтому вам следует протестировать свое приложение с реалистичной нагрузкой, чтобы получить наиболее точные результаты.

После выполнения нагрузочного теста вы можете назначить ресурсам подов рекомендуемые значения, которые обычно соответствуют значению параметра Target.

14.4.1. Ограничения VPA

Вертикальное автомасштабирование имеет несколько ограничений, которые пользователи должны учитывать перед его внедрением в кластер Kubernetes:

- **повторное создание подов при обновлении ресурсов** – всякий раз, когда VPA обновляет ресурсы, под создается заново, что приводит к повторному созданию всех запущенных контейнеров. Заметим, что под может быть повторно создан на другом узле;
- **ограниченная гарантия повторного создания пода** – VPA не может гарантировать, что поды, которые он вытесняет или удаляет для применения рекомендаций (при настройке в режимы Auto и Recreate), будут успешно воссозданы;
- **ограничения для подов без контроллера** – VPA не вытесняет поды, которые не запущены под контроллером. Для таких подов режим Auto эквивалентен режиму Initial. Во всех случаях не рекомендуется создавать поды без контроллера;
- **несовместимость с HPA при работе с ЦП или памятью** – VPA не следует использовать вместе с HPA для масштабирования ЦП или памяти. Однако если ваш HPA основан на пользовательских или внешних метриках, то можно использовать VPA вместе с HPA;
- **неполная обработка событий нехватки памяти** – VPA реагирует на большинство событий нехватки памяти, но не во всех ситуациях;
- **ограничения производительности в больших кластерах** – производительность VPA не тестировалась в больших кластерах. Также нет четкого определения того, что представляет собой большой кластер с точки зрения количества подов или узлов. В целом более крупные кластеры с большим количеством подов и узлов могут испытывать больше ограничений производительности, но точное пороговое значение будет зависеть от многих факторов и может варьироваться от

одного кластера к другому. Поэтому рекомендуется протестировать производительность вашего VPA в вашей конкретной конфигурации кластера, чтобы убедиться, что она соответствует вашим требованиям;

- **рекомендация, превышающая доступные ресурсы** – рекомендация VPA может превышать доступные ресурсы (например, размер узла, доступный размер, доступную квоту) и приводить к тому, что поды переходят в режим ожидания. Частично этот недостаток можно устранить, используя VPA вместе с Cluster Autoscaler;
- **неопределенное поведение с несколькими ресурсами VPA для одного пода** – несколько ресурсов VPA, соответствующих одному поду, дают неопределенное поведение. Поэтому рекомендуется избегать создания нескольких ресурсов VPA для одного пода;
- **количество реплик** – VPA по умолчанию настроен на применение изменений только при наличии не менее двух реплик. Это необходимо для того, чтобы избежать ситуации, когда VPA рекомендует изменение, которое приведет к недоступности вашего приложения. Это поведение можно изменить, настроив поле `minReplicas` в разделе `updatePolicy` спецификации VPA.

Пример:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: stateful-flask-vpa
  namespace: stateful-flask
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: stateful-flask
  updatePolicy:
    updateMode: "Off"
    minReplicas: 1
```

Глава 15

Масштабирование микросервисов, сохраняющих состояние на примере PostgreSQL

15.1. StatefulSet и масштабирование

Ранее мы создали базу данных PostgreSQL с помощью StatefulSet. Ниже приведен манифест, который мы использовали:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:10.1
```

```

    imagePullPolicy: Always
    ports:
      - containerPort: 5432
    envFrom:
      - configMapRef:
          name: postgres-config
    volumeMounts:
      - name: postgreddb-volume
        mountPath: /var/lib/postgresql/data
        subPath: postgres
    env:
      - name: PGDATA
        value: /var/lib/postgresql/data/pgdata
  volumeClaimTemplates:
  - metadata:
      name: postgreddb-volume
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "do-block-storage"
      resources:
        requests:
          storage: 5Gi

```

В приведенном выше коде YAML мы определяем `volumeClaimTemplates` как шаблон для `PersistentVolumeClaim`. Это позволяет нам масштабировать `StatefulSet`, при этом каждая реплика имеет свой собственный `PersistentVolumeClaim`. Однако мы можем столкнуться с проблемами согласованности данных, поскольку данные не реплицируются синхронно с экземплярами.

Например, если мы создаем запись в базе данных с помощью POST-запроса API, который отправляется на первую реплику, масштабирование `StatefulSet` до двух реплик может привести к отправке запроса GET на вторую реплику до того, как данные будут реплицированы. Это приведет к ошибке.

PostgreSQL – это не просто инструмент, а экосистема инструментов. Мы можем рассматривать PostgreSQL как ядро операционной системы; оно обеспечивает базовую функциональность, но вам нужны дополнительные инструменты, чтобы сделать его полезным. Существуют различные инструменты и методы, доступные для решения проблемы согласованности данных, такие как создание собственного кластера PostgreSQL с использованием потоковой репликации, использование инструментов наподобие Patroni¹ или одной из его альтернатив.

Хотя на первый взгляд это выглядит просто, согласованность данных на самом деле является сложной проблемой. Поэтому лучше использовать стабильный инструмент, который уже доступен и поддерживается сообществом, а не изобретать велосипед.

Когда мы работаем с Kubernetes, в нашем распоряжении есть несколько инструментов:

¹ <https://github.com/zalando/patroni>.

- **Zalando PostgreSQL Operator**¹ – оператор Postgres, который создает и управляет кластерами PostgreSQL, работающими в Kubernetes;
- **PostgreSQL HA**², упакованный Bitnami, – это решение для кластера PostgreSQL включает менеджер репликации PostgreSQL, инструмент с открытым исходным кодом для управления репликацией и отказоустойчивостью в кластерах PostgreSQL;
- **PGO**³ – производственная версия PostgreSQL для Kubernetes, от высокодоступных кластеров Postgres до полномасштабной базы данных как услуги;
- **Stolon**⁴ – облачное решение высокодоступной PostgreSQL и другие опции;
- ... и многое другое⁵.

В следующих разделах мы будем экспериментировать со Stolon.

15.2. Stolon: знакомство

Stolon – это облачный менеджер PostgreSQL для обеспечения высокой доступности. Он позволяет вам поддерживать высокодоступный экземпляр PostgreSQL в ваших контейнерах (с интеграцией Kubernetes), а также в других типах инфраструктуры, таких как облачный IaaS и инфраструктуры старого стиля.

Архитектура Stolon состоит из трех основных компонентов:

- **Keeper**: управляет экземпляром PostgreSQL, сходясь к clusterview⁶, вычисляемому ведущим Sentinel;
- **Sentinel**: обнаруживает и контролирует объекты Keeper и Proxy, а также вычисляет оптимальный clusterview;
- **Proxy**: точка доступа клиента. Этот компонент принудительно устанавливает соединения с нужным главным сервером PostgreSQL и принудительно закрывает соединения со старыми главными серверами.

Stolon имеет множество полезных функций, описанных в документации⁷:

- использует потоковую репликацию PostgreSQL;
- устойчив к любому виду секционирования, предпочитает согласованность доступности, пытается поддерживать максимальную доступность;

¹ <https://github.com/zalando/postgres-operator>.

² <https://github.com/bitnami/charts/tree/main/bitnami/postgresql-ha>.

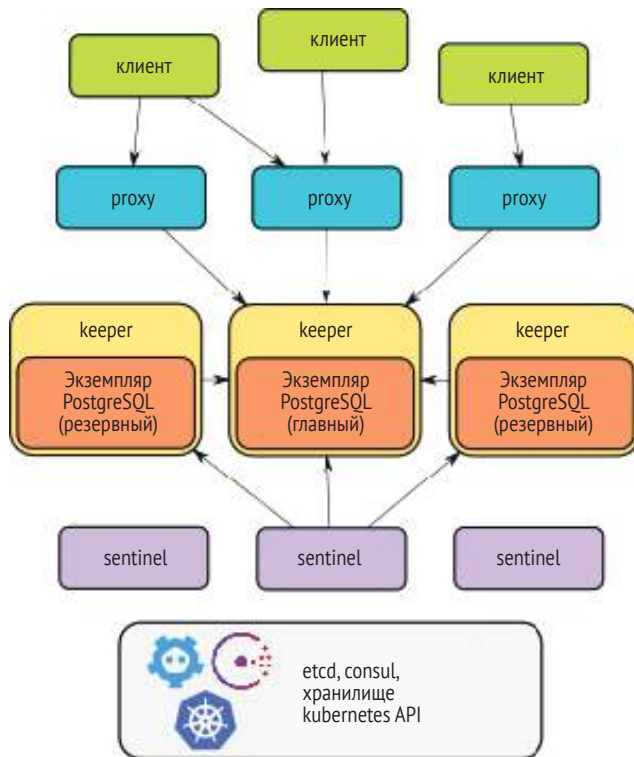
³ <https://github.com/CrunchyData/postgres-operator>.

⁴ <https://github.com/sorintlab/stolon>.

⁵ <https://www.google.com/search?q=postgres+highly+available+on+kubernetes>.

⁶ <https://github.com/sorintlab/stolon/blob/master/doc/architecture.md>.

⁷ <https://github.com/sorintlab/stolon>.



Источник:

<https://github.com/sorintlab/stolon/blob/master/doc/architecture.png>

- обеспечивает интеграцию с Kubernetes для достижения высокой доступности PostgreSQL;
- использует кластерное хранилище, такое как etcd, Consul или сервер API Kubernetes, в качестве высокодоступного хранилища данных и для выбора лидера;
- поддерживает асинхронную (по умолчанию) и синхронную репликации;
- позволяет полностью настроить кластер за считанные минуты и легко его администрировать;
- может выполнять восстановление на определенный момент времени путем интеграции с вашим предпочтительным инструментом резервного копирования/восстановления;
- поддерживает резервный кластер для многосайтовой репликации и миграции с почти нулевым временем простоя;
- поддерживает автоматическое обнаружение сервисов и динамическую перенастройку, обрабатывая процессы PostgreSQL и Stolon, меняющие свои адреса;
- может использовать `pg_rewrite` для быстрой повторной синхронизации экземпляра с текущим мастером.

Мы установим Stolon на наш кластер Kubernetes и оценим его производительность.

15.3. Stolon: установка

Для начала клонируем репозиторий:

```
cd $HOME/stateful-flask
git clone https://github.com/sorintlab/stolon.git
cd stolon/examples/kubernetes
```

Инициализируем кластер при помощи следующей команды:

```
kubectrl run -i -t stolonctl --image=sorintlab/stolon:master-pg10 --restart=Never --rm -- /
usr/local/bin/stolonctl --cluster-name=kube-stolon --store-backend=kubernetes --kube-
resource-kind=configmap init
```

Далее нам нужно создать Sentinel. По умолчанию создается два Sentinel, но вы можете изменить это количество, отредактировав файл `stolon-sentinel.yaml` или масштабировав развертывание позже.

```
kubectrl create -f stolon-sentinel.yaml
```

Создайте файл хранителя секретов:

```
kubectrl create -f secret.yaml
```

В нашем случае секрет содержит пароль, который будет использоваться хранителем для настройки начального пользователя базы данных. Вы можете изменить пароль, отредактировав файл `secret.yaml`. Пароль по умолчанию – `password1`. Если вы хотите изменить пароль, не забудьте закодировать его в Base64.

```
echo -n "<Your_PASSWORD>" | base64
```

Для простоты мы сохраним пароль по умолчанию. Теперь перейдем к созданию объекта `StatefulSet` для `Stolon Keeper` (хранитель `Stolon`):

```
kubectrl create -f stolon-keeper.yaml
```

Этот манифест определяет `StatefulSet` с двумя репликами. Вы можете изменить количество реплик, отредактировав файл `stolon-keeper.yaml` или масштабировав `StatefulSet` позже. Sentinel выберет случайный экземпляр хранителя в качестве главного; этот хранитель инициализирует новый кластер базы данных, в то время как оставшийся станет резервным.

Создайте прокси:

```
kubectl create -f stolon-proxy.yaml
```

Наконец, создайте службу для прокси:

```
kubectl create -f stolon-proxy-service.yaml
```

Все компоненты были установлены в пространстве имен по умолчанию.

Чтобы проверить состояние кластера, выполните следующую команду:

```
kubectl run -i -t stolonctl --image=sorintlab/stolon:master-pg10 --restart=Never --rm -- /usr/local/bin/stolonctl --cluster-name=kube-stolon --store-backend=kubernetes --kube-resource-kind=configmap status
```

15.4. Stolon: применение

Поскольку у нас есть прокси-сервис, мы можем воспользоваться им для подключения к базе данных. Однако прокси-сервис является частью пространства имен по умолчанию, в то время как наше приложение Flask находится в пространстве имен `stateful-flask`.

Чтобы перенаправить трафик на прокси-сервис, можно создать службу `ExternalName` в пространстве имен `stateful-flask`.

Листинг 15.1 ❖ Файл `kubernetes/deployment.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: db
  namespace: stateful-flask
spec:
  type: ExternalName
  externalName: stolon-proxy-service.default.svc.cluster.local
```

Как показано выше, код создает сервис с именем `db` в пространстве имен `stateful-flask`. Этот сервис перенаправляет трафик в сервис `stolon-proxy-service` в пространстве имен по умолчанию.

Чтобы обновить развертывание и

- начать использовать новую службу,
- создать базу данных перед запуском приложения Flask,

вам необходимо обновить файл `kubernetes/deployment.yaml`.

Листинг 15.2 ❖ Файл `kubernetes/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stateful-flask
  namespace: stateful-flask
spec:
  replicas: 1
  selector:
    matchLabels:
      app: stateful-flask
  template:
    metadata:
      labels:
        app: stateful-flask
    spec:
      containers:
        - name: stateful-flask
          image: eon01/stateful-flask:v0
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
          env:
            - name: DB_USER
              value: stolon
            - name: DB_PASSWORD
              value: password1
            - name: DB_HOST
              value: db
            - name: DB_NAME
              value: mydb
            - name: DB_PORT
              value: "5432"
      initContainers:
        - name: init-db
          image: postgres:9.6
          imagePullPolicy: IfNotPresent
          command: ["bash", "-c", "createdb -h db -U \${DB_USER} \${DB_NAME} || true"]
          env:
            - name: PGPASSWORD
              value: password1
            - name: DB_NAME
              value: mydb
            - name: DB_USER
              value: stolon
```

В коде YAML выше мы используем службу `db` для установления соединения с базой данных, а также контейнер `init` для создания базы данных перед запуском приложения Flask.

i Контейнеры `init` – это особый тип контейнера, который запускается перед основным контейнером. Они полезны для выполнения задач инициализации, таких как создание базы данных в нашем случае. Если контейнер `init` сработал правильно, он должен завершиться с кодом выхода 0. Если контейнер `init` завершается с ошибкой, основной контейнер не запустится.

Теперь мы можем применить изменения как к службе, так и к разворачиванию.

```
kubectl apply -f kubernetes/stolon-proxy-service-externalname.yaml
kubectl apply -f kubernetes/deployment.yaml
```

Давайте проверим статус инициализации:

```
export pod=$(kubectl get pods -n stateful-flask -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')
kubectl -n stateful-flask logs -f $pod -c init-db
```

Если все прошло хорошо, вы сможете перейти к следующему шагу – созданию схемы базы данных.

```
export pod=$(kubectl get pods -n stateful-flask -l app=stateful-flask -o jsonpath='{.items[0].metadata.name}')

kubectl -n stateful-flask exec -it $pod -- db init
kubectl -n stateful-flask exec -it $pod -- db migrate
kubectl -n stateful-flask exec -it $pod -- db upgrade
```

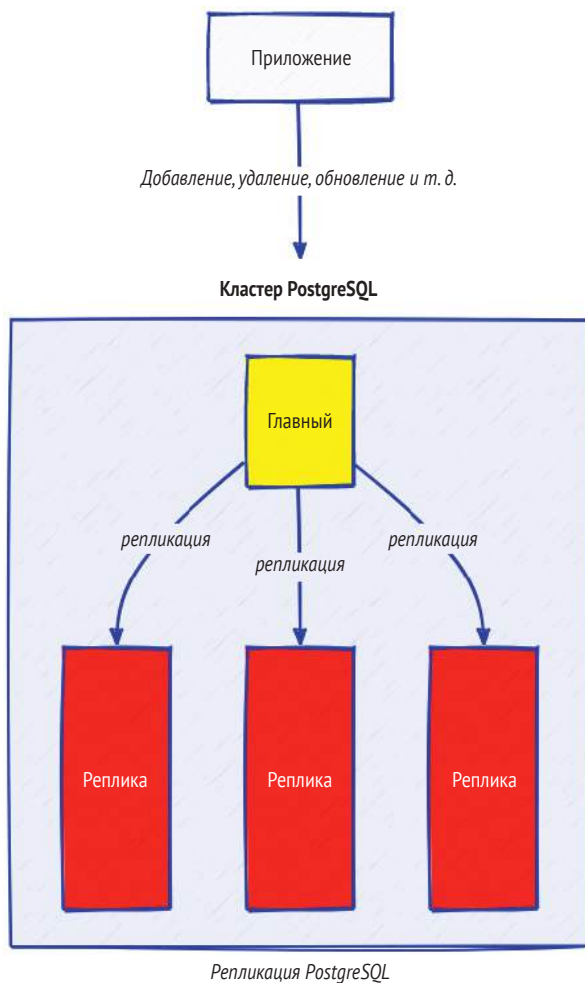
Наконец, мы можем масштабировать разворачивание и `StatefulSet` до необходимого количества реплик.

```
kubectl scale deployment stateful-flask -n stateful-flask --replicas=5
kubectl scale statefulset stolon-keeper -n default --replicas=3
```

Вы также можете добавлять новые задачи в список `todo`.

```
export url="http://$(kubectl get ingress | awk '{print $3}' | tail -n 1)/tasks"

curl -X POST -H "Content-Type: application/json" -d '{"title":"New task 1",
"description":"New description"}' $url
curl -X POST -H "Content-Type: application/json" -d '{"title":"New task 2",
"description":"New description"}' $url
curl -X POST -H "Content-Type: application/json" -d '{"title":"New task 3",
"description":"New description"}' $url
```



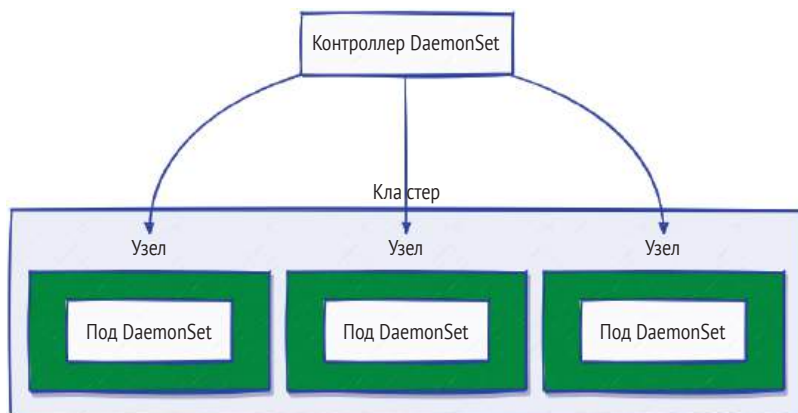
Каждая добавленная задача будет сохранена в реплике главной базы данных и будет доступна всем остальным репликам благодаря структуре Stolon. Подробнее о настройке Stolon можно прочитать на <https://sgotti.dev/post/stolon-introduction/>.

Глава 16

Стратегии развертывания микросервисов: один сервис на узел

16.1. Назначение и варианты использования DaemonSet

DemonSet – это тип контроллера ресурсов в Kubernetes, который обеспечивает запуск пода на каждом узле в кластере. В отличие от других контроллеров ресурсов, которые развертывают несколько реплик пода в кластере, DaemonSet развертывает один под на каждом узле и не более. Это гарантирует, что один экземпляр приложения запущен на каждом узле в кластере.



DaemonSet часто используются для таких задач, как сбор журналов, мониторинг или сетевые прокси на каждом узле.

Типичным примером DaemonSet является Fluentd. Он собирает журналы со всех узлов в кластере и отправляет их в центральный агрегатор журналов.

16.2. Создание и управление DaemonSet

Ниже приведен пример манифеста DaemonSet, который развертывает Fluentd на всех узлах в кластере.

Начните с запуска следующей команды, чтобы создать каталог с именем fluentd в вашем домашнем каталоге:

```
cd $HOME/  
mkdir -p fluentd  
cd fluentd
```

Затем создайте файл манифеста с именем daemonset.yaml:

Листинг 16.1 ❖ Файл daemonset.yaml

```
apiVersion: apps/v1  
kind: DaemonSet  
metadata:  
  name: fluentd-elasticsearch  
  namespace: kube-system  
  labels:  
    k8s-app: fluentd-logging  
spec:  
  selector:  
    matchLabels:  
      name: fluentd-elasticsearch  
  template:  
    metadata:  
      labels:  
        name: fluentd-elasticsearch  
    spec:  
      containers:  
        - name: fluentd-elasticsearch  
          image: quay.io/fluentd_elasticsearch/fluentd:v4  
          volumeMounts:  
            - name: varlog  
              mountPath: /var/log  
      terminationGracePeriodSeconds: 30  
      volumes:  
        - name: varlog  
          hostPath:  
            path: /var/log
```

- вышеприведенный манифест YAML определяет объект DaemonSet, который гарантирует, что все узлы в кластере запускают копию указанного пода. DaemonSet называется fluentd-elasticsearch и находится в пространстве имен kube-system;
- спецификация спес содержит селектор для указания на под, которым будет управлять DaemonSet. В этом случае селектор соответствует поду с меткой name: fluentd-elasticsearch. Этот механизм похож на тот, который применяет ReplicaSet в развертывании;
- под запускает контейнер с именем fluentd-elasticsearch, который использует образ quay.io/fluentd_elasticsearch/fluentd:v4;
- контейнер также монтирует каталог /var/log хоста в каталог /var/log контейнера. Это необходимо, поскольку Fluentd используется для сбора журналов с хоста;
- манифест указывает период завершения в 30 с. Это означает, что при удалении DaemonSet поды будут завершены через 30 с.

Примените манифест DaemonSet с помощью следующей команды:

```
kubectl apply -f daemonset.yaml
```

Затем проверьте, запускают ли поды DaemonSet одну реплику на каждом узле:

```
kubectl get pods -n kube-system -o wide | grep fluentd-elasticsearch
```

В выводе команды седьмой столбец отображает узел, на котором в данный момент запущен под.

Главный узел обычно заблокирован, чтобы предотвратить планирование размещения пода на нем. Поэтому если мы собираемся развернуть DaemonSet на главном узле, мы должны добавить опцию Tolerations в DaemonSet.

Ниже показан краткий пример. Вы узнаете больше о свойствах Taints и Tolerations в следующей главе.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      # Добавление Tolerations, чтобы DaemonSet мог быть развернут
```

```
# на узле плоскости управления
tolerations:
- key: node-role.kubernetes.io/master
  operator: Exists
  effect: NoSchedule
- key: node-role.kubernetes.io/control-plane
  operator: Exists
  effect: NoSchedule
# Конец определения Tolerations
containers:
- name: fluentd-elasticsearch
  image: quay.io/fluentd_elasticsearch/fluentd:v4
  volumeMounts:
- name: varlog
  mountPath: /var/log
terminationGracePeriodSeconds: 30
volumes:
- name: varlog
  hostPath:
    path: /var/log
```

Исключение составляет сценарий, в котором вы используете управляемую службу Kubernetes, такую как DOK DigitalOcean. Управляемые службы Kubernetes обычно имеют ограничения и запреты для обеспечения стабильности, безопасности и производительности узлов плоскости управления. Поэтому если вы используете управляемую службу Kubernetes, вы не сможете развернуть DaemonSet на узлах плоскости управления. Однако это ограничение не распространяется на неуправляемые кластеры Kubernetes.

Глава 17

Стратегии развертывания микросервисов: назначение рабочих нагрузок определенным узлам

17.1. Зачем назначать рабочие нагрузки определенным узлам?

Бывают ситуации, когда вам необходимо назначить некоторые или все рабочие нагрузки строго определенным узлам. Вот несколько примеров таких ситуаций:

- *специальное оборудование*: некоторые приложения нуждаются в особых аппаратных возможностях. В таком случае приходится выполнять эти нагрузки на определенных узлах, имеющих необходимое оборудование;
- *лицензирование*: некоторые лицензии на программное обеспечение ограничивают количество узлов, на которых может работать определенное приложение. Если вы не можете запустить программное приложение более чем на одном узле, вам необходимо назначить рабочую нагрузку определенному узлу;
- *комплаенс*: иногда вам необходимо запустить рабочую нагрузку на определенном узле, чтобы соответствовать нормам регулятора, пра-

вилам или политикам. Например, вам может потребоваться запустить рабочую нагрузку на узле, расположенном в определенном регионе или стране;

- *локальные данные*: если ваше приложение зависит от доступа к данным, хранящимся на определенном узле, вы можете назначить рабочую нагрузку этому узлу, чтобы оптимизировать доступ к данным и минимизировать сетевую задержку;
- *оптимизация производительности*: вы можете запустить рабочую нагрузку на определенном узле, чтобы оптимизировать производительность ваших микросервисов. Этот узел может применяться только для запуска определенного типа оптимизированной рабочей нагрузки;
- *тестирование и отладка*: запуск рабочей нагрузки на определенном узле может помочь вам протестировать и отладить ваше приложение. Например, вы можете запустить рабочую нагрузку на определенном узле, чтобы протестировать новую версию вашего приложения перед его развертыванием в остальной части кластера;
- *устаревшие системы*: некоторые устаревшие системы могут потребовать от вас запуска нагрузки на определенном узле. Например, вам необходимо запустить рабочую нагрузку на узле, на котором работает определенная версия операционной системы;
- *безопасность*: в некоторых случаях необходимо запускать рабочую нагрузку на определенном узле для повышения безопасности. Например, вы можете запустить критически важную рабочую нагрузку на узле, на котором установлено специальное программное обеспечение для усиления защиты ОС или безопасности.

Приведенный выше список не является исчерпывающим. Конечно, существуют и другие сценарии, в которых приходится назначать рабочие нагрузки определенным узлам.

В Kubernetes необходимость выполнения рабочей нагрузки или микросервиса на определенных узлах преобразуется в набор правил, используемых для выбора узлов, на которых будут развернуты поды, запускающие ваши рабочие нагрузки и микросервисы. Эти правила определяются с использованием следующих концепций Kubernetes:

- **Taint и Toleration**: убедитесь, что поды не планируются на узлах с определенными *ограничениями* (taint), если у них нет соответствующих *допусков* (toleration);
- **nodeSelector**: простая функция, которая позволяет планировать поды на узлах с соответствующими метками, указанными пользователем;
- **Node Affinity**: расширенная версия nodeSelector, которая обеспечивает детальный контроль над планированием подов на основе меток узлов.

Далее мы рассмотрим эти три концепции. Оставшиеся две очень похожи и используют почти такие же механизмы.

17.2. Ограничения и допуски

17.2.1. Определение

Концепции ограничений и допусков обычно применяются вместе, чтобы гарантировать, что поды не будут запланированы на неподходящих узлах. Ограничения применяются к узлам, а допуски применяются к подам.

Когда вы добавляете в манифест узла параметр Taint, Kubernetes не допустит на него все поды, кроме тех, у которых есть соответствующий допуск.

Каждый объект Taint состоит из трех компонентов:

- *ключ* (key) – имя, нужное для идентификации ограничения;
- *значение* (value), которое является необязательным и применяется для идентификации ограничения;
- *эффект* (effect) – действие, которое Kubernetes будет выполнять в отношении любых подов, не допускающих это ограничение.

Эффект может быть одним из следующих:

- NoSchedule: Kubernetes не будет планировать поды, которые не допускают данное ограничение;
- PreferNoSchedule: Kubernetes попытается не планировать ни один под, который не допускает данное ограничение, но это не гарантируется;
- NoExecute: Kubernetes исключит любой существующий под, который не допускает Taint.

Вы можете просмотреть ограничения ваших узлов с помощью следующей команды:

```
kubectl get nodes -o=custom-columns=NodeName:.metadata.name,TaintKey:.spec.taints[*].key,TaintValue:.spec.taints[*].value,TaintEffect:.spec.taints[*].effect
```

В качестве альтернативы вы можете создать псевдоним для указанной выше команды:

```
cat <<EOF >> ~/.bashrc
alias taints='kubectl get nodes -o=custom-columns=NodeName:.metadata.name,TaintKey:.spec.taints[*].key,TaintValue:.spec.taints[*].value,TaintEffect:.spec.taints[*].effect'
EOF
```

Далее перезагрузите файл bashrc и выполните псевдоним:

```
source ~/.bashrc
taints
```

По умолчанию контроллеры узлов Kubernetes автоматически снабжают узлы ограничениями при выполнении определенных условий. Вот некоторые из ограничений по умолчанию:

- `node.kubernetes.io/not-ready`: применяется к узлам, которые не готовы;

- `node.kubernetes.io/unreachable`: применяется к узлам, которые недоступны;
- `node.kubernetes.io/memory-pressure`: применяется к узлам, которые находятся под давлением недостатка памяти;
- `node.kubernetes.io/disk-pressure`: применяется к узлам, которые находятся под давлением недостатка дискового пространства;
- `node.kubernetes.io/pid-pressure`: применяется к узлам, которые находятся под давлением PID;
- `node.kubernetes.io/network-unavailable`: применяется к узлам, которые испытывают проблемы с сетью;
- `node.kubernetes.io/unschedulable`: применяется к узлам, которые не могут быть запланированы;
- `node.cloudprovider.kubernetes.io/uninitialized`: применяется к узлам, которые еще не были инициализированы и используются провайдерами облачных услуг.

Вы также можете вручную добавить ограничение к узлу, используя следующую команду:

```
kubecttl taint nodes <node-name> <taint-key>=<taint-value>:<taint-effect>
```

17.2.2. Пример ограничений и допусков

Рассмотрим следующий пример. Добавьте ограничение для всех ваших узлов с помощью следующей команды:

```
kubecttl taint nodes --all mykey=myvalue:NoSchedule
```

Вы можете снова проверить ограничения ваших узлов, используя псевдоним:

```
taints
```

Теперь примените к вашему кластеру следующий DaemonSet:

Листинг 17.1 ❖ Файл daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
```

```

labels:
  name: fluentd-elasticsearch
spec:
  containers:
  - name: fluentd-elasticsearch
    image: quay.io/fluentd_elasticsearch/fluentd:v4
    volumeMounts:
    - name: varlog
      mountPath: /var/log
  terminationGracePeriodSeconds: 30
  volumes:
  - name: varlog
    hostPath:
      path: /var/log

```

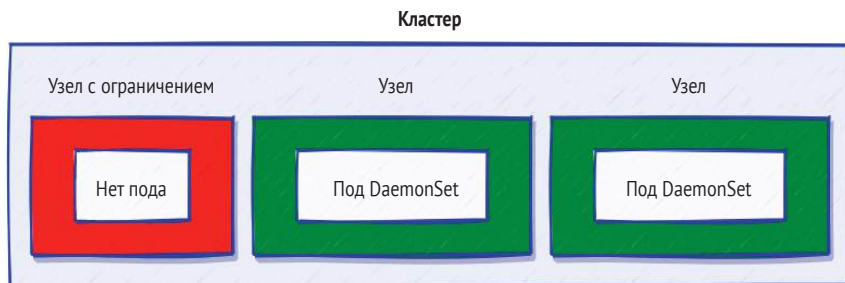
Примените манифест DaemonSet:

```
kubectl apply -f daemonset.yaml
```

Чтобы узнать, где запущены поды DaemonSet, выполните следующую команду:

```
kubectl -n kube-system get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName | grep fluentd-elasticsearch
```

Поды DaemonSet не должны отображаться в выводе. Это происходит потому, что ограничения узлов не пускают на них поды.



Ограничения (Taints) Kubernetes

Чтобы создать исключение для подов DaemonSet с помощью допуска Toleration, используйте следующий манифест YAML:

Листинг 17.2 ❖ Файл daemonset-2.yaml

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:

```

```

k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      tolerations:
        - key: mykey
          operator: Equal
          value: myvalue
          effect: NoSchedule
      containers:
        - name: fluentd-elasticsearch
          image: quay.io/fluentd_elasticsearch/fluentd:v4
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      terminationGracePeriodSeconds: 30
      volumes:
        - name: varlog
          hostPath:
            path: /var/log

```

Теперь примените DaemonSet:

```
kubectl apply -f daemonset.yaml
```

Посмотрите, где запущены поды DaemonSet:

```
kubectl -n kube-system get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.
nodeName | grep fluentd-elasticsearch
```

Вы должны увидеть, что поды DaemonSet запущены на всех узлах.



Если вы хотите удалить ограничения со всех узлов, выполните следующую команду:

```
kubectl taint nodes --all mykey-
```

Убедитесь, что ограничения удалены:

```
taints
```

17.3. nodeSelector

17.3.1. Простейшая форма привязки к узлам

Это, вероятно, самый простой способ ограничить поды определенными узлами, поскольку он основан на простом для понимания селекторе меток. Однако nodeSelector представляет собой упрощенную форму привязки к узлам. Он может выражать только простые требования. Если у вас более сложные требования, вам следует использовать подноценную привязку к узлам.

17.3.2. Пример привязки через nodeSelector

Рассмотрим простой пример. Создайте новое рабочее пространство:

```
cd $HOME/ && mkdir -p nodeSelector && cd nodeSelector
```

Создайте под, который запланирован на узле с меткой disktype: ssd.

Листинг 17.3 ❖ Файл nodeSelector.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      nodeSelector:
        disktype: ssd
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Примените этот манифест:

```
kubectl apply -f nodeSelector.yaml
```

Посмотрите, где запущен под:

```
kubectl get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName
```

Чтобы убедиться, что под запущен на узле с меткой `disktype: ssd`, проверьте, существует ли узел с такой меткой. Если такой узел не существует, под останется в состоянии ожидания.

Обычно метка используется для сопоставления узла, и под планируется на этом узле. Если узла с такой меткой нет, под всегда будет оставаться в состоянии ожидания.

Чтобы просмотреть метки по умолчанию для ваших узлов, выполните следующую команду:

```
kubectl get nodes --show-labels
```

В качестве альтернативы можно отформатировать вывод, чтобы отображались только метки, по одной на строку:

```
kubectl get nodes --show-labels | xargs -L1 | tail -n 1 | tr ", " "\n"
```

Если вывод не содержит метку `disktype=ssd`, вы можете добавить ее ко всем узлам с SSD-дисками с помощью следующей команды:

```
# добавить метку ко всем узлам
kubectl label nodes --all disktype=ssd
# или добавить метку к одному узлу
kubectl label nodes <node-name> disktype=ssd
```

Вы должны наблюдать переход пода из состояния Pending (ожидание) в состояние Running (выполнение).

Не забудьте после этого выполнить очистку:

```
kubectl delete -f nodeSelector.yaml
```

17.4. Сходство и антисходство узлов

17.4.1. Сходство узлов: как nodeSelector, но с большим количеством опций

В Kubernetes администраторы обычно доверяют планировщику автоматический выбор узлов для планирования подов на основе доступности ресурсов. Однако существуют сценарии, в которых необходим ручной выбор, например когда поды нужно разместить на заданных узлах с определенными атрибутами. Эти атрибуты могут быть метками, и в таком случае мы можем просто использовать `nodeSelector`. Однако в других сценариях, где требуются более сложные правила, можно использовать *сходство узлов* (node affinity).

17.4.2. Сходство узлов: пример

Допустим, нам нужно запланировать наши микросервисы на узлах с определенными именами хостов.

Предположим, у нас есть кластер из трех узлов:

- default-f72fj;
- default-f72fo;
- default-f8wx9.

Наша цель – развернуть Nginx на узле с именем хоста default-f8wx9. Для начала создайте новое рабочее пространство.

```
cd $HOME/ && mkdir -p node-affinity && cd node-affinity
```

Далее создайте под, который запланирован на узле с меткой hostname: default-f8wx9.

Листинг 17.4 ❖ Файл affinity.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: In
                    values:
                      - default-f8wx9
```

Примените развертывание:

```
kubectl apply -f affinity.yaml
```

Проверьте, где запущены поды:

```
kubectl get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName | grep nginx
```

Вы должны увидеть под, запущенный на узле с меткой `hostname: default-f8wx9`.

Чтобы добавить еще один узел в список узлов, на которых можно запланировать под, выполните следующую команду:

```
kubectl edit deployment nginx
```

Затем добавьте в манифест следующие строки:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - default-f8wx9
                - default-f72fj
```

Теперь рассмотрим пример привязки узла с использованием метки `topology.kubernetes.io/zone`.

Листинг 17.5 ❖ Файл `affinity-2.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
```

```

nodeSelectorTerms:
- matchExpressions:
  - key: topology.kubernetes.io/zone
    operator: In
    values:
      - FRA
      - LON

```

Приведенный выше пример планирует поды на узлах в зонах FRA (Франкфурт) и LON (Лондон). Он использует метку `topology.kubernetes.io/zone`, которую Kubernetes обычно добавляет к узлам, предоставленным облачным провайдером (в нашем случае DigitalOcean).

Напоминание: вы можете увидеть метки своих узлов с помощью команды `kubecttl get nodes --show-labels`.

Есть и другие операторы, которые можно использовать с привязкой к узлам:

- In: цель `matchExpression` находится в наборе;
- NotIn: цель `matchExpression` отсутствует в наборе;
- Exists: метка существует;
- DoesNotExist: метка не существует;
- Gt: цель `matchExpression` больше;
- Lt: цель `matchExpression` меньше.

17.4.3. Антисходство узлов: пример

Операторы `DoesNotExist` и `NotIn` применяются для создания правил *анти-сходства* (anti-affinity).

Например, ранее мы использовали следующий код YAML:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:

```

```
nodeAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - default-f8wx9
```

Поскольку у нас есть следующие три узла:

- default-f72fj,
- default-f72fo,
- default-f8wx9,

– мы можем применить правило антисходства для достижения того же эффекта (но сначала удалим предыдущее правило):

```
kubectl delete -f affinity.yaml
```

Листинг 17.6 ❖ Файл anti-affinity.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: NotIn
                    values:
                      - default-f72fj
                      - default-f72fo
```

Чтобы узнать, где запущены поды, используйте следующую команду:

```
kubectl get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName | grep
nginx
```

Вы должны увидеть все поды, запущенные на узле с меткой `hostname: default-f8wx9`, и ни одного на других узлах.

17.4.4. Вес сходства

При создании правил сходства и антисходства доступны и другие параметры, такие как параметр `weight` (вес). Давайте равномерно распределим наши рабочие нагрузки по всем узлам в кластере, назначив каждому узлу свой вес.

Листинг 17.7 ❖ Файл `weight.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: In
                    values:
                      - default-f72fj
            - weight: 50
              preference:
                matchExpressions:
                  - key: kubernetes.io/hostname
```

```

      operator: In
      values:
      - default-f72fo
- weight: 25
  preference:
    matchExpressions:
    - key: kubernetes.io/hostname
      operator: In
      values:
      - default-f8wx9

```

Определены три предпочтения узлов, каждое с разным весом:

- первое предпочтение имеет вес 100 и указывает, что развертывание преимущественно планирует под на узле с меткой `kubernetes.io/hostname`, соответствующей `default-f72fj`;
- второе предпочтение имеет вес 50 и указывает, что развертывание преимущественно планирует под на узле с меткой `kubernetes.io/hostname`, соответствующей `default-f72fo`;
- третье предпочтение имеет вес 25 и указывает, что развертывание преимущественно планирует под на узле с меткой `kubernetes.io/hostname`, соответствующей `default-f8wx9`.

Другими словами, планировщик Kubernetes пытается запланировать под на узлах, соответствующих указанным предпочтениям, давая больший вес предпочтениям с более высокими значениями. Однако если предпочтительные узлы недоступны по какой-либо причине, планировщик все равно может запланировать под на других доступных узлах.

Чтобы увидеть, где запущены поды, примените развертывание.

```

kubectl apply -f weight.yaml
kubectl get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName --sort-by=.spec.nodeName | grep nginx

```

Чтобы увидеть, как планировщик планирует поды на узлах, можно масштабировать систему до 10, 20 или более реплик.

```

kubectl scale deployment nginx --replicas=20
kubectl get pods -o=custom-columns=PodName:.metadata.name,NodeName:.spec.nodeName --sort-by=.spec.nodeName | grep nginx

```

17.4.5. Типы сходства и антисходства

Существует два типа правил сходства и антисходства:

- `requiredDuringSchedulingIgnoredDuringExecution`;
- `preferredDuringSchedulingIgnoredDuringExecution`.

Мы уже использовали оба типа в предыдущих примерах. Теперь подробнее рассмотрим различия между ними.

- `requiredDuringSchedulingIgnoredDuringExecution`: под может быть запланирован только в том случае, если правило выполняется;
- `preferredDuringSchedulingIgnoredDuringExecution`: планировщик попытается найти узел, который соответствует правилу. Если узел, соответствующий правилу, недоступен, планировщик все равно запланирует под на другом узле.

В обоих случаях правило метки узла игнорируется во время выполнения, если под уже запущен на узле и метка узла меняется после этого. Под продолжит работать на узле, игнорируя новое правило. Вот почему в названиях есть суффикс `IgnoredDuringExecution`.

Глава 18

Kubernetes: управление обновлениями инфраструктуры и режим обслуживания

18.1. Зачем нам нужно обновлять инфраструктуру?

Есть несколько причин, по которым нам нужно обновлять инфраструктуру, но самая распространенная – это необходимость своевременно вносить в систему обновления безопасности. Кроме того, нам нужно обновлять инфраструктуру, чтобы воспользоваться новыми функциями и исправлениями ошибок.

В некоторых случаях нужная вам функция может быть недоступна в текущей версии Kubernetes. В этих случаях обновление инфраструктуры до последней версии необходимо для использования новой функции.

18.2. Что нужно обновлять?

В кластере есть несколько компонентов, потребность в обновлении которых зависит от вашей инфраструктуры. Например, вы можете обновить версию Kubernetes, операционную систему ваших узлов, среду выполнения контейнера или пул узлов.

Если вы используете управляемую службу Kubernetes, для обновления плоскости управления следуйте инструкциям, предоставленным вашим провайдером облачных услуг. Он сам позаботится об обновлении плоскости управления для вас.

Если вы владеете собственным кластером Kubernetes, то способ обновления плоскости управления зависит от инструмента, который вы используете для управления кластером. Например, если вы используете Kubeadm, вам нужно будет следовать определенным инструкциям¹.

Наиболее распространенным сценарием является обновление рабочих узлов в кластере. Но если вы используете управляемый сервис Kubernetes, то этим обновлением управляют поставщики облачных услуг. Тем не менее есть определенные ситуации, когда вам необходимо обновить рабочие узлы самостоятельно.

Прежде чем исследовать этот случай, давайте рассмотрим, как работает обновление рабочих узлов.

18.3. Обновление рабочих узлов

При обновлении рабочих узлов важно убедиться, что поды, работающие на этих узлах, не будут затронуты процессом обновления. Для этого необходимо выполнить *слив* (draining) узлов перед их обновлением.

Например, рассмотрим кластер с двумя узлами:

- узел 1: default-frp7j;
- узел 2: default-frp7r.

Для начала давайте рассмотрим поды, запущенные на узлах, чтобы абстрагироваться от следующих шагов.

```
export NODE1=default-frp7j
export NODE2=default-frp7r
```

Теперь создадим развертывание и масштабируем его до пяти реплик:

```
kubectrl create deployment hello-world --image=gcr.io/google-samples/hello-app:1.0
kubectrl scale deployment hello-world --replicas=5
```

Если вы хотите увидеть, какие поды запущены на каждом узле, выполните следующую команду:

```
kubectrl get pods -o wide
```

При наличии пяти реплик есть большая вероятность, что наши поды запущены на обоих узлах. Давайте посмотрим, какие поды запущены на каждом узле:

¹ <https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>.

```
kubectl get pods -o wide --field-selector spec.nodeName=NODE1
kubectl get pods -o wide --field-selector spec.nodeName=NODE2
```

Если вы видите, что поды запущены только на одном узле, вы можете масштабировать развертывание до 10 реплик и повторить тестирование. Продолжайте масштабировать развертывание, пока не обнаружите поды, запущенные на обоих узлах.

Предположим, вы хотите обновить NODE2. Для этого сначала необходимо слить узел:

```
kubectl drain $NODE2
```

Слив – это процесс вытеснения всех подов, запущенных на узле. Когда вы сливаете узел, Kubernetes начинает вытеснять все поды, запущенные на узле. Затем поды перепланируются на других доступных узлах в кластере, пока есть достаточно ресурсов. Помните об этом при сливе узлов.

Выполнение команды `kubectl drain $NODE2` обычно приводит к ошибке из-за DaemonSets, работающих на узле. Если вы пользуетесь услугами DigitalOcean, несколько сетевых компонентов работают как DaemonSets. Чтобы правильно слить узел, используйте флаг `-ignore-daemonsets`.

```
kubectl drain $NODE2 --ignore-daemonsets
```

Если вы сейчас проверите узел, то увидите, что на нем нет ни одного пода:

```
kubectl get pods -o wide --field-selector spec.nodeName=$NODE2
```

Теперь узел NODE2 готов к обновлению. Вы можете видеть, что узел находится в состоянии `SchedulingDisabled`. Этот процесс называется постановкой заграждения, или *кордона* (cordon).

```
kubectl get nodes
```

Это означает, что никакой под не будет запланирован на узле, пока вы не отмените кордон. Теперь вы можете безопасно обновить узел.

18.4 Обновление рабочих узлов: отмена кордона

После обновления узла вам необходимо отменить для него кордон. Это означает, что поды снова могут быть запланированы на узле.

```
kubectl uncordon $NODE2
```

Вы должны увидеть, что узел теперь находится в состоянии готовности.

```
kubectl get nodes
```

Поды будут снова запланированы на узле. Вы можете проверить это, выполнив следующую команду:

```
kubect! get pods -o wide --field-selector spec.nodeName=$NODE2
```

Если вы не видите запущенных подов на узле, попробуйте масштабировать развертывание до 10 реплик и проверьте еще раз.

```
kubect! scale deployment hello-world --replicas=10
kubect! get pods -o wide --field-selector spec.nodeName=$NODE2
```

18.5. Обновление пулов узлов

Независимо от того, используете ли вы управляемую службу Kubernetes или запускаете собственный кластер, вам может потребоваться обновить пулы узлов. Это часто необходимо, когда вы хотите изменить тип машины.

Пулы узлов (node pool) Kubernetes – это группы узлов, которые используют одну и ту же конфигурацию. Например, если вы подключены к DigitalOcean, вы можете создать пул узлов с тремя узлами типа s-1vcpu-2gb и еще один пул узлов с тремя узлами типа s-2vcpu-4gb.

Для обновления пула создайте новый пул узлов с новой конфигурацией и перенесите в него свою рабочую нагрузку. После завершения вы можете удалить старый пул узлов.

Предположим, мы создаем новый пул узлов, состоящий из трех узлов типа s-2vcpu-4gb.

```
doctl kubernetes cluster node-pool create <cluster-id|cluster-name> --size s-2vcpu-4gb
--count 3
```

Далее нам нужно перенести нашу рабочую нагрузку в новый пул узлов. Для этого слейте все узлы, привязанные к старому пулу.

```
kubect! drain $NODE1 --ignore-daemonsets
kubect! drain $NODE2 --ignore-daemonsets
```

Теперь вы можете удалить старый Node Pool.

18.6. Обновления с нулевым временем простоя: бюджеты сбоев пода

Когда вы сливаете узлы во время процесса обновления, важно сделать так, чтобы это не повлияло на вашу рабочую нагрузку. Для этого нужно убедиться, что у вас достаточно ресурсов для перепланирования ваших подов на других

узлах. В Kubernetes вы можете определить бюджет сбоев подов (pod disruption budget, PDB), чтобы ограничить количество подов реплицированного приложения, которые могут быть отключены одновременно из-за преднамеренных сбоев. PDB используются не только для обновлений, но и для других случаев, таких как сбой оборудования и паника ядра (kernel panic).

i PDB ограничивает количество подов реплицированного приложения, которые отключены одновременно из-за преднамеренных сбоев, таких как обновление. PDB используется не только для обновлений, но и для других случаев, таких как сбой оборудования, паника ядра и т. д. Иными словами, использование PDB является одним из лучших методов создания высокодоступных приложений и обеспечения нулевого времени простоя обновлений.

Допустим, вы уже разместили развертывание с пятью репликами. Вот как вы можете создать для него PDB:

```
kubectl create pdb hello-world --selector app=hello-world --min-available 3
```

Эта команда создает PDB для развертывания с меткой `app=hello-world`. Она гарантирует, что по крайней мере три пода всегда будут доступны.

Для проверки PDB выполните следующую команду:

```
kubectl get pdb hello-world
```

Если вы хотите использовать YAML, вы должны удалить предыдущий PDB перед применением нового:

```
kubectl delete pdb hello-world
```

Далее можно использовать следующий манифест:

```
kubectl apply -f - <<EOF
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: hello-world
spec:
  minAvailable: 3
  selector:
    matchLabels:
      app: hello-world
EOF
```

В приведенном выше манифесте мы создаем PDB с параметром `minAvailable`, которому присвоено значение 3. Это означает, что по крайней мере три пода будут доступны в любое время. Если вы предпочитаете использовать `maxUnavailable`, примените следующий манифест:

```
kubectl apply -f - <<EOF
apiVersion: policy/v1
```

```
kind: PodDisruptionBudget
metadata:
  name: hello-world
spec:
  maxUnavailable: 2
  selector:
    matchLabels:
      app: hello-world
EOF
```

Математика здесь проста:

- у нас есть пять реплик;
- мы хотим сделать так, чтобы по крайней мере три пода были доступны в любое время;
- следовательно, в любой момент времени у нас могут быть недоступны не более двух подов;
- вот почему мы используем либо `minAvailable: 3`, либо `maxUnavailable: 2`.

Глава 19

Стратегии развертывания микросервисов: управление обновлениями и развертывание приложений

19.1. Изначально облачная система: концепция

Концепция *изначально облачной* (cloud native) системы охватывает обширный перечень приемов и подходов, которые позволяют организациям создавать приложения и управлять их развертыванием одинаково эффективно вне зависимости от масштаба. Облачное приложение, как правило, состоит из нескольких микросервисов, которые упакованы в контейнеры и развернуты как слабосвязанные компоненты. Эти компоненты управляются и организуются с помощью декларативных API.

Понятие облачного подхода намного шире простого проектирования и внедрения приложений. В него входят проектирование и внедрение инфраструктуры, а также стратегии развертывания, используемые для управления жизненным циклом приложения.

Благодаря облачным технологиям и с помощью методик DevOps и DevSecOps организации могут поставлять программные продукты и их обновления быстрее и чаще. Они также могут адаптироваться к быстро меняющейся об-

лачной среде и рынку. Но для достижения этих преимуществ организациям необходимо выбрать правильные стратегии развертывания. Эти стратегии позволят им управлять жизненным циклом приложения в соответствии с их бизнес-целями.

В этой главе будут рассмотрены наиболее распространенные стратегии развертывания.

19.2. Стратегии развертывания

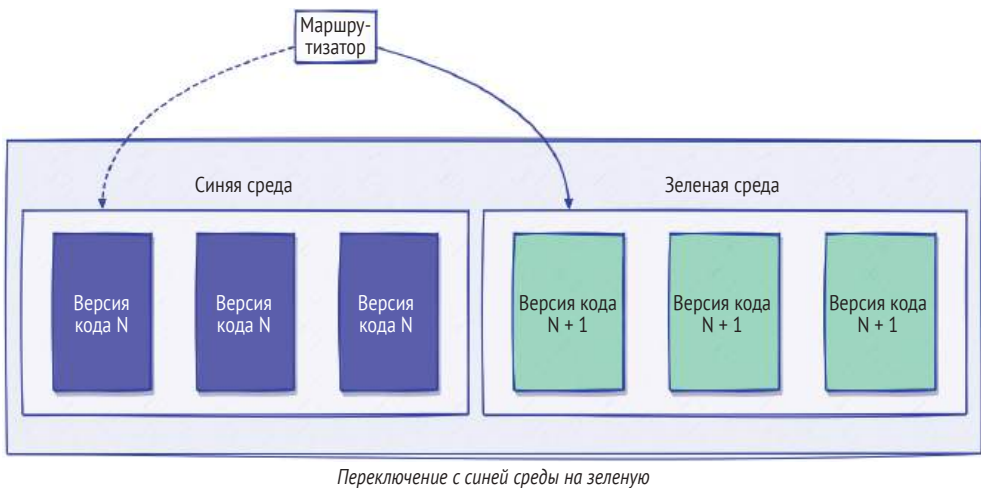
19.2.1. Сине-зеленое развертывание: введение

Стратегия *сине-зеленого развертывания* (blue-green deployment) подразумевает развертывание новой («зеленой») версии приложения при сохранении предыдущей («синей») версии в рабочем состоянии.

Согласно этой стратегии, перед обновлением выстраивается специальная маршрутизация трафика, позволяющая синей среде бесперебойно обслуживать конечных пользователей, пока вы тестируете и развертываете обновления в зеленой среде.

После успешного развертывания и тестирования вы можете перенаправить трафик на зеленую среду без каких-либо заметных изменений для ваших пользователей.

Если в ходе развертывания или рабочего тестирования будут обнаружены какие-либо проблемы, можно легко вернуться к синей среде.



Допустим, у вас есть синяя среда, которая уже находится в рабочем состоянии.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: blue-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: blue-app
  template:
    metadata:
      labels:
        app: blue-app
    spec:
      containers:
        - name: app-container
          image: myregistry/blue-app:1.0
```

Чтобы развернуть новую версию приложения, необходимо реплицировать производственную (синюю) среду для создания зеленой среды.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: green-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: green-app
  template:
    metadata:
      labels:
        app: green-app
    spec:
      containers:
        - name: app-container
          image: myregistry/green-app:1.0
```

После того как в зеленой среде появится новая версия приложения, протестируйте ее. Если тестирование пройдет успешно, перенаправьте рабочий трафик в зеленую среду. Если обнаружатся какие-либо проблемы, вернитесь к синей версии.

Реализацию сине-зеленой стратегии можно упростить с помощью дополнительных инструментов. Вот несколько примеров:

- **Istio:** сервисная сетка (service mesh), которая обеспечивает функции наблюдения, безопасности и управления для ускорения циклов развертывания. Istio – популярный выбор для реализации сине-зеленой стратегии;

- **Spinnaker**: открытая мультиоблачная платформа непрерывной поставки для выпуска изменений программного обеспечения;
- **Jenkins**: сервер автоматизации с открытым исходным кодом, поддерживающий сине-зеленую стратегию;
- **AWS CodeDeploy**: полностью управляемый сервис развертывания, который автоматизирует развертывание программного обеспечения на различных вычислительных платформах, таких как Amazon EC2, AWS Fargate, AWS Lambda, и локальных серверах;
- **Google Cloud Deploy**: сервис развертывания инфраструктуры, который автоматизирует создание и управление ресурсами Google Cloud.

Сине-зеленая стратегия очень часто применяется в различных организациях. Она обеспечивает развертывание без простоев, безопасные откаты, тестирование в производстве и наилучшую изоляцию. Однако поддержание двух идентичных производственных сред может быть дорогостоящим. Более того, развертывание новой версии приложения может занять много времени.

Поэтому сине-зеленые развертывания не подходят для приложений, требующих сложных или масштабных миграций данных.

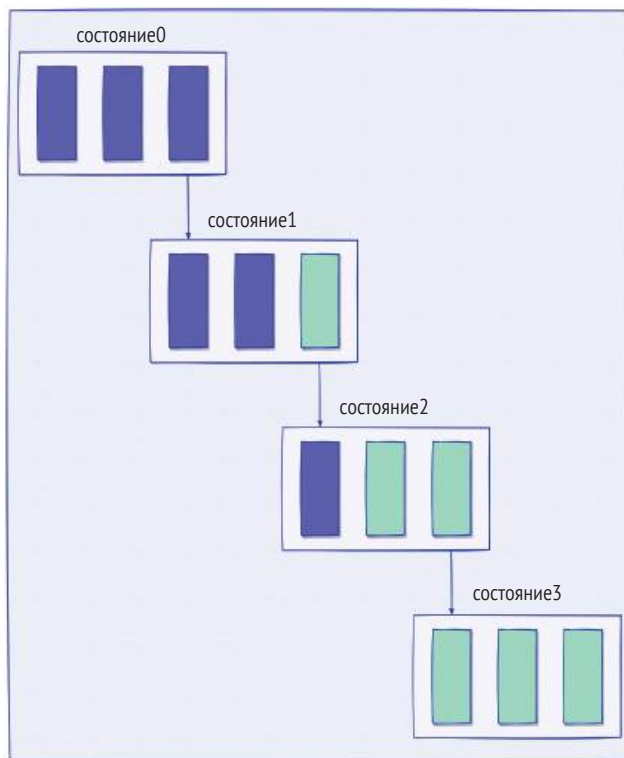
19.2.2. Канареечное развертывание: введение

Канареечная стратегия подразумевает развертывание новой версии приложения для небольшой группы пользователей (обычно 5–10 %) и последующего тестирования в реальных условиях с минимальным ущербом для большинства пользователей. Если новая версия окажется стабильной, ее можно будет развернуть для всех пользователей. Если обнаружатся какие-либо проблемы, предыдущую версию можно будет восстановить, не затрагивая всю базу пользователей.

Канареечное развертывание работает аналогично сине-зеленому, но следует немного иному подходу. Вместо того чтобы иметь вторую полноценную среду, ожидающую переключения трафика после завершения развертывания, обновления сначала развертывают на небольшой группе серверов или узлов и только в случае успеха продолжают развертывание на оставшихся ресурсах.

Канареечная стратегия, в отличие от сине-зеленой, позволяет проводить контролируемое тестирование в рабочей среде, т. е. без создания новой тестовой среды. Благодаря этому удается быстрее развертывать новую версию приложения и снижать затраты на обслуживание. Она обеспечивает развертывание без простоев и позволяет выполнять простой откат. Однако она имеет ту же сложность, что и сине-зеленая стратегия, особенно когда речь идет об управлении сложными миграциями данных.

Канареечное развертывание можно выполнять вручную или с помощью дополнительных инструментов, таких как Istio, Spinnaker, Jenkins, AWS CodeDeploy и Google Cloud Deploy.



*Канареечное развертывание:
частичный выпуск в подмножестве пользователей для тестирования*

19.2.3. Канареечное развертывание: пример использования Istio

В качестве примера выполним канареечное развертывание с помощью Istio.

Сначала нам нужно установить Istio. Чтобы загрузить версию 1.17.2 Istio для архитектуры x86_64, используйте следующую команду (вы можете использовать команду `uname -m`, чтобы получить сведения об архитектуре вашего компьютера):

```
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.17.2 TARGET_ARCH=x86_64 sh -
```

Далее зададим переменной среды `PATH` путь к подкаталогу `bin` каталога установки Istio:

```
cd istio-1.17.2
export PATH=$PWD/bin:$PATH
```

Выполните предварительную проверку вашей среды с помощью следующей команды:

```
istioctl x precheck
```

Продолжите установку Istio, выполнив следующую команду:

```
istioctl install --set profile=demo -y
```

Здесь мы используем профиль `demo`, который является хорошей отправной точкой для большинства развертываний. Вы можете найти больше информации о различных профилях в документации Istio¹.

Последняя команда устанавливает Istio и создает необходимые ресурсы в кластере Kubernetes, включая шлюз Istio Ingress. В результате нам больше не нужен контроллер NGINX Ingress, который был установлен в предыдущей главе. Удалите его:

```
helm delete nginx-ingress
```

Чтобы включить автоматическое внедрение `sidecar`-прокси Envoy при последующем развертывании приложения, добавьте метку пространства имен для указания Istio:

```
kubectl label namespace default istio-injection=enabled
```



Sidecar-контейнеры в Kubernetes – это дополнительные контейнеры, которые запускаются вместе с основным контейнером в поде². Они предназначены для улучшения и расширения функциональности основного контейнера без изменения его кодовой базы.

Sidecar-контейнеры совместно с основным контейнером используют такие ресурсы, как хранилище и сетевые интерфейсы, что позволяет им работать вместе без проблем. Их можно разрабатывать на разных языках, и они обеспечивают повышенную гибкость.

К распространенным вариантам использования `sidecar`-контейнеров относятся совместное использование хранилища или сетей, более эффективное выполнение функций на другом языке программирования и интеграция приложений для ведения журналов и мониторинга, повышающих наблюдаемость и производительность.

В контексте Istio приставка `sidecar`- обозначает дополнительный контейнер, который внедряется в под вместе с основным контейнером приложения. Sidecar-контейнер Istio отвечает за перехват сетевого трафика в основной контейнер и из него. Он обеспечивает расширенные функции сервисной сетки, такие как управление трафиком, безопасность и наблюдаемость. Действуя как прокси, он реализует функциональность плоскости данных Istio.

¹ <https://istio.io/latest/docs/setup/additional-setup/config-profiles/>.

² Sidecar – прицепная боковая люлька у мотоцикла. – *Прим. перев.*

Sidecar-контейнер автоматически внедряется sidecar-инжектором Istio, который изменяет спецификацию пода, чтобы добавить сторонний контейнер. Эта архитектура позволяет Istio предоставлять мощные возможности, такие как маршрутизация трафика, балансировка нагрузки, разрыв цепи и телеметрия, без внесения изменений в код приложения.

Давайте создадим новое рабочее пространство и развернем наше тестовое приложение:

```
cd $HOME && mkdir canary && cd canary
```

Идея состоит в том, чтобы развернуть две версии приложения: v1 и v2. Istio направит 90 % трафика в v1 и 10 % в v2. Новая версия будет протестирована, и если все в порядке, 100 % трафика будет направлено в v2.

Вот первая версия приложения:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloapp-v1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloapp
      version: v1
  template:
    metadata:
      labels:
        app: helloapp
        version: v1
    spec:
      containers:
        - name: hello-app-v1
          image: gcr.io/google-samples/hello-app:1.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
```

Вот вторая версия приложения:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloapp-v2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloapp
      version: v2
  template:
    metadata:
```

```

labels:
  app: helloapp
  version: v2
spec:
  containers:
    - name: hello-app-v2
      image: gcr.io/google-samples/hello-app:2.0
      imagePullPolicy: Always
      ports:
        - containerPort: 8080

```

Мы будем использовать следующий сервис для публикации приложения в сети:

```

apiVersion: v1
kind: Service
metadata:
  name: helloapp
spec:
  selector:
    app: helloapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

```

До этого момента ничего не изменилось по сравнению с тем, что мы делали в предыдущей главе. Теперь у нас просто есть две версии приложения, развернутые и опубликованные в сети через сервис. Если бы у нас работал Ingress Controller, трафик приложения распределялся бы между версиями без какого-либо предпочтения.

Теперь перейдем к следующему шагу и создадим ресурс Istio Gateway. Но сначала нам нужно получить IP-адрес шлюза Istio Ingress:

```

export INGRESS_IP=$(kubectl get svc -n istio-system istio-ingressgateway -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

```

Теперь мы можем использовать следующий манифест YAML:

```

apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: istio-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - "app.${INGRESS_IP}.nip.io"

```

i Gateway (шлюз) – это пользовательский ресурс Istio.

Причина, по которой нам нужно добавить ресурс шлюза в Istio, заключается в определении правил маршрутизации для входящего трафика в сервисную сетку Istio. Хотя у нас уже установлен шлюз Istio Ingress Gateway, он действует как точка входа по умолчанию для внешнего трафика. Однако без явного определения ресурса шлюза Ingress не будет знать, как обрабатывать входящий трафик и направлять его соответствующим сервисам.

Другими словами, Gateway – это расширение Ingress, которое обеспечивает больший контроль и детализацию при настройке правил маршрутизации, а также других функций.

Далее нам нужно создать ресурс `VirtualService`, чтобы определить правила маршрутизации для приложения:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: helloapp
spec:
  hosts:
    - "app.${INGRESS_IP}.nip.io"
  gateways:
    - istio-gateway
  http:
    - route:
        - destination:
            host: helloapp
            subset: v1
            weight: 90
        - destination:
            host: helloapp
            subset: v2
            weight: 10
```

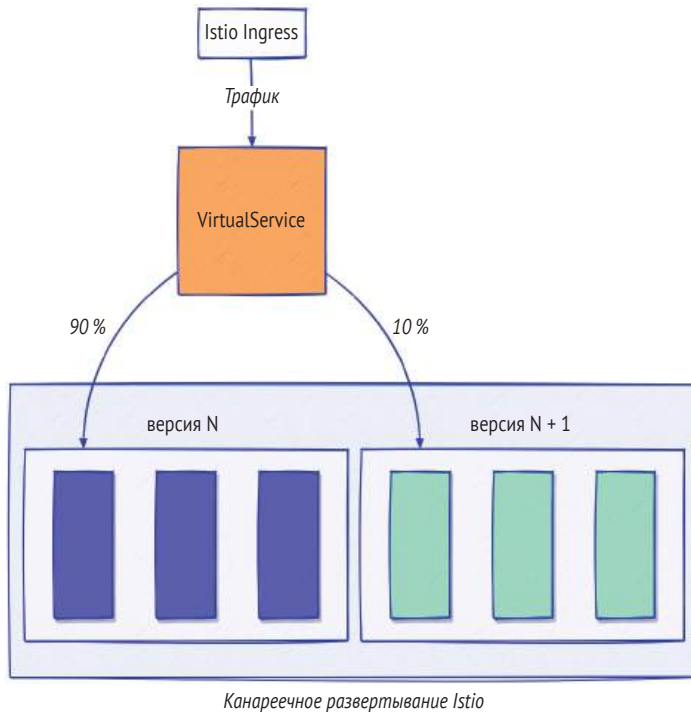
Приведенный выше манифест YAML определяет `VirtualService` в Istio, который применяется для настройки маршрутизации трафика и правил обработки запросов для определенных хостов.

В данном случае `VirtualService` называется `helloapp` и настроен на обработку трафика для хоста `app.${INGRESS_IP}.nip.io`.

`VirtualService` связан со шлюзом `istio-gateway`, следовательно, он будет применяться к трафику, полученному через этот шлюз (который мы уже создали на предыдущем шаге).

Для трафика HTTP `VirtualService` определяет два маршрута с помощью поля `route`:

- первый маршрут направляет 90 % трафика в подмножество `v1` сервиса `helloapp`;
- второй маршрут направляет 10 % трафика в подмножество `v2` того же сервиса `helloapp`.



После определения VirtualService следующим шагом будет создание ресурса DestinationRule. В нем мы определим подмножества приложения:

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: helloapp
spec:
  host: helloapp
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```



DestinationRule в Istio используется для определения политик и настроек маршрутизации трафика в определенные подмножества сервиса.

Показанный выше манифест YAML определяет DestinationRule с именем helloapp. Хост helloapp указан в качестве точки назначения для этого правила.

DestinationRule определяет два подмножества: v1 и v2. Каждое подмножество идентифицируется по имени и снабжено метками, которые соответствуют меткам, определенным в соответствующем VirtualService.

Назначением `DestinationRule` в сочетании с `VirtualService` является обеспечение детального контроля маршрутизации трафика и балансировки нагрузки для различных подмножеств сервиса. В частности, наши подмножества называются `v1` и `v2`. Другими словами, `VirtualService` направляет трафик на основе определенных весов, в то время как `DestinationRule` предоставляет дополнительную конфигурацию для каждого подмножества.

Все приведенные выше манифесты можно объединить в один файл `YAML`:

Листинг 19.1 ❖ Файл `app.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloapp-v1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloapp
      version: v1
  template:
    metadata:
      labels:
        app: helloapp
        version: v1
    spec:
      containers:
        - name: hello-app-v1
          image: gcr.io/google-samples/hello-app:1.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloapp-v2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloapp
      version: v2
  template:
    metadata:
      labels:
        app: helloapp
        version: v2
    spec:
      containers:
        - name: hello-app-v2
          image: gcr.io/google-samples/hello-app:2.0
```

```

        imagePullPolicy: Always
        ports:
          - containerPort: 8080
    ---
apiVersion: v1
kind: Service
metadata:
  name: helloapp
spec:
  selector:
    app: helloapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
    ---
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: istio-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - "app.${INGRESS_IP}.nip.io"
    ---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: helloapp
spec:
  hosts:
    - "app.${INGRESS_IP}.nip.io"
  gateways:
    - istio-gateway
  http:
    - route:
        - destination:
            host: helloapp
            subset: v1
            weight: 90
        - destination:
            host: helloapp
            subset: v2
            weight: 10
    ---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule

```

```
metadata:
  name: helloapp
spec:
  host: helloapp
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

Примените манифест:

```
kubectl apply -f app.yaml
```

Чтобы проверить, как трафик направляется в различные подмножества, воспользуйтесь командой `curl`:

```
while true; do curl -s http://app.${INGRESS_IP}.nip.io ; sleep 1; echo; done
```

Вы должны увидеть вывод приложения с номером версии, меняющимся между `v1` и `v2`:

```
Hello, world!
Version: 1.0.0
Hostname: helloapp-v1-<pod-id>
```

```
Hello, world!
Version: 2.0.0
Hostname: helloapp-v2-<pod-id>
```

Поскольку мы назначили разные веса двум подмножествам, вывод приложения должен состоять в основном из `v1` с небольшой примесью `v2`.

19.2.4. Канареечное развертывание: тестирование в рабочей среде

Можно выполнить тестирование в рабочей среде, используя правило маршрутизации на основе префикса. В следующем примере только запросы с заголовком `internal-testing: True` будут направлены в подмножество `v2`, а все остальные запросы будут направлены в подмножество `v1`.

```
kubectl apply -f - <<EOF
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: helloapp
spec:
  hosts:
```

```

- "app.${INGRESS_IP}.nip.io"
gateways:
- istio-gateway
http:
- match:
  - headers:
    internal-testing:
      exact: "True"
  route:
  - destination:
    host: helloapp
    subset: v2
- route:
  - destination:
    host: helloapp
    subset: v1

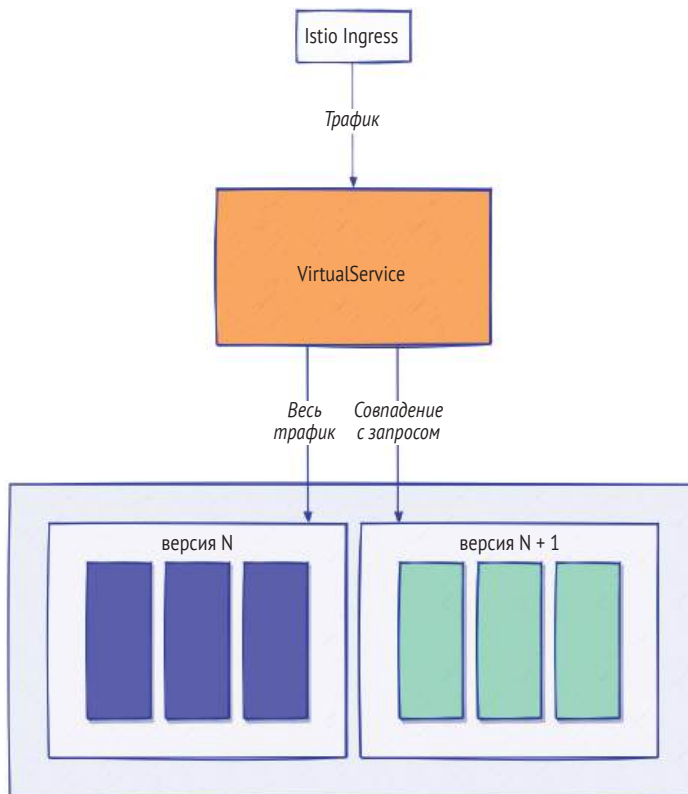
```

Теперь вы можете выполнять тестирование с заголовком `internal-testing: True` и без него:

```

curl -s -H "internal-testing: True" http://app.${INGRESS_IP}.nip.io
curl -s http://app.${INGRESS_IP}.nip.io

```



Канареечное развертывание Istio

Больше правил соответствия можно найти в документации Istio¹.

19.3. Скользящее обновление

Скользящее обновление (rolling update) – это широко используемая стратегия развертывания, которая подразумевает постепенную замену экземпляров старой версии приложения экземплярами новой версии. Обычно она используется для развертывания приложений в среде Kubernetes.

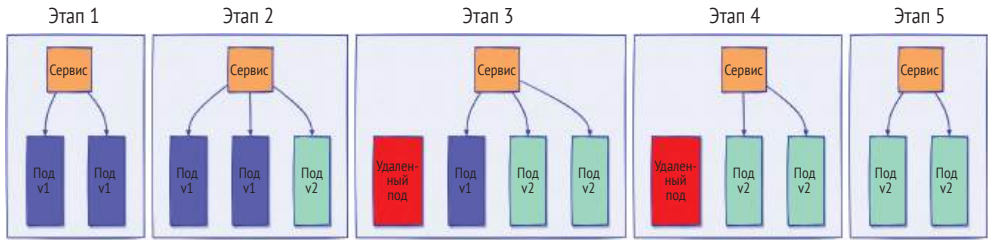
В Kubernetes для реализации стратегии скользящего обновления применяется ресурс развертывания (Deployment). Этот ресурс определяет желаемое состояние приложения, в то время как ReplicaSet поддерживает фактическое состояние приложения.

С помощью стратегии скользящего обновления экземпляры приложения обновляются до новой версии по одному – с минимальным ущербом для общей доступности приложения даже в случае сбоя.

Этапы скользящего обновления выглядят следующим образом:

1. Развертывание начинается с создания экземпляров обновленной версии наряду с существующими экземплярами предыдущей версии.
2. Балансировщик нагрузки или маршрутизатор направляет часть входящего трафика на новые экземпляры, продолжая при этом пересылать часть трафика на существующие экземпляры.
3. Kubernetes отслеживает работоспособность и производительность новых экземпляров, чтобы убедиться, что они работают правильно.
4. Если новые экземпляры работают хорошо и соответствуют желаемым критериям, процесс развертывания постепенно перенаправляет больше трафика на новые экземпляры и уменьшает трафик на старые экземпляры.
5. По мере того как все больше трафика направляется на новые экземпляры, Kubernetes отслеживает возникающие проблемы или ошибки и автоматически откатывается к предыдущей версии, если это необходимо.
6. Этот процесс продолжается до тех пор, пока все экземпляры не будут обновлены до новой версии.
7. После завершения развертывания старые экземпляры удаляются.
8. Развертывание завершено, и теперь работает новая версия. Kubernetes продолжает отслеживать работоспособность и производительность новых экземпляров, чтобы убедиться, что они работают правильно.

¹ <https://istio.io/latest/docs/reference/config/networking/virtual-service/#HTTPMatchRequest>.



В следующем разделе мы развернем новую версию приложения и выполним скользящее обновление.

19.3.1. Скользящие обновления: пример

Для начала удалим Istio из кластера:

```
istioctl uninstall --purge
kubectl delete namespace istio-system
```

Затем установим NGINX Ingress Controller с помощью Helm:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install nginx-ingress ingress-nginx/ingress-nginx --set controller.publishService.enabled=true
```

Создадим новый рабочий каталог:

```
cd $HOME && mkdir -p rollingupdate && cd rollingupdate
```

Убедитесь, что внешний IP Ingress больше не находится в состоянии pending, с помощью команды `kubectl get svc nginx-ingress-nginx-controller`.

Получив внешний IP, мы можем экспортировать его как переменную среды:

```
export INGRESS_IP=$(kubectl get svc nginx-ingress-nginx-controller -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

Затем развернем первую версию приложения, службу Kubernetes и ресурс Ingress:

Листинг 19.2 ❖ Файл app-2.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-app
spec:
  replicas: 3
  selector:
```

```
  matchLabels:
    app: hello-app
template:
  metadata:
    labels:
      app: hello-app
  spec:
    containers:
      - image: gcr.io/google-samples/hello-app:1.0
        imagePullPolicy: Always
        name: hello-app
        ports:
          - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: hello-app
spec:
  selector:
    app: hello-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-app
spec:
  rules:
    - host: app.${INGRESS_IP}.nip.io
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-app
                port:
                  number: 80
  ingressClassName: nginx
```

Чтобы выпустить новую версию (`gcr.io/google-samples/hello-app:2.0`) и пройти весь процесс обновления, необходимо сначала добавить стратегию `RollingUpdate` в манифест развертывания:

```
strategy:
  type: RollingUpdate
rollingUpdate:
  maxSurge: 1
  maxUnavailable: 0
```

Значение параметра `maxSurge`: 1 позволяет развернуть новую версию приложения, постепенно заменяя экземпляры старой версии экземплярами новой версии.

При `maxSurge`: 2 можно развернуть новую версию приложения, постепенно заменяя экземпляры старой версии экземплярами новой версии, одновременно развернув два новых экземпляра.

При `maxUnavailable`: 0 количество недоступных подов во время развертывания не должно превышать 0. Это гарантирует, что развертывание не повлияет на функциональность приложения.

Но при `maxUnavailable`: 1 количество недоступных подов во время развертывания может быть равно 1. Это означает, что функциональность приложения может быть прервана во время развертывания.

Вместо чисел также можно использовать проценты. Например, `maxSurge`: 50% означает, что может быть развернуто 50 % новых экземпляров одновременно.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 50%
```

Чтобы мы могли узнать, что под готов к работе, необходимо добавить в контейнер зонд `readinessProbe`. Как только контейнер будет готов, Kubernetes посчитает соответствующий под готовым и начнет отправлять ему трафик.

```
readinessProbe:
  httpGet:
    path: /
    port: 8080
    scheme: HTTP
  initialDelaySeconds: 20
  timeoutSeconds: 10
  periodSeconds: 10
```

В приведенном выше примере мы указываем Kubernetes проверять путь «/» на порту 8080 каждые 10 с. Первая проверка будет выполнена через 20 с. Если проверка не пройдена, Kubernetes будет ждать 10 с перед повторной проверкой.

Вот как это работает:

- `readinessProbe` ждет `initialDelaySeconds` перед началом проверки готовности пода;
- если операция превышает `timeoutSeconds`, под считается неготовым;
- если проверка прошла успешно, под считается готовым;
- после `successThreshold` успешных проверок под считается готовым;
- если проверка не пройдена `failureThreshold` раз, под считается неготовым;
- эта проверка повторяется каждые `periodSeconds` секунд.

В результате у нас получается новый манифест развертывания:

Листинг 19.3 ❖ Файл v1.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-app
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: hello-app
    spec:
      containers:
        - image: gcr.io/google-samples/hello-app:1.0
          imagePullPolicy: Always
          name: hello-app
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /
              port: 8080
              scheme: HTTP
            initialDelaySeconds: 1
            timeoutSeconds: 2
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: hello-app
  namespace: default
spec:
  selector:
    app: hello-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-app
spec:
  rules:
  - host: app.${INGRESS_IP}.nip.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: hello-app
            port:
              number: 80
  ingressClassName: nginx
```

Применяем манифест:

```
kubectl apply -f v1.yaml
```

Теперь обновим образ приложения до версии 2.0:

```
kubectl set image deployment/hello-app hello-app=gcr.io/google-samples/hello-app:2.0
```

Понаблюдайте, как обновляются поды:

```
kubectl get pods -w
```

Проследите за ходом скользящего обновления, когда старые поды постепенно исчезают, освобождая место для новых:

```
kubectl rollout status deployment/hello-app
```

Вы также можете проверить историю развертывания:

```
kubectl rollout history deployment hello-app
```

Скучаете по старым временам? Путешествия во времени возможны! Вы можете вернуться к предыдущей версии:

```
kubectl rollout undo deployment hello-app
```

Если нужно откатиться к определенной версии, добавьте флаг `--to-revision`:

```
kubectl rollout undo deployment hello-app --to-revision=1
```

Вы также можете приостановить и возобновить развертывание:

```
kubectl rollout pause deployment hello-app
kubectl rollout resume deployment hello-app
```

Если вы хотите добавить причину изменения в историю развертывания, воспользуйтесь командой

```
kubectl patch deployment hello-app -p '{"spec":{"template":{"metadata":{"annotations":{"kubernetes.io/change-cause":"Назрело большое обновление!"}}}}}'
```

Проверьте историю развертывания еще раз, чтобы увидеть причину изменения:

```
kubectl rollout history deployment hello-app
```

Глава 20

Наблюдаемость микросервисов в мире Kubernetes

20.1. Введение

В последние годы микросервисные и распределенные архитектуры стали популярными моделями создания и обслуживания приложений. Хотя этот подход облегчает масштабирование, он также приводит к росту сложности и динамичности в системе. Активное взаимодействие между сервисами в микросервисной архитектуре затрудняет понимание, выявление и устранение ненормального поведения в средах. Для решения этих проблем были предложены такие концепции, как мониторинг и наблюдаемость.

20.2. Что такое мониторинг?

Мониторингом в ИТ называют периодическое отслеживание производительности программного обеспечения и инфраструктуры путем систематического сбора, анализа и применения данных из системы. Цель мониторинга – определить, насколько хорошо ваше программное обеспечение и базовая инфраструктура работают в режиме реального времени, чтобы убедиться, что показатели соответствуют ожиданиям. Мониторинг ИТ-сред подразумевает использование комбинации инструментов и технологий как для определения производительности инфраструктуры, так и для устранения выявленных проблем. Хотя мониторинг широко применяется среди разработчиков, инженеров DevOps и ИТ-специалистов, ему на смену все чаще приходит наблюдаемость.

20.3. Что такое наблюдаемость?

Наблюдаемость (observability) – это обширный набор технологий, охватывающий различные аспекты мониторинга и анализа системы. По определению, наличие наблюдаемости означает возможность наблюдать за системой, понимать, что происходит внутри, и принимать обоснованные решения на основе этого понимания.

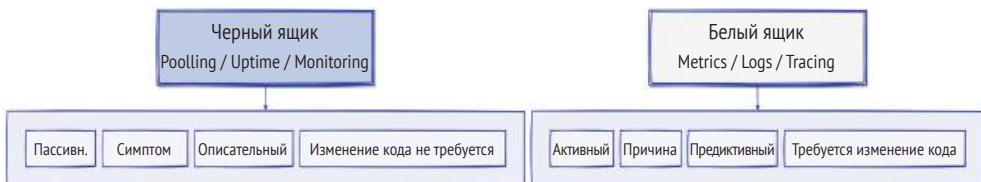
Термин «наблюдаемость» возник в XX веке, но впервые был использован в IT инженерной командой Twitter в 2013 г. в блоге, описывающем, как они контролируют свои системы. С тех пор он быстро приобрел популярность в индустрии программного обеспечения (источники: Encyclopedia, блог Twitter)^{1,2}.

Впоследствии другие компании, такие как Google, Uber и Netflix, внедрили концепцию Site Reliability Engineering (SRE, обеспечение надежности информационных систем), особенно востребованную в области микросервисов и распределенных систем. Инженер Google Роб Эвазук также внес свой вклад в эту работу в книге SRE Book³.

20.4. Мониторинг: белый ящик или черный?

Мониторинг белого ящика подразумевает наблюдение за внутренними показателями приложений, работающих на сервере. Мониторинг этого типа отслеживает количество HTTP-запросов к серверу, MySQL-запросов, запущенных на сервере базы данных, и аналогичные показатели.

Мониторинг черного ящика, в свою очередь, означает наблюдение за сервером и ресурсами сервера, такими как использование ЦП, памяти и средние значения нагрузки. В современных IT-компаниях разработчики приложений, инженеры DevOps и SRE распределяют задачи мониторинга между двумя подходами.



Сравнение мониторинга по типу черного и белого ящиков

¹ <https://www.encyclopedia.com/history/encyclopedias-almanacs-transcripts-and-maps/kalman-rudolf-emil?ref=fau>.

² https://blog.twitter.com/engineering/en_us/a/2013/observability-at-twitter.

³ <https://sre.google/sre-book/monitoring-distributed-systems/>.

Оба типа мониторинга полезны для получения информации о приложении и базовой инфраструктуре и обеспечения оптимальной производительности.

Наблюдаемость не обходится без мониторинга белого ящика. Этот тип мониторинга ориентирован на журналы, метрики и отслеживание внешних событий в системе, которые лежат в основе наблюдаемости.

20.5. Основные компоненты наблюдаемости

Чтобы понять, почему распределенная система, состоящая из различных микросервисов, выдает ошибку или ведет себя определенным нежелательным образом, и добиться наблюдаемости, вам необходимо собрать и изучить данные телеметрии из каждой службы в системе.

Журналы, метрики и трассировка – три типа телеметрических данных, которые обычно называют основными компонентами наблюдаемости.

20.5.1. Журналы

Журналы – это структурированные или неструктурированные сообщения, которые генерирует система при запуске определенного кода. Они представляют исчерпывающие записи, подробно описывающие событие, например ошибку, ее местоположение, время и причину ее возникновения. В среде микросервисов журналы особенно полезны для раскрытия подробностей неизвестных сбоев или непредвиденного поведения.

Анализ данных журнала имеет решающее значение для достижения наблюдаемости. Изучая данные журнала, вы можете устранять неполадки системы, связанные с местоположением, временем и конкретной причиной.

20.5.2. Метрики

Метрики – это групповые значения, которые представляют совокупную производительность системы за определенный период времени. В отличие от журналов, метрики формируют целостное отображение событий и производительности системы.

Вы можете собирать такие метрики, как время безотказной работы системы, количество запросов, время отклика, частота отказов, использование памяти, и другие показатели использования ресурсов с течением времени. Инженеры обычно используют метрики для запуска оповещений или определенных действий, когда значение метрики становится выше или ниже указанного порогового значения.

Метрики также легко сопоставлять между несколькими системами, что позволяет наблюдать тенденции и выявлять проблемы в системе.

20.5.3. Трассировка

Трассировка означает отслеживание первоисточника неисправностей, особенно в распределенных системах. Трассировка записывает путь запроса или действия по мере его перемещения от одного сервиса к другому в микросервисной архитектуре. Это позволяет специалистам выявлять узкие места системы и быстрее решать проблемы.

Трассировка особенно полезна при отладке сложных приложений, поскольку она позволяет понять путь запроса от его начальной точки и определить сервис, послуживший причиной неисправности в микросервисной архитектуре. Хотя первые два компонента, журналы и метрики, предоставляют адекватную информацию о поведении и производительности системы, трассировка расширяет эту информацию, предоставляя полезное представление о жизненном цикле запросов в системе.

20.5.4. Компоненты наблюдаемости на практике

Опираясь на упомянутые выше фрагменты данных, вы можете оценить или определить текущее состояние ИТ-системы без дальнейшего исследования, что делает систему наблюдаемой.

Однако сами по себе компоненты наблюдаемости не являются достаточно действенными.

Команда инженеров Twitter выделила четыре практических шага по реализации наблюдаемости в системе:

- **сбор**: объединение данных телеметрии, журналов, метрик и трассировки с их уникальными идентификаторами и временными метками из различных конечных точек в системе;
- **хранение**: размещение собранных данных в базе данных, отвечающей за фильтрацию, проверку, индексацию и надежное хранение данных для будущего использования;
- **запрос**: извлечение соответствующей информации из системы хранения для использования собранных и сохраненных данных;
- **визуализация**: демонстрация сохраненных данных путем запроса и отображения их в диаграммах и панелях мониторинга для анализа. Анализируя визуализированные данные телеметрии, вы можете достичь наблюдаемости в любой системе.

«Хотя сбор и хранение данных важны, они бесполезны для наших инженеров, если они не визуализированы таким образом, чтобы можно было немедленно рассказать релевантную историю». – Команда наблюдаемости Twitter.

20.6. Четыре золотых сигнала мониторинга

Мониторинг подразумевает отслеживание и запись различных метрик из среды выполнения. Согласно методологии Google¹, можно выделить четыре золотых сигнала мониторинга:

- задержка;
- трафик;
- ошибки;
- насыщенность.

При правильном измерении эти ключевые показатели помогают достичь оптимальной производительности системы.

20.6.1. Задержка

Задержка (latency) – это время, необходимое системе для ответа на запрос. Также важно разделять задержку неудачных и успешных запросов, так как это может помочь в более точной диагностике сбоев.

20.6.2. Трафик

Трафик (traffic) – это мера количества запросов на обслуживание, отправленных в систему за определенный период времени. Характер этих запросов может различаться в зависимости от типа услуги, предоставляемой системой. Например, трафик для веб-службы измеряется в HTTP-запросах в секунду, в то время как для системы потоковой передачи аудио он измеряется скоростью сетевого ввода-вывода или количеством одновременных сеансов. В базе данных или системе хранения «ключ–значение» он измеряется в количестве транзакций или извлечений в секунду. Измерение трафика важно, поскольку оно помогает вам понять рабочую нагрузку на вашу систему.

20.6.3. Ошибки

Механизм регистрации контекста *ошибок* (error), с которыми сталкиваются пользователи, применяемый вместо простой фильтрации ошибок, помогает улучшить систему. Для этого отслеживайте частоту невыполненных запросов и сохраняйте тип запроса и задержку.

¹ <https://sre.google/sre-book/monitoring-distributed-systems/>.

20.6.4. Насыщенность

Насыщенность (saturation) – это мера того, насколько «заполнена» система. Система считается насыщенной, когда ее базовые ресурсы, такие как память, ввод–вывод и ЦП, не могут обрабатывать дальнейшие запросы. Это ценный показатель, поскольку он позволяет вам определить предельный объем трафика или рабочей нагрузки, который может обработать ваша система. Он также помогает предсказать использование ресурсов с течением времени.

20.7. Мониторинг и наблюдаемость: в чем разница?

Мониторинг является функциональным предшественником наблюдаемости. В то время как мониторинг занимается сбором данных, наблюдаемость охватывает сбор, хранение, запрос и визуализацию этих данных. Наблюдаемость облегчает профессионалам понимание причин поведения системы.

Характер собираемых данных также различается. Данные мониторинга обычно ограничиваются определенными локальными метриками, в то время как данные наблюдения предназначены для более глубокого понимания поведения системы.

Мониторинг предоставляет информацию о *наличии* проблем или сбоев в вашей системе, в то время как наблюдаемость помогает вам понять, *что* вызвало сбой, *где* он произошел и *почему*. Система, которая не имеет мониторинга, не может быть наблюдаемой.

Глава 21

.....

Средства наблюдения: Prometheus, Grafana, Loki, Promtail, OpenTelemetry и Jaeger

21.1. Что такое Prometheus?

Prometheus¹ – это система мониторинга и оповещения с открытым исходным кодом, которая записывает метрики в реальном времени в базу данных временных рядов. Разработку начала компания SoundCloud² в 2012 г., проект получил статус открытого исходного кода в 2015 г.

Prometheus – популярный инструмент для мониторинга Kubernetes, микросервисов, контейнерных приложений и других рабочих нагрузок Cloud Native. Это проект с сертификатом фонда Cloud Native Computing Foundation (CNCF), который широко используется такими компаниями, как Google, DigitalOcean, Red Hat, и многими другими.

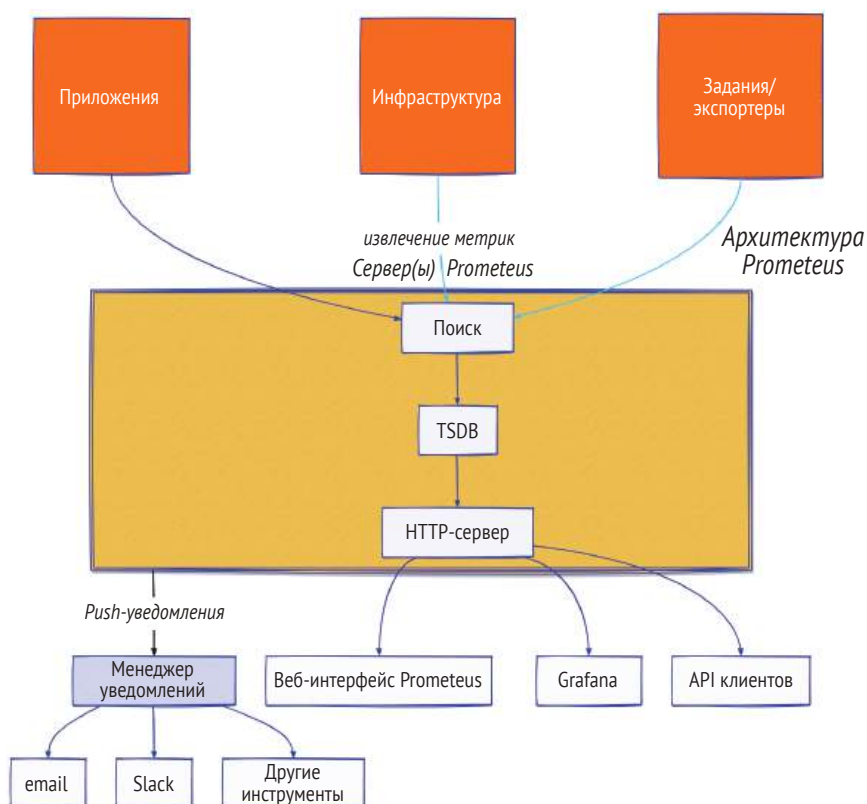
21.2. Как работает Prometheus

Prometheus собирает метрики с регулярными интервалами, оценивает выражения правил, отображает результаты и может запускать оповещения при соблюдении заданных условий. Следующие функции отличают Prometheus от других систем мониторинга и метрик:

¹ <https://github.com/prometheus/prometheus>.

² <https://soundcloud.com/>.

- многомерная модель данных, которая определяет временные ряды по имени метрики и набору измерений «ключ/значение»;
- PromQL¹ – мощный и гибкий язык запросов, использующий эти измерения;
- отсутствие зависимости от распределенного хранилища;
- для сбора временных рядов используется модель HTTP pull;
- целевые объекты обнаруживаются с помощью специальных сервисов обнаружения или статической конфигурации;
- есть поддержка нескольких режимов построения графиков и панелей мониторинга.



Рабочий процесс Prometheus состоит из следующих шагов:

- 1) **сбор данных:** Prometheus собирает метрики из различных целевых объектов, таких как приложения, службы и системы, с помощью запросов (pull-модель). Согласно этой модели, сервер Prometheus периодически извлекает метрики из целей по протоколу HTTP. Цели могут предоставлять метрики в определенном формате, например в формате представления Prometheus или через сторонние экспортеры;

¹ <https://prometheus.io/docs/prometheus/latest/querying/basics/>.

- 2) **хранилище метрик:** хранит собранные метрики в своей базе данных временных рядов. Каждая метрика хранится как точка данных временного ряда с меткой времени и соответствующим значением. Prometheus имеет собственный механизм хранения, оптимизированный для данных временного ряда, что позволяет эффективно запрашивать и извлекать метрики, и является одной из его самых мощных функций;
- 3) **запросы и обработка:** Prometheus располагает собственным мощным языком запросов Prometheus Query Language (PromQL), специально разработанным для анализа и обработки собранных метрик. С помощью PromQL пользователи могут выполнять такие операции, как фильтрация, агрегация, математические вычисления и многое другое. Этот инструмент полезен для извлечения значимых сведений из метрик;
- 4) **оповещения:** Prometheus имеет встроенную систему оповещений, которая позволяет пользователям настраивать оповещения на основе условий или пороговых значений. Инженеры могут настраивать правила оповещений с помощью выражений PromQL. Когда условия, указанные в правилах оповещений, выполняются, Prometheus активирует оповещения и отправляет уведомления по указанным каналам оповещений, таким как Slack или электронная почта;
- 5) **визуализация и мониторинг:** Prometheus предоставляет браузер выражений, называемый Prometheus UI. Он позволяет пользователям визуализировать метрики в реальном времени, исследовать данные и выявлять проблемы. Prometheus также интегрируется с другими инструментами визуализации, такими как Grafana – популярный инструмент визуализации с открытым исходным кодом.

21.3. Установка Prometheus

В этом разделе мы рассмотрим один из способов установки Prometheus в Kubernetes с помощью оператора Prometheus.



Операторы (operator) – это программные расширения Kubernetes, которые используют пользовательские ресурсы для управления приложениями и их компонентами.

Для установки Prometheus мы будем использовать одноименный оператор Prometheus¹. Создайте рабочий каталог и перейдите в него:

```
cd $HOME && mkdir -p monitoring && cd monitoring
```

Клонируйте репозиторий оператора Prometheus:

```
git clone https://github.com/prometheus-operator/kube-prometheus.git
cd kube-prometheus
```

¹ <https://prometheus-operator.dev/>.

Создайте пространство имен и CRD (custom resource definitions, пользовательские определения ресурсов):

```
kubectl create -f manifests/setup
```

Чтобы дождаться создания CRD, отслеживайте ход выполнения с помощью следующей команды:

```
until kubectl get servicemonitors --all-namespaces ; do date; sleep 1; echo ""; done
```

Когда вы видите сообщение `No resources found`, это означает, что CRD были успешно созданы. Теперь мы можем перейти к созданию остальных компонентов.

```
kubectl create -f manifests/
```

21.4. Доступ к веб-интерфейсу Prometheus

Далее мы создадим переадресацию порта для доступа к пользовательскому интерфейсу Prometheus:

```
kubectl -n monitoring port-forward svc/prometheus-k8s 9090 > /dev/null 2>&1 &
```

Вы можете получить доступ к пользовательскому интерфейсу Prometheus с вашего компьютера по адресу <http://localhost:9090>. Если вы используете удаленный компьютер, создайте туннель SSH к удаленному компьютеру и получите доступ к пользовательскому интерфейсу Prometheus с вашего локального компьютера по адресу <http://localhost:9090>.

```
export IP=<IP>
ssh -NfL 9090:localhost:9090 root@$IP
```

Замените `<IP>` на IP-адрес вашего удаленного компьютера.

21.5. Метрики, доступные в Prometheus

После установки Prometheus у вас будет доступ к экземпляру Prometheus, экземпляру Grafana и набору экспортеров Prometheus. Оператор Prometheus также создает `kube-state-metrics`, который является дополнительным агентом, прослушивающим сервер API Kubernetes и генерирующим метрики о состоянии объектов Kubernetes, таких как узлы, поды, развертывания и т. д.

Перечень метрик, доступных с помощью `kube-state-metrics`, можно найти в его официальном репозитории по адресу <https://github.com/kubernetes/kube->

[state-metrics](#). Например, там можно найти ссылку на метрики для подов¹. Ниже приведены некоторые примеры доступных метрик:

- `kube_pod_overhead_memory_bytes`: накладные расходы пода в отношении памяти, связанной с запуском пода;
- `kube_node_status_capacity`: общий объем ресурсов, доступных для узла.

В дополнение к метрикам Prometheus оператор Prometheus также использует другие источники метрик, такие как API Kubernetes², экспортеры черного ящика и экспортеры узлов. Список доступных метрик можно найти в документации Prometheus³.

Чтобы просмотреть полный список метрик, доступных в вашем экземпляре Prometheus, перейдите в пользовательский интерфейс Prometheus и введите следующий запрос в строке поиска: `{__name__=~".+"}`. Он отобразит все доступные метрики в вашем экземпляре Prometheus.

- Синтаксис `{__name__=~".+"}` – это выражение PromQL, которое позволяет фильтровать метрики по имени.
- Оператор `=~` – это сопоставление регулярного выражения, которое означает, что значение метки имени соответствует регулярному выражению `".+"`.
- Регулярное выражение `".+"` соответствует любой строке, содержащей хотя бы один символ.

21.6. Использование Grafana для визуализации метрик Prometheus

Оператор Prometheus также создает экземпляр Grafana для визуализации метрик. Чтобы получить доступ к Grafana, создайте переадресацию порта на службу Grafana:

```
kubectrl -n monitoring port-forward svc/grafana 3000 > /dev/null 2>&1 &
```

Чтобы получить доступ к Grafana на удаленной машине, создайте туннель SSH к ней с локальной машины. После установки туннеля вы сможете получить доступ к Grafana по адресу <http://localhost:3000>.

```
export IP=<IP>
ssh -NfL 3000:localhost:3000 root@$IP
```

По умолчанию имя пользователя и пароль для Grafana – `admin / admin`. Grafana поддерживает несколько источников данных, включая Prometheus. В текущем пакете Prometheus уже настроен как источник данных. Чтобы

¹ <https://github.com/kubernetes/kube-state-metrics/blob/main/docs/pod-metrics.md>.

² <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>.

³ <https://prometheus.io/docs/prometheus/latest/querying/basics/>.

просмотреть список источников данных, нажмите на значок шестеренки на левой панели **Administration** (Администрирование), а затем нажмите на **Data Sources** (Источники данных). В списке источников данных вы должны увидеть Prometheus.

Вы можете настроить пользовательские панели мониторинга, добавив новые или импортировав панели мониторинга с Grafana.com¹.

Чтобы добавить новую панель мониторинга, нажмите значок плюса на левой панели, а затем **Add visualization** (Добавить визуализацию). Выберите Prometheus в качестве источника данных и введите выражение PromQL в поле запроса. Например, чтобы визуализировать использование памяти контейнерами, введите `container_memory_usage_bytes`. Нажмите кнопку **Run query** (Выполнить запрос), чтобы просмотреть результаты.

Чтобы добавить панель мониторинга с Grafana.com, выполните следующие действия:

1. Нажмите на значок плюса на левой панели.
2. Нажмите **Import** (Импорт).
3. В разделе **Import via grafana.com** (Импорт через grafana.com) введите 1860 в поле **Grafana Dashboard** (Панель мониторинга Grafana) и нажмите **Load** (Загрузить).
4. Выберите **Prometheus** в качестве источника данных.
5. Нажмите **Import**, чтобы импортировать панель мониторинга.

Следуя этим инструкциям, вы загрузите панель мониторинга кластера Kubernetes² с Grafana.com.

21.7. Извлечение информации из логов Kubernetes с помощью Promtail

Promtail функционирует аналогично Prometheus, обнаруживая цели путем мониторинга сервера Kubernetes API. Он выполняет запросы меток к Kubernetes API с регулярными интервалами, чтобы найти поды, которые соответствуют селекторам меток. После обнаружения целей он подключается к ним и отправляет содержимое журналов в Loki.

Существует несколько способов³ установки этого инструмента, самый простой из которых – использовать Helm.

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm install promtail grafana/promtail --namespace monitoring
```

¹ <http://grafana.com/>.

² <https://grafana.com/grafana/dashboards/1860>.

³ <https://grafana.com/docs/loki/latest/clients/promtail/>.

Вы можете убедиться, что он работает, выполнив следующие команды:

```
kubectl --namespace monitoring port-forward daemonset/promtail 3101 > /dev/null 2>&1 &
curl http://127.0.0.1:3101/metrics
```

21.8. Стек журналирования Loki

И Loki – это горизонтально масштабируемая высокодоступная многопользовательская система агрегации журналов, напоминающая Prometheus. Она разработана так, чтобы быть очень экономичной и простой в эксплуатации. Она индексирует не содержимое журналов, а набор меток для каждого потока журналов. Grafana является разработчиком Loki и описывает ее как «Prometheus для журналов».

Loki работает совместно с Promtail, который собирает логи из Kubernetes. В совокупности стек ведения журналов состоит из следующих компонентов:

- Promtail: собирает логи из Kubernetes и пересылает их в Loki;
- Loki: хранит логи и позволяет запрашивать их;
- Grafana: визуализирует логи.

Процесс интеграции относительно прост. Сначала нам нужно установить плагин источника данных Loki в Grafana. Затем мы должны настроить его для использования Loki в качестве источника данных.

Для начала установим плагин источника данных Loki с помощью Helm с пользовательской конфигурацией. Выполните следующие действия.

Создайте рабочий каталог и перейдите в него.

```
cd $HOME/monitoring
mkdir loki && cd loki
```

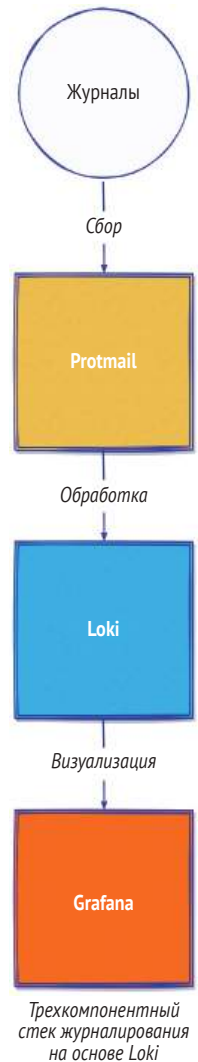
Добавьте репозиторий Loki Helm:

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Создайте файл values.yaml со следующим содержимым:

Листинг 21.1 ❖ Файл values.yaml

```
loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
```



```
storage:
  type: 'filesystem'
singleBinary:
  replicas: 1
```

Приведенный выше манифест устанавливает Loki с одной репликой и использует файловую систему в качестве хранилища. Аутентификация отключена для простоты. Более подробную информацию о параметрах конфигурации можно найти в официальной документации¹.

Установите Loki с Helm в мониторинге пространства имен:

```
helm install --values values.yaml loki grafana/loki --namespace monitoring
```

Если вы хотите установить и запустить Loki с более чем одной репликой, вам следует настроить распределенное хранилище, например S3. Вот пример:

Листинг 21.2 ❖ Файл values-2.yaml

```
loki:
  storage:
    bucketNames:
      chunks: chunks
      ruler: ruler
      admin: admin
    type: s3
    s3:
      endpoint: <endpoint>
      region: <AWS region>
      secretAccessKey: <AWS secret access key>
      accessKeyId: <AWS access key ID>
      s3ForcePathStyle: false
      insecure: false
```

Чтобы добавить Loki в качестве источника данных в Grafana, выполните следующие действия:

1. Перейдите в Grafana и нажмите на значок шестеренки на левой панели **Administration**.
2. Нажмите на **Data Sources**.
3. Нажмите кнопку **Add data source**.
4. Выберите **Loki** в качестве источника данных.
5. В разделе **HTTP** введите `http://loki.monitoring.svc:3100` в поле **URL**. (loki – это имя службы Loki, а monitoring – это пространство имен, в котором установлен Loki.)
6. Нажмите кнопку **Save & Test** (Сохранить и проверить), чтобы сохранить источник данных.

¹ <https://grafana.com/docs/loki/latest/installation/helm/>.

21.9. Использование Loki для запроса и просмотра журналов

Теперь создадим панель мониторинга для визуализации журналов. Сначала нажмите значок плюса на левой панели, затем нажмите **Import**. В разделе **Import via grafana.com** (Импорт через grafana.com) введите 15141 в поле **Grafana.com Dashboard** (Панель мониторинга Grafana.com) и нажмите **Load** (Загрузить). Выберите Loki в качестве источника данных и нажмите **Import**, чтобы загрузить панель просмотра журналов Loki Kubernetes¹.

```
sum(rate({app="my-app", level="warn"}[1m])) / sum(rate({app="my-app", level="error"}[1m]))
```

Этот запрос вычисляет соотношение между долей журналов с меткой level, равной warn, и долей журналов с меткой level, равной error.

Другие операторы, которые вы можете использовать помимо оператора равенства (=):

- != : не равно;
- =~ : регулярное выражение «совпадает»;
- !~ : регулярное выражение «не совпадает».

Операторы сравнения:

- ==: равно;
- !=: не равно;
- >: больше чем;
- >=: больше или равно;
- <: меньше чем;
- <=: меньше или равно.

Логические операторы:

- and (и);
- or (или);
- unless (если не).

Арифметические операторы:

- +: сложение;
- -: вычитание;
- *: умножение;
- /: деление;
- %: остаток от деления;
- ^: возведение в степень.

Примеры:

```
{app!="spring-petclinic"}
```

¹ <https://grafana.com/grafana/dashboards/15141>.

Этот запрос фильтрует журналы из всех подов с меткой `app`, которая не равна `spring-petclinic`.

```
sum(count_over_time({app="my-app"}[1m])) * 100
```

Этот запрос возвращает значения суммы количества журналов с меткой `app`, равной `my-app`, за последнюю минуту, умноженные на 100.

Вы также можете использовать регулярные выражения в значениях меток:

- `{filename=~"/var/log/auth.log|/var/log/syslog"}`: этот запрос фильтрует журналы из файлов `/var/log/auth.log` и `/var/log/syslog`;
- `{app=~"spring-petclinic|my-app"}`: этот запрос фильтрует журналы из всех подов с меткой `app`, равной `spring-petclinic` или `my-app`.

Для более точной фильтрации можно применить различные фильтры, доступные в LogQL:

- `|`: этот фильтр выбирает только те журналы, которые соответствуют заданному фильтру метки;
- `!=`: этот фильтр выбирает только те журналы, которые не соответствуют заданному фильтру метки;
- `|~`: этот фильтр выбирает только те журналы, которые соответствуют заданному фильтру метки с использованием регулярного выражения;
- `!~`: этот фильтр выбирает только те журналы, которые не соответствуют заданному фильтру метки с использованием регулярного выражения.

Примеры:

```
{app="spring-petclinic"} | "error"
```

Этот запрос фильтрует журналы всех подов, у которых метка `app` равна `spring-petclinic`, и выбирает только те, которые содержат строку `error`.

```
{app="spring-petclinic"} |~ "error|exception"
```

Этот запрос фильтрует журналы всех подов, имеющих метку `app`, равную `spring-petclinic`, и выбирает только те, которые содержат строки `error` или `exception`.

```
{app="spring-petclinic"} |~ "error|exception" != "404"
```

Этот запрос фильтрует журналы всех подов, имеющих метку `app`, равную `spring-petclinic`. Он выбирает только те, которые содержат строки `error` или `exception`, но не `404`.

Чтобы извлечь значимые метрики из ваших журналов, вы можете использовать скалярные векторы, такие как:

- `count_over_time`: показывает общее количество строк журнала для диапазона времени;
- `gate`: аналогично `count_over_time`, но преобразуется в количество записей в секунду;

- `bytes_over_time`: количество байтов в каждом потоке журнала в диапазоне;
- `bytes_rate`: аналогично `bytes_over_time`, но преобразовано в количество байтов в секунду.

Примеры:

```
count_over_time({app="spring-petclinic"}[60m])
```

Этот запрос подсчитывает количество журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час.

```
rate({app="spring-petclinic"}[60m])
```

Этот запрос вычисляет долю журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час.

```
bytes_over_time({app="spring-petclinic"}[60m])
```

Этот запрос вычисляет количество байтов журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час.

```
bytes_rate({app="spring-petclinic"}[60m])
```

Этот запрос вычисляет долю в байтах журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час.

Доступны также следующие функции для агрегации журналов:

- `sum`: вычисляет сумму всех векторов в диапазоне в заданное время;
- `min`: показывает минимальное значение из всех векторов в диапазоне в заданное время;
- `max`: показывает максимальное значение из всех векторов в диапазоне в заданное время;
- `avg`: вычисляет среднее значение из всех векторов в диапазоне в заданное время;
- `stddev`: вычисляет стандартное отклонение значений из всех векторов в диапазоне в заданное время;
- `stdvar`: вычисляет стандартную дисперсию значений из всех векторов в диапазоне в заданное время;
- `count`: подсчитывает количество элементов во всех векторах в диапазоне в заданное время;
- `bottomk`: выбирает k наименьших значений во всех векторах в диапазоне в заданное время;
- `topk`: выбирает k наибольших значений во всех векторах в диапазоне в заданное время.

Примеры:

```
sum(count_over_time({app="spring-petclinic"}[60m]))
```

Этот запрос подсчитывает количество журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час и суммирует результат.

```
avg(count_over_time({app="spring-petclinic"}[60m]))
```

Этот запрос подсчитывает количество журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час и вычисляет среднее значение результата.

Чтобы сгруппировать ряд векторов по метке, можно использовать оператор `by`:

```
sum(count_over_time({app="spring-petclinic"}[60m])) by (filename)
```

Этот запрос подсчитывает количество журналов всех подов с меткой `app`, равной `spring-petclinic`, за последний час и суммирует результат по метке `filename`.

21.10. Использование Jaeger и OpenTelemetry для распределенной трассировки

Jaeger и OpenTelemetry – это взаимодополняющие инструменты, которые позволяют отслеживать запросы по нескольким службам.

i OpenTelemetry – это фреймворк для наблюдения за облачным программным обеспечением. Он был образован слиянием OpenCensus и OpenTracing и теперь содержит API, библиотеки, агенты и службы для сбора распределенных трассировок и метрик из ваших приложений. OpenTelemetry предоставляет не зависящий от провайдера способ обслуживания приложений, сбора телеметрических данных и отправки их в различные системы мониторинга. Он поддерживает сбор метрик, журналов и распределенных трассировок.

i Jaeger – это распределенная система трассировки, созданная по мотивам Dapper и OpenZipkin. Она используется для мониторинга и устранения неполадок в распределенных системах на основе микросервисов и является проектом CNCF. Jaeger – один из популярных бэкендов, который может получать и хранить распределенные трассировки, созданные приложениями с OpenTelemetry.

OpenTelemetry используется для отслеживания приложения с целью захвата распределенных трассировок и других данных телеметрии. Затем эти захваченные трассировки экспортируются в Jaeger, который обрабатывает и сохраняет их для последующего анализа.

Коллектор OpenTelemetry – это компонент, который может получать трассировки из различных источников, выполнять агрегацию, фильтрацию

и другую обработку, а затем экспортировать трассировки в Jaeger или другую бэкенд-систему. Коллектор выступает в качестве центральной точки для сбора и распространения данных телеметрии.

Можно сказать, что OpenTelemetry предоставляет средства для инструментирования приложений и захвата распределенных трассировок, в то время как Jaeger выступает в качестве бэкенда для получения, хранения и анализа этих трассировок.

Сначала создайте рабочий каталог и перейдите в него:

```
mkdir -p $HOME/monitoring/jaeger && cd $HOME/monitoring/jaeger
```

Прежде чем продолжить, убедитесь, что вы установили Cert Manager в своем кластере:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.6.3/cert-manager.yaml
```

Согласно официальной документации, начиная с версии 1.31 оператор Jaeger использует веб-перехватчики для проверки пользовательских ресурсов Jaeger (custom resource, CR). Для этого должен быть установлен Cert Manager.

Создайте новое пространство имен observability:

```
kubectl create namespace observability
```

Установите оператор Jaeger:

```
kubectl apply -f https://github.com/jaegertracing/jaeger-operator/releases/download/v1.45.0/jaeger-operator.yaml -n observability
```

Разверните экземпляр Jaeger:

```
kubectl apply -f - <<EOF
---
apiVersion: jaegertracing.io/v1
kind: Jaeger
metadata:
  name: simplest
EOF
```

Это самый простой экземпляр Jaeger, который вы можете развернуть. Код создаст экземпляр Jaeger с конфигурацией по умолчанию. Более подробную информацию о параметрах конфигурации можно найти в официальной документации¹.

У экземпляра Jaeger есть множество настраиваемых параметров конфигурации. Например, вы можете включить аутентификацию, настроить серверную часть хранилища (например, Elasticsearch или Cassandra) и многое другое. Если вы хотите развернуть более продвинутый экземпляр Jaeger с Elasticsearch в качестве бэкенда хранилища, воспользуйтесь Helm:

¹ <https://www.jaegertracing.io/docs/1.25/operator/>.

```
helm repo add jaegertracing https://jaegertracing.github.io/helm-charts
helm repo update
helm install jaeger jaegertracing/jaeger \
--set provisionDataStore.cassandra=false \
--set provisionDataStore.elasticsearch=true \
--set storage.type=elasticsearch
```

Чтобы использовать Cassandra в качестве бэкенда хранилища, выполните следующую команду:

```
helm install jaeger jaegertracing/jaeger \
--set provisionDataStore.elasticsearch=false \
--set provisionDataStore.cassandra=true \
--set storage.type=cassandra
```

Cassandra и Elasticsearch разворачиваются как часть экземпляра Jaeger. В качестве альтернативы вы можете использовать существующий экземпляр Cassandra или Elasticsearch, указав данные о подключении в конфигурации. Для получения дополнительной информации о параметрах конфигурации хранилища данных обратитесь к официальной документации¹.

Если вы остановились на простейшем экземпляре Jaeger, то можете перенаправить порт пользовательского интерфейса Jaeger на локальную машину.

```
kubectrl port-forward svc/simplest-query 16686:16686 > /dev/null &2>1
```

Затем разверните оператор OpenTelemetry:

```
kubectrl apply -f https://github.com/open-telemetry/opentelemetry-operator/releases/
download/v0.76.1/opentelemetry-operator.yaml
```

Создайте экземпляр коллектора OpenTelemetry:

Листинг 21.3 ❖ Файл v1alpha1.yaml

```
apiVersion: opentelemetry.io/v1alpha1
kind: OpenTelemetryCollector
metadata:
  name: otel
spec:
  config: |
    receivers:
      otlp:
        protocols:
          grpc:
          http:
    processors:
      memory_limiter:
        check_interval: 5s
        limit_percentage: 90
        spike_limit_percentage: 95
    batch:
```

¹ <https://github.com/jaegertracing/helm-charts/tree/main/charts/jaeger>.

```

    send_batch_size: 8192
    timeout: 5s
  exporters:
    logging:
  jaeger:
    endpoint: "simplest-collector:14250"
    tls:
      insecure: true
  service:
    pipelines:
      traces:
        receivers: [otlp]
        processors: []
        exporters: [jaeger]

```

Примените этот манифест:

```
kubectl apply -f v1alpha1.yaml
```

i Коллектор OTel получает трассировки из разных источников, включая OpenTelemetry SDK, OpenTelemetry Collector и другие источники. Затем он выполняет агрегацию, фильтрацию и другую обработку трассировок и экспортирует их в Jaeger.

Предыдущий код YAML развертывает коллектор OTel, определяя приемники (receiver), процессоры (processor), экспортеры (exporter) и сервисы (service):

- `receivers` определяет приемники данных телеметрии. В нашем примере это приемник OTLP¹, который может получать данные телеметрии через протоколы gRPC и HTTP;
- `processors` определяет процессоры данных, применяемые к собранным данным телеметрии. В нашем примере процессоры не определены;
- `exporters` определяет пункты назначения, куда должны экспортироваться обработанные данные телеметрии. В нашем примере к ним относятся экспортер журналов и экспортер Jaeger. Экспортер журналов отправляет данные телеметрии в систему журналов, в то время как экспортер Jaeger отправляет данные в коллектор Jaeger (экземпляр Jaeger, который мы развернули ранее);
- `service` определяет конфигурацию службы для коллектора OpenTelemetry. Раздел `pipelines` определяет конвейер обработки для собранных данных телеметрии. В нашем коллекторе есть один конвейер с именем `traces`, который получает данные от приемника OTLP, не применяет процессоры и экспортирует данные в экспортер Jaeger.

Одной из самых интересных особенностей OpenTelemetry является возможность автоматического инструментирования² вашего приложения. Давайте посмотрим, как это работает.

¹ <https://github.com/open-telemetry/opentelemetry-collector/blob/main/receiver/otlpreceiver/README.md>.

² [https://ru.wikipedia.org/wiki/Инструментирование_\(программирование\)](https://ru.wikipedia.org/wiki/Инструментирование_(программирование)).

```
kubectl apply -f - <<EOF
---
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: my-instrumentation
spec:
  exporter:
    endpoint: http://otel-collector:4317
  propagators:
    - tracecontext
    - baggage
    - b3
  sampler:
    type: parentbased_traceidratio
    argument: "0.5"
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java:1.26.0
EOF
```

Ресурс OpenTelemetry Instrumentation, созданный выше, будет автоматически инструментировать приложение Spring, которое мы собираемся развернуть. Он настраивает конечную точку экспортера (<http://otel-collector:4317>), распространители (tracecontext, baggage и b3) и семплер. Он также указывает образ автоинструментирования для приложений Java (ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java:1.26.0).

Поскольку наш коллектор OpenTelemetry развернут в том же кластере, мы можем использовать его имя сервиса в качестве конечной точки экспортера. Если вы используете внешний коллектор OpenTelemetry, укажите его внешний IP-адрес или доменное имя.

В OpenTelemetry мы используем концепцию *сигналов* для обозначения различных типов телеметрических данных. Существует четыре типа сигналов: trace, metrics, logs и baggages.

Трассировки (trace) дают обзор того, что происходит в вашем приложении. Они представляют собой последовательность событий, которые связаны вместе для формирования трассировки. Каждое событие в трассировке называется *интервалом* (span). Другими словами, интервал является составной частью трассировки. Интервал может использоваться для представления запроса, вызова функции, запроса к базе данных или любого другого события в вашем приложении.

Трассировка может иметь несколько интервалов. Например, запрос к вашему приложению может быть представлен как трассировка с несколькими интервалами. Каждый интервал может представлять вызов функции, запрос к базе данных, а затем HTTP-запрос. Чтобы интервалы были сопоставлены и собраны вместе для формирования трассировки, нам требуется распространение контекста.

Распространение контекста (context propagation) является ключевой концепцией в распределенной трассировке и определяет процесс передачи контекста запроса от одного сервиса к другому.

Контекст представляет информацию, необходимую для корреляции диапазонов и связывания их с трассировкой. Например, когда сервис А вызывает сервис В, интервал из сервиса А становится родительским интервалом для следующего интервала, созданного в сервисе В. Распространение – это механизм, который передает контекст между сервисами, позволяя собирать распределенную трассировку. Он сериализует и десериализует контекст интервала, позволяя передавать соответствующую информацию о трассировке из одной службы в другую. Этот механизм называется `tracecontext`.

Распространитель багажа (`baggage propagator`) применяется для распространения элементов багажа. *Багаж* (`baggages`) – это механизм для передачи данных между интервалами. Он похож на концепцию заголовков HTTP, но используется для передачи данных между интервалами вместо передачи данных между запросами и ответами HTTP.

Наконец, *распространитель b3* (`b3 propagator`) применяется для распространения контекста B3. B3 – это распределенный протокол трассировки, который определяет формат контекста трассировки. Он поддерживается многими системами трассировки, включая Jaeger, Zipkin и AWS X-Ray.

Семплер `parentbased_traceidratio` используется для выборки трасс на основе идентификатора родительской трассы. Это вероятностный семплер, который делает выборку трассы, если идентификатор родительской трассы меньше указанного соотношения. В нашем примере мы устанавливаем соотношение на 0.5, что означает, что будет выбрано 50 % трасс. Это полезно для сокращения объема данных, отправляемых на бэкенд. Как правило, не следует отправлять все трассы на бэкенд, так как будет слишком много данных для обработки. Например, если у вас есть сервис, который получает 1000 запросов в секунду, и все они успешны, незачем генерировать 1000 трасс в секунду. Вместо этого вы можете сделать выборку 50 % трасс и отправить их на бэкенд.

В заключение мы настраиваем образ для использования в автоинструментации. В нашем примере мы используем образ автоинструментации Java. Существуют также образы автоинструментации для других языков, включая Python, Node.js и .NET.

Теперь мы можем развернуть наше приложение:

```
kubectl apply -f - <<EOF
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-petclinic
spec:
  selector:
    matchLabels:
      app: spring-petclinic
  replicas: 1
  template:
    metadata:
      labels:
        app: spring-petclinic
    annotations:
```

```

    sidecar.opentelemetry.io/inject: "true"
    instrumentation.opentelemetry.io/inject-java: "true"
  spec:
    containers:
      - name: app
        image: ghcr.io/pavolloffay/spring-petclinic:latest
EOF

```

EOF

Приведенный выше код YAML разворачивает приложение Spring PetClinic¹.

В коде присутствуют следующие аннотации для внедрения коллектора OpenTelemetry и агента автоинструментации Java в под, на котором запущено приложение:

- `sidecar.opentelemetry.io/inject`;
- `instrumentation.opentelemetry.io/inject-java`.

Теперь вы можете получить описание созданного пода:

```

export pod=$(kubectl get pod -l app=spring-petclinic -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $pod

```

Вы должны увидеть в поде два контейнера:

- контейнер `app` – это основной контейнер, который запускает приложение Spring PetClinic;
- контейнер `sidecar`.

В приложении также есть новые переменные среды, используемые для инструментации:

```
kubectl exec $pod -- printenv | grep OTEL
```

Теперь мы можем получить доступ к пользовательскому интерфейсу Jaeger, чтобы увидеть трассировки. Выполните следующую команду, чтобы перенаправить порт 16686 сервиса `simplest-query` на локальную машину, если не сделали этого раньше:

```
kubectl port-forward svc/simplest-query 16686:16686 > /dev/null &2>1
```

Затем, если вы хотите получить доступ к пользовательскому интерфейсу Jaeger, откройте следующий URL в своем браузере: `http://localhost:16686` (после ssh-туннелирования на машину разработки).

```

export IP=<IP>
ssh -NfL 16686:localhost:16686 root@$IP

```

Вы должны увидеть в пользовательском интерфейсе Jaeger два сервиса: `spring-petclinic` и `jaeger-query`. Первый – это приложение, которое мы развернули, а второй – это сам Jaeger Query.



Автоинструментация с OpenTelemetry и Jaeger

¹ <https://github.com/spring-projects/spring-petclinic>.

Глава 22

GitOps: непрерывная поставка в облачном стиле

22.1. GitOps: введение и определения

С ростом популярности подхода «все как код» Git стал фактическим стандартом систем контроля версий и управления исходным кодом. GitOps – это новый подход к управлению инфраструктурой и поставке приложений, в соответствии с которым Git выступает как единый источник кода для конфигураций среды и развертываний приложений.

В среде GitOps каждое изменение проверяется с помощью процессов проверки кода и последовательно применяется к среде с использованием инструментов CI/CD (непрерывная интеграция / непрерывное развертывание), таких как Jenkins, Travis CI или CircleCI.

Используя Git как единый источник кода, команды разработки и эксплуатации могут строго соблюдать процесс утверждения и проверки изменений перед их применением, что повышает безопасность и стабильность среды. Git, являясь центральным компонентом процесса, также обеспечивает полное отслеживание всех изменений, будь то изменение конфигурации, новый выпуск, откат, исправление безопасности или новый компонент инфраструктуры. Фактически методику GitOps можно применять для управления различными средами, включая серверные инфраструктуры, контейнерные платформы и среды микросервисов.

22.2. Преимущества и недостатки GitOps

Команды, использующие GitOps, определяют желаемое состояние в Git, а затем применяют автоматизацию, чтобы фактическое состояние системы всег-

да отражало желаемое состояние. Все обновления предлагаются в форме *запросов на внесение* (pull request) и коллективно проверяются, что позволяет командам сотрудничать и предоставлять друг другу обратную связь до того, как изменения будут применены к среде.

Развертывания могут автоматически запускаться изменениями в Git, благодаря чему среда всегда находится в актуальном состоянии. Это также позволяет легко откатывать изменения, отменяя их в Git. GitOps отслеживает все изменения, предоставляя полный аудиторский след, который может быть полезен с точки зрения комплаенса и безопасности.

GitOps также упрощает управление несколькими средами, используя одни и те же инструменты и процессы.

Хотя GitOps относительно прост, у него есть некоторые недостатки. Командам необходимо использовать Git как своего рода источник эталонной информации, что не всегда легко осуществимо. GitOps также требует высокого уровня автоматизации, а это бывает слишком сложно для неопытных команд.

22.3. Инструменты GitOps

GitOps – это новый подход к управлению инфраструктурой и поставке приложений, и его экосистема все еще развивается. Тем не менее уже есть несколько инструментов, пригодных для внедрения GitOps в вашей среде.

Наиболее популярные инструменты для внедрения GitOps:

- **Flux**¹: инструмент, который автоматизирует развертывание приложений и инфраструктуры, используя Git как единый источник кода. Его можно использовать для управления кластерами Kubernetes, серверными инфраструктурами и средами микросервисов;
- **Argo CD**²: инструмент, который автоматизирует развертывание приложений и инфраструктуры, используя Git как единый источник информации. Его можно использовать для управления кластерами Kubernetes, серверными инфраструктурами и средами микросервисов. Мы рассмотрим этот инструмент в следующей главе;
- **Jenkins X**³: это инструмент, который автоматизирует развертывание приложений и инфраструктуры, используя Git как единый источник информации. Его можно использовать для управления кластерами Kubernetes, серверными инфраструктурами и средами микросервисов;

¹ <https://fluxcd.io/>.

² <https://argoproj.github.io/argo-cd/>.

³ <https://jenkins-x.io/>.

- **GitLab**¹: это полноценная платформа DevOps, поставляемая как единое приложение. Она предоставляет полную цепочку инструментов DevOps из коробки и может применяться для внедрения GitOps в вашей среде;
- **Werf**²: инструмент, который автоматизирует развертывание приложений и инфраструктуры, используя Git как единый источник информации. Его можно использовать для управления кластерами Kubernetes, серверными инфраструктурами и средами микросервисов.

¹ <https://about.gitlab.com/>.

² <https://werf.io/>.

Глава 23

Пример рабочего процесса GitOps с использованием Argo CD

23.1. Знакомство с Argo CD

Argo CD – это инструмент *непрерывного развертывания* (continuous deployment, CD) с открытым исходным кодом для сред Kubernetes, основанный на методике GitOps. Он позволяет разработчикам развертывать свои приложения в кластере Kubernetes, просто отправляя их в определенную ветку Git. Это событие запускает конвейер непрерывного развертывания, который обновляет ресурсы Kubernetes в соответствии с изменениями в исходном коде.

Для управления и развертывания приложений в кластере Kubernetes Argo CD использует такие концепции, как приложения, среды и синхронизации. Он предлагает пользовательский интерфейс и REST API для управления и мониторинга приложений, развернутых в кластере Kubernetes. Кроме того, он предоставляет инструмент командной строки, который можно интегрировать с инструментами CI/CD и использовать для управления и мониторинга приложений, развернутых в кластере Kubernetes.

23.2. Argo CD: установка и настройка

Argo CD можно установить в кластере Kubernetes с помощью следующей команды:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Чтобы получить доступ к пользовательскому интерфейсу Argo CD, вам нужно создать переадресацию порта на сервер Argo CD (или предоставить доступ к сервису с помощью LoadBalancer или Ingress):

```
kubectrl port-forward svc/argocd-server -n argocd 8080:443 > /dev/null 2>&1 &
```

Теперь мы можем создать туннель SSH для доступа к пользовательскому интерфейсу Argo CD с нашей локальной машины:

```
export ip=<IP>
ssh -NfL 8080:localhost:8080 root@$ip
```

Имя пользователя по умолчанию – admin, а пароль можно получить с помощью такой команды:

```
kubectrl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d; echo
```

Чтобы установить инструмент командной строки (CLI) Argo CD, выполните следующие команды:

```
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/
download/argocd-linux-amd64
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
rm argocd-linux-amd64
```

Подключите CLI к серверу Argo CD:

```
export password=$(kubectrl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.
data.password}" | base64 -d; echo)
argocd login localhost:8080 --username admin --password $password --insecure
```

Теперь, когда мы установили Argo CD CLI и подключили его к серверу Argo CD, мы можем добавить к этому серверу наш кластер. Однако этот шаг требуется только тогда, когда Argo CD не запущен на том же кластере, которым мы хотим управлять. В нашем случае мы можем пропустить этот шаг.

```
export cluster=$(kubectrl config get-contexts -o name)
argocd cluster add $cluster -y
```

Мы можем убедиться, что наш кластер был добавлен, с помощью следующей команды:

```
argocd cluster list
```

23.3. Argo CD: создание приложения

Мы будем использовать готовый репозиторий GitHub¹ для развертывания простого приложения Flask на нашем кластере Kubernetes с помощью Argo

¹ <https://github.com/eon01/argocd-examples/>.

CD. Клонировать этот репозиторий, чтобы начать работу. Репозиторий содержит несколько примеров приложений, которые можно развернуть с помощью Argo CD; каждый пример хранится в отдельной папке.

Начнем с создания пространства имен для нашего приложения:

```
kubectl create namespace flask-app
```

Теперь мы можем развернуть манифест приложения с помощью следующей команды (одна строка):

```
argocd app create flask-app
--repo https://github.com/eon01/argocd-examples/
--path flask-app
--dest-server https://kubernetes.default.svc
--dest-namespace flask-app
--revision main
--sync-policy automated
```

Эта команда создаст приложение с именем flask-app, используя манифест, хранящийся в каталоге flask-app репозитория GitHub. Приложение будет развернуто в пространстве имен flask-app кластера Kubernetes. Используется ветка main. В команду добавлен флаг --sync-policy automated для включения автоматической синхронизации приложения при обнаружении изменения в репозитории Git.

Папка flask-app содержит файл kubernetes.yaml, который определяет ресурсы, необходимые для развертывания приложения.

Листинг 23.1 ❖ Файл kubernetes.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 1
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - image: eon01/stateless-flask:v0
          name: flask-app
          ports:
            - containerPort: 5000
      resources:
        limits:
```

```

    cpu: 100m
    memory: 128Mi
  requests:
    cpu: 100m
    memory: 128Mi
  readinessProbe:
    httpGet:
      path: /tasks
      port: 5000
    initialDelaySeconds: 5
    periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: flask-app
spec:
  ports:
  - port: 5000
    protocol: TCP
    targetPort: 5000
  selector:
    app: flask-app
  type: LoadBalancer

```

23.4. Argo CD: автоматическая синхронизация и самовосстановление

Поскольку мы активировали автоматическую синхронизацию приложения, любые изменения, внесенные в манифест приложения, вызовут запуск синхронизации. Например, мы можем изменить количество реплик с 1 на 2 в файле `kubernetes.yaml` и отправить изменение в репозиторий GitHub. Это вызовет синхронизацию приложения и обновит количество реплик до 2. По умолчанию Argo CD опрашивает репозиторий Git каждые 3 минуты, чтобы проверить наличие изменений.

Не забудьте сделать коммит и отправить изменения в репозиторий GitHub:

```
git add .
git commit -m "update replicas"
git push
```

Концепция желаемого состояния занимает центральное место в идеологии Argo CD. *Желаемое состояние* (desired state) – это состояние, определенное в репозитории Git приложения. Argo CD будет постоянно сравнивать желаемое состояние с фактическим и автоматически синхронизировать приложение, если между двумя состояниями есть разница.

Однако иногда политика автоматической синхронизации также подразумевает удаление ресурсов. Чтобы предотвратить аварии, по умолчанию Argo CD не удаляет ресурсы при синхронизации. Но если вы точно знаете, что делаете, то можете включить автоматическое удаление ресурсов с помощью следующей команды:

```
argocd app set $APPNAME --auto-prune
```

Другой механизм безопасности, который включен по умолчанию, – это защита от пустых ресурсов. Если вы хотите разрешить пустые ресурсы, отключите механизм безопасности с помощью следующей команды:

```
argocd app set $APPNAME --allow-empty
```

Другой полезный механизм – это возможность вернуться в желаемое состояние, если текущее состояние было изменено. Например, если вы вручную измените количество реплик на 3, Argo CD автоматически вернет изменение на 2. Вы можете включить этот механизм с помощью следующей команды:

```
argocd app set $APPNAME --self-heal
```

Чтобы активировать автоматическую синхронизацию, не дожидаясь следующего опроса, выполните команду

```
argocd app sync $APPNAME
```

В качестве альтернативы вы можете настроить веб-перехватчик в вашем репозитории GitHub, который будет срабатывать при обнаружении изменения. Веб-перехватчик должен использовать следующий URL: `https://<argocd>/api/webhook`, где `<argocd>` – это общедоступный домен, указывающий на ваш экземпляр Argo CD. Вы можете найти больше информации о веб-перехватчиках в документации¹.

23.5. Откат изменений с помощью Argo CD

Для просмотра истории изменений приложения выполните следующую команду:

```
argocd app history $APPNAME
```

Вы можете выполнить откат к предыдущей версии с помощью следующей команды:

```
argocd app rollback $APPNAME <revision-number>
```

Однако откат невозможно выполнить для приложения с включенной автоматической синхронизацией. Перед выполнением отката необходимо отключить автоматическую синхронизацию:

```
argocd app set $APPNAME --sync-policy none
```

Для включения автоматической синхронизации после отката введите следующую команду:

```
argocd app set $APPNAME --sync-policy automated
```

23.6. Декларативный способ использования Argo CD

Ранее мы создали приложение с помощью следующей команды (одна строка):

```
argocd app create flask-app  
  --repo https://github.com/eon01/argocd-examples/  
  --path flask-app  
  --dest-server https://kubernetes.default.svc  
  --dest-namespace flask-app
```

¹ <https://argo-cd.readthedocs.io/en/stable/operator-manual/webhook/>.

```
--revision main
--sync-policy automated
--self-heal
--auto-prune
--allow-empty
```

Это *императивный* способ создания приложения, но поскольку мы придерживаемся идеологии «все как код», также возможно создать приложение с использованием *декларативного* подхода. Это означает, что мы можем определить приложение в файле манифеста, а затем применить файл манифеста для создания приложения.

Давайте удалим предыдущее приложение с помощью команды

```
argocd app delete flask-app
```

и повторно создадим его с использованием декларативного подхода:

```
kubectl apply -f - <<EOF
---
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    server: https://kubernetes.default.svc
  project: default
  source:
    path: flask-app
    repoURL: https://github.com/eon01/argocd-examples/
    targetRevision: main
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true
EOF
```

23.7. Управление конфигурацией с помощью Argo CD

Если вы используете непрерывное развертывание приложений, вам нужно будет управлять их конфигурацией. Существует несколько способов управления конфигурацией приложений. Некоторые из них встроены в Argo CD,

например Helm¹, Jsonnet² и Kustomize³, но вы также можете написать свой собственный плагин управления конфигурацией⁴ и использовать его с Argo CD.

i Helm использует *чарты* (chart) для определения структуры приложения. Чарт – это набор файлов, описывающих связанный набор ресурсов Kubernetes.

Чарт Helm состоит из следующих файлов:

- Chart.yaml: содержит метаданные чарта;
- values.yaml: содержит значения по умолчанию для чарта;
- templates/: содержит шаблоны ресурсов Kubernetes;
- _helpers.tpl: содержит вспомогательные функции, используемые в шаблонах ресурсов Kubernetes.

Helm использует язык шаблонов Go для генерации файлов ресурсов Kubernetes на основе предоставленных значений. Вы можете определять переменные в шаблонах с помощью синтаксиса `{{ .Values.variableName }}`, где `variableName` соответствует ключу в файле `values.yaml`.

Для генерации различных конфигураций на основе определенных условий или для итерации по спискам значений в шаблонах можно применять условия и циклы.

Помощники (helper) – это многократно используемые функции шаблонов, которые могут упростить сложную логику или вычисления. Пользовательские вспомогательные функции можно определить в файле `_helpers.tpl`, расположенном в корневом каталоге чарта. Вспомогательные функции можно применять в шаблонах для выполнения таких действий, как манипуляция строками, вычисления или условные проверки.

Рассмотрим примеры совместной работы Helm с Argo CD. У нас уже есть другой вложенный каталог в том же репозитории GitHub, который содержит чарт Helm. Давайте обновим приложение, чтобы использовать чарт Helm вместо манифестов Kubernetes.

Листинг 23.2 ❖ Файл Chart.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    server: https://kubernetes.default.svc
```

¹ <https://helm.sh/>.

² <https://jsonnet.org/>.

³ <https://kustomize.io/>.

⁴ <https://argo-cd.readthedocs.io/en/stable/operator-manual/config-management-plugins/>.

```
project: default
source:
  repoURL: https://github.com/eon01/argocd-examples
  targetRevision: main
  path: flask-app-helm
  helm:
    valueFiles:
      - values.yaml
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: true
```

Файл Chart.yaml определяет следующие метаданные:

```
# Версия API чарта Helm
apiVersion: v2
# Имя чарта Helm
name: flask-app-helm
# Описание чарта Helm
description: Чарт Helm Kubernetes для изучения Argo CD
# Тип чарта (например, приложение, библиотека)
type: application
# Версия чарта Helm
version: 0.1.0
# Версия приложения, разворачиваемого чартом Helm
appVersion: "1.0"
```

Так выглядит файл deployment.yaml, который определяет разворачивание приложения:

Листинг 23.3 ❖ Файл deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ template "flask-app.fullname" . }}
  labels:
    app: {{ template "flask-app.fullname" . }}
    chart: {{ template "flask-app.chart" . }}
    release: {{ .Release.Name }}
    heritage: {{ .Release.Service }}
spec:
  replicas: {{ .Values.replicaCount }}
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: {{ template "flask-app.fullname" . }}
      release: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ template "flask-app.fullname" . }}
```

```

    release: {{ .Release.Name }}
  spec:
    containers:
      - image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        name: {{ .Chart.Name }}
        ports:
          - containerPort: 5000
        resources:
          limits:
            cpu: 100m
            memory: 128Mi
          requests:
            cpu: 100m
            memory: 128Mi
        readinessProbe:
          httpGet:
            path: /tasks
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 10

```

Нам также нужен файл `service.yaml`, который определяет сервис:

Листинг 23.4 ❖ Файл `service.yaml`

```

apiVersion: v1
kind: Service
metadata:
  name: {{ template "flask-app.fullname" . }}
  labels:
    app: {{ template "flask-app.fullname" . }}
    chart: {{ template "flask-app.chart" . }}
    release: {{ .Release.Name }}
    heritage: {{ .Release.Service }}
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 5000
  selector:
    matchLabels:
      app: {{ template "flask-app.fullname" . }}
      release: {{ .Release.Name }}
  type: {{ .Values.service.type }}

```

Ниже показан файл `_helpers.tpl`, который определяет повторно используемую логику для вызова из наших шаблонов.

Листинг 23.5 ❖ Файл `_helpers.tpl`

```

{{/*

```

Этот файл содержит вспомогательные шаблоны и функции, которые можно использовать в нескольких шаблонах в текущем чарте.

```

*/*}}

{/* Определение вспомогательной функции для генерации полного имени приложения */}}
{{- define "flask-app.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name -}}
{{- end -}}

{/* Определите вспомогательную функцию для получения имени чарта */}}
{{- define "flask-app.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version -}}
{{- end -}}

```

Наконец, в следующем листинге показано содержимое файла `values.yaml`, который определяет значения по умолчанию для нашего приложения:

Листинг 23.6 ❖ Файл `values.yaml`

```

replicaCount: 3

image:
  repository: eon01/stateless-flask:v0
  tag: latest

service:
  type: LoadBalancer

```

Кратко поясню, как работает язык шаблонов Helm:

- переменные `{{ .Values.replicaCount }}` заменяются соответствующими значениями, определенными в файле `values.yaml`. Объект `.Values` позволяет получить доступ к значениям, определенным в файле `values.yaml` диаграммы, и использовать их в ваших шаблонах;
- переменные `{{ .Release.Service }}` заменяются значениями из объекта выпуска, созданного Helm. Объект `.Release` предоставляет доступ к информации о текущем выпуске, такой как имя выпуска, пространство имен выпуска и временная метка выпуска. Поле `.Release.Service` ссылается на поле сервиса объекта релиза, которому по умолчанию задано значение `Helm`;
- переменные `{{ template "flask-app.fullname" . }}` заменяются значениями, возвращаемыми вспомогательными функциями, определенными в файле `_helpers.tpl`. Вспомогательные функции позволяют повторно используемую логику шаблонов и вызывать их из ваших шаблонов. В данном случае вспомогательная функция `flask-app.fullname` возвращает полное имя приложения, объединяя имя релиза и имя чарта.



Язык шаблонов не является уникальной особенностью Helm; это комбинация языка шаблонов Go, дополнительных функций и оболочек, которые предоставляют доступ к определенным объектам в шаблонах. Поэтому многие ресурсы и руководства, доступные для шаблонов Go, могут быть ценными ссылками при изучении шаблонизации Helm.

Итак, мы развернули приложение с помощью Helm, используя следующий манифест YAML:

Листинг 23.7 ❖ Файл helm-1.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    server: https://kubernetes.default.svc
  project: default
  source:
    repoURL: https://github.com/eon01/argocd-examples
    targetRevision: main
    path: flask-app-helm
    helm:
      valueFiles:
        - values.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true
```

Как видите, файл значений, который Argo CD будет использовать для развертывания приложения, – это файл `values.yaml`, который должен быть указан в разделе `helm` поля `source`.

В качестве альтернативы мы можем определить значения непосредственно в поле `values` раздела `source`:

Листинг 23.7 ❖ Файл helm-2.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    server: https://kubernetes.default.svc
  project: default
  source:
    repoURL: https://github.com/eon01/argocd-examples
    targetRevision: main
    path: flask-app-helm
    helm:
      values:
```

```
replicaCount: 3
image:
  repository: eon01/stateless-flask:v0
  tag: latest
service:
  type: LoadBalancer
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: true
```

23.8. Argo CD: управление различными средами

Как было показано ранее, вы можете управлять различными кластерами, используя один и тот же экземпляр Argo CD. Это очень полезно, когда вы хотите управлять различными средами одного и того же приложения. Например, у вас может быть среда разработки, промежуточная среда и производственная среда.

Чтобы добавить кластер, используйте следующую команду:

```
export cluster=<cluster-name>
argocd cluster add $cluster -y
```

Например, у нас есть производственный кластер с именем `production` и промежуточный кластер с именем `staging`. Мы можем добавить их в Argo CD с помощью следующих команд:

```
export cluster=production
argocd cluster add $cluster -y
```

```
export cluster=staging
argocd cluster add $cluster -y
```

В пользовательском интерфейсе Argo CD вы можете увидеть различные кластеры на вкладке **Settings** (Настройки). Можно также получить список кластеров с помощью команды

```
argocd cluster list
```

Экспортируйте имя кластера, который вы хотите использовать, в зависимости от среды, в которой вы хотите развернуть приложение.

```
export CLUSTER=<cluster-name>
```

Разверните приложение в этом кластере из манифеста `app.yaml` с помощью команды `kubectl apply -f app.yaml`:

Листинг 23.8 ❖ Файл app.yaml

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    # $CLUSTER - имя кластера, в котором вы хотите развернуть приложение
    server: $CLUSTER
  project: default
  source:
    repoURL: https://github.com/eon01/argocd-examples
    targetRevision: main
    path: flask-app-helm
    helm:
      valueFiles:
        - values.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true

```

Кроме того, вы можете создать разные значения для каждой среды. Например, вы можете создать файлы `values-production.yaml` и `values-staging.yaml`, а затем выполнить следующую команду для развертывания приложения в производственной среде:

```
kubectl apply -f app-2.yaml
```

Листинг 23.9 ❖ Файл app-2.yaml

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    # $CLUSTER - имя кластера, в котором вы хотите развернуть приложение
    server: $CLUSTER
  project: default
  source:
    repoURL: https://github.com/eon01/argocd-examples
    targetRevision: main
    path: flask-app-helm
    helm:
      valueFiles:
        - values-production.yaml # или values-staging.yaml ... и т. д.

```

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: true
```

23.9. Argo CD: хуки развертывания

Можно определить хуки (hook), которые будут выполнены до или после развертывания приложения. Например, вы можете выполнить скрипт, который отправляет уведомление в канал Slack или электронное письмо вашей команде.

Чтобы определить хук, вы должны определить *задание* (job). Например, вы можете определить задание, которое отправляет электронное письмо:

Листинг 23.10 ❖ Файл hook.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  generateName: email-
  namespace: argocd
  annotations:
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: email
          image: bytemark/smtp
          command: ["/bin/sh"]
          args: ["-c", "echo 'Hello World!' | mail -s 'Hello World!' <email>"]
          env:
            - name: RELAY_HOST
              value: <smtp-host>
            - name: RELAY_PORT
              value: <smtp-port>
            - name: RELAY_USERNAME
              value: <smtp-username>
            - name: RELAY_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: smtp-password
                  key: password
          restartPolicy: Never
      backoffLimit: 4
  ---
apiVersion: v1
```

```
kind: Secret
metadata:
  name: smtp-password
  namespace: argocd
type: Opaque
stringData:
  password: <smtp-password>
```

Затем вы можете повторно развернуть приложение, чтобы запустить хук:

Листинг 23.11 ❖ Файл app-3.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: flask-app
  namespace: argocd
spec:
  destination:
    namespace: flask-app
    server: https://kubernetes.default.svc
  project: default
  source:
    repoURL: https://github.com/eon01/argocd-examples
    targetRevision: main
    path: flask-app-helm
    helm:
      valueFiles:
        - values.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true
```

Вы должны увидеть задание после выполнения предыдущей команды:

```
kubectl get jobs -n argocd
```

Можно также увидеть созданные журналы контейнера:

```
kubectl logs -n argocd email-<pod-id>--<container-id>
```

Точно так же, как и выше, мы можем создать задание, которое будет отправлять уведомление в канал Slack:

Листинг 23.12 ❖ Файл job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  generateName: slack-
  namespace: argocd
```

```

annotations:
  argocd.argoproj.io/hook: PostSync
  argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
      - name: slack
        image: curlimages/curl
        command:
        - "curl"
        - "-X"
        - "POST"
        - "--data-urlencode"
        - "payload={\"channel\": \"#<channel-name>\", \"username\": \"hello\", \"text\": \"App Sync failed\", \"icon_emoji\": \":ghost:\"}"
        - "https://hooks.slack.com/services/..."
      restartPolicy: Never
    backoffLimit: 2

```

В дополнение к хукам PostSync вы можете определить другие хуки, такие как:

- PreSync;
- SyncFail;
- Skip;
- Sync.

А вместо HookSucceeded вы можете использовать HookFailed или BeforeHook-Creation.

Глава 24

Создание конвейеров CI/CD для микросервисов

24.1. Непрерывная интеграция, поставка и развертывание микросервисов

В микросервисной архитектуре несколько сервисов работают вместе, чтобы обеспечить бизнес-функциональность. Система достигает своих целей посредством совместной работы этих сервисов. Если хотя бы один сервис работает неправильно, это может повлиять на всю систему. Поэтому важно иметь конвейер CI/CD для каждого сервиса. Каждый сервис может иметь свой собственный репозиторий, или у вас может быть монорепозиторий для всех сервисов. Это лишь вопрос предпочтений. Однако важно независимо создавать и тестировать каждый сервис с новыми изменениями. Кроме того, система также должна тестироваться целиком всякий раз, когда добавляется новый сервис или изменяется существующий.

Перед тестированием должны быть установлены все библиотеки и зависимости. После прохождения тестов сервис должен быть упакован и развернут в промежуточной среде, которая максимально приближена к производству. Если в промежуточной среде нет проблем, сервис может быть развернут в производственной среде.

Так выглядит типичный конвейер CI/CD для микросервиса. В зависимости от ваших потребностей, архитектуры, взаимодействия команд, бизнес-требований, инфраструктуры и других факторов конвейер может быть сложнее.

24.2. Инструменты CI/CD

На рынке представлено много разных инструментов CI/CD, пригодных для создания микросервисного конвейера. Эти инструменты имеют разные функции, возможности, кривые обучения и недостатки. Некоторые из них имеют открытый исходный код, а другие – нет. Некоторые из них облачные, а другие – нет. Кроме того, они могут работать обособленно или в сочетании с другими инструментами. Например, вы можете создать конвейер с Jenkins и использовать Argo CD для развертывания. Вы можете использовать действия GitHub для сборки и тестирования и использовать Jenkins для развертывания и т. д.

Далее мы рассмотрим некоторые из самых популярных инструментов CI/CD.

24.2.1. Jenkins

Один из самых популярных инструментов CI/CD. Он имеет открытый исходный код и большое сообщество. Его легко устанавливать и настраивать, и у него есть множество плагинов, которые вы можете использовать для расширения его функциональности. Jenkins написан на Java и является кросс-платформенным.

Однако Jenkins иногда может быть слишком ресурсоемким. Его также не легко масштабировать. Кроме того, вам могут потребоваться некоторые знания Groovy для создания расширенных конвейеров.

24.2.2. Spinnaker

Spinnaker – это инструмент CI/CD с открытым исходным кодом, созданный Netflix. Написанный на Java, он является кросс-платформенным и разработан для работы в облаке. Spinnaker работает с Kubernetes, AWS, Google Cloud, Azure и другими провайдерами облачных услуг.

Spinnaker предоставляет два основных набора функций:

- управление приложениями: функции управления для просмотра и управления вашими облачными ресурсами;
- развертывание приложений: предоставляет функции развертывания для построения и управления рабочими процессами непрерывной поставки.

Spinnaker состоит из нескольких микросервисов, которые работают вместе для реализации вышеуказанных функций.

Хотя Spinnaker является мощным инструментом, его изучение и использование может вызвать затруднения у новичков. Кроме того, он больше ориентирован на развертывание.

24.2.3. Argo CD

Argo CD – это декларативный инструмент непрерывной поставки GitOps с открытым исходным кодом для Kubernetes. Он часто используется для управления развертываниями микросервисов, особенно когда микросервисы упакованы в контейнеры Docker и организованы с помощью Kubernetes.

Argo CD написан на Go, не зависит от платформы, прост в установке и настройке. Он также оснащен удобным пользовательским интерфейсом, который можно применять вместо CLI.

Для использования всех возможностей Argo CD необходимо знать Kubernetes, Helm, Kustomize и другие инструменты. Он больше ориентирован на развертывание, чем на остальные аспекты процесса разработки.

24.2.4. GitHub Actions

GitHub Actions – это инструмент CI/CD, который интегрируется с GitHub. Он использует декларативный подход YAML, популярный среди разработчиков облачных экосистем. Разработчики могут определять рабочие процессы, которые запускаются такими событиями Git, как push, pull request и issue.

GitHub Actions прост в использовании и бесплатен для публичных репозиторий. Благодаря популярности GitHub как инструмента управления исходным кодом GitHub Actions имеет широкое сообщество со множеством примеров, учебных пособий и рабочих процессов, доступных на его площадке.

Однако он не является инструментом с открытым исходным кодом и не полностью бесплатен. Дополнительное использование сверх бесплатного уровня в 2000 минут GitHub Actions в месяц и 500 МБ хранилища необходимо оплачивать. Более подробную информацию о ценах можно найти на сайте¹.

Кроме того, GitHub Actions не подходит, если ваш исходный код размещен в другом месте.

24.2.5. GitLab CI/CD

GitLab предоставляет надежную систему непрерывной интеграции и развертывания, встроенную в его платформу. Благодаря своим богатым возможностям GitLab CI/CD можно настроить для удовлетворения потребностей практически любого рабочего процесса разработки. Вот некоторые из его функций:

- Auto DevOps: набор шаблонов CI/CD по умолчанию для популярных языков и фреймворков;

¹ <https://docs.github.com/en/github/setting-up-and-managing-billing-and-payments-on-github/about-billing-for-github-actions>.

- ChatOps: способ запуска и получения результатов конвейеров CI/CD из окна чата;
- интеграция с облачными сервисами: GitLab CI/CD использует OpenID Connect для аутентификации пользователей и авторизации доступа к облачным провайдерам.

Бесплатная версия допускает 400 вычислительных единиц в месяц и 5 ГБ хранилища. Если этого мало, вы можете перейти на платный план. Более подробную информацию о ценах можно найти на сайте¹.

24.3. Создание конвейера CI/CD для микросервиса

В следующих разделах мы создадим конвейер CI/CD для микросервиса. Наша цель – упростить процесс, чтобы вам было легче его понять. Как только вы усвоите основы, все станет проще. Если у вас есть особые потребности, вы можете обратиться к документации, чтобы найти правильные решения. Я рекомендую начать с малого, а затем расширять и настраивать ваши конвейеры. Излишняя сложность может замаскировать причины потенциальных сбоев, а простота помогает обнаружить их при первом запуске.

Мы будем использовать GitHub для размещения исходного кода и GitHub Actions для создания конвейера. Конвейер начнется с выполнения модульных тестов. Если тесты пройдены, после сборки приложения будет создан образ Docker, который отправится в Docker Hub с тегом latest (последний). Наконец, пакет Helm будет создан и развернут в Kubernetes с помощью Argo CD.

Этот процесс схематически показан на диаграмме на стр. 259.

24.3.1. Установка и настройка Argo CD

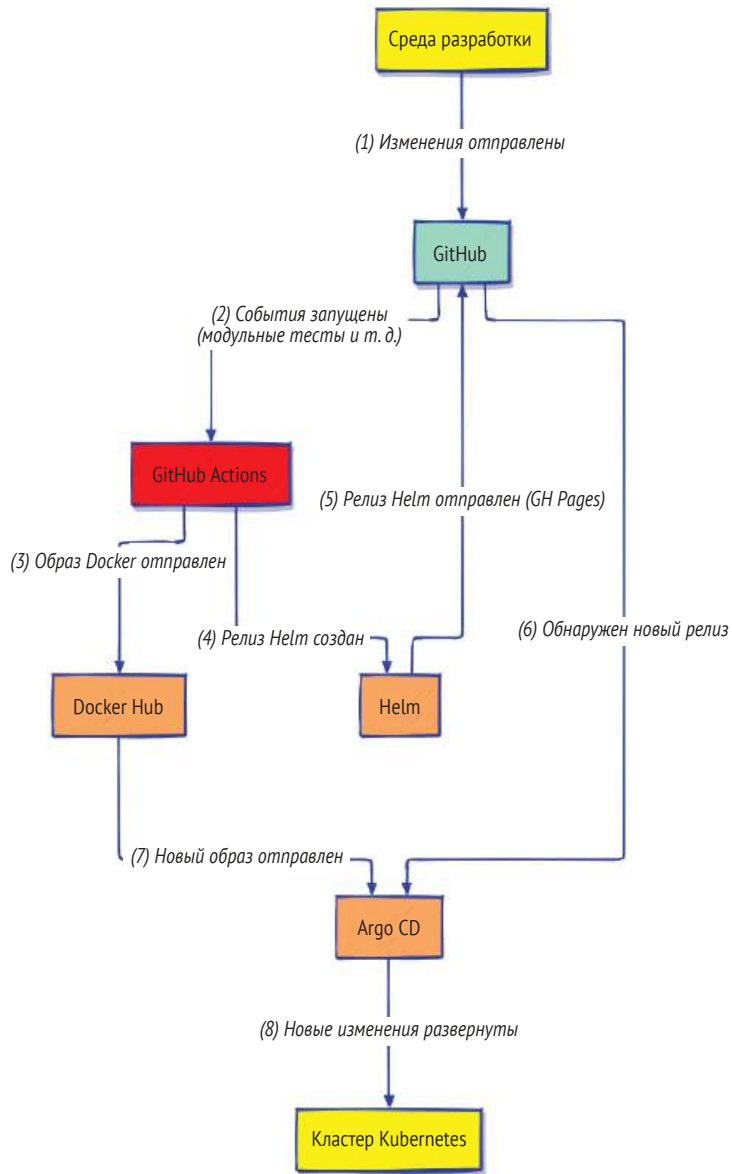
Начнем с развертывания контроллера Ingress. Мы будем использовать контроллер Ingress NGINX:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install nginx-ingress ingress-nginx/ingress-nginx --set controller.publishService.enabled=true
```

Сохраните IP-адрес контроллера Ingress:

```
export INGRESS_IP=$(kubectl -n default get svc | grep ingress | grep LoadBalancer | awk '{print $4}')
```

¹ <https://about.gitlab.com/pricing/>.



CI/CD с GitOps, GitHub, Argo CD и Helm

Установите Argo CD:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Создайте и примените манифест Ingress для Argo CD:

Листинг 24.1 ❖ Файл ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd-server-ingress
  namespace: argocd
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
spec:
  rules:
    - http:
        paths:
          - pathType: Prefix
            path: /
            backend:
              service:
                name: argocd-server
                port:
                  name: https
        host: argocd.${INGRESS_IP}.nip.io
  tls:
    - hosts:
        - argocd.${INGRESS_IP}.nip.io
      secretName: argocd-secret
```

Теперь вы можете получить публичный URL Argo CD:

```
echo https://argocd.${INGRESS_IP}.nip.io
```

По умолчанию используется логин `admin`, а пароль можно получить с помощью следующей команды:

```
kubectll -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d; echo
```

24.3.2. Создание репозитория GitHub для нашего микросервиса

Для начала создайте репозиторий GitHub. На своей машине разработки установите необходимые инструменты для микросервиса Flask.

```
apt get update && apt install -y python3-pip
```

```
pip install virtualenvwrapper
export WORKON_HOME=~/.Envs
```

```
mkdir -p $WORKON_HOME
export VIRTUALENVWRAPPER_PYTHON='/usr/bin/python3'
source /usr/local/bin/virtualenvwrapper.sh
```

Создайте виртуальную среду и установите необходимые зависимости:

```
mkvirtualenv ci-cd-with-argocd-helm-and-github-actions
```

Далее создайте папки для своего приложения Flask и установите его зависимости.

```
mkdir -p ci-cd-with-argocd-helm-and-github-actions
cd ci-cd-with-argocd-helm-and-github-actions
mkdir -p app
mkdir -p charts
pip install Flask==2.2.3
pip install Flask-Testing==0.8.1
pip freeze > app/requirements.txtbash
```

Используйте следующий код для создания простого приложения Flask:

Листинг 24.2 ❖ Файл app/app.py

```
""" Простое приложение todo """
from flask import Flask, jsonify, request

app = Flask(__name__)

tasks = []

@app.route('/tasks', methods=['GET'])
def get_tasks():
    """
    Получаем все задачи
    """
    return jsonify({'tasks': tasks})

# Получение отдельной задачи
@app.route('/tasks', methods=['POST'])
def add_task():
    """
    Добавление задачи
    """
    task = {
        'id': len(tasks) + 1,
        'title': request.json['title'],
        'description': request.json['description'],
    }
    tasks.append(task)
    return jsonify(task), 201

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Создадим несколько модульных тестов для нашего приложения:

Листинг 24.3 ❖ Файл app/test_app.py

```
""" Тестирование приложения """

import unittest
import json
from app import app

class TestApp(unittest.TestCase):
    """
    Тест приложения
    """

    def setUp(self):
        """
        Настройка тестового клиента
        """
        self.app = app.test_client()

    def test_get_tasks(self):
        """
        Проверка получения всех задач
        """
        response = self.app.get('/tasks')
        self.assertEqual(response.status_code, 200)

    def test_add_task(self):
        """
        Проверка добавления задачи
        """
        response = self.app.post('/tasks', json={
            'title': 'Test task',
            'description': 'Test description'
        })
        self.assertEqual(response.status_code, 201)
        self.assertEqual(json.loads(response.get_data()), {
            'id': 1,
            'title': 'Test task',
            'description': 'Test description'
        })

if __name__ == '__main__':
    unittest.main()
```

Мы также можем протестировать приложение вручную, чтобы убедиться, что оно работает, как и ожидалось:

```
python app/test_app.py
```

Вы должны увидеть следующий вывод:

```
Ran 2 tests in 0.009s
```

```
OK
```

Далее мы создадим Dockerfile:

Листинг 24.4 ❖ Файл app/Dockerfile

```
# Использовать официальную среду выполнения Python в качестве родительского образа
FROM python:3.9-slim-buster
# Установить рабочий каталог /app
WORKDIR /app
# Скопировать содержимое текущего каталога в контейнер в /app
COPY . /app
# Установить все необходимые пакеты, указанные в requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
# Сделайте порт 5000 общедоступным за пределами этого контейнера
EXPOSE 5000
# Определить переменную окружения
CMD ["python", "app.py"]
```

В этой книге репозиторий называется eon01/ci-cd-with-argocd-helm-and-github-actions. Если вы хотите следовать инструкциям, используйте свое собственное имя репозитория либо клонируйте репозиторий из моей учетной записи GitHub¹.

Чтобы отправить коммит в репозиторий из командной строки:

```
export user_name=<your username>
export repository_name="ci-cd-with-argocd-helm-and-github-actions"

git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:${user_name}/${repository_name}.git
git push -u origin main
```

Прежде чем приступить к этому шагу, вы должны были сгенерировать ключ SSH и добавить его в свою учетную запись GitHub с правами чтения/записи. Если вы этого еще не сделали, следуйте инструкциям, приведенным в документации².

24.3.3. Создание учетной записи Docker Hub

Чтобы отправить образ Docker в систему контроля версий и использовать его в нашем кластере Kubernetes, нам нужно создать учетную запись Docker Hub. Вы можете создать ее на сайте³.

В оставшейся части этого руководства предполагается, что ваше имя пользователя Docker Hub совпадает с вашим именем пользователя GitHub.

¹ <https://github.com/eon01/ci-cd-with-argocd-helm-and-github-actions>.

² <https://docs.github.com/en/github/authenticating-to-github/connecting-to-github-with-ssh>.

³ <https://hub.docker.com/signup>.

24.3.4. Настройка GitHub Actions

Фактически настройка GitHub Actions уже завершена.

Прежде чем продолжить, нам осталось настроить несколько секретов в репозитории GitHub. GitHub Actions будет использовать эти секреты для отправки нашего образа Docker в Docker Hub. Для этого перейдите в свой репозиторий GitHub и нажмите **Settings** (Настройки), затем **Secrets and variables** (Секреты и переменные). Выберите **Actions** (Действия), потом нажмите **New repository secret** (Новый секрет репозитория) и добавьте следующие секреты:

- DOCKER_USERNAME: ваше имя пользователя Docker Hub;
- DOCKER_PASSWORD: ваш пароль Docker Hub.

Действия GitHub – это рабочие процессы, запускаемые событиями. В нашем случае мы хотим запустить рабочий процесс при отправке обновления в главную ветку нашего репозитория.

Каждый рабочий процесс определяется в файле YAML. В нашем случае мы будем использовать следующий файл:

`.github/workflows/main.yaml`

Создайте структуру каталогов:

```
mkdir -p .github/workflows
```

Создайте файл рабочего процесса:

Листинг 24.5 ❖ Файл `.github/workflows/main.yaml`

name: Test, build, push Docker image and release Helm chart
on:

```
  push:
    branches: [main, master]
  workflow_dispatch:
jobs:
  perform_unit_tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-python@v2
      - run: pip install -r app/requirements.txt
      - run: python app/test_app.py
  build_and_push_to_docker_hub:
    runs-on: ubuntu-latest
    needs: perform_unit_tests
    steps:
      - uses: actions/checkout@v2
      - run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin
      - run: docker build -t "${{ secrets.DOCKER_USERNAME }}/${{repository_name}}:latest" -f app/Dockerfile app
```

```

- run: docker push \${{ secrets.DOCKER_USERNAME }}/${repository_name}:latest
release_helm_chart:
permissions:
  contents: write
runs-on: ubuntu-latest
needs: build_and_push_to_docker_hub
steps:
  - name: Checkout
    uses: actions/checkout@v3
    with:
      fetch-depth: 0
  - name: Configure Git
    run: |
      git config user.name "\${GITHUB_ACTOR}"
      git config user.email "\${GITHUB_ACTOR}@users.noreply.github.com"
  - name: Install Helm
    uses: azure/setup-helm@v3
  - name: Run chart-releaser
    uses: helm/chart-releaser-action@v1.5.0
  env:
    CR_TOKEN: "\${{ secrets.GITHUB_TOKEN }}"

```

Приведенный выше код определяет рабочий процесс GitHub Actions для автоматизации тестирования, сборки и отправки образов Docker и выпуска чартов Helm:

- `name`: эта строка указывает имя рабочего процесса;
- `on`: эта часть определяет события, которые запускают рабочий процесс. В данном случае любая отправка обновления в ветку `main` или `master` или ручной запуск рабочего процесса (`workflow_dispatch`) инициирует рабочий процесс.

Рабочий процесс состоит из трех заданий:

- 1) `perform_unit_tests`: это задание отвечает за запуск модульных тестов:
 - оно указывает, что задание должно выполняться в последней версии Ubuntu;
 - оно проверяет код с помощью `actions/checkout@v2` и настраивает Python;
 - затем оно устанавливает требования приложения и запускает модульные тесты;
- 2) `build_and_push_to_docker_hub`: это задание создает образ Docker и отправляет его в Docker Hub:
 - это задание также выполняется в последней версии Ubuntu и зависит от успешного завершения задания `perform_unit_tests`;
 - оно проверяет код;
 - оно входит в Docker, используя секреты `DOCKER_USERNAME` и `DOCKER_PASSWORD`;
 - затем оно создает образ Docker с помощью Dockerfile, расположенного в каталоге `app`, и отправляет его в Docker Hub;
 - вместо Docker Hub можно использовать собственный реестр Docker;

- 3) `release_helm_chart`: эта задача выпускает чарт Helm:
- настраивает разрешения задания, чтобы разрешить доступ на запись к содержимому репозитория;
 - проверяет код и задает для Git значение GitHub Actor в качестве имени пользователя и адреса электронной почты;
 - устанавливает Helm с помощью действия `azure/setup-helm`;
 - выпускает чарт с помощью действия `helm/chart-releaser-action` с `GITHUB_TOKEN` для аутентификации.

Добавьте новые файлы в репозиторий и отправьте их в Git:

```
git add .
git commit -m "Add CI/CD workflow"
git push origin main
```

24.3.5. Создание чарта Helm

Helm поможет нам упаковать и развернуть приложение в Kubernetes. Всякий раз, когда возникает необходимость развернуть новую версию нашего приложения, нам просто нужно обновить чарт Helm и отправить его в репозиторий GitHub. Затем GitHub Actions создаст новый релиз с последней версией приложения.

Вам нужно будет установить Helm на свой компьютер. Инструкции по установке можно найти на сайте¹.

Для начала создайте новый чарт Helm:

```
mkdir -p charts/cicd-flask
helm create charts/cicd-flask
```

Чтобы изменить значения по умолчанию нашего чарта Helm, отредактируйте файл `values.yaml`. Используйте следующие значения:

Листинг 24.6 ❖ Файл `charts/cicd-flask/values.yaml`

```
replicaCount: 1
image:
  repository: ${user_name}/${repository_name}
  pullPolicy: IfNotPresent
  tag: "latest"
imagePullSecrets: []
nameOverride: ""
fullnameOverride: ""
serviceAccount:
  create: true
  annotations: {}
  name: ""
service:
```

¹ <https://helm.sh/docs/intro/install/>.

```

    type: ClusterIP
    port: 5000
  ingress:
    enabled: true
    className: ""
    annotations:
      kubernetes.io/ingress.class: nginx
    hosts:
      - host: flask.${INGRESS_IP}.nip.io
        paths:
          - path: /tasks
            pathType: ImplementationSpecific
    tls: []
  autoscaling:
    enabled: false
    minReplicas: 1
    maxReplicas: 100
    targetCPUUtilizationPercentage: 80

```

Теперь создадим еще один файл значений для использования в нашей промежуточной среде:

```
cp charts/cicd-flask/values.yaml charts/cicd-flask/values-staging.yaml
```

Что касается значений в файле `values-staging.yaml`, вы можете, среди прочего, настроить количество реплик или тег образа.

Теперь отредактируйте файл `deployment.yaml`, чтобы обновить зонды `livenessProbe` и `readinessProbe` по умолчанию:

Листинг 24.7 ❖ Файл `charts/cicd-flask/templates/deployment.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "cicd-flask.fullname" . }}
  labels:
    {{- include "cicd-flask.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "cicd-flask.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      {{- with .Values.podAnnotations }}
      annotations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
    labels:
      {{- include "cicd-flask.selectorLabels" . | nindent 8 }}

```

```
spec:
  {{- with .Values.imagePullSecrets }}
  imagePullSecrets:
    {{- toYaml . | nindent 8 }}
  {{- end }}
  serviceAccountName: {{ include "cicd-flask.serviceAccountName" . \
}}
  securityContext:
    {{- toYaml .Values.podSecurityContext | nindent 8 }}
  containers:
    - name: {{ .Chart.Name }}
      securityContext:
        {{- toYaml .Values.securityContext | nindent 12 }}
      image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.
AppVersion }}"
      imagePullPolicy: {{ .Values.image.pullPolicy }}
      ports:
        - name: http
          containerPort: {{ .Values.service.port }}
          protocol: TCP
      livenessProbe:
        httpGet:
          path: /tasks
          port: http
      readinessProbe:
        httpGet:
          path: /tasks
          port: http
      resources:
        {{- toYaml .Values.resources | nindent 12 }}
    {{- with .Values.nodeSelector }}
    nodeSelector:
      {{- toYaml . | nindent 8 }}
    {{- end }}
    {{- with .Values.affinity }}
    affinity:
      {{- toYaml . | nindent 8 }}
    {{- end }}
    {{- with .Values.tolerations }}
    tolerations:
      {{- toYaml . | nindent 8 }}
    {{- end }}
```

После редактирования чарта Helm нам обычно нужно упаковать его и отправить в репозиторий Helm. Для этого пригодятся такие инструменты, как ChartMuseum¹. Но в этой книге мы опубликуем чарт Helm на сайте GitHub Pages.

Так или иначе, мы уже настроили наш рабочий процесс GitHub для публикации чарта.

¹ <https://chartmuseum.com/>.

Чтобы получить доступ к пакетам с помощью URL, нам нужно преобразовать репозиторий GitHub в сайт GitHub Pages. Выполните следующие действия.

Создайте новую пустую ветку с именем `gh-pages` и отправьте ее в репозиторий.

```
git checkout --orphan gh-pages
git rm -rf .
git commit -m "Initial commit" --allow-empty
git push --set-upstream origin gh-pages
```

Чтобы развернуть свой сайт с помощью GitHub Pages, выполните следующие действия:

- 1) перейдите в настройки репозитория и прокрутите вниз до раздела **GitHub Pages**;
- 2) в разделе **Source** выберите **Deploy from a branch** (Развернуть из ветки);
- 3) выберите ветку `gh-pages` и каталог `/` (корневой);
- 4) предоставьте своему рабочему процессу права на чтение и запись в репозиторий, перейдя в **Actions** (Действия) в настройках репозитория, нажав **General** (Общие) и выбрав **Read and write permissions** (Разрешения на чтение и запись) в разделе **Workflow permissions** (Разрешения рабочего процесса).

Добавьте новые файлы в репозиторий.

```
git add .
git commit -m "Add Helm chart"
git push
```

24.3.6. Создание приложения Argo CD

Создайте ресурс Argo CD Application, который даст указание Argo CD развернуть наше приложение с помощью созданного ранее чарта Helm. Он будет искать чарт Helm на сайте GitHub Pages, который мы создали ранее, используя файл `index.yaml`. Этот файл содержит список пакетов, доступных в репозитории, и был сгенерирован рабочим процессом GitHub с помощью действия `Helm Chart Releaser`.

Обратите внимание, что мы используем шаблон `https://${user_name}.github.io/${repository_name}/` при указании адреса репозитория Helm. Это связано с тем, что мы размещаем наш чарт Helm на GitHub Pages. Если адрес вашего репозитория – `https://github.com/you/app`, то адрес репозитория Helm будет `https://you.github.io/app/`.

Мы также используем файл `values-staging.yaml` в качестве файла значений для нашей промежуточной среды. Если вы хотите выполнить развертывание в производстве, используйте вместо него файл `values.yaml`. Вы также можете создать новые файлы значений для других сред.

```
kubectl apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cicd-flask
  namespace: argocd
spec:
  destination:
    namespace: default
    server: https://kubernetes.default.svc
  project: default
  source:
    chart: cicd-flask
    repoURL: https://{user_name}.github.io/{repository_name}/
    targetRevision: 0.1.0
    helm:
      valueFiles:
        - values-staging.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true
```

В этом примере мы используем DigitalOcean. Однако существует известная проблема с CiliumIdentity CRD (описанная на GitHub), которая препятствует разворачиванию приложения¹. Чтобы решить эту проблему, нам нужно добавить исключение в Argo CD ConfigMap.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/name: argocd-cm
    app.kubernetes.io/part-of: argocd
  name: argocd-cm
  namespace: argocd
data:
  resource.exclusions: |
    - apiGroups:
      - cilium.io
      kinds:
      - CiliumIdentity
      clusters:
      - "*"

```

Если вы используете другого поставщика облачных услуг, вам может не потребоваться это делать.

¹ <https://github.com/argoproj/argo-cd/issues/10456>.

24.3.7. Автоматизация развертывания новых версий

Argo CD постоянно отслеживает репозиторий Git и автоматически развертывает любые изменения в приложении. Чтобы проверить это, мы обновим тег образа в файле `values-staging.yaml`.

Вместо одной реплики у нас будет три.

Листинг 24.8 ❖ Файл `charts/cicd-flask/values-staging.yaml`

```
# Меняем количество реплик на 3
replicaCount: 3
image:
  repository: ${user_name}/${repository_name}
  pullPolicy: IfNotPresent
  tag: "latest"
imagePullSecrets: []
nameOverride: ""
fullNameOverride: ""
serviceAccount:
  create: true
  annotations: {}
  name: ""
service:
  type: ClusterIP
  port: 5000
ingress:
  enabled: true
  className: ""
  annotations:
    kubernetes.io/ingress.class: nginx
  hosts:
    - host: flask.${INGRESS_IP}.nip.io
      paths:
        - path: /tasks
          pathType: ImplementationSpecific
  tls: []
autoscaling:
  enabled: false
  minReplicas: 1
  maxReplicas: 100
  targetCPUUtilizationPercentage: 80
```

Обновим версию релиза в файле `charts/cicd-flask/Chart.yaml`:

```
sed -i 's/version: .*/version: 0.2.0/g' charts/cicd-flask/Chart.yaml
```

Добавим новые файлы в репозиторий:

```
git add .
git commit -m "Update values-staging.yaml"
git push
```

Обновим приложение Argo CD:

```
kubectl apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cicd-flask
  namespace: argocd
spec:
  destination:
    namespace: default
    server: https://kubernetes.default.svc
  project: default
  source:
    chart: cicd-flask
    repoURL: https://${user_name}.github.io/${repository_name}/
    targetRevision: 0.2.0
    helm:
      valueFiles:
        - values-staging.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: true
EOF
```

Вы должны увидеть новую версию развернутого приложения.

Вот что происходит на протяжении всего процесса:

- вы отправляете изменения в главную ветку репозитория;
- запускается рабочий процесс GitHub;
- рабочий процесс создает образ Docker и отправляет его в Docker Hub;
- рабочий процесс упаковывает чарт Helm и отправляет его на сайт GitHub Pages;
- создается новый файл `index.yaml`, который ссылается на новую версию чарта Helm и архивный файл;
- Argo CD обнаруживает изменения в файле `index.yaml` и развертывает новую версию приложения;
- приложение развертывается с новыми изменениями.

Рекомендуется создавать новый релиз всякий раз, когда вы вносите изменения в чарт. Это связано с тем, что релизы чартов должны быть неизменяемыми, чтобы избежать любых проблем и сохранить воспроизводимость развертываний.

Глава 25

Послесловие

25.1. Что дальше?

Итак, эта книга подошла к концу! Надеюсь, что путешествие в мир облачных микросервисов и Kubernetes дало вам ценные идеи и практические знания.

Мы рассмотрели различные темы, связанные с созданием, развертыванием и управлением микросервисами с использованием Kubernetes, и углубились в такие важные концепции, как контейнеризация, архитектура, стратегии управления ресурсами, автоматическое масштабирование, стратегии развертывания, наблюдаемость и GitOps.

К настоящему моменту у вас должно быть четкое понимание принципов и передовых методов разработки и эксплуатации масштабируемых, устойчивых и эффективных облачных микросервисов в среде Kubernetes.

Помните, что путь к совершенству лежит через практику, и лучший способ учиться – это делать. Поэтому если вы еще не работали с кодом, я рекомендую вам начать с примеров в книге и поэкспериментировать с концепциями и методами, представленными в ней.

Надеюсь, вам было приятно читать эту книгу. Продолжайте учиться и оставайтесь любознательными!

25.2. Благодарность

Спасибо, что присоединились ко мне в этом увлекательном проекте, и желаю вам всего наилучшего в ваших будущих начинаниях.

25.3. Об авторе

Аймен Эль Амри – инженер-программист, автор и предприниматель. Он основал сообщество разработчиков FAUN – платформу, которая помогает разработчикам учиться и расти в карьере. Он также является автором нескольких книг по разработке программного обеспечения и облачным вычислениям. Вы можете найти его в Twitter (X) и LinkedIn^{1,2}.

25.4. Присоединяйтесь к сообществу

Если вам понравилось это руководство, вы можете присоединиться к сообществу FAUN³, чтобы получать уведомления о будущих бесплатных и платных руководствах, курсах и других ресурсах.

25.5 Обратная связь

Я с нетерпением жду ваших идей и отзывов об этой новой книге. Ваши сообщения помогут мне совершенствовать и дополнять ее.

Я буду рад, если вы поделитесь своим отзывом, отправив мне письмо на адрес aymen@faun.dev. Ваш опыт и слова могут послужить примером для будущих читателей. И конечно, я бы с удовольствием поделился вашим отзывом с более широким сообществом моих читателей.

¹ <https://twitter.com/eon01>.

² <https://www.linkedin.com/in/elamriaymen/>.

³ <https://faun.dev/join>.

Предметный указатель

- Автомасштабирование кластера, 141
- Желаемое состояние, 242
- Журнал, 213
- Журналирование, 19
- Задание, 252
- Закон
 - возрастающей сложности, 13
 - постоянных изменений, 13
 - снижения качества, 13
- Зонд, 105
 - готовности, 105
 - жизнеспособности, 105
- Качество обслуживания, 118
- Манифест зависимостей, 15
- Методология двенадцати факторов, 14
- Метрика, 213
- Микросервис, 21
- Мониторинг, 211
 - белый ящик, 212
 - черный ящик, 212
- Наблюдаемость, 212
- Непрерывное развертывание, 238
- Облачные приложения, 24
- Оператор Kubernetes , 219
- Переменные среды, 98
- Плоскость управления, 40
- Под, 58
 - автомасштабирование
 - вертикальное, 132, 146
 - горизонтальное, 132
 - допуск, 170
 - журнал, 61
- Привязка портов, 17
- Развертывание, 58, 62
 - канареечное, 193
 - сине-зеленое, 191
- Распространитель
 - багажа, 233
 - контекста, 232
 - S3, 233
- Ресурс
 - выделяемый, 117
 - несжимаемый, 116
 - резервированный, 117
 - сжимаемый, 116
- Секреты, 98
- Сервис, 62
 - внешний, 15
 - реестр, 125
- Скачок ЦП, 117
- Скользящее обновление, 204
- Том, 79
 - заявка, 79
 - постоянный, 79
- Трассировка, 214
- Узел
 - антисходство, 179
 - допуск, 50
 - кордон, 186
 - ограничение, 170
 - окраска, 49
 - пул, 49, 187
 - слив, 185
 - сходство, 176
- Утилизируемость, 19
- Хук, 252
- Чарт, 245

Аймен Эль Амри

Облачные микросервисы

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Перевод *Яценков В. С.*
Корректор *Синяева Г. И.*
Верстка *Чаннова А. А.*
Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 22,43. Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**



«Рекомендую всем, кто действительно хочет быть многопрофильным архитектором».

Александр Бардаш, главный архитектор интеграционных платформ в МТС

Данная книга раскроет весь потенциал Kubernetes – самой популярной платформы оркестровки контейнеров в мире. Вы узнаете, как можно использовать Kubernetes для создания надежной, масштабируемой и отказоустойчивой микросервисной архитектуры. Подробно раскрыты особенности применения микросервисов, стратегий высокой доступности, CI/CD, GitOps и наблюдаемости в мире облачных технологий. Представлены практические примеры применения таких инструментов, как Docker, Rancher, Terraform, Operators, Helm, Prometheus, Istio, Grafana, OpenTelemetry, Jaeger, Loki и другие.

Основные темы:

- суть облачного подхода к созданию программного обеспечения и микросервисов;
- понимание архитектуры Kubernetes и ее основных компонентов;
- запуск Kubernetes локально и в облаке;
- использование Rancher для управления контейнерами и Kubernetes;
- управление сохранением данных в Kubernetes;
- различные типы сервисов в Kubernetes и назначение каждого из них;
- применение Operators, Helm, Terraform и других инструментов для создания и управления кластерами Kubernetes;
- стратегии разворачивания обновлений: сине-зеленая, канареечная, скользящая;
- использование Istio для внедрения сервисной сетки в Kubernetes;
- реализация технологий наблюдаемости и GitOps в Kubernetes с помощью Prometheus, Grafana, Jaeger, Loki, OpenTelemetry, Argo CD и Ansible.

Издание предназначено инженерам DevOps, архитекторам ПО, а также будет полезно системным администраторам и разработчикам микросервисов.



Аймен Эль Амри – инженер-программист, предприниматель и автор с обширным опытом в различных областях технологий, включая DevOps & Cloud Native, Cloud Architecture, Python, NLP. Автор нескольких книг и курсов по разработке ПО и облачным вычислениям. Основатель таких проектов, как eralabs, FAUN и MarketToDev. Его талант преподавателя проявляется в умении упрощать сложные концепции до понятного языка и предлагать реальные примеры, которые близки аудиторией.



ISBN 978-5-93700-324-9



9 785937 003249 >